book 17

the interview repertoire

working answers for frequent questions

Raeliosol

version 6.0, compiled 10 2026

Contents

1. the 2026 interview landscape 8
   1.1. how the chapters are labeled 8
   1.2. what this book is not 8
   1.3. how to drill 9
   1.4. the 2026 facts, pinned 9
2. paradigms: oop, fp, mvc, mvvm, ecs 11
   2.1. oop in go: methods without classes [EWC] 11
   2.2. oop in c#: the keyword version [EWC] 12
   2.3. fp: pure functions, closures, composition [EWC] 14
   2.4. mvc, mvvm, and ecs: where state lives [DRILL] 16
3. data structures, implemented 18
   3.1. the dynamic array [TDD] 18
   3.2. the linked list [TDD] 20
   3.3. the hash map [TDD] 21
   3.4. the binary search tree [TDD] 23
   3.5. the heap [TDD] 24
   3.6. the trie [TDD] 25
   3.7. the graph [TDD] 27
4. graphs: dfs, bfs, topological sort 30
   4.1. dfs and bfs, one shape apart [TDD] 30
   4.2. shortest path on an unweighted graph [TDD] 31
   4.3. kahn's topological sort [TDD] 32
5. classics part 1: warmup, arrays, hashing 35
   5.1. the string reverser and the palindrome [TDD] 35
   5.2. primes, by trial and by sieve [TDD] 36
   5.3. permutations, generated and detected [TDD] 36
   5.4. linked list operations [TDD] 37
   5.5. two sum, both ways [TDD] 38
   5.6. isolated islands [TDD] 40
6. classics part 2: dynamic programming and tries 42
   6.1. the ladder: recursion, memo, table [TDD] 42
   6.2. the trie earns its keep: autocomplete [TDD] 43
   6.3. string manipulation [TDD] 44
7. javascript core: scope, closures, event loop 47
   7.1. hoisting and the temporal dead zone [EWC] 47
   7.2. closures [EWC] 48
   7.3. precedence, coercion, and equality [EWC] 48
   7.4. null, undefined, NaN, and the optionals [EWC] 49
   7.5. prototypes [EWC] 50
   7.6. the event loop [EWC] 51
8. javascript and the dom 53
   8.1. what the dom is [EWC] 53
   8.2. center a div, four honest ways [EWC] 54
   8.3. centering edge cases: the group, the overflow, the choice [TDD] 55
   8.4. class versus arrow, map filter reduce, async await, es6 [EWC] 57
9. javascript from scratch 60
   9.1. word count in one loop [TDD] 60
   9.2. the event emitter [TDD] 60

9.3. map, filter, reduce [TDD] 61
9.4. a promise subset [TDD] 62
9.5. the combinator ladder: race, allSettled, any [TDD] 64
9.6. a signals core [TDD] 66
9.7. signals edge cases: the diamond, disposal, and the branch switch [TDD] 67
10. data plumbing: csv to mongodb 72
10.1. parsing csv without lying about it [TDD] 72
10.2. documents, batches, and the one-method interface [TDD] 74
10.3. the real run, docker gated [DRILL] 76
11. react core: components, props, virtual dom 78
11.1. function components versus class components [EWC] 78
11.2. props, children, and drilling [EWC] 78
11.3. the virtual dom [EWC] 79
12. react hooks and routing 81
12.1. the built-in hooks that render server-side [EWC] 81
12.2. a custom hook is just a function that calls hooks [EWC] 83
12.3. react-router, one route table [EWC] 84
12.4. react 19: actions, use, and the compiler [EWC] 84
13. react state: context, redux, graphql 86
13.1. context versus a store [EWC] 86
13.2. what rtk query automates [EWC] 87
13.3. graphql, rest, and n+1 [EWC] 88
14. react build: a listing page, test driven 90
14.1. beat one: red, the page does not exist [TDD] 90
14.2. beat two: green, three states as early returns [TDD] 91
14.3. beat three: the virtual dom from scratch [TDD] 92
15. go runtime: scheduler, gc, patterns 96
15.1. parallelism is not concurrency [EWC] 96
15.2. the scheduler versus the os [EWC] 97
15.3. the garbage collector, and 1.26's green tea [DRILL] 97
15.4. concurrent patterns: pool, pipeline, cancellation [EWC] 97
15.5. closures and the loop variable [EWC] 99
15.6. generics and their constraint vocabulary [EWC] 99
15.7. context, timeouts, and errors as values [EWC] 100
15.8. the singleton under contention [TDD] 101
16. go testing: mocks and dependency injection 103
16.1. the four layers [TDD] 103
16.2. mocks without a mocking framework [TDD] 106
16.3. what each layer's suite proves [TDD] 107
16.4. the golden master, ported small [EWC] 108
16.5. determinism discipline for cross-language twins [DRILL] 112
17. database answers 113
17.1. acid and base [DRILL] 113
17.2. money is cents [TDD] 113
17.3. normalization, proven on both shapes [TDD] 115
17.4. indexing, proven without a stopwatch [TDD] 116
17.5. optimistic versus pessimistic locking [TDD] 117
17.6. cursors, limit offset, and json [DRILL] 119
17.7. mongo geo: the predicate underneath [TDD] 119
18. network answers 122
18.1. the stack in one breath [DRILL] 122

18.2. http, the protocol with semantics [EWC] 123
18.3. websockets from the metal [TDD] 123
18.4. https, certificates, symmetric versus asymmetric [DRILL] 126
18.5. graphql versus rest, said precisely [DRILL] 126
19. system design answers 127
19.1. scaling with and without the cloud [DRILL] 127
19.2. gateway, reverse proxy, kubernetes [DRILL] 127
19.3. bottlenecks: tracing, logging, metrics [DRILL] 128
19.4. cap, event-driven, orchestration [DRILL] 128
19.5. the 300k concurrent users sketch [DRILL] 128
19.6. queue before the database write [DRILL] 129
19.7. the remaining inventory, one paragraph each [DRILL] 130
20. go build part 1: fetchers and rest apis 131
20.1. the parallel fetcher, five workers [TDD] 131
20.2. the rest api: stdlib, sqlite, cache, rate limit [TDD] 133
20.3. the client-side limiter: a fake clock, no sleeps [TDD] 134
20.4. the schema generator/parser [TDD] 138
21. go build part 2: storefront and live features 140
21.1. the storefront and the reservation transaction [TDD] 140
21.2. the feed, bolted on step by step [TDD] 141
21.3. graceful shutdown and the live deploy [TDD] 142
22. go streaming: jetstream batch pipelines 145
22.1. the embedded server, and two windows bugs [TDD] 145
22.2. batch produce, durable consume [TDD] 146
22.3. the offset is the ack floor [TDD] 148
23. algorithm patterns: window, pointers, search, greedy, bits 150
23.1. longest substring without repeating characters [TDD] 150
23.2. two pointers: sorted pair sum, in-place dedupe [TDD] 151
23.3. binary search and the rotated variant [TDD] 152
23.4. greedy: jump game and interval scheduling [TDD] 153
23.5. backtracking: subsets and permutations [TDD] 155
23.6. bit manipulation: single number, counting bits [TDD] 156
23.7. sorting internals, which sort when [DRILL] 157
24. weighted graphs: dijkstra, union-find, mst 159
24.1. dijkstra, heap-driven [TDD] 159
24.2. union-find with path compression [TDD] 161
24.3. kruskal and prim, the mst pair [TDD] 162
24.4. when bellman-ford and floyd-warshall [DRILL] 164
25. distributed systems answers 165
25.1. replication: leaders, followers, and lag [DRILL] 165
25.2. sharding and partitioning strategies [DRILL] 165
25.3. consensus and raft, honestly [DRILL] 166
25.4. crdts versus distributed transactions [DRILL] 166
25.5. delivery semantics and idempotency [DRILL] 167
26. systems and runtime answers 168
26.1. stack, heap, and allocators [DRILL] 168
26.2. virtual memory, pages, and the tlb [DRILL] 168
26.3. processes versus threads [DRILL] 169
26.4. epoll and the event loop [DRILL] 169
26.5. c networking, sockets end to end [DRILL] 170
27. mathematics answers 171

27.1. asymptotics and solving recurrences [DRILL] ... 171
27.2. probability you can compute at the board [DRILL] ... 171
27.3. counting and combinatorics [DRILL] ... 172
27.4. reading statistics: baselines, p-values, overfitting [DRILL] ... 172
27.5. the statistics workspace: exact binomial, auc, bootstrap, verdicts [TDD] ... 173
27.6. backtesting without lookahead [DRILL] ... 179
27.7. guarded promotion and multiple testing in a game tool [DRILL] ... 179
28. data mining answers ... 181
28.1. similarity and distance measures [DRILL] ... 181
28.2. clustering: k-means, hierarchical, dbscan [DRILL] ... 181
28.3. itemsets: apriori and fp-growth [DRILL] ... 182
28.4. cosine neighbors and smoothed bayes [TDD] ... 182
28.5. the levelwise pass, counted and pruned [TDD] ... 185
28.6. pagerank and the pipeline view [DRILL] ... 187
29. python answers ... 189
29.1. the gil and what it actually locks [DRILL] ... 189
29.2. the data model: dunders and protocols [DRILL] ... 189
29.3. decorators, closures, scopes [DRILL] ... 190
29.4. asyncio versus threads [DRILL] ... 190
30. lua answers ... 192
30.1. metatables and metamethods [DRILL] ... 192
30.2. coroutines versus javascript async [DRILL] ... 192
30.3. closures and environments against javascript [DRILL] ... 193
31. c sharp answers ... 194
31.1. async await machinery: state machines and context [DRILL] ... 194
31.2. linq as an embedded dsl [DRILL] ... 194
31.3. value versus reference, boxing [DRILL] ... 195
31.4. the gc generations against go's [DRILL] ... 195
32. css answers: box model, cascade, layout ... 197
32.1. the box model and the two box sizes [EWC] ... 197
32.2. the cascade, specificity, and selector matching [TDD] ... 198
32.3. flex and grid, the layout answers [EWC] ... 201
32.4. responsive and media queries [TDD] ... 203
32.5. the spoken css answers [DRILL] ... 205
33. the rendering pipeline and accessibility ... 207
33.1. parse to dom and cssom [EWC] ... 207
33.2. layout, paint, composite [DRILL] ... 207
33.3. reflow versus repaint versus compositing [TDD] ... 207
33.4. the accessibility tree and semantic html [TDD] ... 208
33.5. aria, contrast, and keyboard paths [TDD] ... 210
33.6. the spoken rendering answers [DRILL] ... 211
34. security answers: the named taxonomy ... 213
34.1. the taxonomy, owasp-shaped [DRILL] ... 213
34.2. xss and csrf [TDD] ... 213
34.3. cors and the same-origin rule [TDD] ... 215
34.4. sql injection [TDD] ... 217
34.5. security headers: csp, hsts, and friends [TDD] ... 218
34.6. authn, authz, tokens: oauth and oidc at interview altitude [DRILL] ... 220
34.7. attack rounds [DRILL] ... 221
35. back of the envelope, said aloud ... 222
35.1. the method [DRILL] ... 222

35.2. the numbers every round expects [DRILL] ... 222
35.3. storage and memory, worked [DRILL] ... 223
35.4. qps, fanout, headroom, worked [DRILL] ... 223
35.5. five boards in ten minutes [DRILL] ... 223
36. system design prompts, four worked ... 225
36.1. how to run a design prompt [DRILL] ... 225
36.2. url shortener [TDD] ... 225
36.3. rate limiter [DRILL] ... 228
36.4. news feed [DRILL] ... 229
36.5. chat [DRILL] ... 230
36.6. long polling, sse, websockets: the delivery ladder [DRILL] ... 231
37. behavioral answers, from the register ... 233
37.1. star, but honest [DRILL] ... 233
37.2. the story cards [DRILL] ... 233
37.3. tell me about yourself [DRILL] ... 235
37.4. conflict, failure, ownership [DRILL] ... 235
37.5. why leaving, why us, weakness [DRILL] ... 236
37.6. the drill and the traps [DRILL] ... 237
38. the observed round, take-homes, debugging ... 239
38.1. the first five minutes [DRILL] ... 239
38.2. narration while writing [DRILL] ... 239
38.3. blanking recovery [DRILL] ... 240
38.4. ai-assist norms [DRILL] ... 240
38.5. timed mocks from this book [DRILL] ... 240
38.6. take-home discipline [DRILL] ... 241
38.7. the debugging round [DRILL] ... 241
39. offers and negotiation ... 243
39.1. reading an offer [DRILL] ... 243
39.2. anchoring from the record [DRILL] ... 243
39.3. competing timelines [DRILL] ... 244
39.4. the scripts [DRILL] ... 244
39.5. closing the loop after a no [DRILL] ... 245
40. the transfer answer: java and rust shops ... 246
40.1. the transfer method [DRILL] ... 246
40.2. java, from the record [DRILL] ... 246
40.3. rust, from the metal [DRILL] ... 247
40.4. the traps [DRILL] ... 248
41. the one billion row challenge ... 249
41.1. the challenge and its rules [DRILL] ... 249
41.2. the exact arithmetic [TDD] ... 250
41.3. the sqlite lane [TDD] ... 252
41.4. c: the window and the table [EWC] ... 254
41.5. rust: byte walks and an even table [EWC] ... 257
41.6. go: map against table, measured [EWC] ... 260
41.7. java: byte windows and a pinned charset [EWC] ... 262
41.8. c sharp: byte keys and snapped cuts [EWC] ... 264
41.9. javascript: the map that stays [EWC] ... 266
41.10. python: bytes windows, processes at the end [EWC] ... 268
41.11. lua: interning is the hash map [EWC] ... 269
41.12. the full-scale table [TDD] ... 271
41.13. the divergences that stay [TDD] ... 273

42. appendices: question index and sources 274
42.1. the question to section index 274
42.2. the workspaces behind the labels 283
42.3. pinned sources 284

1. the 2026 interview landscape

The interview loop this book drills for changed shape between 2024 and 2026. AI-assisted screening spread first, then the live rounds absorbed it: several major employers now run AI-enabled coding rounds or comprehension-focused experiments, and every serious loop assumes the candidate has an assistant one keystroke away. What did not shrink is the live problem solving norm. Medium difficulty data structure problems, solved with narration, still gate the generalist software loop. On what the narration is worth, the pinned data cuts both ways and this book takes the honest reading: a 100k interview analysis by interviewing.io found code and problem solving scores dominate pass decisions and communication is the smallest rubric factor, and the same narration is how a problem solving score gets observed at all, which is what a loop does now that unassisted code alone proves so much less.

Three consequences organize this book.

First, every answer ships code. A spoken answer that cannot survive "show me" is a liability in a loop where the interviewer can ask for the repo. The classic questions, from two sum through the event loop through a rate limited REST service, are implemented here with their tests, in the languages the questions get asked in: Go and C# for the algorithm half, plain JavaScript and a real React workspace for the frontend half, Go for the systems half.

Second, system design weight rose as pure coding signal fell. The design chapters treat scaling, gateways, queues, and the 300K concurrent user sketch as first class drills with drawn topologies, not as trivia.

Third, the spoken layer is an artifact. Every implementable carries the narration that goes with it: what to say while writing, what to concede early, and what the follow-up trap is. The repertoire format is answer, code, drill.

1.1. how the chapters are labeled

Every chapter carries one of three labels on its sections.

- **TDD**, test driven. The section exists as code first: a failing test, the minimal implementation, the refactor. These live in the sample and capstone workspaces this book carries and run under `make verify`.
- **EWC**, explained with code. The section walks existing code line by line. The code runs under the same gate, the explanation is the deliverable.
- **DRILL**, spoken answer. No new code. The section is the answer to say out loud, the follow-ups to expect, and the floor chapter that backs it, in this book or in the handbooks it cross references.

	tdd test driven	ewc with code	drill spoken
evidence	a failing test first	gated code, walked	a spoken answer
the gate	new code, red first	pre-existing suite	no new code
deliverable	the green suite	the explanation	script, follow-ups

figure 1: the three labels are a contract about where the evidence lives

NOTE: the question inventory is audited, not claimed

The appendices hold the full question to section index. Every question in the inventory this book was commissioned from maps to at least one labeled section, and the mapping is a table in book 17, ch 42 (the interview repertoire, appendices: question index and sources), so a missing answer is a visible gap in a compiled document, not a quiet omission.

1.2. what this book is not

It is not a problem archive. The inventory is the frequent sixty or so, the questions that recur across loops, stacked into 158 index rows across the 40 practice chapters, each with working code where code is the honest answer and a drilled script where talking is. Depth questions, the ones that separate a practitioner from a memorizer, cross

reference the handbooks: the language manuals for books 5 through 8 floor the syntax questions, book 10, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking) and its neighbors floor the complexity claims, book 13, ch 15 (infrastructure: docker, databases, and testing, jetstream: streams, consumers, kv, kafka compared) floors the streaming answers, and the defense cookbook, book 16, owns the resume itself. Part 3, chapters 23 through 31, is that promise kept in reverse: the algorithm whiteboard staples gated under tdd in two go workspaces, and seven spoken-answer chapters floored on the handbooks they name, the distributed patterns, the c manual, mathematics, knowledge discovery, and the python, lua, and c# manuals, so the corpus's core concepts each carry a drilled answer here without re-carrying their code. Part 4, chapters 32 through 40, is the loop completed: the web platform in css, rendering, and security answers, estimation at the back of the envelope and four worked design prompts, and the human round, behavioral and the observed screen, take-homes and debugging, negotiation, and the java and rust transfer, so every stage of a real loop from the first screen to the signed offer carries a drilled answer here.

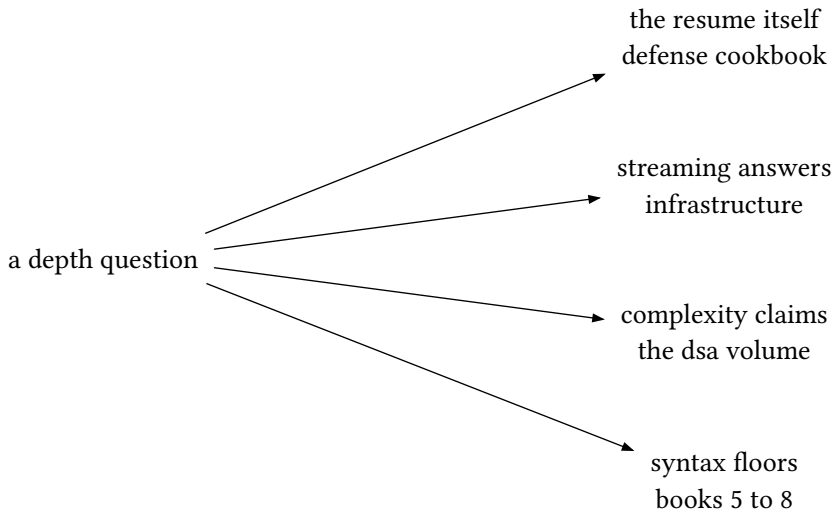
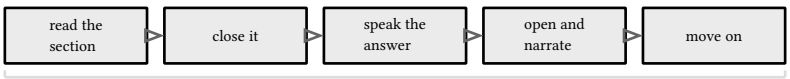


figure 2: every depth question has an owner, the frequent sixty or so stay here

1.3. how to drill

One pass per chapter, aloud, same rule as the defense cookbook. Read the section once, close it, speak the answer at an imaginary interviewer, open the code and read it as you would narrate it live, then move on. The night before, read only the appendices index and the DRILL sections. Everything else is either internalized by then or it is not.



night before: drill sections and the appendices index only

figure 3: the rehearsal loop, one pass per chapter, aloud

1.4. the 2026 facts, pinned

The meta claims above are not folklore. Meta piloted an AI-enabled coding round in october 2025 that replaces one of the two onsite coding rounds, sixty minutes with an assistant expected, per interviewing.io's walkthrough. Google was reported in may 2026 to be piloting a comprehension round with an AI assistant, rolling out through the second half of the year. Karat's 2026 trends report, published january 2026 from 400 engineering leaders, has the numbers that frame the whole book: 71 percent say AI makes technical skills harder to assess, take-homes and automated code tests degrade fastest as signal, live technical interviews still run at 79 percent in the US, and 62 percent of organizations still ban AI in interviews while estimating over half of candidates use it anyway. System design weight rose as the coding signal fell, with 2026 guides reporting a higher passing bar and AI infrastructure

prompts inside design rounds. Every URL and access date for these claims sits in the sources appendix of book 17, ch 42 (the interview repertoire, appendices: question index and sources), verified 2026-09-10.

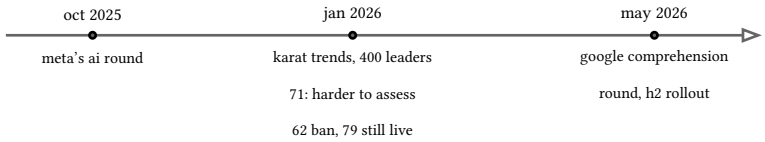


figure 4: ai-assisted rounds, pinned: oct 2025 into 2026

2. paradigms: oop, fp, mvc, mvvm, ecs

The paradigm question is a vocabulary check that turns into a depth probe. The safe answer names the four oop pillars and moves on. The answer that scores walks the same ground in two languages with different mechanisms for it, then names the architecture patterns as variations of where state lives. This chapter drills both halves, and every file it walks runs under `make verify` in the Go and C# workspaces of this chapter.

2.1. oop in go: methods without classes [EWC]

Go has no classes, yet answers every oop question. The four pillars are encapsulation, abstraction, inheritance, and polymorphism: this section answers encapsulation and polymorphism without the keywords, the C# section adds real inheritance with its override, and abstraction is the interface story in both. Encapsulation is a lowercase name, construction is a plain function, and polymorphism is a method set satisfied implicitly:

```go
// Counter is the interface consumers see. Go has no class keyword:
// an interface is a method set, and any type carrying those methods
// satisfies it, with no declaration on the type itself.
type Counter interface {
  Count() int
}

// inventory keeps its slice unexported, so the only writes come through
// methods. Encapsulation is a naming convention, not a keyword.
type inventory struct {
  items []string
}

// NewInventory is the constructor idiom: a plain function, because Go
// has no constructors, returning the initialized value.
func NewInventory(items ...string) *inventory {
  inv := &inventory{items: make([]string, 0, len(items))}
  inv.items = append(inv.items, items...)
  return inv
}

// Count has a pointer receiver so it reads the same struct Add mutates.
// A value receiver would snapshot the struct per call.
func (i *inventory) Count() int { return len(i.items) }

func (i *inventory) Add(item string) { i.items = append(i.items, item) }
```

listing 1: the interface, the unexported struct, the constructor function, the pointer receiver (source: interview-repertoire/samples/ch02-go/oop.go, lines 5-30)

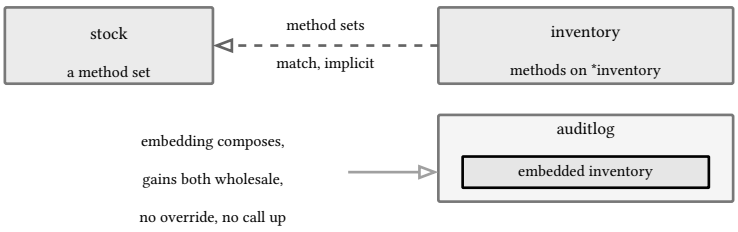


figure 5: structural satisfaction: the method sets match, the type never says so

The two facts interviewers listen for live in the receivers and the embedding. A pointer receiver means `Count` reads the same struct `Add` mutates, and a value receiver would not. Embedding is composition, not inheritance: `auditLog`

gains `Count` and `Add` without redeclaring them, but there is no override, no virtual dispatch, and no way for the embedded type to call back up:

```go
// auditLog demonstrates embedding, Go's composition answer to
// inheritance. It carries an inventory and gains Count and Add without
// redeclaring them, then adds its own method. There is no override:
// the outer method always wins, and the embedded one stays callable.
type auditLog struct {
  inventory
  loggedAt time.Time
}

func NewAuditLog(items ...string) *auditLog {
  return &auditLog{inventory: *NewInventory(items...), loggedAt: time.Now()}
}

// LastAudit does not shadow anything; it reads the embedded timestamp.
func (a *auditLog) LastAudit() time.Time { return a.loggedAt }

// polymorphism is calling Count without knowing the concrete type.
func TotalCount(cs ...Counter) int {
  total := 0
  for _, c := range cs {
    total += c.Count()
  }
  return total
}
```

listing 2: embedding gains methods wholesale, the outer type adds its own (source: interview-repertoire/samples/ch02-go/oop.go, lines 32-55)

The test suite pins the semantics: the embedded method stays reachable as `audit.inventory.Count`, and `TotalCount` accepts both concrete types through the interface without either one declaring it. When the follow-up comes, the honest concession is that Go trades subclass polymorphism for interface polymorphism on purpose, and the template method shape an override enables does not translate.

2.2. oop in c#: the keyword version [EWC]

C# has the keywords the Go answer describes. The same inventory becomes a class hierarchy with real inheritance, a `virtual` method overridden through `base.Add`, and an interface satisfied by declaration rather than implicitly:

```
// The class hierarchy answer: Inventory declares its own state and
// behavior, AuditLog inherits and extends it. C# has real inheritance
// with virtual dispatch, the part Go deliberately omits.
public class Inventory : ICounter
{
    private readonly List<string> _items = [];

    public Inventory(IEnumerable<string> items) => _items.AddRange(items);

    public int Count => _items.Count;

    public virtual void Add(string item) => _items.Add(item);
}

// AuditLog overrides Add but calls back into the base implementation,
// the template method shape Go's embedding cannot express.
public sealed class AuditLog : Inventory
{
    public AuditLog(IEnumerable<string> items) : base(items) =>
        LastAudit = DateTime.UtcNow;

    public DateTime LastAudit { get; }

    public override void Add(string item)
    {
        base.Add(item);
        Console.WriteLine($"{LastAudit:HH:mm:ss} added {item}");
    }
}
```

listing 3: inheritance with a real override calling into the base implementation (source: interview-repertoire/samples/ch02-cs/src/Oop.cs, lines 3-31)

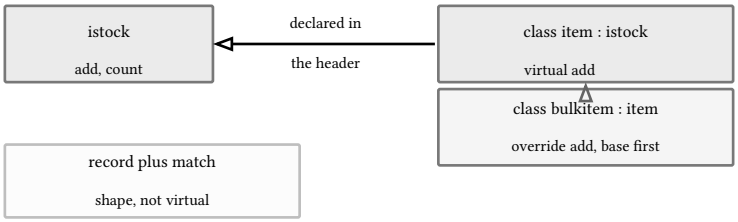


figure 6: nominal satisfaction: the header says it, the override calls base

The one-line contrast worth saying out loud: in Go, satisfaction is structural, the type never names the interface it implements. In C#, satisfaction is nominal, and the compiler message when a type forgets the interface is the first thing the tests in this workspace caught.

The C# answer also carries the fp-leaning half of the language: records are immutable data with structural equality, and pattern matching over a closed hierarchy replaces adding a virtual method per behavior:

```cs
// The record is C#'s fp-leaning data type: immutable by default,
// structural equality, with positional syntax.
public record LineItem(string Sku, int Cents);

// Pattern matching over a hierarchy, the polymorphism alternative
// that avoids adding a virtual method for every new behavior.
public abstract record Discount
{
    public sealed record None() : Discount;
    public sealed record Percent(int Value) : Discount;
    public sealed record Fixed(int Cents) : Discount;
}

public static class Pricing
{
    public static int Apply(Discount discount, int cents) => discount switch
    {
        Discount.None => cents,
        Discount.Percent p => cents - cents * p.Value / 100,
        Discount.Fixed f => Math.Max(0, cents - f.Cents),
        _ => throw new NotSupportedException(),
    };
}
```

listing 4: a record, a closed discount hierarchy, and the match that replaces a virtual method (source: interview-repertoire/samples/ch02-cs/src/Oop.cs, lines 54-77)

2.3. fp: pure functions, closures, composition [EWC]

Functional programming in an interview is three claims: pure functions are easier to reason about, closures capture state without a class, and higher order functions compose. Both workspaces implement the same trio.

```go
// Map, Filter, and Reduce are pure: same inputs, same output, no
// mutation, no side effects. Go is not an fp language, but its
// function values and generics carry the core of the style.
func Map[T, U any](xs []T, f func(T) U) []U {
  out := make([]U, 0, len(xs))
  for _, x := range xs {
    out = append(out, f(x))
  }
  return out
}

func Filter[T any](xs []T, keep func(T) bool) []T {
  out := make([]T, 0, len(xs))
  for _, x := range xs {
    if keep(x) {
      out = append(out, x)
    }
  }
  return out
}

func Reduce[T, U any](xs []T, init U, f func(U, T) U) U {
  acc := init
  for _, x := range xs {
    acc = f(acc, x)
  }
  return acc
}
```

listing 5: map, filter, reduce over generics, pure by construction (source: interview-repertoire/samples/ch02-go/fp.go, lines 3-30)

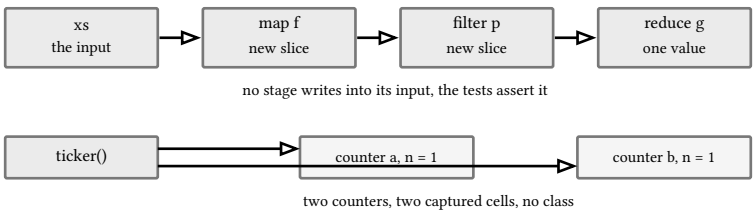


figure 7: pure stages never mutate, a closure is a function plus its own cell

```go
// ticker returns a closure: a function that captures n by reference,
// so each call sees the previous call's increment. This is the state
// an object would otherwise hold in a field.
func Ticker(start, step int) func() int {
  n := start
  return func() int {
    n += step
    return n
  }
}

// HigherOrder shows functions as arguments and return values at once:
// compose builds a pipeline that runs f, then g.
func Compose[T, U, V any](f func(T) U, g func(U) V) func(T) V {
  return func(t T) V { return g(f(t)) }
}
```

listing 6: a closure holding state, and compose returning a function from two functions (source: interview-repertoire/samples/ch02-go/fp.go, lines 32-47)

The closure test is the one that lands: two counters from the same factory stay independent, because each call of `Ticker` creates its own `n`. The C# side is the same trio with LINQ spelling, `Where` `Select` `Aggregate`, plus the identical closure over a captured local. The trap to concede early: `Filter` in the Go workspace allocates a new slice and never mutates its input, and the test proves it. A candidate whose filter writes into the argument has answered the purity question wrong while saying the right words.

2.4. mvc, mvvm, and ecs: where state lives [DRILL]

The three architecture patterns are the same question at three altitudes. Say it that way and the follow-ups answer themselves.

MVC, model view controller, is a server pattern: the model owns state and rules, the view renders it, the controller translates requests into model calls and picks a view. State lives in the model, mutation flows through the controller, and every mainstream web framework, ASP.NET included, is a variation on it.

MVVM, model view viewmodel, is the client descendant: the viewmodel exposes observable state and commands, the view binds to it declaratively, and the binding layer replaces the controller's hand-written glue. It is the pattern behind WPF and behind the signal-based state of book 17, ch 9 (the interview repertoire, javascript from scratch), where the framework re-renders on state change instead of the developer pushing updates.

ECS, entity component system, inverts all of it: state leaves objects entirely. Entities are ids, components are plain data rows, systems are functions over all rows matching a shape. There is no inheritance and no per-object behavior, which is what makes the cache-friendly iteration of book 15, ch 2 (game systems and architecture, ecs theory) and its implementation chapter possible.

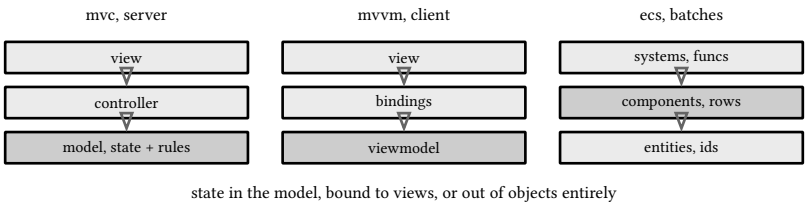


figure 8: one question at three altitudes: where state lives

Follow-ups to expect: which pattern for a CRUD dashboard, answer MVC, because the framework gives it to you. Which for a live editor, answer MVVM or signals, because observable state beats manual dom sync. Which for thousands of moving objects, answer ECS, because per-object method calls lose to batch iteration.

sources: none cited online; both workspaces verified by `go test` and `dotnet test` through `make verify`, 7 Go tests and 12 C# tests, chapter id `ch02-go` and `ch02-cs`.

3. data structures, implemented

“Implement a hash map” and its siblings are the medium-difficulty loop staples. This chapter carries the seven structures the inventory names, dynamic array, linked list, hash map, binary search tree, heap, trie, and graph, each implemented from zero with its tests, in Go and in C#. Both suites are test-first and run under `make verify`. The complexity claims are floored by book 10, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking), which builds the instruments a cost claim must survive, and the per-structure chapters of the same book carry the depth this chapter drills from memory.

3.1. the dynamic array [TDD]

The claim to narrate: append is constant on average because growth doubles capacity, so the copies over n appends stay under 2n. The suite pins go’s actual growth policy, strict doubling while small and a gentler factor above a few hundred slots, rather than the folk version that doubles forever:

```go
// so the total copy cost over n appends is under 2n.
type Vector[T any] struct {
  data []T
}

func NewVector[T any]() *Vector[T] { return &Vector[T]{} }

func (v *Vector[T]) Len() int { return len(v.data) }

func (v *Vector[T]) Get(i int) T {
  if i < 0 || i ≥ len(v.data) {
    panic("index out of range")
  }
  return v.data[i]
}

func (v *Vector[T]) Push(x T) {
  v.data = append(v.data, x)
}

// Pop removes and returns the last element in constant time.
func (v *Vector[T]) Pop() T {
  if len(v.data) == 0 {
    panic("pop from empty vector")
  }
  last := len(v.data) - 1
  x := v.data[last]
  v.data = v.data[:last]
  return x
}

// Insert shifts everything right of i, an O(n) operation, which is
// the honest cost the array charges for middle edits.
func (v *Vector[T]) Insert(i int, x T) {
  if i < 0 || i > len(v.data) {
    panic("index out of range")
  }
  v.data = append(v.data, x) // grow by one, the value is a placeholder
  copy(v.data[i+1:], v.data[i:])
  v.data[i] = x
}

// Capacity reports the backing slice length, so tests can watch it
// double: 1, 2, 4, 8, and so on under go's growth policy.
func (v *Vector[T]) Capacity() int { return cap(v.data) }
```

listing 7: push, pop, insert, and the capacity probe the doubling test reads (source: interview-repertoire/samples/ch03-go/dynamic.go, lines 5-49)

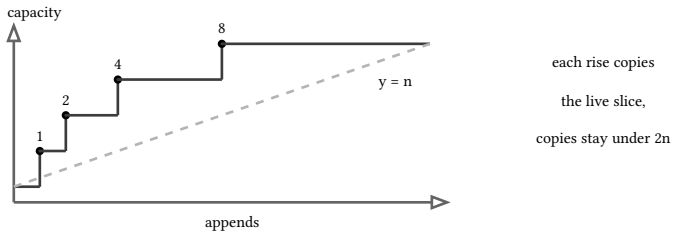


figure 9: the capacity stair-step against n, why append is constant on average

The C# mirror is the same contract over `T[]` with `Array.Resize`, and the insert operation states its honest cost: shifting the tail is linear, and the test says so instead of hiding it.

3.2. the linked list [TDD]

The three-pointer reverse is the centerpiece, `prev`, `cur`, `next`, one pass, constant extra space:

```
// Reverse rewires the chain in one pass with three pointers, O(n)
// time, O(1) space. The classic interview shape: prev, cur, next.
func Reverse[T comparable](head *Node[T]) *Node[T] {
  var prev *Node[T]
  cur := head
  for cur ≠ nil {
    next := cur.Next
    cur.Next = prev
    prev = cur
    cur = next
  }
  return prev
}

// ContainsValue is the linear scan; no index arithmetic exists here.
func ContainsValue[T comparable](head *Node[T], value T) bool {
  for n := head; n ≠ nil; n = n.Next {
    if n.Value == value {
      return true
    }
  }
  return false
}

// Remove unlinks the first match and returns the new head, O(n).
func Remove[T comparable](head *Node[T], value T) *Node[T] {
  dummy := &Node[T]{Next: head}
  prev := dummy
  for n := head; n ≠ nil; n = n.Next {
    if n.Value == value {
      prev.Next = n.Next
      break
    }
    prev = n
  }
  return dummy.Next
}
```

listing 8: reverse rewires in place; the scan and the unlink (source: interview-repertoire/samples/ch03-go/linked.go, lines 24-60)

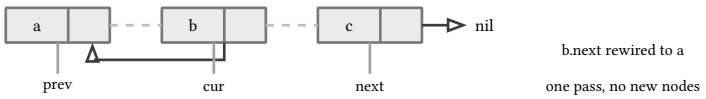


figure 10: the three-pointer reverse rewires next in place

The trap in the room: remove on a head node. The dummy head in `Remove` deletes the special case, and the Go table test walks middle, head, tail, and missing in one function.

3.3. the hash map [TDD]

Separate chaining with growth: buckets are small vectors, the load factor crossing 0.75 doubles the table, and every entry re-buckets. The hash itself comes from the stdlib, `maphash.Comparable`, the seeded hash go's own map uses, folded by the bucket count:

```go
// Map is a hash map with separate chaining: buckets are dynamic
// arrays of entries, collisions stack inside a bucket, and the table
// doubles its bucket count when the load factor passes 0.75.
type Map[K comparable, V any] struct {
  buckets [][]entry[K, V]
  size    int
  seed    maphash.Seed
}

type entry[K comparable, V any] struct {
  key K
  val V
}

const initialBuckets = 8

func NewMap[K comparable, V any]() *Map[K, V] {
  return &Map[K, V]{
    buckets: make([][]entry[K, V], initialBuckets),
    seed:    maphash.MakeSeed(),
  }
}

func (m *Map[K, V]) Len() int { return m.size }

// bucket picks a slot with maphash.Comparable, the stdlib's seeded
// hash for comparable keys, folded down by the bucket count.
func (m *Map[K, V]) bucket(key K) int {
  h := maphash.Comparable(m.seed, key)
  return int(h % uint64(len(m.buckets)))
}

func (m *Map[K, V]) Put(key K, val V) {
  m.rehashIfNeeded()
  i := m.bucket(key)
  for j, e := range m.buckets[i] {
    if e.key == key {
      m.buckets[i][j].val = val
      return
    }
  }
  m.buckets[i] = append(m.buckets[i], entry[K, V]{key: key, val: val})
  m.size++
}

func (m *Map[K, V]) Get(key K) (V, bool) {
  i := m.bucket(key)
  for _, e := range m.buckets[i] {
    if e.key == key {
      return e.val, true
    }
  }
  var zero V
  return zero, false
}
```

listing 9: chained buckets, the seeded hash, put and get (source: interview-repertoire/samples/ch03-go/hashmap.go, lines 5-59)



figure 11: chained buckets, doubling at load factor 0.75

```go
// rehashIfNeeded doubles the bucket slice and re-buckets every entry
// when the load factor crosses the threshold, the moment the average
// bucket stops being O(1).
func (m *Map[K, V]) rehashIfNeeded() {
  if float64(m.size+1)/float64(len(m.buckets)) <= 0.75 {
    return
  }
  old := m.buckets
  m.buckets = make([][]entry[K, V], len(old)*2)
  m.size = 0
  for _, b := range old {
    for _, e := range b {
      m.Put(e.key, e.val)
    }
  }
}
```

listing 10: the doubling rehash, the moment average constant time is preserved (source: interview-repertoire/samples/ch03-go/hashmap.go, lines 72-87)

The 500-key growth test is the line to quote: the load check runs before every put, so inserts 7, 13, 25, 49, 97, 193 and 385 each double the table, seven rehashes that land 500 keys in 1024 buckets, Len still 500, every 37th key still reading back and one absent key still missing, which is the claim rehashing exists to keep. The C# twin hashes through `GetHashCode` with the unsigned fold, because dotnet hashes can be negative and a naive modulo goes negative for about half of all hash codes, so the first negative index throws instead of bucketing.

3.4. the binary search tree [TDD]

In-order traversal emitting sorted keys is the property interviewers probe, and the two height tests are the honest counterweight: sorted insertion makes a chain of height 6 from 7 nodes, balanced insertion makes height 2, same algorithm:

```
    walk(n.Left)
    out = append(out, n.Key)
    walk(n.Right)
  }
  walk(root)
  return out
}

// Height is the longest root to leaf edge count; -1 for empty. An
// unbalanced insertion order makes this linear, the BST's weakness.
func Height(root *TreeNode) int {
  if root == nil {
    return -1
  }
  l := Height(root.Left)
  r := Height(root.Right)
  if l > r {
    return l + 1
  }
  return r + 1
}

// Delete removes a key and returns the new subtree root. Two children
```

listing 11: in-order walk and height, the balance story in two functions (source: interview-repertoire/samples/ch03-go/bst.go, lines 46-68)

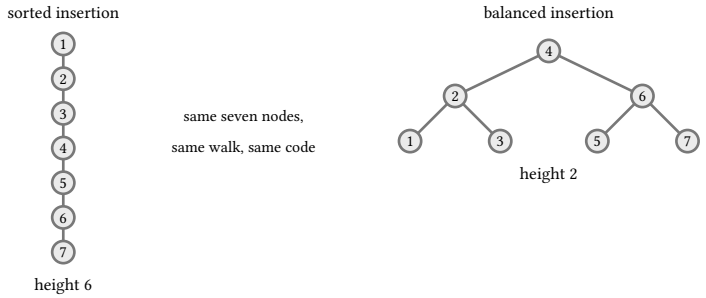


figure 12: same algorithm, two shapes: height 6 against height 2

Delete carries the fiddly case: two children means swapping in the in-order successor and recursing. The suite exercises leaf, one-child, and two-child deletes and asserts the traversal stays sorted after each.

3.5. the heap [TDD]

A min-heap over one slice, children of i at `2i+1` and `2i+2`, no pointers, push sifts up and pop sifts down:

```go
// Push appends then sifts up, swapping with the parent while smaller.
// Worst case O(log n), which is the depth of the implicit tree.
func (h *Heap) Push(x int) {
  h.data = append(h.data, x)
  i := len(h.data) - 1
  for i > 0 {
    parent := (i - 1) / 2
    if h.data[parent] <= h.data[i] {
      break
    }
    h.data[parent], h.data[i] = h.data[i], h.data[parent]
    i = parent
  }
}
```

listing 12: sift up to the root (source: interview-repertoire/samples/ch03-go/heap.go, lines 21-34)

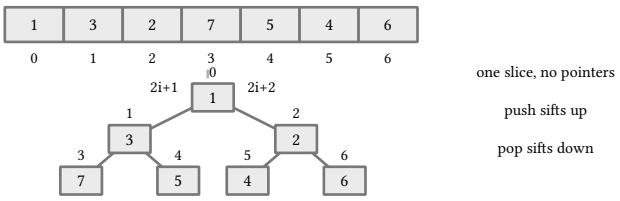


figure 13: one slice as a tree: children of i at 2i+1 and 2i+2

The randomized test is the one that matters: 300 random pushes drained must equal `slices.Sort` on the same input, which catches every off-by-one the small cases miss.

3.6. the trie [TDD]

Prefix trees answer the autocomplete question in word length, independent of corpus size. `Leaf` marks whole words, prefixes are the nodes themselves:

```go
func ContainsWord(root *TrieNode, word string) bool {
  n := walk(root, word)
  return n != nil && n.Leaf
}

// HasPrefix answers whether any stored word starts with the prefix,
// which is the autocomplete question.
func HasPrefix(root *TrieNode, prefix string) bool {
  return walk(root, prefix) != nil
}

func walk(root *TrieNode, s string) *TrieNode {
  n := root
  for _, r := range s {
    child, ok := n.Children[r]
    if !ok {
      return nil
    }
    n = child
  }
  return n
}

// WordsUnder enumerates every word below a node, the backend of a
// suggestion list.
func WordsUnder(root *TrieNode, prefix string) []string {
  start := walk(root, prefix)
  out := []string{}
  var visit func(*TrieNode, []rune)
  visit = func(n *TrieNode, acc []rune) {
    if n == nil {
      return
    }
    if n.Leaf {
      out = append(out, string(acc))
    }
    for r, child := range n.Children {
      visit(child, append(acc, r))
    }
  }
  visit(start, []rune(prefix))
  return out
}
```

listing 13: word search, prefix search, and the suggestion walk (source: interview-repertoire/samples/ch03-go/trie.go, lines 29-71)

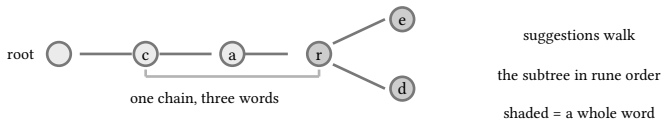


figure 14: shared prefixes are shared nodes, shaded leaves mark whole words

The negative cases in the test are the scoring ones: `ca` is a prefix but not a word, `carts` is a word extension past `car`, and both must report exactly false.

3.7. the graph [TDD]

The adjacency list is the representation to reach for: a map from node id to neighbor ids, undirected edges stored both ways:

```go
// Graph is an adjacency list: one map from node id to its neighbor
// ids. Undirected edges are stored both ways.
type Graph struct {
  adj map[int][]int
}

func NewGraph() *Graph { return &Graph{adj: map[int][]int{}} }

func (g *Graph) AddNode(id int) {
  if _, ok := g.adj[id]; !ok {
    g.adj[id] = nil
  }
}

func (g *Graph) AddEdge(from, to int) {
  g.AddNode(from)
  g.AddNode(to)
  g.adj[from] = append(g.adj[from], to)
  g.adj[to] = append(g.adj[to], from)
}

func (g *Graph) Neighbors(id int) []int {
  ns := append([]int(nil), g.adj[id]...)
  sort.Ints(ns)
  return ns
}

func (g *Graph) Nodes() []int {
  ids := make([]int, 0, len(g.adj))
  for id := range g.adj {
    ids = append(ids, id)
  }
  sort.Ints(ids)
  return ids
}

func (g *Graph) HasEdge(from, to int) bool {
  for _, n := range g.adj[from] {
    if n == to {
      return true
    }
  }
  return false
}

// EdgeCount reports undirected edges once each: half the sum of all
// adjacency lengths, the correction interviews like to probe.
func (g *Graph) EdgeCount() int {
  total := 0
  for _, ns := range g.adj {
    total += len(ns)
  }
  return total / 2
}
```

listing 14: the adjacency map and its honest edge counting (source: interview-repertoire/samples/ch03-go/graph.go, lines 5-58)

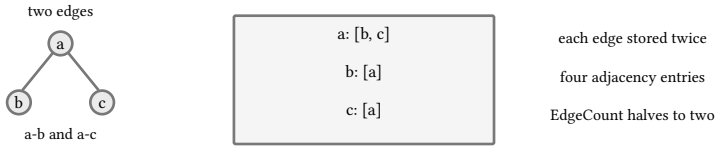


figure 15: undirected edges live in both adjacency lists

The suite pins one honesty point interviewers like: a plain adjacency list keeps a repeated `AddEdge` as two stored edges, and `EdgeCount` says so rather than silently deduplicating.

sources: complexity instruments and per-structure depth floor to book 10, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking) and its neighbors. Verified by `go test` and `dotnet test` through `make verify`, 25 Go tests and 22 C# tests, chapter ids `ch03-go` and `ch03-cs`.

4. graphs: dfs, bfs, topological sort

Traversal is the graph interview. Two orders cover the questions: depth first for reachability, structure, and cycle detection, breadth first for distance and layering, and topological sort is bfs with a counter attached. All of it runs in the `ch04-go` module under `make verify`.

4.1. dfs and bfs, one shape apart [TDD]

The two traversals differ in one data structure: dfs recurses, the call stack is the frontier, bfs loops over a queue. Both visit the same set, and the suite proves it by sorting both orders and comparing:

```
    }
    seen[id] = true
    order = append(order, id)
    for _, n := range g.Neighbors(id) {
      visit(n)
    }
  }
  visit(start)
  return order
}

// Bfs visits in rings, nearest first, using a slice as the queue:
// head indexes the front, append pushes the back. On an unweighted
// graph this order doubles as shortest path discovery.
func Bfs(g *Graph, start int) []int {
  seen := map[int]bool{start: true}
  queue := []int{start}
  order := []int{}
  for head := 0; head < len(queue); head++ {
    id := queue[head]
    order = append(order, id)
    for _, n := range g.Neighbors(id) {
      if !seen[n] {
        seen[n] = true
        queue = append(queue, n)
      }
    }
  }
  return order
}
```

listing 15: dfs recurses, bfs walks the queue by index (source: interview-repertoire/samples/ch04-go/traverse.go, lines 53-82)

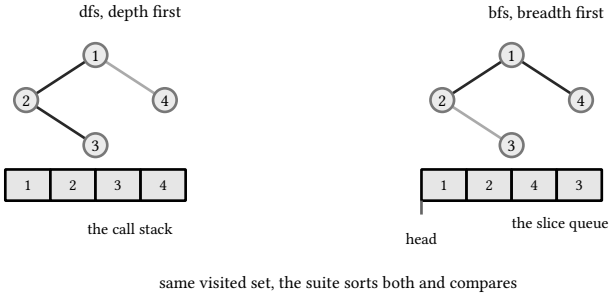


figure 16: two frontiers, one visited set: the call stack against the head-indexed queue

The bfs here is the slice-queue idiom worth narrating: `head` indexes the front, append pushes the back, and the slice is its own queue without a container type. On the house graph the two orders come out `[1 2 3 4]` and `[1 2 4 3]`, same set, different shape.

4.2. shortest path on an unweighted graph [TDD]

Bfs with parent pointers is shortest path whenever edges are unweighted: the first time a node is seen is via a shortest path, so recording the parent and walking back reconstructs it:

```go
// ShortestPath returns the minimum edge count from start to target
// plus the path itself, BFS with parent pointers. False when the
// target is unreachable.
func ShortestPath(g *Graph, start, target int) ([]int, bool) {
  if start == target {
    return []int{start}, true
  }
  parent := map[int]int{start: start}
  queue := []int{start}
  for head := 0; head < len(queue); head++ {
    id := queue[head]
    for _, n := range g.Neighbors(id) {
      if _, ok := parent[n]; ok {
        continue
      }
      parent[n] = id
      if n == target {
        path := []int{n}
        for cur := n; cur ≠ start; {
          cur = parent[cur]
          path = append([]int{cur}, path...)
        }
        return path, true
      }
      queue = append(queue, n)
    }
  }
  return nil, false
}
```

listing 16: parent map, first-seen recording, path reconstruction (source: interview-repertoire/samples/ch04-go/traverse.go, lines 84-112)

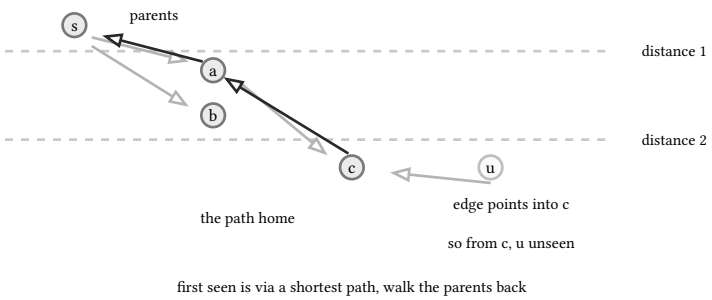


figure 17: first seen is via a shortest path, the parents walk it home

The test pins the direction subtlety: upstream search in a directed graph correctly reports unreachable, the asymmetric case candidates forget.

4.3. kahn's topological sort [TDD]

Topological order is in-degree bookkeeping: count incoming edges, repeatedly emit a zero in-degree node and subtract its outgoing edges. When the loop consumes every node an order exists, when it stalls the remainder sits on a cycle:

```go
// TopoSort is Kahn's algorithm: count in-degrees, repeatedly take a
// zero in-degree node, subtract its outgoing edges. When the loop
// consumes every node the order exists; when it stalls first, the
// remainder sits on a cycle, and the second return says so.
func TopoSort(g *Graph) ([]int, bool) {
  indeg := map[int]int{}
  for _, id := range g.Nodes() {
    indeg[id] = 0
  }
  for _, from := range g.Nodes() {
    for _, to := range g.Neighbors(from) {
      indeg[to]++
    }
  }

  // the ready set stays sorted so the output order is deterministic,
  // a convenience for tests, not a requirement of the algorithm
  var ready []int
  for _, id := range g.Nodes() {
    if indeg[id] == 0 {
      ready = append(ready, id)
    }
  }
  sort.Ints(ready)

  order := []int{}
  for len(ready) > 0 {
    id := ready[0]
    ready = ready[1:]
    order = append(order, id)
    next := g.Neighbors(id)
    sort.Ints(next)
    for _, n := range next {
      indeg[n]--
      if indeg[n] == 0 {
        ready = append(ready, n)
        sort.Ints(ready)
      }
    }
  }
  if len(order) != len(indeg) {
    return nil, false
  }
  return order, true
}

// CourseSchedule answers the interview phrasing directly: n courses
// numbered 0..n-1, prerequisites as (course, prereq) pairs, true
// exactly when a completion order exists.
func CourseSchedule(n int, prereqs [][2]int) bool {
  g := &Graph{}
```

listing 17: in-degree counts, the ready set, and the stall that means a cycle (source: interview-repertoire/samples/ch04-go/topo.go, lines 5-55)

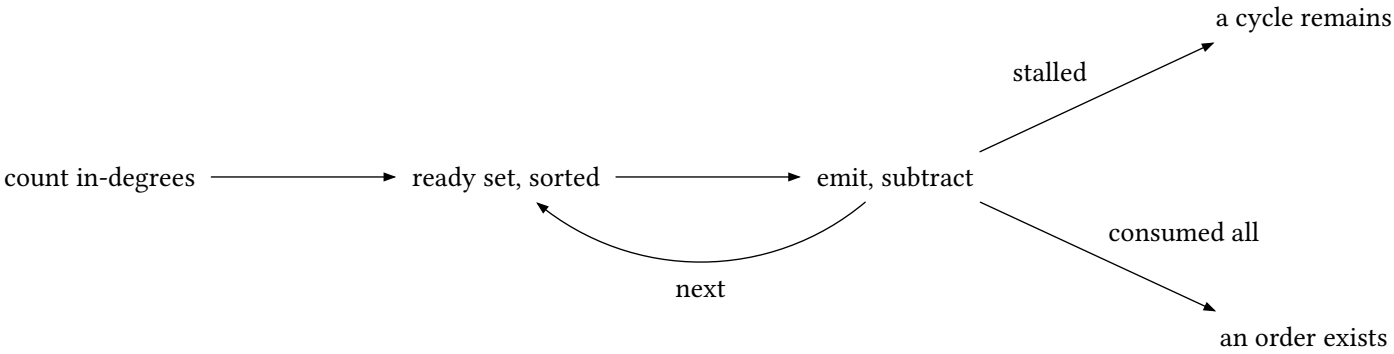


figure 18: in-degrees drain through the sorted ready set, a stall is the cycle

The deterministic ready set, kept sorted, makes the test assert the exact order `[4 5 0 2 3 1]` and every edge's position, a stronger check than "is a valid order". The course-schedule phrasing of the same question sits below it, prerequisites as edges from prereq to course, `true` exactly when the sort exists.

NOTE: say the complexity, then prove you know what n is

Dfs and bfs are both O(V + E), vertices plus edges. The follow-up that scores is naming the space term: dfs carries the recursion depth, worst case the vertex count on a path graph, bfs carries the frontier width, worst case also the vertex count on a star. Neither is free, and the two worst cases are different graphs.

sources: verified by `go test` through `make verify`, 10 tests in `ch04-go`. Depth on traversal families floors to book 10, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking) and the graph chapters that follow it.

5. classics part 1: warmup, arrays, hashing

The warmup classics are table stakes: solve them clean, narrate the cost, and bank the time for the harder rounds. Every problem here exists in Go and in C#, tests first, one problem per file, under `make verify`.

5.1. the string reverser and the palindrome [TDD]

The reverser's whole trick is runes versus bytes. Reversing a byte slice splits multi byte characters, reversing code points does not, and the test feeds `héllo` through to prove the difference:

```go
// ReverseString returns the string reversed by rune, not by byte.
// Reversing bytes splits multi byte characters; reversing code points
// is the answer interviewers listen for.
func ReverseString(s string) string {
  rs := []rune(s)
  out := make([]rune, 0, len(rs))
  for i := len(rs) - 1; i ≥ 0; i-- {
    out = append(out, rs[i])
  }
  return string(out)
}

// IsPalindrome ignores case and everything that is not a letter or
// digit, then compares outside in.
func IsPalindrome(s string) bool {
  rs := []rune(s)
  left, right := 0, len(rs)-1
  for left < right {
    for left < right && !isAlnum(rs[left]) {
      left++
    }
    for left < right && !isAlnum(rs[right]) {
      right--
    }
    if lower(rs[left]) ≠ lower(rs[right]) {
      return false
    }
```

listing 18: rune-wise reverse, then the outside-in palindrome with cleaning (source: interview-repertoire/samples/ch05-go/strings.go, lines 3-29)

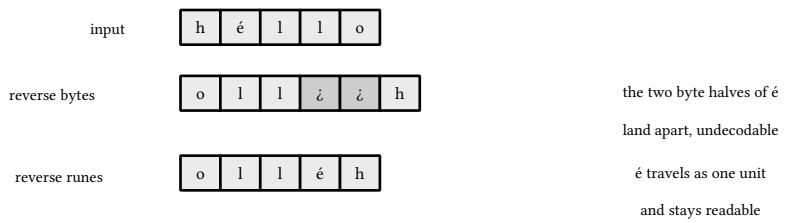


figure 19: runes are code points: the pair stays whole, the bytes do not

The palindrome is the two-pointer pattern with a cleaning twist: skip non-alphanumerics in place, compare lowercased pairs, never allocate a filtered copy. The `0P` case in the table is the one that catches solutions that compare letters only.

5.2. primes, by trial and by sieve [TDD]

Trial division to the square root with evens skipped is the answer for one number. The sieve is the answer for all of them, and the suite cross-checks the two implementations under 1000 so neither can drift:

```
      return false
    }
  }
  return true
}

// Sieve marks composites and keeps the unmarked. Every composite
// under n is crossed out by a prime no greater than its square root,
// so the sieve costs roughly n log log n.
func Sieve(n int) []int {
  if n < 2 {
    return nil
  }
  marked := make([]bool, n+1)
  primes := []int{}
  for p := 2; p <= n; p++ {
    if marked[p] {
      continue
    }
    primes = append(primes, p)
    for m := p * p; m <= n; m += p {
      marked[m] = true
    }
```

listing 19: the sieve marks composites and keeps the unmarked (source: interview-repertoire/samples/ch05-go/primes.go, lines 20-42)



figure 20: each prime starts crossing at p times p, the smaller multiples are gone

The line worth narrating: marking starts at `p * p`, because every smaller multiple of p already got marked by a smaller prime.

5.3. permutations, generated and detected [TDD]

Heap's algorithm generates every ordering with one swap per step, no ordering ever rebuilt from scratch, and the count test demands exactly factorial many distinct outputs, empty input included:

```go
func Permutations(xs []int) [][]int {
  out := [][]int{}
  a := append([]int(nil), xs...)
  var generate func(k int)
  generate = func(k int) {
    if k <= 1 { // k==1 is one element, k==0 is the empty permutation
      out = append(out, append([]int(nil), a...))
      return
    }
    for i := 0; i < k; i++ {
      generate(k - 1)
      if k%2 == 0 {
        a[i], a[k-1] = a[k-1], a[i]
      } else {
        a[0], a[k-1] = a[k-1], a[0]
      }
    }
  }
  generate(len(a))
  return out
```

listing 20: heap's algorithm, single-swap generation (source: interview-repertoire/samples/ch05-go/perm.go, lines 5-24)

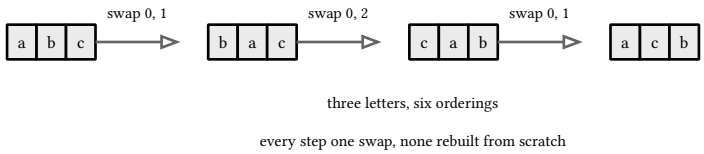


figure 21: one swap per step, factorial many orderings, nothing rebuilt

The anagram question is the detection half, rune counting in one map, early exit the moment a count goes negative.

5.4. linked list operations [TDD]

Reverse, middle, cycle detection: three two-pointer dances over the same node type. The tortoise and hare cycle test builds an actual loop, 1 → 2 → 3 → 2, plus a self-loop single node, the two shapes a misspelled check survives by accident:

```
  return prev
}

// Middle returns the second middle for even lengths, found by two
// pointers moving at 1x and 2x without counting.
func Middle(head *Node) *Node {
  slow, fast := head, head
  for fast ≠ nil && fast.Next ≠ nil {
    slow = slow.Next
    fast = fast.Next.Next
  }
  return slow
}

// HasCycle is Floyd's tortoise and hare: if a loop exists the 2x
// pointer eventually laps the 1x pointer inside it.
func HasCycle(head *Node) bool {
  slow, fast := head, head
  for fast ≠ nil && fast.Next ≠ nil {
    slow = slow.Next
    fast = fast.Next.Next
    if slow == fast {
      return true
    }
  }
  return false
}
```

listing 21: middle by 1x and 2x pointers, floyd cycle detection (source: interview-repertoire/samples/ch05-go/listops.go, lines 39-66)

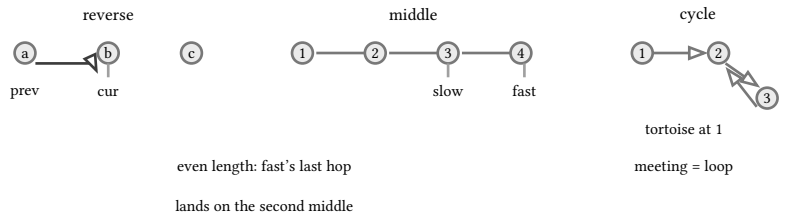


figure 22: three two-pointer dances over one node type

The `Middle` contract to state unprompted: for even lengths it returns the second middle, because the fast pointer's last hop lands there.

5.5. two sum, both ways [TDD]

The hash map answer is one pass: each number asks whether its complement already passed by, and the second operand of any pair is always seen before the pair completes. The sorted-input follow-up is two pointers walking inward:

```go
// is either already in the map or it is not, and the second operand
// of any pair is always seen before the pair completes.
func TwoSum(nums []int, target int) ([2]int, bool) {
  seen := map[int]int{} // value -> index
  for i, n := range nums {
    if j, ok := seen[target-n]; ok {
      return [2]int{j, i}, true
    }
    seen[n] = i
  }
  return [2]int{}, false
}

// TwoSumSorted runs the two pointer walk over ascending input, the
// follow-up interviewers ask after the hash map answer lands.
func TwoSumSorted(nums []int, target int) ([2]int, bool) {
  left, right := 0, len(nums)-1
  for left < right {
    sum := nums[left] + nums[right]
    if sum == target {
      return [2]int{left, right}, true
    }
    if sum < target {
      left++
    } else {
      right--
    }
  }
```

listing 22: one pass with a map, then the two-pointer walk (source: interview-repertoire/samples/ch05-go/twosum.go, lines 5-32)

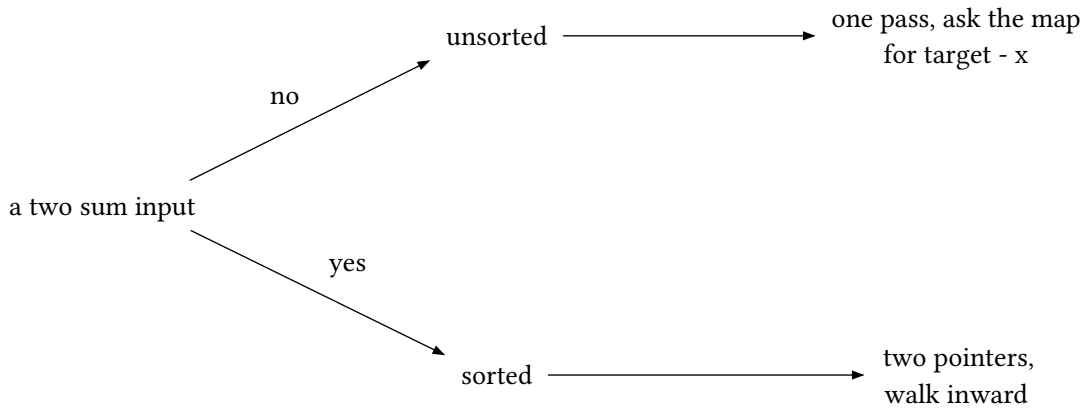


figure 23: sorted input walks two pointers inward, otherwise the complement map in one pass

The duplicate-pair test, `[3 3]` targeting 6, kills the variant that stores the index and then looks up the same slot twice. The C# mirror returns `(int First, int Second)?`, the nullable tuple spelling of “no answer”, shown beside its map walk:

```
    // One pass, one dictionary: value -> index.
    public static (int First, int Second)? TwoSum(int[] nums, int target)
    {
        var seen = new Dictionary<int, int>();
        for (var i = 0; i < nums.Length; i++)
        {
            if (seen.TryGetValue(target - nums[i], out var j))
            {
                return (j, i);
            }
            seen[nums[i]] = i;
        }
        return null;
    }
```

listing 23: the C# one pass, nullable tuple result (source: interview-repertoire/samples/ch05-cs/src/Searches.cs, lines 5-18)

5.6. isolated islands [TDD]

Island counting is flood fill wearing a costume: scan the grid, and every time the scan steps on land it has not visited, increment and sink the whole island. Each cell is touched a constant number of times, so the cost is linear in cells:

```go
// steps on. Each cell is visited a constant number of times.
func CountIslands(grid []string) int {
  if len(grid) == 0 {
    return 0
  }
  cells := make([][]byte, len(grid))
  for i, row := range grid {
    cells[i] = []byte(row)
  }
  islands := 0
  var sink func(r, c int)
  sink = func(r, c int) {
    if r < 0 || r ≥ len(cells) || c < 0 || c ≥ len(cells[r]) {
      return
    }
    if cells[r][c] ≠ '1' {
      return
    }
    cells[r][c] = '0'
    sink(r+1, c)
    sink(r-1, c)
    sink(r, c+1)
    sink(r, c-1)
  }
  for r := range cells {
    for c := range cells[r] {
      if cells[r][c] == '1' {
        islands++
        sink(r, c)
      }
    }
  }
  return islands
}
```

listing 24: scan, sink, count: dfs on a grid (source: interview-repertoire/samples/ch05-go/islands.go, lines 5-38)

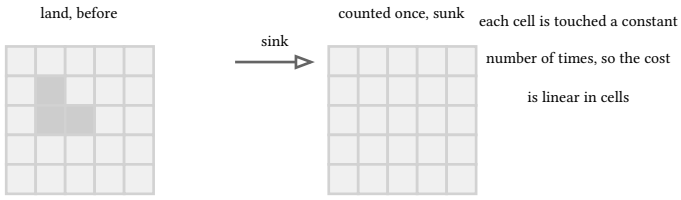


figure 24: the scan counts once per island, the sink claims the whole thing

The suite guards the representation copy: the input strings are copied to byte slices before any sinking, and the caller's grid comes back untouched.

sources: verified by `go test` and `dotnet test` through `make verify`, 17 Go tests and 29 C# tests, chapter ids `ch05-go` and `ch05-cs`.

6. classics part 2: dynamic programming and tries

Part 2 is the difficulty step up: the dp ladder in three refinements, the trie as a paying structure rather than a trivia answer, and the string manipulations that recur across loops. Go and C# both, tests first, under `make verify`.

6.1. the ladder: recursion, memo, table [TDD]

“How many ways to climb n stairs taking 1 or 2 steps” is the fibonacci in disguise, and the answer worth giving is the walk from the naive recursion to the O(1)-space table, narrating each refinement:

```go
// the nth Fibonacci number of calls.
func ClimbStairsNaive(n int) int {
  if n < 0 {
    return 0
  }
  if n <= 1 {
    return 1
  }
  return ClimbStairsNaive(n-1) + ClimbStairsNaive(n-2)
}

// ClimbStairsMemo keeps the recursion but caches answers, so each n
// computes once: O(n) time, O(n) space.
func ClimbStairsMemo(n int) int {
  memo := map[int]int{}
  var climb func(int) int
  climb = func(k int) int {
    if k <= 1 {
      return 1
    }
    if v, ok := memo[k]; ok {
      return v
    }
    memo[k] = climb(k-1) + climb(k-2)
    return memo[k]
  }
  return climb(n)
}

// ClimbStairsTable is bottom up: two rolling slots hold the last two
// counts while the loop climbs to n. O(n) time, O(1) space, the
// version to say out loud.
func ClimbStairsTable(n int) int {
  if n < 0 {
    return 0
  }
  prev, cur := 1, 1
  for i := 2; i <= n; i++ {
    prev, cur = cur, prev+cur
  }
```

listing 25: the exponential recursion, the memoized recursion, the two-slot table (source: interview-repertoire/samples/ch06-go/ladder.go, lines 5-44)

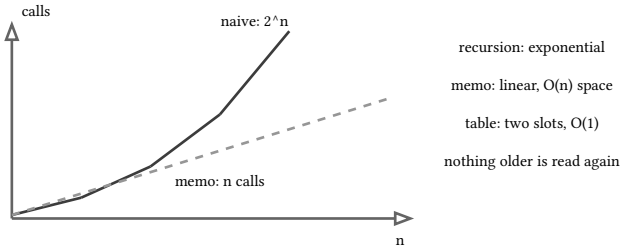


figure 25: one recurrence, three costs: exponential, cached, two rolling slots

The suite runs all three against the fibonacci prefix to 15 and then hands n = 60 to the table only, where the naive version would make trillions of calls. The priced variant, `MinCostClimbingStairs`, is the same recurrence with a cost array and a two-start choice, and its two fixtures are the standard ones.

The narration to give while writing the table version: `prev` and `cur` hold the last two answers, the loop climbs from rung 2, and the space dropped from O(n) to O(1) because nothing older is ever read again. That last sentence is the dp understanding the interviewer is actually checking.

6.2. the trie earns its keep: autocomplete [TDD]

Chapter 3 built the trie as a structure. Here it carries an application: a vocabulary that answers prefix queries with suggestions in alphabetical order, up to a limit:

```
      n.children[r] = child
    }
    n = child
  }
  n.leaf = true
}

// Suggest returns up to limit stored words that extend prefix, in
// alphabetical order so the output is deterministic.
func (ac *AutoComplete) Suggest(prefix string, limit int) []string {
  n := ac.root
  for _, r := range prefix {
    child, ok := n.children[r]
    if !ok {
      return nil
    }
    n = child
  }
  out := []string{}
  var visit func(*trieNode, []rune)
  visit = func(node *trieNode, acc []rune) {
    if len(out) ≥ limit {
      return
    }
    if node.leaf {
      out = append(out, string(acc))
    }
    runes := make([]rune, 0, len(node.children))
    for r := range node.children {
      runes = append(runes, r)
    }
    sort.Slice(runes, func(i, j int) bool { return runes[i] < runes[j] })
    for _, r := range runes {
```

listing 26: suggest walks to the prefix node then enumerates in rune order (source: interview-repertoire/samples/ch06-go/autocomplete.go, lines 36-68)

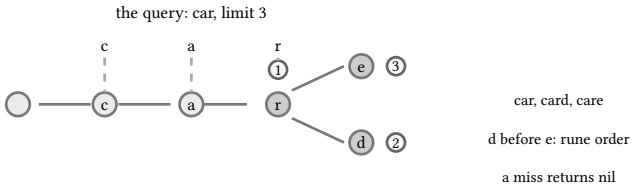


figure 26: walk to the prefix node, enumerate in rune order, cap at the limit

The deterministic output, alphabetical under a sorted rune walk, is what lets the test assert `Suggest("car", 3)` equals exactly `[car card care]` against a six word vocabulary. The miss case returns nil, not an empty slice, and the test distinguishes them.

6.3. string manipulation [TDD]

Run length encoding with counts always emitted, first unique character in two passes, and anagram grouping by sorted signature:

```go
// Compress is run length encoding: "aaabbc" becomes "a3b2c1". The
// encoding always emits a count, so a string of single letters grows;
// the tests pin that behavior instead of pretending it never does.
func Compress(s string) string {
  if s == "" {
    return ""
  }
  out := []byte{}
  run := 1
  for i := 1; i <= len(s); i++ {
    if i < len(s) && s[i] == s[i-1] {
      run++
      continue
    }
    out = append(out, s[i-1])
    digits := []byte{}
    for n := run; n > 0; n /= 10 {
      digits = append([]byte{byte('0' + n%10)}, digits...)
    }
    out = append(out, digits...)
    run = 1
  }
  return string(out)
```

listing 27: run length encoding, counts always written (source: interview-repertoire/samples/ch06-go/strings.go, lines 5-27)

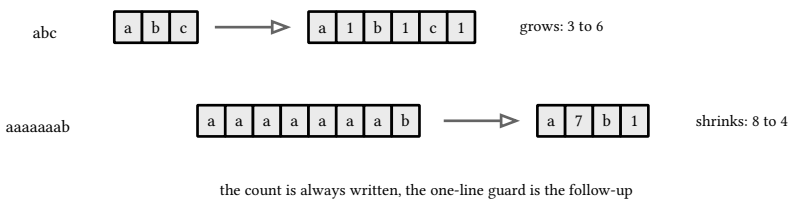


figure 27: counts always emitted: singles grow, runs shrink

The honesty callout the tests encode: `Compress` always emits a count, so a string of single letters grows, `abc` becomes `a1b1c1`. The version that returns the original when the encoding would not shrink is a one-line guard candidates offer as the follow-up, and the fixture table pins the unguarded behavior first.

```go
// FirstUniqueChar returns the index of the first character that
// appears exactly once, -1 when there is none. Two passes: count,
// then find.
func FirstUniqueChar(s string) int {
  counts := map[rune]int{}
  for _, r := range s {
    counts[r]++
  }
  for i, r := range s {
    if counts[r] == 1 {
      return i
    }
  }
  return -1
}

// GroupAnagrams buckets words by their sorted rune signature; every
// bucket is an anagram class.
func GroupAnagrams(words []string) [][]string {
  buckets := map[string][]string{}
  for _, w := range words {
    key := []rune(w)
    sort.Slice(key, func(i, j int) bool { return key[i] < key[j] })
    buckets[string(key)] = append(buckets[string(key)], w)
  }
  out := [][]string{}
  for _, group := range buckets {
```

listing 28: two-pass first unique, bucketed anagrams with deterministic ordering (source: interview-repertoire/samples/ch06-go/strings.go, lines 29-56)

Grouped anagrams come back sorted inside and across buckets, because a hash map's iteration order is not a contract, in go or in C#, and a test that depends on it is a flake in waiting. The longest common prefix by column scan rounds out the set, and the C# twin runs the same fixtures with `StringBuilder` and `SortedDictionary` spelling.

sources: verified by `go test` and `dotnet test` through `make verify`, 10 Go tests and 13 C# tests, chapter ids `ch06-go` and `ch06-cs`. The dp family's wider ladder floors to book 10, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking) neighbors on dynamic programming.

7. javascript core: scope, closures, event loop

The js core questions are conceptual with executable answers. This chapter's workspace, `ch07-js`, is plain javascript under `node --test`, no build step, no framework, 22 tests, and every demo returns its evidence instead of printing it so the suite can pin it.

7.1. hoisting and the temporal dead zone [EWC]

Hoisting is a compile-time fact: declarations move to the top of their scope, but what moves depends on the keyword. `var` hoists the binding with value undefined, function declarations hoist whole, and `let` with `const` hoist the binding into a dead zone where reading throws:

```
// "undefined" above the assignment, the value below it.
export function varHoisting() {
  const before = typeof flavor;
  var flavor = "mint";
  const after = flavor;
  return { before, after };
}

// let and const hoist too, but into the temporal dead zone: reading
// before the declaration throws, it does not read undefined.
export function letTDZ() {
  let threw = false;
  {
    try {
      const probe = hidden; // eslint-disable-line no-use-before-define
      void probe;
    } catch {
      threw = true;
    }
    let hidden = 1;
  }
  return threw;
}

// Function declarations hoist whole: body included, callable above
// the line where they are written.
export function functionHoisting() {
  return whole();
  function whole() {
    return "callable before declaration";
  }
}

// let is block scoped, var is function scoped: two counters in
// sibling blocks share the var, not the let.
export function blockVsFunctionScope() {
  const seen = [];
  for (var i = 0; i < 2; i++) seen.push(`var-${i}`);
  for (let j = 0; j < 2; j++) seen.push(`let-${j}`);
```

listing 29: var reads undefined above its line, let throws, functions call from above (source: interview-repertoire/samples/ch07-js/src/scope.mjs, lines 5-43)

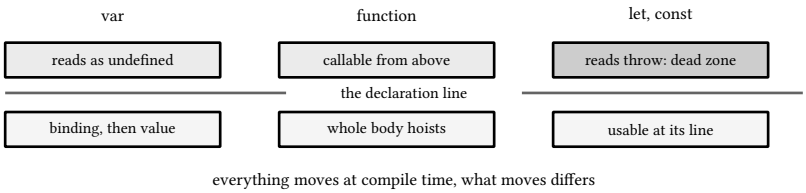


figure 28: what moves at compile time, per keyword

The block scope demo is the one that still surprises: two sibling loops, one `var`, one `let`, and after both run the `var` binding leaked to the function while the `let` binding is gone. The `constBinding` demo separates binding from value, the array mutates, the reassignment throws.

7.2. closures [EWC]

A closure is a function plus the environment it captured. The counter factory gives each returned function its own `n`, and the loop-capture demo is the classic gotcha now inverted by spec: since es2015 semantics gave `let` loops per-iteration bindings, the `let` callbacks each see their own index while a `var` loop still shares one:

```
// var in a loop shares one binding, so every callback sees the final
// value; let gives each iteration its own binding.
export function loopCapture() {
  const varSeen = [];
  const letSeen = [];
  for (var vi = 0; vi < 3; vi++) varSeen.push(() => vi);
  for (let li = 0; li < 3; li++) letSeen.push(() => li);
  return { varSeen: varSeen.map((f) => f()), letSeen: letSeen.map((f) => f()) };
}

// Private state: the balance is reachable only through the two
// closures returned with it.
export function makeWallet(opening) {
  let balance = opening;
  return {
    deposit: (n) => (balance += n),
    balance: () => balance,
  };
}
```

listing 30: the shared var binding versus per-iteration let (source: interview-repertoire/samples/ch07-js/src/closures.mjs, lines 10-30)

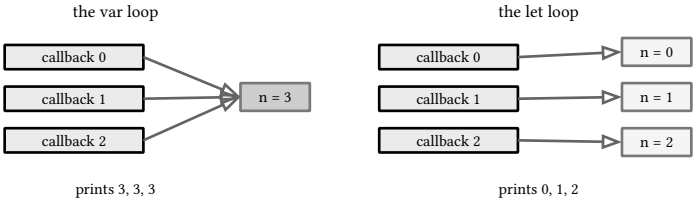


figure 29: one shared cell prints 3, per-iteration cells print 0 1 2

Say it plainly in the room: the `var` callbacks all close over the same variable and print 3, the `let` callbacks close over three variables and print 0, 1, 2. The wallet demo is privacy, the balance reachable only through the two returned functions.

7.3. precedence, coercion, and equality [EWC]

`1 + "1"` is `"11"` because plus prefers concatenation when either side is a string, `"3" * "2"` is 6 because multiplication has only the numeric path, and `[] + []` is the empty string. The demo file returns the whole table at once:

```
    boolean: typeof true,
    object: typeof {},
    function: typeof (() => {}),
    null: typeof null,
    undefined: typeof undefined,
  };
}

// Unary plus and multiplication coerce before addition concatenates.
export function coercion() {
  return {
    plusStringNumber: 1 + "1",
    minusStringNumber: 1 - "1",
    multiplyStringNumber: "3" * "2",
    plusPlus: 1 + 1 + "1",
```

listing 31: the coercion table, plus binds looser than the arithmetic operators (source: interview-repertoire/samples/ch07-js/src/precedence.mjs, lines 27-41)

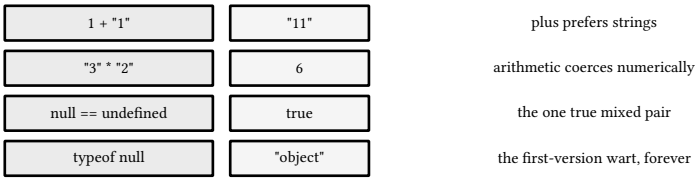


figure 30: plus prefers strings, arithmetic coerces, == coerces and === does not

Equality lives beside it: == coerces, === does not, and the one pair that surprises under == is `null == undefined` being true while every other mixed comparison is coercion at work. The `typeof` table carries the famous wart, `typeof null ===
"object"`, a first-version-js bug the language can never remove.

7.4. null, undefined, NaN, and the optionals [EWC]

Null is assigned absence, undefined is unassigned absence, and NaN is the only value not equal to itself, which is why `Number.isNaN` exists beside the coercing global `isNaN`:

```
    selfInequality: NaN !== NaN,
    parsed: Number("nope"),
    infected: NaN + 1,
    isNanCoerces: isNaN("nope"),
    numberIsNan: Number.isNaN("nope"),
    sanitizes: Number.isNaN(Number("nope")),
  };
}

// Optional chaining ?. short circuits on null and undefined; the
// nullish coalescing ?? defaults only those two, where || defaults
// every falsy value.
export function optionals(user) {
  return {
    city: user?.address?.city,
    cityOrDefault: user?.address?.city ?? "unknown",
    nicknameOrZero: user?.nickname || "anonymous",
    nicknameNullish: user?.nickname ?? "anonymous",
  };
}

// Default parameters apply only for undefined, not for null.
export function defaults(x = 10) {
  return x;
}
```

listing 32: the three absences, the self-inequality, and the ?. ?? pair (source: interview-repertoire/samples/ch07-js/src/values.mjs, lines 21-46)

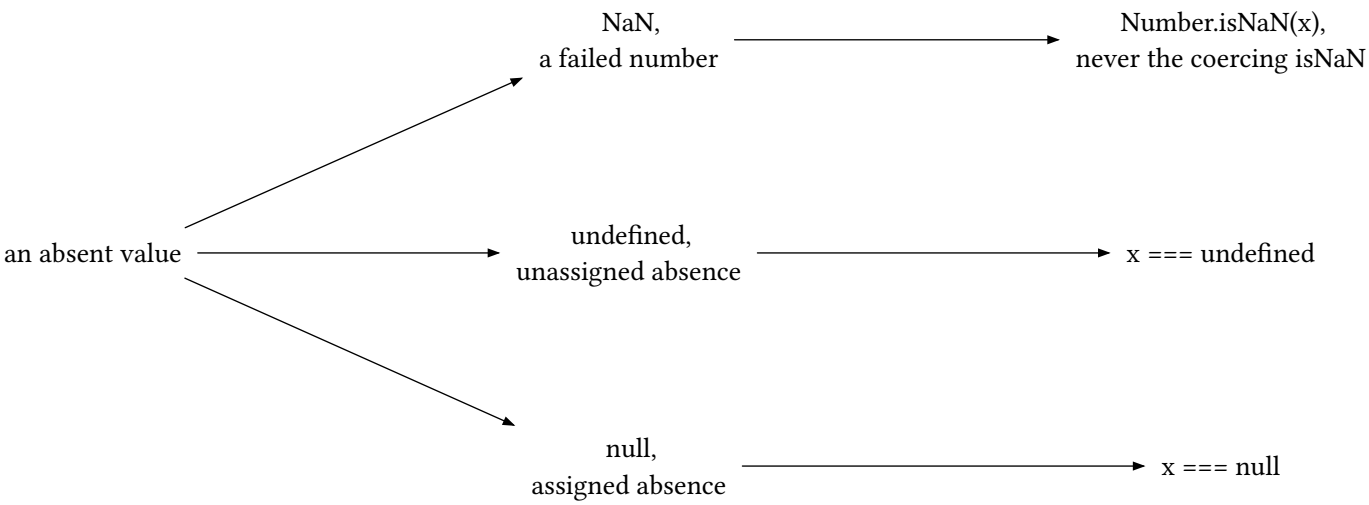


figure 31: three absences, three distinct tests, ?? touches only two of them

The optionals are the modern answer to the absences: `?.` short-circuits a chain on null and undefined, and `??` defaults only those two. The test's sharpest row: an empty-string nickname is replaced by `||` but kept by `??`, because empty string is falsy but not nullish.

7.5. prototypes [EWC]

Property reads walk the prototype chain, writes never do. The demo builds the chain with `Object.create`, then mutates through the child and checks the parent stayed whole:

```
    hello() {
      return `hi ${this.name}`;
    }
    static species() {
      return "greeter";
    }
  }
  const g = new Greeter("go");
  return {
    hello: g.hello(),
    methodOnPrototype: Greeter.prototype.hello === Object.getPrototypeOf(g).hello,
    staticOnConstructor: Greeter.species(),
    instanceHasNoMethod: Object.hasOwn(g, "hello") === false,
  };
}
```

listing 33: reads walk up, writes stay on the receiver (source: interview-repertoire/samples/ch07-js/src/prototypes.mjs, lines 33-47)

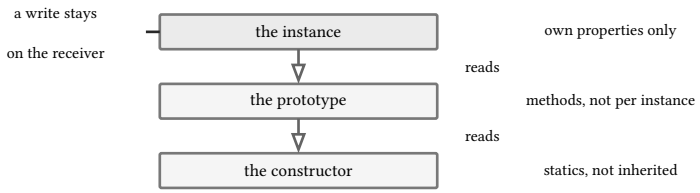


figure 32: reads walk the chain, writes stay on the receiver

The `classShape` demo pins where the moving parts sit: methods on `Greeter.prototype`, statics on the constructor itself, and neither on the instance. Class syntax is sugar over those links, and the test proves the sugar does not move them.

7.6. the event loop [EWC]

One call stack, one microtask queue drained to empty after every macrotask, and timers queued as macrotasks. The demo records rather than prints:

```
// still have not run by the time the function returns. The return
// snapshots the log, because the live array keeps mutating as queued
// work fires later.
export async function microtasksBeforeTimers() {
  const events = [];
  setTimeout(() => events.push("timer"), 0);
  queueMicrotask(() => events.push("microtask-1"));
  await Promise.resolve();
  events.push("after-await");
  queueMicrotask(() => events.push("microtask-2"));
  return [...events];
}

// Awaiting a resolved promise resumes before any timer: two awaits
// in a row are two round trips through the microtask queue, and the
```

listing 34: microtasks scheduled before an await run before the resume, the timer still pending (source: interview-repertoire/samples/ch07-js/src/eventloop.mjs, lines 6-20)

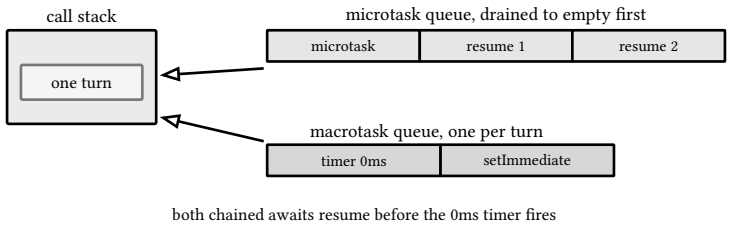


figure 33: one call stack, the microtask queue drained to empty between macrotask turns

The two facts the tests pin: a microtask queued before an `await` runs before the resumption, and two chained awaits still both return before a `setTimeout(0)` fires, because awaits ride the microtask queue and timers wait for a macrotask turn. The third demo records what node's own docs warn: from the main module, `setImmediate` versus `setTimeout(0)` is unspecified order, and the test asserts only the guaranteed part, the microtask in front of both.

PITFALL: the returned array keeps mutating

The first draft of the microtask demo returned the live events array, and the assertion saw entries pushed after the function returned, because the caller's own `await` yields to the same microtask queue. The fix is the snapshot, `return [...events]`, and the lesson travels: a log you hand to an async caller is a log they read later.

sources: verified by `npm run verify` under node 26.3.0, 22 tests in `ch07-js`. Language semantics floor to book 7, ch 2 (javascript es2026 and typescript 7, values and lexical structure) and the typescript handbook it cites.

8. javascript and the dom

The dom questions split cleanly: half are about the browser's document model and css, half are plain javascript wearing a dom costume. The `ch08-js` workspace runs the javascript half under `node --test`, 16 tests, and keeps the dom-specific answers as a real stylesheet, a centering decision model, and the tree-model demos the node runtime can honestly execute.

8.1. what the dom is [EWC]

The document object model is a tree of nodes the script can read and write, and every api, `querySelector`, `createElement`, `addEventListener`, is a walk or a mutation of that tree. The workspace models the tree as plain data and implements the canonical query by hand:

```
// The document tree as data: a node has a tag, attributes, children,
// and text. querySelector walks a shape exactly like this one.
export function buildTree() {
  return {
    tag: "html",
    children: [
      {
        tag: "body",
        children: [
          { tag: "header", children: [{ tag: "h1", text: "shop" }] },
          {
            tag: "main",
            attributes: { id: "listing" },
            children: [
              { tag: "ul", children: [{ tag: "li", text: "sword" }, { tag: "li", text: "shield" }] },
            ],
          },
        ],
      },
    ],
  };
}

// querySelector('#listing li') by hand: id lookup, then descendant
// search for the tag. The browser does the same walk with indexes.
export function querySelector(root, selector) {
  const tokens = selector.split(" ");
  const last = tokens[tokens.length - 1];
  const idToken = tokens.find((t) => t.startsWith("#"));
  let scope = root;
  if (idToken) {
```

listing 35: the document as a tree, and querySelector as an id scope plus a tag walk (source: interview-repertoire/samples/ch08-js/src/dom-shape.mjs, lines 5-35)

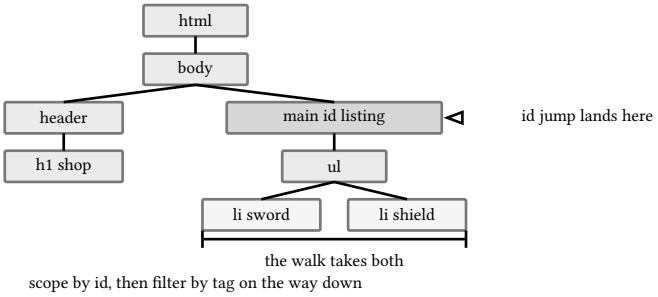


figure 34: the id jump scopes the search, then every li below it is a descendant hit

The point the demo makes without a browser: `#listing li` is an id lookup to scope, then a descendant walk for the tag, and the browser's real implementation differs by using indexes, not by being a different idea.

8.2. center a div, four honest ways [EWC]

The css lives in a real stylesheet file, one panel per technique:

```
/* 1. flex: the parent becomes a flex container and centers on both
   axes. The everyday answer. */
.center-flex {
  display: flex;
  justify-content: center; /* horizontal */
  align-items: center; /* vertical */
}

/* 2. grid: one cell, centered twice, fewer properties than flex. */
.center-grid {
  display: grid;
  place-items: center;
}

/* 3. absolute plus transform: the child positions its own center
   onto the parent's center, works when the parent is relative and
   the child has a known or unknown size. */
.center-absolute {
  position: relative;
}
.center-absolute > .box {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
}

/* 4. auto margins: the child owns the centering, the parent only
   agrees to be a flex or grid container, because auto margins
   absorb the free space on both axes. */
.center-margin {
  display: flex;
}
.center-margin > .box {
  margin: auto;
}
```

listing 36: flex, grid, absolute plus transform, auto margins (source: interview-repertoire/samples/ch08-js/src/center-a-div.css, lines 11-46)

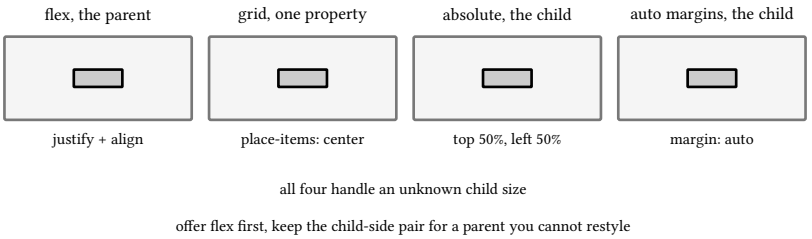


figure 35: four models: the parent flexes, the grid places, the child translates, the child absorbs

Each technique is one ownership decision, and the four question walk settles which to offer.

Flex. What: the parent becomes a flex container and distributes its free space. Why: it is the default layout tool, so offering it first reads as fluency rather than recall. How: `display: flex` on the parent, `justify-content: center` for the inline axis, `align-items: center` for the block axis. When: one item, and the parent is yours to restyle.

Grid. What: the same two-axis centering in one declaration. Why: fewer moving parts, and the shorthand names the concept. How: `display: grid` then `place-items: center`, which spans `align-items` plus `justify-items`. When: the same shape as flex, offered as the shorter spelling.

Absolute plus transform. What: the child carries its own center onto the parent's center. Why: it survives the one constraint the parent-side pair cannot, a parent whose `display` must not change. How: `position: relative` on the parent, then `top: 50%; left: 50%` on the child, whose percentages resolve against the parent's box, plus `translate(-50%, -50%)`, whose percentages resolve against the child's own box, the two bases meeting in the middle. When: a parent you may annotate but not restyle, a child of unknown size.

Auto margins. What: the child absorbs every unit of free space around itself. Why: it is the child-side answer when the parent is already a flex or grid container someone else owns. How: `margin: auto` on the child, nothing else. When: one child among siblings inside a container a library or a teammate wrote.

The trap to volunteer: all four handle an unknown child size. The one spelling that does need a known size is the older absolute variant, `inset: 0` with `margin: auto`, which centers only when width and height are set, and naming why the transform spelling replaced it is the flex that ends the follow-up.

The spoken answer: offer flex first, `justify-content` and `align-items` on the parent, grid `place-items` as the shorter spelling, the absolute plus transform pair when the parent's display is frozen, and `margin: auto` when the child must own the move inside a container someone else wrote.

8.3. centering edge cases: the group, the overflow, the choice [TDD]

Two follow-ups end most centering questions, and both are decisions rather than declarations, so the workspace pins the choice as a model you can run and the panels as real css:

```
export function pickCentering(scenario) {
  const itemCount = scenario?.itemCount ?? 1;
  const overflowRisk = scenario?.overflowRisk ?? false;
  let technique;
  let why;
  if (itemCount > 1) {
    if (scenario?.parentDisplayFrozen) {
      technique = "absolute transform";
      why =
        "the group shortcut needs a grid parent and the display is frozen, so the items wrap in one element
and the wrapper translates";
    } else {
      technique = "place-content";
      why =
```

```
        "the items center as one track, place-items would center each inside its own cell";
    }
  } else if (scenario?.parentDisplayFrozen) {
    technique = "absolute transform";
    why =
      "the parent keeps its display, only position relative is added, and translate absorbs an unknown
child size";
  } else if (scenario?.childOnly) {
    technique = "margin auto";
    why =
      "the parent already flexes or grids, auto margins on the child absorb the free space on both axes";
  } else {
    technique = "flex";
    why =
      "justify-content and align-items on the parent, grid place-items is the same answer in one property";
  }
  const overflow = overflowRisk
    ? {
        modifier: "safe",
        why:
          "plain center clips both ends of an oversized item out of reach, safe center falls back to start
so the overflow stays reachable",
      }
    : null;
  return { technique, why, overflow };
}

// The classifier for the classic follow-up: place-items and
// place-content differ in what they center, and both are shorthand
// pairs over the two axes.
export function placeItemsVsPlaceContent(property) {
  if (property === "place-items") {
    return {
      centers: "each item",
      where: "inside its own cell, one item per track",
      spans: "align-items plus justify-items",
    };
  }
  if (property === "place-content") {
    return {
      centers: "the whole track",
      where: "the items move as one group inside the container",
      spans: "align-content plus justify-content",
    };
  }
  throw new Error(`not a placement property: ${String(property)}`);
}
```

listing 37: pickCentering walks the ownership ladder, placeItemsVsPlaceContent settles the follow-up (source: interview-repertoire/samples/ch08-js/src/centering.mjs, lines 10-67)

```
/* The group rule: many items centered as one track. place-content
   moves the track, place-items would center each item inside its
   own cell instead. */
.center-group {
  display: grid;
  place-content: center;
}

/* The overflow rule: a centered item larger than its container
   clips both ends under plain center, unreachable by scroll. safe
   abandons center for start exactly when that would happen, and is
   a no-op when the item fits. */
.center-safe {
  display: flex;
  justify-content: safe center;
}
```

listing 38: the group panel and the overflow panel (source: interview-repertoire/samples/ch08-js/src/center-a-div.css, lines 48-63)

The group edge. What: `place-content: center` centers the items as one track. Why: `place-items` centers each item inside its own cell, so a group asked to sit in the middle needs its track moved, not its items. How: `display: grid` with `place-content: center`, the shorthand spanning `align-content` plus `justify-content`. When: more than one item with the parent's display yours to set, and the interviewer says group, cluster, or row. A frozen parent display forfeits the shortcut, the items wrap in one element and the wrapper takes the transform pair.

The overflow edge. What: `safe center` abandons center for start exactly when the item is larger than its container. Why: plain center on an oversized item clips both ends out of scroll's reach, the leading edge becomes unreachable on the axis that overflows. How: `justify-content: safe center`, the modifier riding the justify-content value, the one alignment row the sources pin. When: user generated or image sized content that can outgrow its box, a modal body, a carousel slide. The rule and the fallback are pinned to the mdn row in the sources appendix.

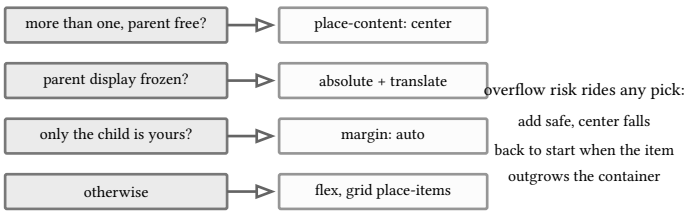


figure 36: which centering tool when: the ladder the model walks

The step by step, said aloud for any centering question: count the items, because more than one under a free parent display means the track moves with `place-content` while a frozen one means the group wraps in one element first, ask whether the content can outgrow the box, because that adds the `safe` modifier, ask whose declarations you own, because the parent means flex or grid and the child means transform or auto margins, then name the properties and stop.

8.4. class versus arrow, map filter reduce, async await, es6 [EWC]

Class methods read `this` from the call site, arrow functions close over the enclosing `this`, and the difference is observable without a dom: a detached method call throws in strict mode, while `bind` and arrow wrappers keep working:

```
export function bindingDemo() {
  class Cart {
    constructor() {
      this.items = [];
    }
    add(item) {
      this.items.push(item);
      return this.items.length;
    }
    get addArrow() {
      return (item) => this.add(item); // this captured at construction
    }
  }
  const cart = new Cart();
  const loose = cart.add;
  let looseThrew = false;
  try {
    loose("sword"); // this is undefined in the plain call
  } catch {
    looseThrew = true;
  }
  const bound = cart.add.bind(cart);
  const arrow = cart.addArrow;
  return {
    looseThrew,
    boundWorks: bound("shield") === 1, // first successful push
    arrowWorks: arrow("potion") === 2, // second
    items: cart.items,
  };
}

// timers lose this the same way: an arrow callback keeps it, a
// regular function callback needs bind.
export function timerBinding() {
  class Ticker {
```

listing 39: the detached call throws, bind and the arrow survive (source: interview-repertoire/samples/ch08-js/src/functions.mjs, lines 5-40)

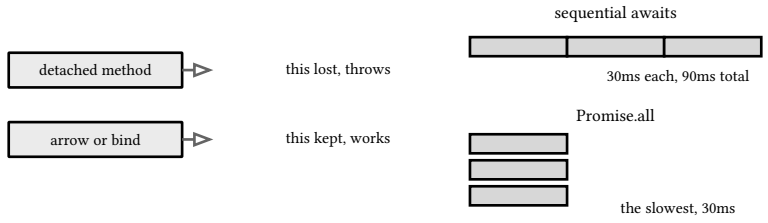


figure 37: this binds at the call site or lexically, awaits sum or take the slowest

The same rule is why react class components needed `bind` in constructors and function components do not, the bridge to book 17, ch 11 (the interview repertoire, react core: components, props, virtual dom). The array and async half of the question runs as a pipeline and two fetch orders:

```
  return items
    .filter((i) => i.stock > 0)
    .map((i) => ({ ...i, price: i.cents / 100 }))
    .reduce((acc, i) => ({ ...acc, [i.sku]: i.price }), {});
}

export async function fetchInOrder(urls, get) {
  const out = [];
  for (const url of urls) {
    // sequential awaits: order preserved, total time is the sum
    out.push(await get(url));
  }
  return out;
}

export async function fetchAll(urls, get) {
  // Promise.all: concurrent, order preserved by array position,
  // total time is the slowest
  return Promise.all(urls.map((url) => get(url)));
}
```

listing 40: the filter map reduce pipeline, sequential awaits, Promise.all (source: interview-repertoire/samples/ch08-js/src/arrays-async.mjs, lines 5-24)

The comparison to narrate: sequential awaits preserve call order at the cost of the sum of latencies, `Promise.all` preserves result order at the cost of the slowest request. The es6 shapes, destructuring, template literals, spread, computed keys, sit in one small function the suite pins.

sources: verified by `npm run verify` under node 26.3.0, 16 tests in `ch08-js`. The stylesheet is a real file, listed verbatim, not executed by the suite. The safe alignment fallback is pinned to the mdn justify-content row in the sources appendix.

9. javascript from scratch

"Write X from scratch" checks whether the candidate has used the primitive or only the convenience. This chapter implements five of them, test driven, in `ch09-js`: word count in one loop, an event emitter, map filter reduce, a promise subset that native async/await can drive, and a signals core, with the combinator ladder beside the promise and the diamond, disposal, and the branch switch closing the signals. 40 tests, no dependencies.

9.1. word count in one loop [TDD]

One pass over the characters, one state machine: a word opens on the first separator-to-letter transition and closes on the first letter-to-separator one after it. No split, no regex, no intermediate array:

```
export function wordCount(text) {
  let count = 0;
  let inWord = false;
  for (const ch of text) {
    const letter = ch !== " " && ch !== "\t" && ch !== "\n" && ch !== "\r";
    if (letter && !inWord) {
      count++;
      inWord = true;
    } else if (!letter) {
      inWord = false;
    }
  }
  return count;
}
```

listing 41: the single-loop counter (source: interview-repertoire/samples/ch09-js/src/wordcount.mjs, lines 5-18)

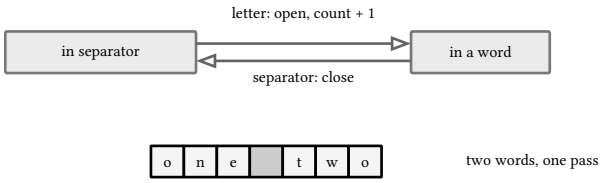


figure 38: a state machine over characters, no split and no regex

The follow-up answer ships in the same file: `wordsOf` collects the words with the same single loop, because an interviewer who asks for a count is two minutes from asking for the words.

9.2. the event emitter [TDD]

The node shape: `on`, `off`, `once`, `emit`, handlers in registration order, and `once` implemented as a wrapper that removes itself:

```
export class Emitter {
  #handlers = new Map();

  on(event, handler) {
    if (!this.#handlers.has(event)) {
      this.#handlers.set(event, []);
    }
    this.#handlers.get(event).push(handler);
    return () => this.off(event, handler); // unsubscribe handle
  }

  off(event, handler) {
    const list = this.#handlers.get(event);
    if (!list) return;
    const i = list.indexOf(handler);
    if (i =/= -1) list.splice(i, 1);
  }

  once(event, handler) {
    const wrap = (payload) => {
      this.off(event, wrap);
      handler(payload);
    };
    this.on(event, wrap);
  }

  emit(event, payload) {
    const list = this.#handlers.get(event);
    if (!list || list.length === 0) return false;
    // copy: a handler may unsubscribe mid-emit without corrupting
    // the loop that is running
    for (const handler of [...list]) {
      handler(payload);
```

listing 42: on, off, once as a self-removing wrapper, emit over a copy (source: interview-repertoire/samples/ch09-js/src/emitter.mjs, lines 5-37)

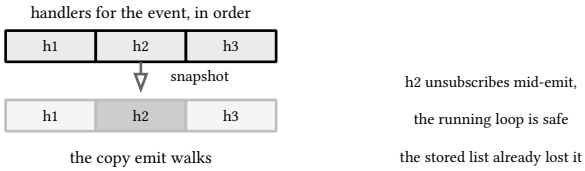


figure 39: registration-ordered list, emit iterates a copy

The copy in `emit` is the detail that scores: a handler that unsubscribes mid-emit must not corrupt the loop that is running, and the test does exactly that. The returned unsubscribe handle is the modern ergonomics worth offering unprompted.

9.3. map, filter, reduce [TDD]

The spec behaviors beyond the obvious: the index and array arguments, holes skipped by `i in xs`, and reduce's two signatures, with an initial value folding from it, without one adopting the first element and throwing on empty:

```
export function myMap(xs, f) {
  const out = new Array(xs.length);
  for (let i = 0; i < xs.length; i++) {
    if (i in xs) out[i] = f(xs[i], i, xs);
  }
  return out;
}

export function myFilter(xs, keep) {
  const out = [];
  for (let i = 0; i < xs.length; i++) {
    if (i in xs && keep(xs[i], i, xs)) out.push(xs[i]);
  }
  return out;
}

export function myReduce(xs, f, init) {
  let acc = init;
  let start = 0;
  if (arguments.length === 2) {
    // no initial value: the first element becomes the accumulator
    if (xs.length === 0) throw new TypeError("Reduce of empty array with no initial value");
    acc = xs[0];
    start = 1;
  }
  for (let i = start; i < xs.length; i++) {
    if (i in xs) acc = f(acc, xs[i], i, xs);
  }
  return acc;
}
```

listing 43: the three loops with the spec's edge behavior (source: interview-repertoire/samples/ch09-js/src/arraymethods.mjs, lines 5-35)

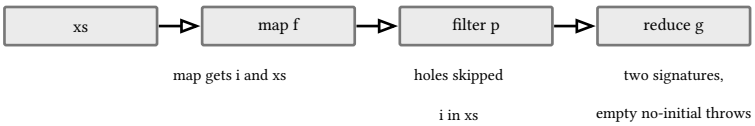


figure 40: the spec edges ride the same three loops

The empty-array-no-initial-value test pins the exact TypeError message of the native implementation, which is a flex that costs one line.

9.4. a promise subset [TDD]

The teaching core of the A+ spec: settle once, callbacks always microtasked, chaining through a returned promise, and the resolution procedure that waits on thenables instead of storing them:

```
  constructor(executor) {
    const reject = (value) => this.#settle(REJECTED, value);
    // the resolution procedure of the spec: resolving with a
    // thenable waits for it instead of storing it, which is what
    // makes promise chaining and MiniPromise.all work
    const resolve = (value) => {
      if (value && typeof value.then === "function") {
        value.then(resolve, reject);
        return;
      }
      this.#settle(FULFILLED, value);
    };
    try {
      executor(resolve, reject);
    } catch (err) {
      reject(err);
    }
  }

  #settle(state, value) {
    if (this.#state =!= PENDING) return; // first settle wins
    this.#state = state;
    this.#value = value;
    for (const cb of this.#callbacks) queueMicrotask(() => cb(state, value));
    this.#callbacks = [];
  }
```

listing 44: first settle wins, thenables are assimilated, callbacks ride queueMicrotask (source: interview-repertoire/samples/ch09-js/src/minipromise.mjs, lines 15-40)

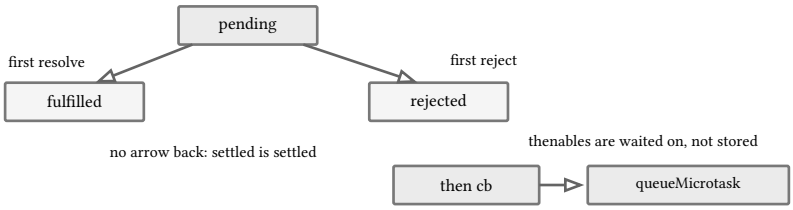


figure 41: settle once, callbacks always microtasked, thenables assimilated

```
  #onDone(onFulfilled, onRejected) {
    return new MiniPromise((resolve, reject) => {
      const deliver = (state, value) => {
        const handler = state === FULFILLED ? onFulfilled : onRejected;
        if (typeof handler !== "function") {
          // pass through: resolve with the same value, or keep rejecting
          if (state === FULFILLED) resolve(value);
          else reject(value);
          return;
        }
        try {
          resolve(handler(value));
        } catch (err) {
          reject(err);
        }
      };
      if (this.#state === PENDING) {
        this.#callbacks.push(deliver);
      } else {
        queueMicrotask(() => deliver(this.#state, this.#value));
      }
    });
  }

  then(onFulfilled, onRejected) {
    return this.#onDone(onFulfilled, onRejected);
  }

  catch(onRejected) {
    return this.#onDone(undefined, onRejected);
  }
```

listing 45: the chained promise, pass-through on missing handlers, catch as sugar (source: interview-repertoire/samples/ch09-js/src/minipromise.mjs, lines 42-72)

Two red runs taught this file its shape, and both are interview stories. A thenable stored instead of assimilated made `MiniPromise.all` resolve with the promise object where its value belonged. A single-argument `then` swallowed the rejection path of `all`, and the fail-fast test hung forever, which is what a swallowed rejection looks like from the outside. The final demo is the payoff line for the whole exercise: a native `async` function awaits a chain of MiniPromises, because `await` only requires the thenable protocol, and the test drives three stages through it.

9.5. the combinator ladder: race, allSettled, any [TDD]

`all` has three siblings, and interviewers ask for the four as a table. Each is one sentence of semantics plus one accounting rule, and the workspace runs all four:

```
  // race: the first settle wins, whichever flavor it is. The
  // settle-once gate makes later arrivals no-ops, and an empty
  // input never settles at all, which is the spec's own answer.
  static race(promises) {
    return new MiniPromise((resolve, reject) => {
      for (const p of promises) MiniPromise.resolved(p).then(resolve, reject);
    });
  }

  // allSettled: rejection is data, not failure. Every input lands
```

```
  // in the output at its own index, as a status pair.
  static allSettled(promises) {
    return new MiniPromise((resolve) => {
      const list = [...promises]; // any iterable, materialized once
      const out = new Array(list.length);
      let remaining = list.length;
      if (remaining === 0) {
        resolve(out);
        return;
      }
      list.forEach((p, i) => {
        MiniPromise.resolved(p).then(
          (value) => {
            out[i] = { status: "fulfilled", value };
            if (--remaining === 0) resolve(out);
          },
          (reason) => {
            out[i] = { status: "rejected", reason };
            if (--remaining === 0) resolve(out);
          },
        );
      });
    });
  }

  // any: the mirror of all, the first fulfillment wins and the
  // rejections accumulate. Only when every input has rejected does
  // the AggregateError leave with the errors in input order.
  static any(promises) {
    return new MiniPromise((resolve, reject) => {
      const list = [...promises]; // any iterable, materialized once
      const errors = new Array(list.length);
      let remaining = list.length;
      if (remaining === 0) {
        reject(new AggregateError([], "all promises were rejected"));
        return;
      }
      list.forEach((p, i) => {
        MiniPromise.resolved(p).then(resolve, (reason) => {
          errors[i] = reason;
          if (--remaining === 0) {
            reject(new AggregateError(errors, "all promises were rejected"));
          }
        });
      });
    });
  }
```

listing 46: race takes the first settle, allSettled treats rejection as data, any accumulates rejections into an AggregateError (source: interview-repertoire/samples/ch09-js/src/minipromise.mjs, lines 100-156)

What each is: `race` settles with the first input to settle, fulfillment or rejection alike. `allSettled` always fulfills, with every input's outcome as a status pair at its own index. `any` is `all` mirrored, the first fulfillment wins, and rejection is the failure mode only when every input has rejected.

Why they exist: the four cover the two axes an async collection cares about, first versus all, and success versus failure. `all` is all plus success. `race` is first plus either. `allSettled` is all plus either, with rejection demoted to data. `any` is first plus success.

How, walked for `any` because its accounting is the one people get wrong: every input is assimilated through `MiniPromise.resolved` exactly as in `all`, but a fulfillment resolves the whole thing on arrival, while each rejection only writes its error into a slot and decrements the count, and only at zero does the `AggregateError` leave with the errors in input order. The empty input rejects immediately with an empty errors list. `race` on an empty input never settles at all, which is the spec's own answer and the trap worth volunteering, because what `race` of nothing does has no intuitive guess. Every combinator also takes any iterable, like the native ones, the input materialized once with `[...promises]`, because the accounting needs indexes and a Set's `forEach` hands its value back twice, never an index.

When: `race` for a timeout raced against the real work, `allSettled` when a batch must report every outcome, `any` for redundant replicas where one success is enough.

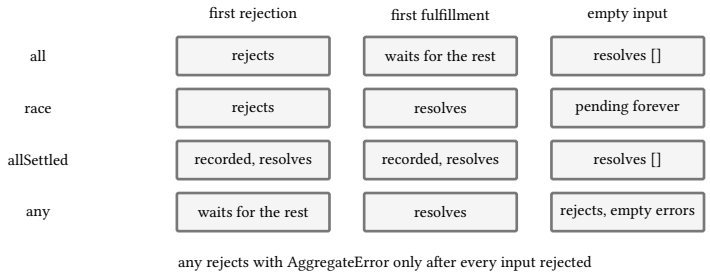


figure 42: the combinator truth table: which settle wins, and what an empty input does

The tests pin the table against node itself: the `allSettled` statuses and the `any` error lists are compared with the native `Promise` combinators over the same inputs, so the accounting, not just the shape, matches the spec.

9.6. a signals core [TDD]

The tc39 signals proposal shape reduced to its moving parts: state with subscribers, computed with dependency tracking while it evaluates, effects as the evaluation root:

```
export function createState(initial) {
  return {
    _value: initial,
    _subscribers: new Set(),
    get() {
      if (evaluating) {
        this._subscribers.add(evaluating);
        evaluating._reads.add(this); // disposal walks the edge back
      }
      return this._value;
    },
    set(next) {
      if (Object.is(next, this._value)) return; // same value: no notification
      this._value = next;
      for (const sub of [...this._subscribers]) sub.notify();
      drain(); // the wave is over, every effect runs at most once
    },
  };
}
```

listing 47: state: track readers, notify on change, skip same-value writes under SameValue (source: interview-repertoire/samples/ch09-js/src/signal.mjs, lines 44-62)

```
export function createComputed(compute) {
  const computed = {
    _value: undefined,
    _valid: false,
    _reads: new Set(), // re-collected on every recompute
    _subscribers: new Set(),
    get() {
      if (!this._valid) {
        this._value = evaluate(this, compute);
        this._valid = true;
      }
      // whoever is reading this computed (an effect or another
      // computed) becomes its subscriber, which is the edge that
      // propagates invalidation downstream
      if (evaluating) {
        this._subscribers.add(evaluating);
        evaluating._reads.add(this); // disposal walks the edge back
      }
      return this._value;
    },
    notify() {
      if (this._valid) {
        this._valid = false; // dirty: the next read recomputes
        for (const sub of [...this._subscribers]) sub.notify();
      }
    },
  };
  return computed;
}
```

listing 48: computed: evaluate under the tracking flag, subscribe the reader, invalidate on notify (source: interview-repertoire/samples/ch09-js/src/signal.mjs, lines 64-92)

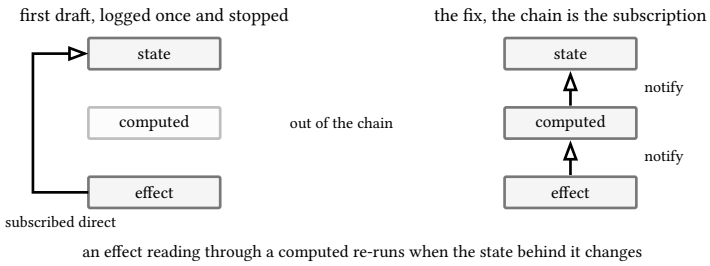


figure 43: the subscription edge, through the computed, never around it

The observable the tests pin: an unrelated state change does not recompute a computed, a same-value write fires nothing under SameValue comparison, an effect reading through a computed re-runs when the state behind it changes, and a conditional read re-tracks on every run so the branch not taken stops notifying. The subscription edge is the oldest of those stories, and the first draft of this file got it wrong, effects subscribed to the state directly and never to the computed between them, so the demo logged once and stopped. The status of the real proposal, stage 1 with stage 2 blockers as the active work, is pinned in book 17, ch 12 (the interview repertoire, react hooks and routing).

9.7. signals edge cases: the diamond, disposal, and the branch switch [TDD]

Three edges finish the signals story, and two of them found real bugs in this file's own first draft, which is the interview story worth telling.

The diamond. What: one state feeding two computed branches that a third computed joins, with one effect on the join. Why it is the test: a naive push design invalidates branch one, the effect runs immediately, the join recomputes from one fresh branch and one stale, and the effect logs a value that never existed. That is the glitch, and the first draft of this file produced it: with `a` at 1 the join reads 5, one `set` to 5 made the effect log 9 on the way to 13, because it ran after the first branch landed and before the second. How the fix works: invalidation only marks nodes dirty and batches the effect runs, `drain()` fires once at the end of the whole `set`, so the join recomputes exactly once from both fresh branches and the effect logs `[5, 13]`. When to say it: the follow-up about diamond dependencies is answered with this batch, plus the honest boundary, real frameworks also order recomputation topologically and dedupe across microtasks, and those semantics are part of the stage 2 blockers the pinned proposal row names.

Disposal. What: an effect's `dispose()` removes it from every node it subscribed to and kills any run still waiting in the batch. Why: an effect that outlives its page or component leaks, because every set keeps notifying it. How: the effect records its read edges on every run, disposal walks them back, and a disposed flag checked at drain time kills the pended run, because the wave is a snapshot an editor cannot reach. When: unmount, cleanup, any effect with a lifetime shorter than the graph.

The branch switch. What: a conditional read swaps the subscription edges to the branch actually taken, on every run. Why: an implementation that builds its edges once, on the first run, goes wrong in both directions, it stops hearing the state it has started reading and keeps hearing from the one it has left, a missed update and a stale notification out of one bug. How: every evaluation, effect or computed, re-collects its reads, and a dependency left behind loses its subscriber. When: any read behind a conditional, a flag choosing between two sources, a derived value whose formula changes with a mode.

```
// Effects run once per wave, after the whole invalidation has
// finished, never inline mid-propagation. The first draft ran
// them inline and the diamond test logged an intermediate value,
// both branches had not landed yet, which is exactly the glitch
// real frameworks batch away. The disposed flag carries the other
// half of the batch story: a wave is a snapshot, an effect disposed
// mid-wave cannot be removed from it, so the check happens at run
// time.
function drain() {
  while (pending =/= null && pending.size > 0) {
    const wave = pending;
    pending = null;
    for (const eff of wave) if (!eff._disposed) eff.run();
  }
}

// Every evaluation re-collects its reads. A conditional read swaps
// the edges to the branch actually taken, and a dependency left
// behind loses its subscriber. The first draft built every edge
// once, on the first run, so an effect that switched branches went
// wrong in both directions, deaf to the state it had started
// reading and still listening to the one it had left.
function evaluate(node, compute) {
  const stale = node._reads;
  node._reads = new Set();
  const outer = evaluating;
  evaluating = node;
  const value = compute();
  evaluating = outer;
  for (const dep of stale) {
    if (!node._reads.has(dep)) dep._subscribers.delete(node);
  }
  return value;
}
```

listing 49: the batch and the evaluation wrapper: one wave per set, every run re-collects its reads (source: interview-repertoire/samples/ch09-js/src/signal.mjs, lines 9-42)

```
export function createEffect(fn) {
  const effect = {
    _reads: new Set(), // every node that subscribed this effect
    _disposed: false,
    notify() {
      (pending ??= new Set()).add(effect); // batched, deduped per wave
    },
    dispose() {
      this._disposed = true; // a pended run dies with the effect
      for (const node of this._reads) node._subscribers.delete(this);
      this._reads.clear();
    },
    run() {
      evaluate(this, fn); // every run re-collects the read edges
    },
  };
  effect.run();
  return effect;
}
```

listing 50: the effect: notify batches and dedupes, dispose flags and walks the read edges back (source: interview-repertoire/samples/ch09-js/src/signal.mjs, lines 94-112)

```
export function diamond() {
  const log = [];
  const runs = { effect: 0, join: 0 };
  const a = createState(1);
  const b = createComputed(() => a.get() + 1);
  const c = createComputed(() => a.get() + 2);
  const d = createComputed(() => {
    runs.join += 1;
    return b.get() + c.get();
  });
  createEffect(() => {
    runs.effect += 1;
    log.push(d.get());
  });
  a.set(5);
  return { log, runs };
}
```

listing 51: the diamond demo, its counters the tests pin (source: interview-repertoire/samples/ch09-js/src/signal.mjs, lines 130-146)

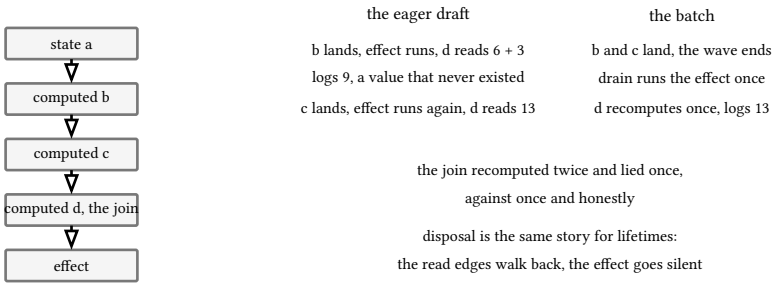


figure 44: one set, one join recompute: the eager draft logged a value that never existed

The step by step for a signals question, said aloud: name the three roles, state, computed, effect, name the two edges, dependency tracking on the way down and invalidation on the way up, then the diamond answer, one batch per set so nothing observes a half updated graph, then disposal when the listener has a lifetime, and the branch switch, reads re-collect on every run so a conditional cannot leak a stale edge.

sources: verified by `npm run verify` under node 26.3.0, 40 tests in `ch09-js`. Proposal shape read from the tc39 signals repository, access date in the sources appendix.

10. data plumbing: csv to mongodb

The migration question is really three questions: parse the source honestly, shape the rows into documents worth storing, and insert in batches a real server will accept. The `ch10-js` workspace runs the first two thirds under `npm run verify` against a recording fake, 10 tests, and the last third is a mongosh script gated behind a real server from book 13, ch 13 (infrastructure: docker, databases, and testing, mongodb and aggregation pipelines), whose compose capstone starts one.

10.1. parsing csv without lying about it [TDD]

Csv is a format with quoting rules, and the tests exercise them: commas inside quotes, newlines inside quotes, doubled quotes as escapes, CRLF tolerated, no trailing newline required:

```
  const rows = [];
  let field = "";
  let row = [];
  let inQuotes = false;
  let i = 0;
  while (i < text.length) {
    const ch = text[i];
    if (inQuotes) {
      if (ch === '"') {
        if (text[i + 1] === '"') {
          field += '"';
          i += 2;
          continue;
        }
        inQuotes = false;
        i++;
        continue;
      }
      field += ch;
      i++;
      continue;
    }
    if (ch === '"') {
      inQuotes = true;
      i++;
      continue;
    }
    if (ch === ",") {
      row.push(field);
      field = "";
      i++;
      continue;
    }
    if (ch === "\n" || ch === "\r") {
      if (ch === "\r" && text[i + 1] === "\n") i++;
      row.push(field);
      field = "";
      if (row.length > 1 || row[0] !== "") rows.push(row);
      row = [];
      i++;
      continue;
    }
    field += ch;
    i++;
  }
  // last field and last row when the file lacks a trailing newline
  if (field !== "" || row.length > 0) {
    row.push(field);
    rows.push(row);
  }
  return rows;
```

listing 52: the one-pass parser: quote state, delimiter, and the trailing row (source: interview-repertoire/samples/ch10-js/src/csv.mjs, lines 5-55)

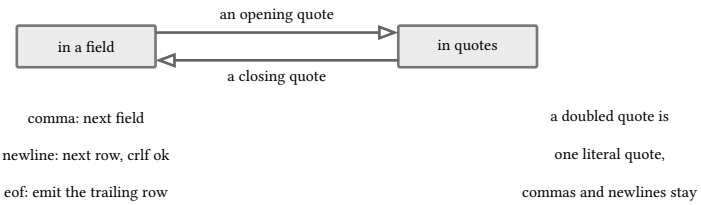


figure 45: the quote state machine: doubles escape, the trailing row still emits

The bug class this parser exists to avoid is the `split(",")` opening gambit, which dies on the first quoted comma. The fixture file under `csv/fixtures/products.csv` carries quoted tags so the parse is never tested on toy rows alone.

10.2. documents, batches, and the one-method interface [TDD]

The migration core codes against a collection interface with a single method, `insertMany`, because that is exactly what the mongodb driver exposes, which is why the same code runs against the fake and against a real server:

```
// satisfy for the migration to run against a real server.

export function coerce(value) {
  if (value === "") return null;
  if (value === "true") return true;
  if (value === "false") return false;
  if (/^-?\d+$/.test(value)) return Number.parseInt(value, 10);
  if (/^-?\d+\.\d+$/.test(value)) return Number.parseFloat(value);
  return value;
}

export function toDocuments(objects, schema = {}) {
  return objects.map((o) => {
    const doc = {};
    for (const [key, raw] of Object.entries(o)) {
      const type = schema[key];
      doc[key] = type ? type(raw) : coerce(raw);
    }
    // the migration stamp every import carries
    doc._importedAt = new Date("2026-09-10T00:00:00Z");
    if (doc.priceCents !== undefined) {
      doc.price = { cents: doc.priceCents, currency: "usd" };
      delete doc.priceCents;
    }
    return doc;
  });
}

export async function migrate(collection, documents, batchSize = 100) {
  let inserted = 0;
  for (let i = 0; i < documents.length; i += batchSize) {
    const batch = documents.slice(i, i + batchSize);
    await collection.insertMany(batch.map(cloneDoc));
    inserted += batch.length;
  }
  return { inserted, batches: Math.ceil(documents.length / batchSize) };
}

// Mongo mutates the documents it inserts (adds _id); the fake below
// does the same, so the migration hands each batch a fresh copy.
function cloneDoc(doc) {
  return structuredClone(doc);
```

listing 53: type coercion, the money reshape, stamped imports, batched inserts (source: interview-repertoire/samples/ch10-js/src/migrate.mjs, lines 5-46)

Three decisions worth narrating in the room. Csv values are all strings, so coercion happens once at the boundary, with a schema override for columns like a leading-zero sku. Money lands as an integer cents field reshaped into a nested `{cents, currency}` document, the same no-floats rule as book 17, ch 17 (the interview repertoire, database answers). And every document carries an import stamp, because a migration without provenance is a table nobody trusts.

The fake records batches, assigns `_id` exactly the way the driver does, and can fail on a chosen batch number:

```
export class FakeCollection {
  docs = [];
  batches = [];
  failOnBatch = -1; // 1-indexed batch number that throws

  async insertMany(batch) {
    const batchNo = this.batches.length + 1;
    if (batchNo === this.failOnBatch) {
      throw new Error(`insertMany failed on batch ${batchNo}`);
    }
    for (const doc of batch) {
      doc._id = `oid-${this.docs.length + 1}`;
      this.docs.push(doc);
    }
    this.batches.push({ batchNo, size: batch.length });
    return { acknowledged: true, insertedCount: batch.length };
  }
}
```

listing 54: the recording fake with failure injection (source: interview-repertoire/samples/ch10-js/src/fake.mjs, lines 5-24)

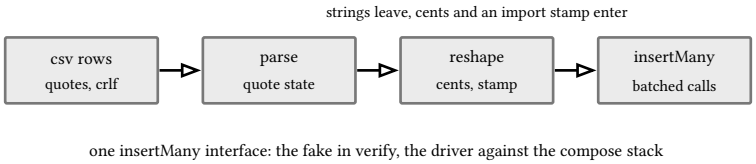


figure 46: csv rows to reshaped documents to batched calls, one interface for the fake and the real driver

The failure test asserts the honest middle state: batch 1 landed, batch 2 raised, and nothing was swallowed. A migration that hides a failed batch has converted a data problem into a data mystery.

10.3. the real run, docker gated [DRILL]

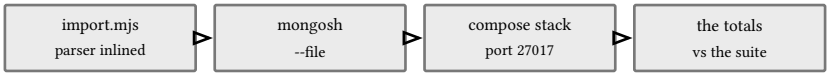
The real import is a mongosh script, listed from the workspace, that mirrors the parser and coercer inline because mongosh runs one standalone script rather than the package's ESM modules:

```
// The parser and coercer mirror src/csv.mjs and src/migrate.mjs
// inline, because mongosh runs one standalone script, not the
// package's ESM modules. It reads csv/fixtures/products.csv, inserts
// in batches of 100, and prints counts to compare with the suite.

const csv = require("fs").readFileSync("csv/fixtures/products.csv", "utf8");

function parseCsv(text) {
  const rows = [];
  let field = "";
  let row = [];
  let inQuotes = false;
  for (let i = 0; i < text.length; i++) {
    const ch = text[i];
    if (inQuotes) {
```

listing 55: the gated script: same fixture, same batches, counts printed at the end (source: interview-repertoire/samples/ch10-js/real/import.mjs, lines 6-20)



outside npm run verify on purpose: the gate stays offline

figure 47: the same migration inlined for mongosh, totals compared with the offline suite

Run it against the compose stack of book 13: `docker compose up -d`, then `mongosh mongodb://localhost:27017 shop --file real/import.mjs`, and compare the printed totals with the suite's. It is deliberately outside `npm run verify`: the gate stays offline and deterministic, and the network path is a separate, named step, the same split book 13 uses for its docker-gated suites.

sources: batch insert and driver behavior from the mongodb manual, pinned in book 13, ch 13 (infrastructure: docker, databases, and testing, mongodb and aggregation pipelines) and its sources. Verified by `npm run verify` under node 26.3.0, 10 tests in `ch10-js`.

11. react core: components, props, virtual dom

The react workspace, `ch12-react`, named for the hooks chapter and shared by chapters 11 through 14, is a real react install, react 19, react-dom 19, react-router 7, committed lockfile, driven by `node --test` over `react-dom/server` rendering. That constraint shapes what this chapter can honestly claim: everything here is proven by rendered markup, the same guarantee a server render gives, and the effects-and-browser half of react floors to the hooks chapter that follows.

11.1. function components versus class components [EWC]

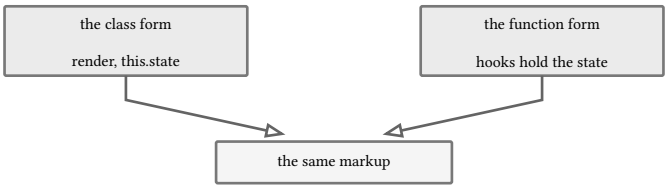
A function component is a function from props to elements. A class component is a class with a render method and instance state. The workspace renders both to identical markup:

```
  return h("p", { class: "greeting" }, `hello ${name}`);
}

// The class form: render() returns elements, state lives on this.
export class CounterClass extends Component {
  constructor(props) {
    super(props);
    this.state = { count: props.start ?? 0 };
  }
  render() {
    return h("p", { class: "counter" }, `count ${this.state.count}`);
  }
}

// Props flow down: the parent composes children as data.
export function Card({ title, children, footer }) {
```

listing 56: the same output from a function and from a class (source: interview-repertoire/samples/ch12-react/src/core/components.js, lines 7-22)



since 16.8 the function form owns the state story

figure 48: both are props to elements, hooks moved the state into functions

The spoken history that scores: class components carried the state and lifecycle story before hooks existed, function components plus hooks took it over in 16.8, and new react code is function-only because the class form buys nothing, not even `this` safety, as book 17, ch 8 (the interview repertoire, javascript and the dom) showed with the detached-method throw. Say the one thing classes still own in old codebases, `componentDidCatch` error boundaries, and concede it.

11.2. props, children, and drilling [EWC]

Props flow down, `children` is a prop like any other, and the composition demo renders both:

```
    "section",
    { class: "card" },
    h("h2", null, title),
    h("div", { class: "body" }, children),
    footer ? h("footer", null, footer) : null,
  );
}

// Prop drilling is the same flow repeated: Theme passes what Page
// needs down to Button because only Button uses it.
export function Theme({ tone, label }) {
  return h(Page, { tone, label });
}

function Page({ tone, label }) {
  return h("div", { class: "page" }, h(Button, { tone, label }));
}

function Button({ tone, label }) {
  return h("button", { class: `btn btn-${tone}` }, label);
}
```

listing 57: children through composition, and the drilling chain (source: interview-repertoire/samples/ch12-react/src/core/components.js, lines 24-44)

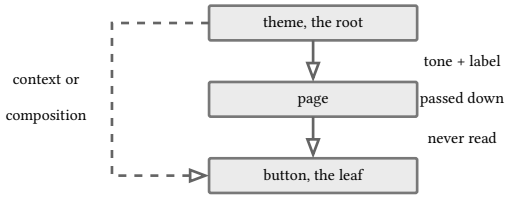


figure 49: drilling repeats the downward flow until a leaf uses what the root had

Prop drilling is the same downward flow repeated until a leaf uses something only the root had. The `Theme` to `Button` chain passes `tone` and `label` through a `Page` that never reads them, and the test asserts the exact markup drilling produces. The fixes are the context of book 17, ch 12 (the interview repertoire, react hooks and routing) when the value is ambient, or component composition when the problem is a parent in the way.

11.3. the virtual dom [EWC]

Elements are plain objects, not markup. Type, props, children, and that is the whole premise:

```
// objects describing markup, not markup.
export function elementIsData() {
  const el = h("li", { key: 1 }, "sword");
  return { isObject: typeof el === "object", type: el.type, hasProps: "props" in el };
}
```

listing 58: an element is data (source: interview-repertoire/samples/ch12-react/src/core/components.js, lines 47-52)

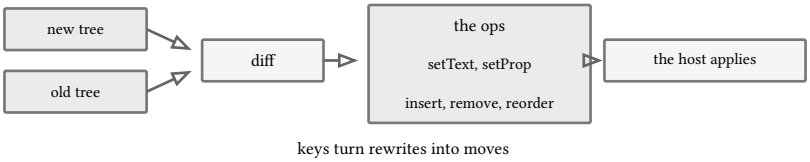


figure 50: elements are data, the diff emits ops, the host applies them

The interview version in one breath: components return element trees, react diffs the new tree against the old one, and applies the minimal set of dom mutations the diff implies, so the developer describes the whole interface and the library pays for only what changed. The from-scratch implementation of that diff, with keyed reordering and patch ops a host applies, is the second half of book 17, ch 14 (the interview repertoire, react build: a listing page, test driven), built in plain js with tests that count nodes.

sources: verified by `npm run verify` under node 26.3.0 with react 19.3, 5 tests in the `src/core` suite of `ch12-react`. React 19 release facts are pinned in the sources appendix from the react blog, accessed 2026-09-09.

12. react hooks and routing

“List the hooks and say what each is for” is a breadth check with a depth trap attached, and 2019 through 2026 added a whole react 19 layer to it. The `ch12-react` workspace covers both under `npm run verify`, with every claim rendered to markup through `react-dom/server`.

12.1. the built-in hooks that render server-side [EWC]

State, reducer, context, memoization, and identity, each in its smallest honest form:

```
// useState: one value and its setter. The initial render shows the
// seed; updates happen after events, which a server render never
// sees, so the tests pin the first paint.
export function Stepper({ start = 0 }) {
  const [step, setStep] = useState(start);
  return h("p", { "data-step": String(step) },
    h("button", { onClick: () => setStep(step + 1) }, "next"));
}

// useReducer: state transitions named as actions, the shape that
// scales past useState.
export function cartReducer(state, action) {
  switch (action.type) {
    case "add":
      return { items: state.items + 1 };
    case "clear":
      return { items: 0 };
    default:
      return state;
  }
}

export function CartBadge() {
  const [state, dispatch] = useReducer(cartReducer, { items: 0 });
  return h("p", { "data-items": String(state.items) },
    h("button", { onClick: () => dispatch({ type: "add" }) }, "add"),
    h("button", { onClick: () => dispatch({ type: "clear" }) }, "clear"));
}

// context: values that skip the prop drilling chain.
export const CartContext = createContext({ items: 0, label: "cart" });

export function CartLabel() {
  const { items, label } = useContext(CartContext);
  return h("p", { class: "badge" }, `${label}: ${items}`);
}

// useMemo: cache a computation between renders when dependencies
// hold. On the server it runs once, which is exactly what the first
// paint costs.
export function PricedTotal({ prices }) {
  const total = useMemo(() => prices.reduce((a, b) => a + b, 0), [prices]);
  return h("p", { "data-total": String(total) }, `total ${total}`);
}

// useId: stable unique ids across the tree, the accessible-label
// hook, hydration safe where Math.random is not.
export function Field({ label }) {
  const id = useId();
  return h("label", { for: id }, h("input", { id }), label);
}
```

listing 59: useState seeding, useReducer transitions, context consumption, useMemo, useId (source: interview-repertoire/samples/ch12-react/src/hooks/hooks.js, lines 13-63)

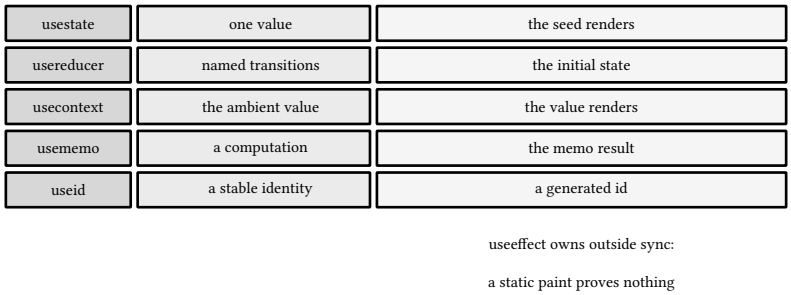


figure 51: what each hook owns and what a static render can prove

The narration map, one line each. `useState` for one value, `useReducer` when transitions deserve names, `useContext` to skip the drilling chain, `useMemo` to cache a computation between renders, `useCallback` for a stable function identity, `useRef` for a value the render must not see change, `useEffect` for synchronization with the outside world. The honest framing for the last one, from the react docs themselves: effects are an escape hatch from react, not a lifecycle feature.

NOTE: what a server render can and cannot prove

The suite pins what renders on first paint: seeded state, initial reducer state, context values, memo results, generated ids. What it cannot pin is updates after events, because `renderToStaticMarkup` never fires them. That is a real boundary, not a dodge: effects and updates belong to a browser runtime, and the from-scratch signal of book 17, ch 9 (the interview repertoire, javascript from scratch) demonstrates the same reactive core without one.

12.2. a custom hook is just a function that calls hooks [EWC]

The useToggle demo owns a boolean and returns it with props for the caller, reusable per component:

```
// A custom hook is a function that calls hooks. This one owns a
// boolean and returns it with its setter, reusable per component.
export function useToggle(initial = false) {
  const [on, setOn] = useState(initial);
  return {
    on,
    toggleProps: { "aria-pressed": String(on) },
    toggle: () => setOn((v) => !v),
  };
}

export function Switch({ start }) {
  const { on, toggleProps } = useToggle(start);
  return h("button", { ...toggleProps, role: "switch" }, on ? "on" : "off");
}
```

listing 60: useToggle: stateful behavior in a callable package (source: interview-repertoire/samples/ch12-react/src/hooks/hooks.js, lines 65-79)

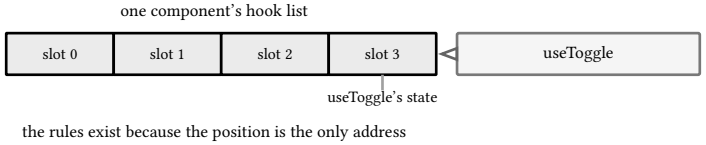


figure 52: hook state lives in a position-indexed list, order is the address

The two rules to state unprompted: hooks call only from components or other hooks, and they call unconditionally in the same order every render. The rules exist because hook state lives in a position-indexed list per component instance, which is also the answer to “why can’t I put it in an if”.

12.3. react-router, one route table [EWC]

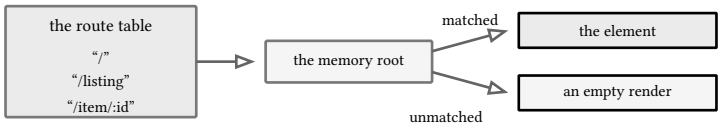
Routes as data, `MemoryRouter` for anything but the browser, nested paths and params:

```
export function App() {
  return h(MemoryRouter, { initialEntries: ["/"] }, h(Routes, null,
    h(Route, { path: "/", element: h(Home) }),
    h(Route, { path: "/products", element: h(Products) }),
    h(Route, { path: "/products/:sku", element: h(ProductDetail) }),
  ));
}

function Home() {
  return h("main", null,
    h("h1", null, "shop"),
    h("nav", null, h(Link, { to: "/products" }, "all products")));
}

function Products() {
  return h("main", null,
    h("h1", null, "products"),
    h("ul", null,
      h("li", { key: "a" }, h(Link, { to: "/products/sword-01" }, "sword")),
      h("li", { key: "b" }, h(Link, { to: "/products/hat-01" }, "hat"))));
}
```

listing 61: the route table and the in-memory root (source: interview-repertoire/samples/ch12-react/src/router/app.js, lines 6-27)



the renderAt seam mounts any url, no browser history

figure 53: routes as data under one in-memory root, any url mounted

The `renderAt` seam is the testable part worth narrating: the same table mounted at any in-memory url, no browser history involved. The suite asserts the root, the listing, a param route, and the unknown-path empty render, which is what the router actually does with an unmatched path.

12.4. react 19: actions, use, and the compiler [EWC]

React 19, stable december 2024, made async transitions first class. An action is any async function, and `useActionState` wraps one with pending state and a returned form action:

```
// An Action is any async function: react 19 lets transitions and
// form handlers await them natively, with pending state managed by
// the framework instead of hand-written isLoading flags.
export async function saveName(name) {
  await new Promise((r) => setTimeout(r, 1));
  if (name === "error") throw new Error("name rejected");
  return `saved ${name}`;
}

// useActionState wraps an action: [state, formAction, isPending].
// The first server render shows initialState and isPending false.
export function NameForm({ action = saveName }) {
  const [state, formAction, isPending] = useActionState(
    async (prev, formData) => {
      const result = await action(formData.get("name"));
      return result;
```

listing 62: an action, and useActionState around it (source: interview-repertoire/samples/ch12-react/src/react19/actions.js, lines 14-30)

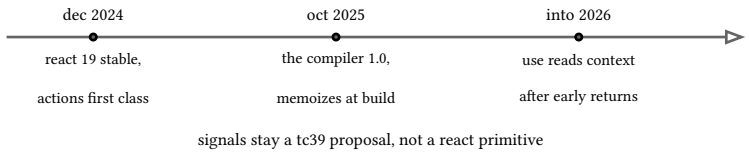


figure 54: react 19 made async first class, the compiler memoizes at build

`useOptimistic` renders a value that assumes success and reverts on completion, `use` reads a promise or a context anywhere below a provider, including after early returns, which `useContext` forbids. The workspace renders the initial states of all of them, and the `use(Context)` case runs for real:

```
// use() unwraps a context anywhere below the provider, including
// after early returns, which useContext forbids.
export function UserBadge({ hidden = false }) {
  if (hidden) return null;
  const { name, plan } = use(UserContext);
  return h("span", { class: `badge-${plan}` }, name);
```

listing 63: use reading context after an early return (source: interview-repertoire/samples/ch12-react/src/react19/actions.js, lines 47-53)

The compiler is its own fact, stable at 1.0 since october 2025: it memoizes at build time so the manual `useMemo` and `useCallback` call sites disappear from new code. The status of signals belongs to the ecosystem, not to react: the tc39 proposal sits at stage 1, with stage 2 blockers as the active work item, and react's own answer to fine-grained reactivity remains the compiler plus `use`, not a signal primitive. Both facts are pinned in the sources appendix with their access dates.

sources: react 19 release post and the react compiler 1.0 announcement at react.dev, tc39 proposal-signals repository, the release post and the tc39 repository accessed 2026-09-09, the compiler announcement 2026-09-10. Verified by `npm run verify` under node 26.3.0 with react 19.3 and react-router 7, 6 tests in `src/hooks`, 4 in `src/router`, 4 in `src/react19`.

13. react state: context, redux, graphql

State management questions are really caching questions wearing a state hat. The workspace builds the three answers in miniature: context for ambient values, a redux-shaped store bound through `useSyncExternalStore`, and a request cache with dedupe and invalidation, the part rtk query automates for a living.

13.1. context versus a store [EWC]

Context is a dependency injection mechanism: one provider, many consumers, and a value change re-renders every consumer. A store is observable state plus dispatch, and react binds to it through `useSyncExternalStore`:

```
  const listeners = new Set();
  return {
    getState: () => state,
    subscribe(listener) {
      listeners.add(listener);
      return () => listeners.delete(listener);
    },
    dispatch(action) {
      state = reducer(state, action);
      for (const l of [...listeners]) l();
    },
  };
}

// The react binding. getServerSnapshot covers the server render;
// both snapshots read the same source.
export function StoreProvider({ store, context = StoreContext, children }) {
  return h(context.Provider, { value: store }, children);
}

export const StoreContext = createContext(null);

export function useStore(context = StoreContext) {
  const store = useContext(context);
  const state = useSyncExternalStore(
    store.subscribe,
    store.getState,
    store.getState,
  );
  return [state, store.dispatch];
}

// A request cache with dedupe and invalidation: the part of rtk query
// this workspace rebuilds. Identical keys share one flight; a
// failure clears the slot; invalidate forces the next read to fly.
export function createRequestCache(fetchImpl) {
  const inflight = new Map();
  const fresh = new Map();
```

listing 64: a redux-shaped store, and the react binding over it (source: interview-repertoire/samples/ch12-react/src/state/store.js, lines 9-46)

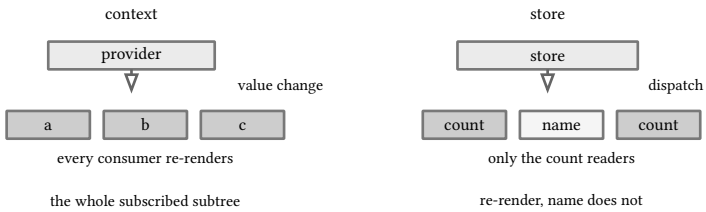


figure 55: context versus a store: who re-renders on a change

The line that scores: reach for context when the value is ambient and changes rarely, theme, locale, the current user. Reach for a store when many components read slices of fast-changing state, because a context value change re-renders the whole consumer subtree while a store subscription re-renders exactly the components whose slice changed. The suite proves the binding: one dispatch, the readout moves, and the same store serves a second component without threading.

13.2. what rtk query automates [EWC]

Redux toolkit query is a request cache with dedupe, invalidation, and lifecycle flags. The workspace rebuilds the core of it in forty lines so the answer is not a product name:

```
      if (fresh.has(key)) return fresh.get(key);
      if (!inflight.has(key)) {
        const p = fetchImpl(key).then(
          (v) => {
            inflight.delete(key);
            fresh.set(key, v);
            return v;
          },
          (e) => {
            inflight.delete(key);
            throw e;
          },
        );
        inflight.set(key, p);
      }
      return inflight.get(key);
    },
    invalidate(key) {
      fresh.delete(key);
    },
    _fresh: fresh,
    _inflight: inflight,
  };
}

// The n+1 question as data: 10 users means 1 + 10 queries naive, or
// 2 batched. The numbers below are what the answer has to say.
export function nPlusOne(users, fetchImpl, cache) {
  return {
    naive: 1 + users.length, // users list, then one post query per user
    batched: 2, // users list, then one posts query with all ids
    async naivePosts() {
      const out = [];
      for (const u of users) out.push(await cache.load(`u${u}`));
      return out;
    },
```

listing 65: dedupe by inflight promise, fresh cache, failure eviction, invalidation (source: interview-repertoire/samples/ch12-react/src/state/store.js, lines 49-84)

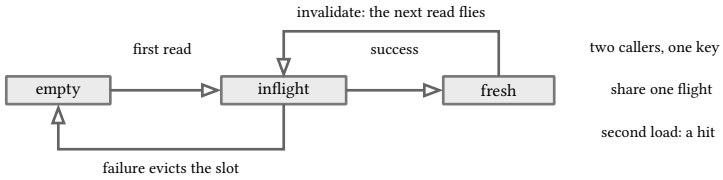


figure 56: the request cache state machine rtk query automates

The tests pin the four behaviors an interviewer is really asking about: two callers loading the same key share one flight, a second load hits the fresh cache without flying, a failure evicts instead of poisoning the slot, and invalidate forces the next read to fly. Say the rtk query names for these, same key sharing, cache lifetime tags, refetch on invalidation, and the miniature maps onto the product one to one.

13.3. graphql, rest, and n+1 [EWC]

The n+1 question is arithmetic, and the workspace makes it count: ten users naive is one users query plus ten post queries, and batching the posts by user ids is two queries total:

```
// The n+1 question as data: 10 users means 1 + 10 queries naive, or
// 2 batched. The numbers below are what the answer has to say.
export function nPlusOne(users, fetchImpl, cache) {
  return {
    naive: 1 + users.length, // users list, then one post query per user
    batched: 2, // users list, then one posts query with all ids
    async naivePosts() {
      const out = [];
      for (const u of users) out.push(await cache.load(`u${u}`));
      return out;
    },
    async batchedPosts() {
      const key = `users:${users.join(",")}`;
      return [await cache.load(key)];
    },
  };
}
```

listing 66: the naive count, the batched count, and both loaders against the cache (source: interview-repertoire/samples/ch12-react/src/state/store.js, lines 74-90)

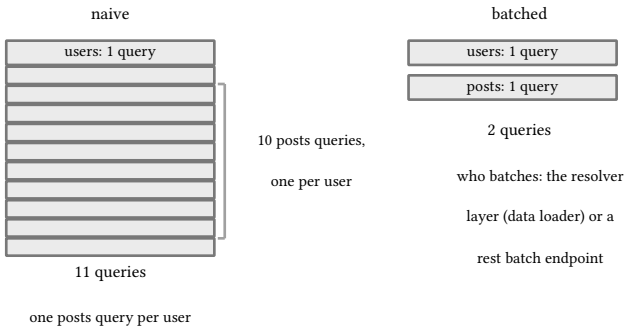


figure 57: ten users naive is 11 queries, batched is 2

The fake network in the test records every key loaded, so the assertion is on observable fetches, not on a promise the code makes in a comment. The graphql half of the question, said plainly: graphql moves the n+1 problem server-side, the client asks once and the resolver layer batches, data loader style, exactly like the `batchedPosts` key the test watches. Rest does the same with a batch endpoint or a compound resource, and the either-or framing of the question is usually false.

sources: verified by `npm run verify` under node 26.3.0, 5 tests in `src/state` of `ch12-react`. Store binding through useSyncExternalStore per the react api docs, react.dev, accessed 2026-09-10.

14. react build: a listing page, test driven

This is the book's flagship exercise: a listing page built against a public api, test driven, then a virtual dom written from scratch in plain javascript. The public api is jsonplaceholder, and the suite runs offline against a verbatim capture of `/posts?userId=1` taken 2026-09-10, with an `npm run live` script that re-runs the same assertions against the network on demand. The red-green beats below are the actual sequence the workspace went through.

14.1. beat one: red, the page does not exist [TDD]

The first commit is the test file alone: render `ListingPage` from api data, one card per post, a count attribute, an error state, an empty state, and a cache that hits the fixture url once. Run against nothing, it fails to even import:

```
node --test src/listing/*.test.js
Error [ERR_MODULE_NOT_FOUND]: Cannot find module '.../listing.js'
```

fragment, illustrative, not executed

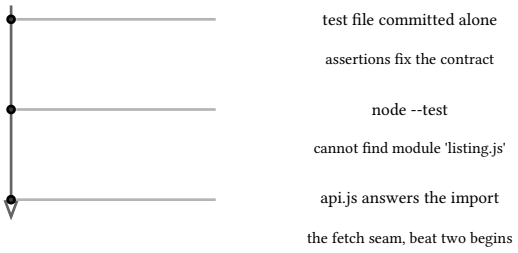


figure 58: the honest red: the test file alone fixes the contract

That is the honest red. The api contract the test fixes in place, a fetch seam plus a one-slot cache, is the second file:

```
  const cache = new Map();

  async function load(url) {
    const res = await fetchImpl(url);
    if (!res.ok) {
      const err = new Error(`failed to load posts (http ${res.status})`);
      err.status = res.status;
      throw err;
    }
    return res.json();
  }

  return {
    async listByUser(userId) {
      const url = `https://jsonplaceholder.typicode.com/posts?userId=${userId}`;
      if (cache.has(url)) return cache.get(url);
      const promise = load(url);
      cache.set(url, promise);
      // a failed load must not poison the cache
      promise.catch(() => cache.delete(url));
      return promise;
    },
    _cache: cache,
  };
}
```

listing 67: the injectable fetch, the deduping cache, failure eviction (source: interview-repertoire/samples/ch12-react/src/listing/api.js, lines 5-30)

14.2. beat two: green, three states as early returns [TDD]

The minimal component that passes is three early returns, error, empty, content, and the content is a mapped card list:

```
  if (posts.error) {
    return h("p", { class: "error" }, posts.error.message);
  }
  if (posts.length === 0) {
    return h("p", { class: "empty" }, "no posts yet");
  }
  return h(
    "section",
    { class: "listing" },
    h("h1", null, "posts"),
    h("p", { class: "count", "data-count": String(posts.length) },
      `${posts.length} posts by user ${posts[0].userId}`),
    posts.map((p) => h(PostCard, { key: p.id, post: p })),
  );
}

function PostCard({ post }) {
  return h(
    "article",
    { class: "card" },
    h("h2", null, `#${post.id} ${post.title}`),
    h("p", null, post.body.split("\n")[0]),
  );
}
```

listing 68: the page and the card, data in, markup out (source: interview-repertoire/samples/ch12-react/src/listing/listing.js, lines 7-31)

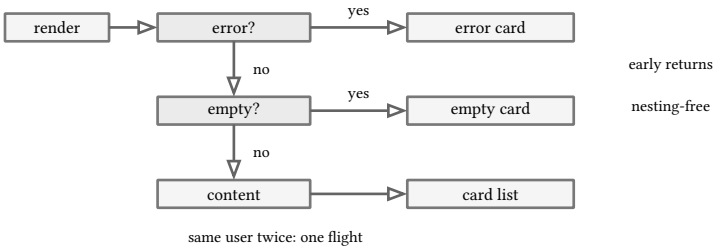


figure 59: green is three early returns and a mapped card list

The loader contract the test encodes, data on success and {error} on failure, is the shape a real data layer returns to a boundary. All five tests pass on this file, including the one that matters most for the interview story: two loads of the same user hit the network once, which is the request cache of book 17, ch 13 (the interview repertoire, react state: context, redux, graphql) earning its keep inside a component test.

VERIFY: the fixtures are real

The ten posts under src/listing/fixtures/posts.json are the live response from https://jsonplaceholder.typicode.com/posts?userId=1, fetched 2026-09-10, byte for byte. The live script runs the same render assertions against the endpoint, and it passed the day the fixture was captured. Offline suite, online proof, one file apart.

14.3. beat three: the virtual dom from scratch [TDD]

The second half of the exercise drops react entirely. h builds plain objects, renderHost mounts them on an in-memory host whose shape mirrors what a real renderer targets, and diff walks old against new producing patch ops:

```
// writing createElement/setText/replaceChild against document.

export function h(type, props = {}, ...children) {
  return { type, props, children: children.flat() };
}

export function text(value) {
  return { text: String(value) };
}

// The host: createHostElement mirrors document.createElement closely
// enough that the patch algorithm is host-agnostic. The key lives on
// the host node as _key, the way react keeps it on the fiber rather
// than the dom node.
export function createElementHost(type) {
  return { type, props: {}, children: [], _text: null, _key: null };
}

export function renderHost(vnode) {
  if (vnode == null || vnode === false) return null;
  if (typeof vnode === "object" && "text" in vnode) {
    const el = createElementHost("#text");
    el._text = vnode.text;
    return el;
  }
  if (typeof vnode.type === "function") {
    return renderHost(vnode.type({ ...vnode.props, children: vnode.children }));
  }
  const el = createElementHost(vnode.type);
  el._key = vnode.props?.key ?? null;
  for (const [k, v] of Object.entries(vnode.props ?? {})) {
    if (k === "key" || k.startsWith("on")) continue;
    el.props[k] = v;
  }
  el.children = vnode.children.map(renderHost).filter(Boolean);
  return el;
}
```

listing 69: vnodes as data, function components expanding, keys living on the host node (source: interview-repertoire/samples/ch12-react/src/vdom/vdom.js, lines 5-41)

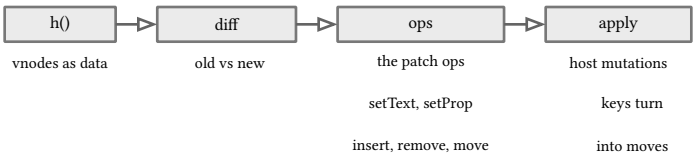


figure 60: h builds vnodes, diff emits ops, the host applies the change

```
// diff produces a patch list: [op, path, payload]. Ops are setText,
// setProp, removeProp, replace, insert, remove. Paths address the
// host tree by child indexes from the root.
export function diff(oldV, newV, path = []) {
  const patches = [];
  if (oldV === newV) return patches;
  if (oldV == null || oldV === false) {
    if (newV != null && newV !== false) patches.push({ op: "insert", path, vnode: newV });
    return patches;
```

```
  }
  if (newV == null || newV === false) {
    patches.push({ op: "remove", path });
    return patches;
  }
  const oldText = typeof oldV === "object" && "text" in oldV ? oldV.text : null;
  const newText = typeof newV === "object" && "text" in newV ? newV.text : null;
  if (oldText =/= null || newText =/= null) {
    if (oldText =/= newText) patches.push({ op: "replace", path, vnode: newV });
    return patches;
  }
  const oldType = typeof oldV.type === "function" ? "fn" : oldV.type;
  const newType = typeof newV.type === "function" ? "fn" : newV.type;
  if (oldType =/= newType) {
    patches.push({ op: "replace", path, vnode: newV });
    return patches;
  }
  // props
  const oldProps = oldV.props ?? {};
  const newProps = newV.props ?? {};
  for (const k of Object.keys(newProps)) {
    if (k === "key" || k.startsWith("on")) continue;
    if (oldProps[k] =/= newProps[k]) {
      patches.push({ op: "setProp", path, key: k, value: newProps[k] });
    }
  }
  for (const k of Object.keys(oldProps)) {
    if (k === "key" || k.startsWith("on")) continue;
    if (!(k in newProps)) patches.push({ op: "removeProp", path, key: k });
  }
  // children, keyed when both sides carry keys, positional otherwise
  const oldKids = oldV.children ?? [];
  const newKids = newV.children ?? [];
  if (oldKids.some((k) => k?.props?.key =/= null) && newKids.some((k) => k?.props?.key =/= null)) {
    patches.push(...diffKeyed(oldKids, newKids, path));
    return patches;
  }
  const shared = Math.min(oldKids.length, newKids.length);
  for (let i = 0; i < shared; i++) {
    patches.push(...diff(oldKids[i], newKids[i], [...path, i]));
  }
  for (let i = shared; i < oldKids.length; i++) {
    patches.push({ op: "remove", path: [...path, i] });
  }
  for (let i = shared; i < newKids.length; i++) {
    patches.push({ op: "insert", path: [...path, i], vnode: newKids[i] });
  }
  return patches;
```

listing 70: the diff: props, positional children, keyed children (source: interview-repertoire/samples/ch12-react/src/vdom/vdom.js, lines 43-99)

The tests are where the virtual dom claims become numbers: a text change is exactly one patch, a prop change exactly one setProp, and the node count before and after an insert or remove is counted, text nodes included. The keyed reorder test is the one that earned its keep, because the first implementation diffed keyed children positionally, which rewrites content instead of moving nodes, exactly the bug keys exist to prevent. The fixed diffKeyed emits drop, append, move, then per-pair diffs at final positions:

```
// Keyed children diff to a small op set: drop vanished keys, append
// new keys, move survivors into the new order, then diff each
// survivor pair at its final index. Paths in the pair diffs address
// positions AFTER the reorder, which is the order apply sees.
function diffKeyed(oldKids, newKids, path) {
  const patches = [];
  const oldByKey = new Map(oldKids.map((k) => [k.props.key, k]));
  const newKeys = new Set(newKids.map((k) => k.props.key));
  for (const [key, vnode] of oldByKey) {
    if (!newKeys.has(key)) patches.push({ op: "removeKeyed", path, key });
  }
  for (const nk of newKids) {
    if (!oldByKey.has(nk.props.key)) {
      patches.push({ op: "insertKeyed", path, vnode: nk });
    }
  }
  patches.push({ op: "reorder", path, keys: newKids.map((k) => k.props.key) });
  newKids.forEach((nk, ni) => {
    const old = oldByKey.get(nk.props.key);
    if (old) patches.push(...diff(old, nk, [...path, ni]));
  });
  return patches;
}
```

listing 71: keyed diff: drop vanished keys, append new ones, reorder, diff pairs at final indexes (source: interview-repertoire/samples/ch12-react/src/vdom/vdom.js, lines 102-124)

Say the summary out loud, because it is the answer to the original question: a virtual dom is a diff engine over element trees, its value is paying dom mutations only for real changes, and keys are the identity hint that turns rewrites into moves.

sources: jsonplaceholder.typicode.com, fixture captured 2026-09-10. Verified by `npm run verify` under node 26.3.0, 5 tests in `src/listing`, 8 in `src/vdom`, plus the on-demand live script which passed against the network the same day.

15. go runtime: scheduler, gc, patterns

The go systems half of the loop starts here. The ch15-go module carries the runnable answers, 16 tests including one run under -race, and the release facts are pinned to the go 1.26 notes in the sources appendix, accessed 2026-09-09.

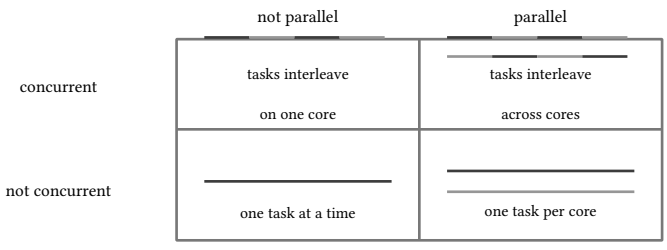
15.1. parallelism is not concurrency [EWC]

Parallelism is two things at once, which needs cores. Concurrency is a program structured as independently runnable pieces, which needs nothing but the structure. GOMAXPROCS is where the two meet, the number of OS threads the go scheduler keeps runnable goroutines on:

```go
// Parallelism is doing two things at once, which needs two cores.
// Concurrency is structuring a program as independently runnable
// pieces, which needs nothing. GOMAXPROCS is how many OS threads the
// Go scheduler keeps runnable goroutines on at once.
func Procs() int { return runtime.GOMAXPROCS(0) }

// Goroutines are cheap because the runtime schedules them itself,
// multiplexing M goroutines onto N OS threads. Ten thousand of them
// start and rendezvous well inside a test.
func Spawn(n int) int {
  var wg sync.WaitGroup
  started := make(chan struct{}, n)
  for i := 0; i < n; i++ {
    wg.Add(1)
    go func() {
      defer wg.Done()
      started <- struct{}{}
    }()
  }
  wg.Wait()
  close(started)
  count := 0
  for range started {
    count++
  }
  return count
}
```

listing 72: the definition pair, and ten thousand goroutines rendezvousing inside a test (source: interview-repertoire/samples/ch15-go/scheduler.go, lines 8-34)



GOMAXPROCS: how many cores the scheduler uses

figure 61: parallelism is cores, concurrency is structure

The demo to narrate from the test: ten thousand goroutines spawn, signal, and drain in milliseconds, because a goroutine is a few kilobytes and a runtime scheduling decision, not an OS thread the kernel allocates.

15.2. the scheduler versus the os [EWC]

The model to say out loud: goroutines are M, OS threads are N, and the go scheduler multiplexes M onto N with work stealing. The full GMP walk is book 4, ch 9 (go 1.27, runtime). A goroutine blocking on a channel or mutex parks in the runtime and its thread runs someone else. A goroutine blocking in a syscall hands its thread to the kernel and the runtime spins up or reuses another. That split, runtime parking versus kernel blocking, is the answer to "why are goroutines cheap" and the follow-up "when are they not", where the honest list is cgo calls, file io, and anything that parks the thread instead of the goroutine.

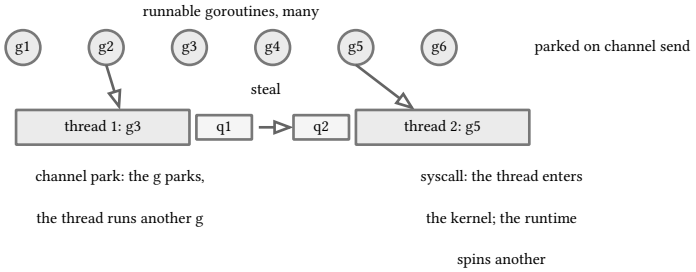


figure 62: M goroutines multiplexed onto N threads: park versus syscall

Go 1.26 added scheduler observability rather than a new policy: `runtime/metrics` gained goroutine counts by state, threads known to the runtime, and total goroutines created, which is what the probes of book 4, ch 9 (go 1.27, runtime) render live.

15.3. the garbage collector, and 1.26's green tea [DRILL]

The baseline answer: go's gc is concurrent, tri-color mark and sweep, non-generational, with sub-millisecond stop-the-world pauses at both ends of the cycle. It tunes by pacing, `GOGC` and now `GOMEMLIMIT`, rather than by generations, and escape analysis is the compiler's part, keeping what it can on the stack.

The 1.26 fact worth citing: the green tea collector, experimental in 1.25, is the default in 1.26, with the release notes claiming a 10 to 40 percent reduction in gc overhead for gc-heavy programs, and about ten percent more on recent intel and amd cores because small-object scanning uses vector instructions. The mechanism to name when asked: green tea does its small-object scanning in cache-friendly batches interleaved through the mark cycle, work chosen for memory locality, which is where the overhead reduction comes from. The opt-out is `GOEXPERIMENT=nogreenteagc`, and it is scheduled for removal in 1.27, which is the sentence that makes it real. The same notes add an experimental goroutine leak detector under `GOEXPERIMENT=goroutineleakprofile`, the gc-reachability idea: a goroutine blocked on a channel nothing else can reach can never wake.

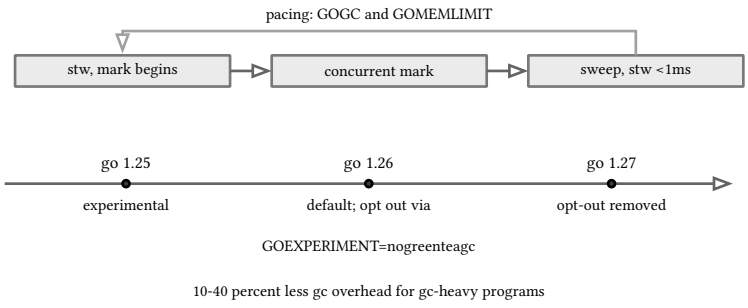


figure 63: tri-color mark and sweep, the green tea default timeline

15.4. concurrent patterns: pool, pipeline, cancellation [EWC]

The worker pool with context cancellation and first-error-wins is the pattern interviews mean by "how would you process a queue":

```go
// would you process a queue": fixed workers pulling a channel until
// it closes or the context cancels, first error wins, everyone goes
// home through the WaitGroup.
func WorkerPool[T any](ctx context.Context, work []T, workers int, do func(T) error) error {
  jobs := make(chan T)
  errs := make(chan error, 1)
  var wg sync.WaitGroup
  var once sync.Once

  fail := func(err error) {
    once.Do(func() { errs <- err })
  }

  ctx, cancel := context.WithCancel(ctx)
  defer cancel()

  for i := 0; i < workers; i++ {
    wg.Add(1)
    go func() {
      defer wg.Done()
      for job := range jobs {
        if err := do(job); err != nil {
          fail(err)
          cancel()
          return
        }
      }
    }()
  }
feed:
  for _, job := range work {
    select {
    case jobs <- job:
    case <-ctx.Done():
      break feed
    }
  }
  close(jobs)
  wg.Wait()
  select {
  case err := <-errs:
    return err
  default:
    return ctx.Err()
  }
}

// ClosureCapture is the 1.22 semantic: every loop iteration gets its
// own copy of the variable, so closures spawned inside the loop see
// the value of their own pass. Before 1.22 all closures shared one
// variable and every callback saw the final value, the classic
// gotcha this function now cannot reproduce.
```

listing 73: fixed workers, a jobs channel, cancel on first error, wait, report (source: interview-repertoire/samples/ch15-go/patterns.go, lines 9-60)

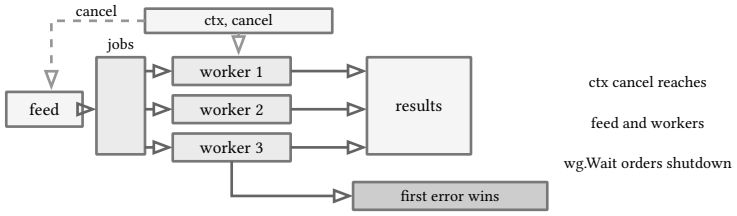


figure 64: the pool: fixed workers, first error cancels, wait orders shutdown

The three tests cover the healthy path, the error path, and the pre-canceled context, and the narration is the structure: the feed loop selects on the context while pushing, workers drain until the channel closes, `once` makes the first error the only one, and `wg.Wait` orders shutdown. The fan-out fan-in beside it preserves input order by writing to indexed slots, which is the counterpoint to "results arrive whenever".

15.5. closures and the loop variable [EWC]

Since go 1.22, each loop iteration owns its own copy of the loop variable, so closures spawned inside the loop see their own pass. The demo returns exactly `[0 1 2 3 4]` where pre-1.22 go returned five copies of the final value, and that history is worth volunteering, because interview questions written before 1.22 expect the gotcha answer:

```
  var wg sync.WaitGroup
  seen := make([]int, 5)
  for i := 0; i < 5; i++ {
    wg.Add(1)
    go func() {
      defer wg.Done()
      seen[i] = i
    }()
  }
  wg.Wait()
  return seen
}
```

listing 74: per-iteration capture, the 1.22 semantics (source: interview-repertoire/samples/ch15-go/patterns.go, lines 62-74)

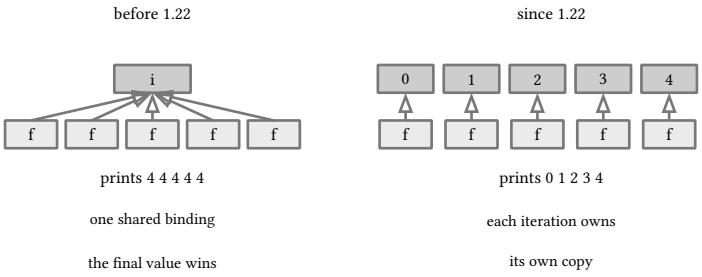


figure 65: one shared binding against per-iteration copies, the 1.22 fix

15.6. generics and their constraint vocabulary [EWC]

Type parameters with a real constraint, a generic cache over `comparable` keys, and a helper that would have been duplicated per-type before 1.18:

```
  "sync"
)

// Sum works on any ordered type: the constraint is a type set, the
// compiler checks every operation against it, and one function
// serves int, float64, and string orderings.
func Sum[T cmp.Ordered](xs []T) T {
  var zero T
  for _, x := range xs {
    zero += x
  }
  return zero
}

// Cache is the generic container with a method constraint-free
// enough to hold anything: map plus mutex, zero copy on read hit.
type Cache[K comparable, V any] struct {
  mu    sync.Mutex
  items map[K]V
}
```

listing 75: an ordered constraint sum, a locked generic cache (source: interview-repertoire/samples/ch15-go/generics.go, lines 5-25)

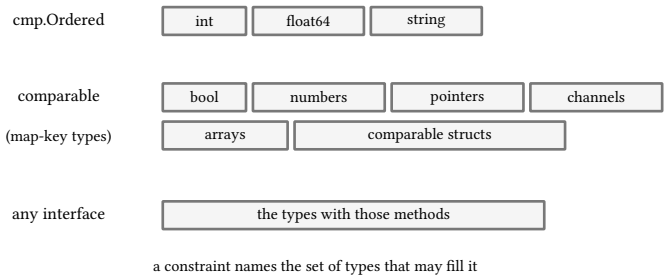


figure 66: constraints are type sets: ordered, comparable, methods

The follow-up that separates practitioners: constraints are type sets, `cmp.Ordered` is a named set, `comparable` is exactly the types usable as map keys, and a constraint can be any interface, including methods. Generics do not replace interfaces, they parametrize data structures and algorithms, interfaces remain the boundaries.

15.7. context, timeouts, and errors as values [EWC]

Timeouts belong to the caller and travel in the context. Error wrapping is `%w`, inspection is `errors.Is` and `errors.As`, and go 1.26 adds `errors.AsType`, the generic one-expression form of `As`:

```go
// WrapOnce shows the error idiom: %w chains the cause, errors.Is
// finds it, and Go 1.26's errors.AsType[E] extracts the typed cause
// without the target pointer dance of errors.As.
type RateLimited struct {
  RetryAfter time.Duration
}

func (e *RateLimited) Error() string {
  return fmt.Sprintf("rate limited, retry after %s", e.RetryAfter)
}

func WrapOnce() error {
  return fmt.Errorf("list items: %w", &RateLimited{RetryAfter: 3 * time.Second})
}

func TypedCause(err error) *RateLimited {
  var rl *RateLimited
  if errors.As(err, &rl) {
    return rl
  }
  return nil
}

// TypedCauseAsType returns the typed cause in one expression: go
// 1.26's errors.AsType[E] finds the first match in the chain and
// reports it, nil and false when the type never appears.
func TypedCauseAsType(err error) (*RateLimited, bool) {
  return errors.AsType[*RateLimited](err)
}

// ChainTimeout wraps a deadline error with the operation name, the
// shape production logs want: context, then cause, then name.
func ChainTimeout(ctx context.Context) error {
  <-ctx.Done()
```

listing 76: a typed error, %w wrapping, errors.As beside the 1.26 AsType (source: interview-repertoire/samples/ch15-go/context.go, lines 29-62)

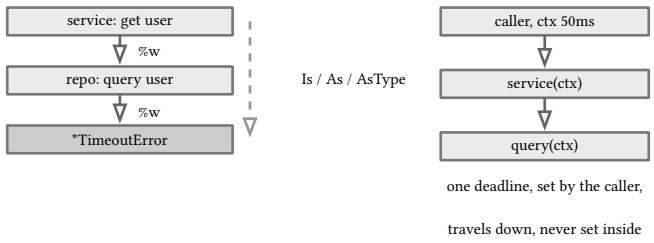


figure 67: errors chain through %w, timeouts travel down the context

The suite proves both extraction paths find the typed cause and that a foreign error returns false, and `ChainTimeout` pins the message shape production logs want, operation name first, deadline cause inside.

15.8. the singleton under contention [TDD]

`sync.Once` is the whole answer: one construction, however many goroutines race for it. The test runs a swarm of 64 through a start gate and asserts every slot received the same pointer with `ID == 1`:

```
// on the Once.
func Instance() *singleton {
  once.Do(func() {
    nextID++
    instance = &singleton{ID: nextID, Expensive: "built"}
  })
  return instance
}

// Reset drops the singleton so tests can start from zero.
func Reset() {
  instance = nil
  nextID = 0
  once = sync.Once{}
}
```

listing 77: once.Do builds, everyone shares the result (source: interview-repertoire/samples/ch15-go/once.go, lines 22-37)

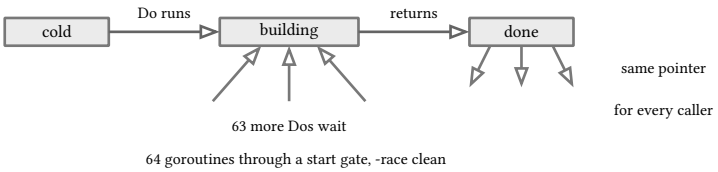


figure 68: once.Do: one build, every caller shares the result

Run under `go test -race`, the same test also proves no data race in the lazy path, and it passed under the detector in this tree. The concession to volunteer: the once-singleton is process-wide state, tests reset it through `Reset`, and real code prefers explicit construction at main with the dependency passed down, which is the `go-testing` chapter's whole thesis.

sources: go 1.26 release notes at go.dev, green tea defaults, goroutine leak detector experiment, scheduler metrics, and errors.AsType, accessed 2026-09-09. Verified by `go vet` and `go test` through `make verify`, 16 tests in `ch15-go`, plus the race detector pass on the singleton test.

16. go testing: mocks and dependency injection

“How do you test code that talks to a database” is answered by architecture, not by a mocking library. The `ch16-go` module builds the standard four layers, store, service, handler, callback, each tested with the layer below it replaced by a fake, and the whole stack exercised once against real sqlite. 8 tests under `make verify`, engine 3.53 through the pinned driver.

16.1. the four layers [TDD]

The store is sql and nothing else. The service holds the rules and depends on two interfaces, the store and a notifier. The handler decodes, calls, encodes, and depends on a narrow service interface. The notifier is the callback layer, the email or queue hook the service fires after a state change:

```go
// Store is the db layer: sql rows in and out, no business rules.
// The interface is what the service layer sees, which is also what
// the mock implements.
type Store interface {
  CreateItem(sku, name string, cents int) (int64, error)
  GetItem(id int64) (Item, error)
  ListItems() ([]Item, error)
  UpdatePrice(id int64, cents int) error
  Close() error
}

// Item is the row shape shared by all layers.
type Item struct {
  ID    int64  `json:"id"`
  SKU   string `json:"sku"`
  Name  string `json:"name"`
  Cents int    `json:"cents"`
}

// OpenStore opens the sqlite file and asserts the engine is at least
// 3.53, the version this book pins across every runtime.
func OpenStore(dsn string) (*SqliteStore, error) {
  db, err := sql.Open("sqlite", dsn)
  if err ≠ nil {
    return nil, err
  }
  var version string
  if err := db.QueryRow("select sqlite_version()").Scan(&version); err ≠ nil {
    db.Close()
    return nil, err
  }
  if version < "3.53" {
    db.Close()
    return nil, fmt.Errorf("sqlite %s is older than the pinned 3.53 line", version)
  }
  if _, err := db.Exec(`create table if not exists items (
    id integer primary key,
    sku text not null unique,
    name text not null,
    cents integer not null check (cents ≥ 0)
  )`); err ≠ nil {
    db.Close()
    return nil, err
  }
  return &SqliteStore{db: db, Version: version}, nil
}
```

listing 78: the store interface is the seam, open asserts the pinned engine (source: interview-repertoire/samples/ch16-go/store.go, lines 10-55)

```
  store  Store
  notify Notifier
}

func NewService(store Store, notify Notifier) *Service {
  if notify == nil {
    notify = noopNotifier{}
  }
  return &Service{store: store, notify: notify}
}

var (
  ErrInvalidPrice = errors.New("price must be a positive number of cents")
  ErrInvalidSKU   = errors.New("sku must not be empty")
  ErrNotFound     = errors.New("item not found")
)

func (s *Service) AddItem(ctx context.Context, sku, name string, cents int) (Item, error) {
  if sku == "" {
    return Item{}, ErrInvalidSKU
  }
  if cents <= 0 {
    return Item{}, ErrInvalidPrice
  }
  id, err := s.store.CreateItem(sku, name, cents)
  if err ≠ nil {
    return Item{}, fmt.Errorf("create %q: %w", sku, err)
  }
  return s.store.GetItem(id)
}

// Reprice applies the price rule and fires the callback on success
// only. The callback error propagates, because a silent notification
// failure is a lie the caller deserves to know about. Store-level
// missing rows surface as the domain ErrNotFound, which keeps
// database/sql out of every layer above this one.
func (s *Service) Reprice(ctx context.Context, id int64, cents int) error {
```

listing 79: validation, orchestration, domain errors, the callback fired on success only (source: interview-repertoire/samples/ch16-go/service.go, lines 22-58)

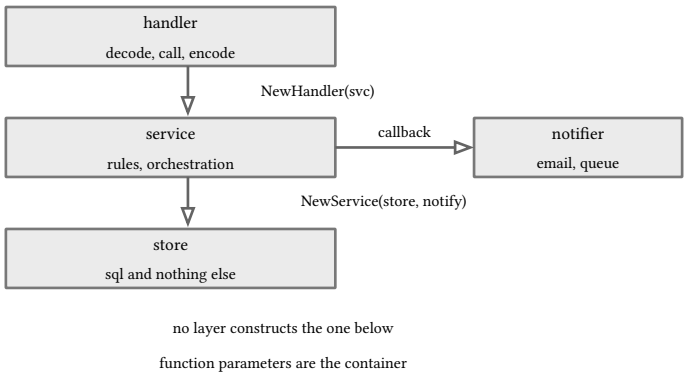


figure 69: four layers wired by constructor injection, no layer builds another

The dependency injection is the constructors: `NewService(store, notify)` takes interfaces, `NewHandler(svc)` takes another one, and no layer constructs the layer below it. That is the

entire di pattern in go, and the honest answer to “do you need a di framework” is no, function parameters are the container.

16.2. mocks without a mocking framework [TDD]

The fakes file is the answer to “what do you mock”: an in-memory store with failure injection, a recording notifier, and a scripted service:

```
// The fakes, collected in one file: a mock store, a recording
// notifier, and a scripted service. Each exists so the layer above
// can be tested without the layer below.

// MockStore is an in-memory Store with failure injection.
type MockStore struct {
  Items    []Item
  NextID   int64
  GetErr   error
  WriteErr error
}

func NewMockStore(seed ...Item) *MockStore {
  m := &MockStore{NextID: 1}
  for _, it := range seed {
    it.ID = m.NextID
    m.NextID++
    m.Items = append(m.Items, it)
  }
  return m
}

func (m *MockStore) CreateItem(sku, name string, cents int) (int64, error) {
  if m.WriteErr ≠ nil {
    return 0, m.WriteErr
  }
  id := m.NextID
  m.NextID++
  m.Items = append(m.Items, Item{ID: id, SKU: sku, Name: name, Cents: cents})
  return id, nil
}

func (m *MockStore) GetItem(id int64) (Item, error) {
  if m.GetErr ≠ nil {
    return Item{}, m.GetErr
  }
  for _, it := range m.Items {
    if it.ID == id {
```

listing 80: the mock store and its failure injection (source: interview-repertoire/samples/ch16-go/fakes.go, lines 8-45)

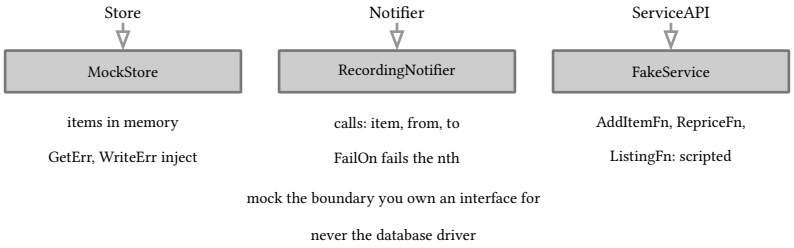


figure 70: three fakes behind the three interfaces the code already owns

The rules the tests encode. Mock the boundary you own an interface for, never the database driver. Assert interactions that are part of the contract, the callback's from and to values, and not incidental call counts. Inject failures, `GetErr` and `WriteErr`, because the error paths are the ones production finds and hand-rolled tests skip.

16.3. what each layer's suite proves [TDD]

The store tests run against a real file in a temp dir, reopen it, and demand the same rows back, because a store test against a fake proves nothing about persistence. The service tests use the mock store and prove validation, the missing-row translation to `ErrNotFound`, and that a failing callback propagates instead of vanishing. The handler tests script the service and prove routing, encoding, and the error-to-status mapping, 400 for rules, 404 for missing, no database anywhere:

```
// Handler is the http layer: decode, call the service, encode. It
// depends on a narrow service interface, so tests inject the service
// directly without any database behind it.
type ServiceAPI interface {
  AddItem(ctx context.Context, sku, name string, cents int) (Item, error)
  Reprice(ctx context.Context, id int64, cents int) error
  Listing(ctx context.Context) ([]Item, error)
}

type Handler struct {
  svc ServiceAPI
}

func NewHandler(svc ServiceAPI) *Handler { return &Handler{svc: svc} }

func (h *Handler) ServeHTTP(w http.ResponseWriter, r *http.Request) {
  switch {
  case r.URL.Path == "/items" && r.Method == http.MethodGet:
    h.list(w, r)
  case r.URL.Path == "/items" && r.Method == http.MethodPost:
    h.create(w, r)
  case r.URL.Path == "/items/price" && r.Method == http.MethodPost:
    h.reprice(w, r)
  default:
    http.NotFound(w, r)
  }
}

func (h *Handler) list(w http.ResponseWriter, r *http.Request) {
  items, err := h.svc.Listing(r.Context())
  if err != nil {
    writeError(w, http.StatusInternalServerError, err.Error())
    return
  }
  writeJSON(w, http.StatusOK, map[string]any{"items": items})
}
```

listing 81: one ServeHTTP, three routes, encoding at the edges (source: interview-repertoire/samples/ch16-go/handler.go, lines 11-46)

layer	its suite asserts	stands in below
store	reopen the file, the same rows back	real sqlite, temp dir
service	rules, ErrNotFound, callback errors propagate	MockStore
handler	routing, encoding, errors map to status	FakeService
full stack	one price change over http, end to end	no fakes at all

figure 71: what each suite proves, and which fake stands in below

The domain-error mapping is the piece worth narrating: the store returns `sql.ErrNoRows`, the service translates it to `ErrNotFound`, the handler maps that to 404, and `database/sql` never appears above the service layer. Layer-locked imports are what make the layers testable apart.

VERIFY: the full stack test

The last test wires real sqlite, the service, the notifier, and the handler together and drives a price change over http. The notifier receives one event with the new total, which is the end-to-end claim the isolated suites cannot make alone.

16.4. the golden master, ported small [EWC]

The behavioral chapter's migration card captures a legacy service's outputs as golden masters, ports, and diffs, and this section is that harness shrunk until one file holds it. A golden master is committed bytes: the generator runs, its output is compared against a file in `testdata`, and a change in either surfaces as the first line that moved. The port renders a catalog report from a map and gates it against `testdata/catalog.golden`, three tab-separated lines in sorted sku order, hat-01, map-00, sword-01:

```
// updateGoldens rewrites testdata/catalog.golden instead of comparing,
// the corpus pattern: the committed bytes are the contract, regenerated
// with `go test -update`, never edited by hand.
var updateGoldens = flag.Bool("update", false, "rewrite the golden under testdata")

// goldenCatalog is a map on purpose: go randomizes map iteration order,
// so the renderer below must sort its keys or the committed bytes would
// flake across runs. The sort stays load-bearing because the fixture
// refuses to be a slice.
func goldenCatalog() map[string]Item {
  return map[string]Item{
    "sword-01": {ID: 1, SKU: "sword-01", Name: "iron sword", Cents: 499},
    "map-00":   {ID: 2, SKU: "map-00", Name: "region map", Cents: 250},
    "hat-01":   {ID: 3, SKU: "hat-01", Name: "sun hat", Cents: 400},
  }
}

// renderCatalog is the generator the golden gates: keys sorted, one
// fixed tab-separated line per item, so the bytes out are a pure
// function of the value in, whatever order the map handed over.
func renderCatalog(catalog map[string]Item) string {
  skus := make([]string, 0, len(catalog))
  for sku := range catalog {
    skus = append(skus, sku)
  }
  sort.Strings(skus)
  var b strings.Builder
  for _, sku := range skus {
    it := catalog[sku]
    b.WriteString(sku)
    b.WriteByte('\t')
    b.WriteString(strconv.FormatInt(it.ID, 10))
    b.WriteByte('\t')
    b.WriteString(strconv.Itoa(it.Cents))
    b.WriteByte('\n')
  }
  return b.String()
}
```

listing 82: the update flag, the map fixture that forces the sort, the renderer (source: interview-repertoire/samples/ch16-go/golden_test.go, lines 13-50)

The walk, line by line. The flag is a plain `flag.Bool` parsed by go test itself, false in every normal run, true only when the suite is invoked with `-update`. `goldenCatalog` returns a map on purpose, because go randomizes map iteration order and a renderer that walked the map as given would commit bytes that flake run to run. The sort inside `renderCatalog` is what makes the bytes out a pure function of the value in, rule one of the determinism drill below. The format is deliberately boring, one line per item, sku, id, cents, tab-separated, because a golden nobody can read is a golden nobody trusts.

```go
// The golden master, ported small: compare generated bytes against the
// committed file, report the first divergence, rewrite only through the
// flag.
func TestCatalogGoldenMatchesCommittedBytes(t *testing.T) {
  got := renderCatalog(goldenCatalog())
  path := filepath.Join("testdata", "catalog.golden")
  if *updateGoldens {
    if err := os.MkdirAll("testdata", 0o755); err ≠ nil {
      t.Fatal(err)
    }
    if err := os.WriteFile(path, []byte(got), 0o644); err ≠ nil {
      t.Fatal(err)
    }
    return
  }
  want, err := os.ReadFile(path)
  if err ≠ nil {
    t.Fatalf("read golden: %v (run go test -update to generate)", err)
  }
  if got ≠ string(want) {
    t.Fatalf("catalog drifted from testdata/catalog.golden:\n%s",
      firstDivergence(string(want), got))
  }
}
```

listing 83: compare by default, rewrite through the flag, the drift named (source: interview-repertoire/samples/ch16-go/golden_test.go, lines 52-75)

The test body is two branches. Under `-update` it makes `testdata` and writes the bytes, and that is the only way the golden ever changes, a golden edited by hand being a contract nobody signed. The default branch reads the committed file, and the missing-file error names the regeneration command, because a golden test that fails with "no such file" and no remedy is a speed bump, not a gate. The comparison is plain inequality on the whole string, bytes for bytes, and the failure message is `firstDivergence`, never a dump:

```
// firstDivergence names the first line where want and got disagree,
// both lines quoted, so a drifted golden reads as one short diff
// instead of two walls of bytes.
func firstDivergence(want, got string) string {
  wl := strings.Split(strings.TrimSuffix(want, "\n"), "\n")
  gl := strings.Split(strings.TrimSuffix(got, "\n"), "\n")
  for i := 0; i < max(len(wl), len(gl)); i++ {
    var w, g string
    if i < len(wl) {
      w = wl[i]
    }
    if i < len(gl) {
      g = gl[i]
    }
    if w ≠ g {
      return "line " + strconv.Itoa(i+1) +
        "\n  want: " + strconv.Quote(w) +
        "\n  got:  " + strconv.Quote(g)
    }
  }
  return ""
}
```

listing 84: the first differing line, want and got quoted, one short diff (source: interview-repertoire/samples/ch16-go/golden_test.go, lines 77-98)

`firstDivergence` walks both texts line by line, stops at the first pair that disagrees, and quotes both through `strconv.Quote` so tabs and trailing spaces are visible. A wall of generated bytes helps nobody, the first divergent line is the diagnosis.

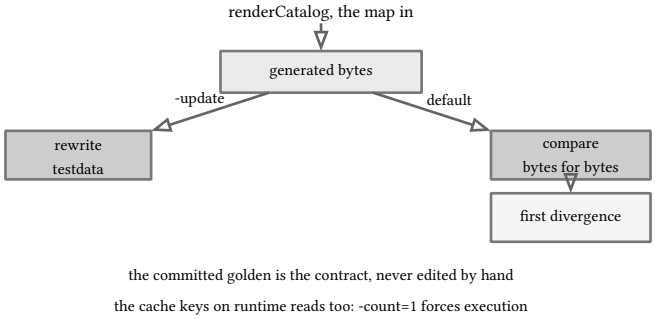


figure 72: two branches: compare by default, rewrite through the flag

VERIFY: the cache and why -count=1

Go's test cache keys on the files a test reads: the runtime records every open and hashes it into the key, so a golden edited with no code change busts the cache and plain `go test` re-runs and fails. What a warm cache will still answer is a run whose binary and inputs are all unchanged, and a run about to trust a golden wants execution, not recall: `-count=1` makes the run prove the suite actually ran on the tree in front of it, warm cache or not.

This is the same shape the go book's offline engine gates its parity artifact with, one flag, one compare, one divergence report, one committed fixture two suites in two languages assert, book 4, ch 33 (go 1.27, the offline engine and its js twin) for the full-size version.

16.5. determinism discipline for cross-language twins [DRILL]

The spoken answer, when the follow-up is “a go engine and its javascript twin, how do you know they agree”: one committed fixture, two suites, bit-identical output, and three rules that make the bits line up. Sorted keys first: go randomizes map iteration order, so every rendered artifact sorts its keys and the bytes out become a pure function of the value in, the same sort the golden port above leans on. Fixed term ordering second: a float sum depends on the order of its terms, so both languages evaluate one written expression tree, the same terms in the same order with the same operations, the accumulation mirrored line for line rather than formula for formula. One committed fixture third: the go side emits the payload and golden-gates its bytes, the twin reads the same committed file and asserts the same constants with plain ==, and parity is transitive, go equals the artifact, the artifact equals what the twin reads.

Why bit-identical rather than close-enough: IEEE 754 arithmetic is deterministic per operation, so identical operation sequences give identical doubles, and an epsilon in the assertion can only widen the blind spot, a tolerance passes two scores that differ, and a differing bit means the operation trees diverged somewhere real. Close-enough hides exactly the bug the twin exists to catch. The keep-it-honest half is a hand-computed fixture riding beside the generated one, every value a quarter or an eighth, so the expected scores are doubles a pencil can verify in both languages. The material is mined from dota-helper’s emit and picker layers and taught full size in book 4, ch 33 (go 1.27, the offline engine and its js twin).

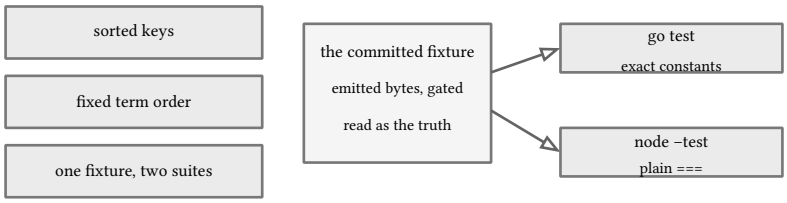


figure 73: the three rules, one fixture, two suites, plain equality on both sides

The traps to have ready. Float sum order: reordering the terms changes the double, `(a + b) + c` and `a + (b + c)` can disagree in the last bits, which is why the twin mirrors the tree, not the formula. Locale: number formatting is pinned, never delegated to whatever the runtime’s formatter picks, the engine rounds its packed cells at 4 decimals by printing and parsing back so both sides rescore against the artifact’s digits, and a locale that swaps separators would fork the bytes silently. Map iteration order: the trap the golden port above already carries, go hands map entries over in a new order every run, and the sort is the whole defense. All three traps share one shape, a value that depends on an order nobody wrote down.

sources: verified by `go vet` and `go test` through `make verify`, 8 tests in `ch16-go` over sqlite 3.53 through modernc.org/sqlite v1.58.0, the golden run passed `-count=1` so it executes rather than recalls, and regenerated only through `go test -update`. Deeper testing doctrine floors to book 13, ch 16 (infrastructure: docker, databases, and testing, testing without docker) and its mocks chapter.

17. database answers

The database round mixes definitional questions, acid, base, normal forms, isolation levels, with precision questions, money types, json, geo, cursors. This chapter's rule is the book's rule: the points this tree can prove on sqlite 3.53 are demonstrated, the postgres-specific answers are framed as postgres, honestly, and the one mongo question is implemented as the predicate it actually runs. 9 tests in `ch17-go` under `make verify`.

17.1. acid and base [DRILL]

Acid is four guarantees a transaction makes: atomicity, all or nothing, consistency, invariants hold across the boundary, isolation, concurrent transactions do not see each other's intermediate state, durability, a commit survives the crash that follows it. Base is the eventually consistent counterpart, basically available, soft state, eventually consistent, the tradeoff distributed stores make when partitions force a choice between availability and consistency, the cap theorem's corner.

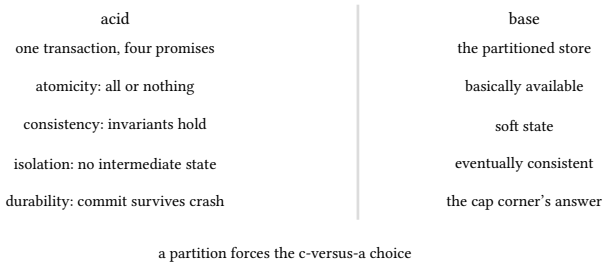


figure 74: acid's four promises against base's partition trade

The sqlite framing worth saying: sqlite is acid by default, and the isolation chapter of book 13, ch 7 (infrastructure: docker, databases, and testing, sqlite: transactions and isolation) demonstrates the file-level locking that buys it. Postgres is acid with configurable isolation, and the isolation levels question, read committed, repeatable read, serializable, is really the anomalies question, dirty reads, non-repeatable reads, phantom rows, each level closing one more.

17.2. money is cents [TDD]

Floating point cannot represent 0.1, so float money accumulates error at the only place error is unacceptable. The demo needs one go subtlety narrated: constant arithmetic is exact, so `0.1 + 0.2` as constants folds to the nearest float64 to 0.3 and the drift disappears, the drift demo uses variables:

```go
// Money is cents, never floats. The test proves why: 0.1 and 0.2
// have no exact binary representation, so float addition drifts
// while integer cents stay exact. The addends must be variables:
// go's constant arithmetic is exact and would fold the expression
// to precisely the float64 nearest 0.3, which is its own lesson.
func FloatMoneyDrift() float64 {
  a, b := 0.1, 0.2
  return a + b
}

func CentsTotal(cents ...int) int {
  total := 0
  for _, c := range cents {
    total += c
  }
  return total
}

// FormatCents renders cents for humans, the only place division
// ever touches money.
func FormatCents(cents int) string {
  sign := ""
  if cents < 0 {
    sign = "-"
    cents = -cents
  }
  return fmt.Sprintf("%s%d.%02d", sign, cents/100, cents%100)
}

// SplitCents divides an amount into n parts without losing or
// inventing a cent: the remainder is spread over the first shares.
func SplitCents(total, n int) []int {
  if n <= 0 {
    return nil
  }
  base := total / n
```

listing 85: the drift, integer totals, formatting at the edge, lossless splits (source: interview-repertoire/samples/ch17-go/money.go, lines 5-40)

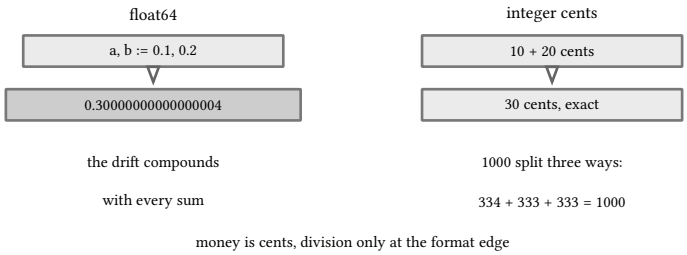


figure 75: float drifts, cents stay exact, splits sum back

The split function is the interview favorite hidden in here: dividing 1000 cents three ways sums back to exactly 1000 because the remainder spreads over the first shares. The postgres framing: `numeric` for money in postgres, never `float4` or `float8`, and integer cents is the sqlite-native answer this tree uses everywhere, demonstrated across the capstones of books 13 and 14.

17.3. normalization, proven on both shapes [TDD]

The demo database stores the same facts twice, once as a wide denormalized row, once as customers and orders with a reference: the wide table:

```
for _, ddl := range []string{
  `create table orders_wide (
    id integer primary key,
    customer_name text,
    customer_email text,
    city text,
    sku text,
    quantity integer
  )`,
  `create table customers (
    id integer primary key,
    name text not null,
    email text not null unique,
    city text not null
  )`,
  `create table orders (
    id integer primary key,
    customer_id integer not null references customers(id),
    sku text not null,
    quantity integer not null
  )`,
  `create index idx_orders_sku on orders(sku)`,
} {
  if _, err := db.Exec(ddl); err ≠ nil {
    db.Close()
    return nil, err
  }
}
```

listing 86: both schemas side by side, the wide row and the normalized pair (source: interview-repertoire/samples/ch17-go/indexes.go, lines 28-55)

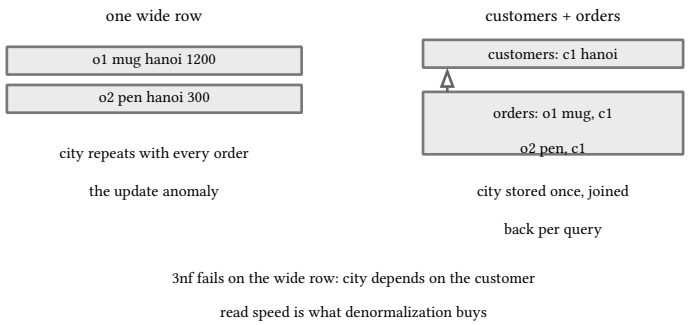


figure 76: the same facts as one wide row against customers plus orders

```go
// CityOrdersWide returns the denormalized answer: city lives in the
// wide row because someone copied the spreadsheet.
func (a *AnswerDB) CityOrdersWide(city string) (int, error) {
  var n int
  err := a.db.QueryRow(
    `select count(*) from orders_wide where city = ?`, city).Scan(&n)
  return n, err
}

// CityOrdersNormalized returns the same answer through the join the
// normalized schema buys: city lives once, with the customer, and
// orders reference the customer instead of repeating the city.
func (a *AnswerDB) CityOrdersNormalized(city string) (int, error) {
  var n int
  err := a.db.QueryRow(
    `select count(*) from orders o
     join customers c on c.id = o.customer_id
     where c.city = ?`, city).Scan(&n)
  return n, err
}
```

listing 87: the wide scan and the join, the same question answered twice (source: interview-repertoire/samples/ch17-go/indexes.go, lines 88-107)

Both answer "orders in hanoi" and the tests demand the same number from both. The spoken normalization ladder: first normal form, atomic values, second, no non-key dependencies on part of a key, third, none on any non-key attribute, and the wide table fails third when city depends on customer, not on order. The tradeoff sentence that finishes the answer: normalization removes update anomalies, denormalization buys read speed, and mature systems normalize the source of truth and denormalize deliberately at materialized edges.

17.4. indexing, proven without a stopwatch [TDD]

The index question is answered with `EXPLAIN QUERY PLAN`, not with timing:

```go
// PlanFor returns the query plan text, which is how the index answer
// gets proven without timing anything.
func (a *AnswerDB) PlanFor(query string, args ...any) ([]string, error) {
  rows, err := a.db.Query("explain query plan "+query, args...)
  if err ≠ nil {
    return nil, err
  }
  defer rows.Close()
  var plans []string
  for rows.Next() {
    var id, parent, notused int
    var detail string
    if err := rows.Scan(&id, &parent, &notused, &detail); err ≠ nil {
      return nil, err
    }
    plans = append(plans, detail)
  }
  return plans, rows.Err()
}

// UsesIndex reports whether the plan for the query mentions an
// index, covering or not, the deterministic proof the suite pins.
func UsesIndex(plans []string) bool {
  for _, p := range plans {
    up := strings.ToUpper(p)
    if strings.Contains(up, "USING INDEX") || strings.Contains(up, "USING COVERING INDEX") {
      return true
    }
  }
  return false
}
```

listing 88: the plan query, and the index matcher over the plan text (source: interview-repertoire/samples/ch17-go/indexes.go, lines 109-139)

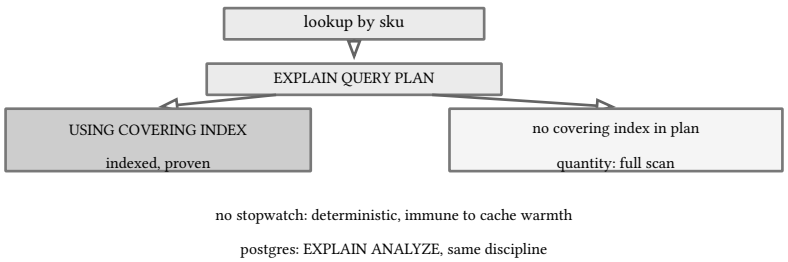


figure 77: the plan text is the proof, not the timing

The suite asserts the sku lookup's plan says `USING COVERING INDEX` and that the unindexed quantity lookup's plan says nothing of the kind. That is the deterministic version of "the index is used", immune to cache warmth and machine load, and the postgres equivalent, `EXPLAIN ANALYZE`, is the same discipline with real row counts.

17.5. optimistic versus pessimistic locking [TDD]

Pessimistic locks first, writers queue, the database is the referee. Optimistic writes anyway and checks at commit, the conflicting loser retries. The demo runs the pessimistic story on sqlite: one connection holds a write transaction, a second's `BEGIN IMMEDIATE` fails busy after the timeout:

```go
// TwoWriterLocking demonstrates the pessimistic story on sqlite: one
// connection opens a transaction and inserts (which takes the write
// lock), and a second connection's BEGIN IMMEDIATE must fail busy
// once the busy timeout expires. That busy error is what
// optimistic-versus-pessimistic conversations are about: the lock
// holder finished, the waiter still has to deal with contention.
func TwoWriterLocking(dsn string) error {
  first, err := sql.Open("sqlite", dsn)
  if err ≠ nil {
    return err
  }
  defer first.Close()
  second, err := sql.Open("sqlite", dsn)
  if err ≠ nil {
    return err
  }
  defer second.Close()
  for _, db := range []*sql.DB{first, second} {
    if _, err := db.Exec("pragma busy_timeout = 50"); err ≠ nil {
      return err
    }
  }
  if _, err := first.Exec("create table if not exists t (v integer)"); err ≠ nil {
    return err
  }

  tx, err := first.Begin()
  if err ≠ nil {
    return err
  }
  defer tx.Rollback()
  if _, err := tx.Exec("insert into t (v) values (1)"); err ≠ nil {
    return err
  }
  _, blocked := second.Exec("begin immediate")
  return blocked
}
```

listing 89: two connections, one held write lock, the busy error and its translation (source: interview-repertoire/samples/ch17-go/locking.go, lines 8-45)

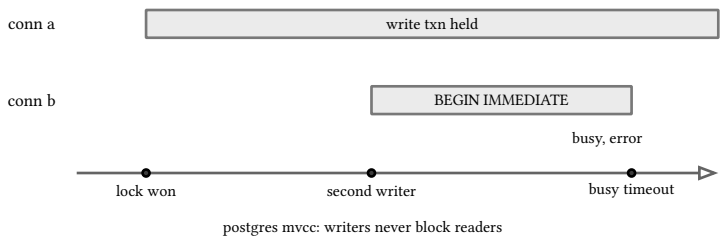


figure 78: the second writer waits out the timeout, then fails busy

The postgres framing to volunteer: postgres mvcc, multi-version concurrency control, readers work from a snapshot, is optimistic at the storage layer, writers never block readers, and explicit `SELECT FOR UPDATE` is the pessimistic opt-in. The choice rule: high contention on hot rows favors pessimistic, low contention with many readers favors optimistic, and retries belong at the caller.

17.6. cursors, limit offset, and json [DRILL]

Deep `LIMIT OFFSET` pagination degrades linearly, the database walks and discards the offset, and the fix is keyset pagination, `WHERE id > $last ORDER BY id LIMIT n`, an index range scan. Cursors are the database's wire-protocol version of the same idea, postgres cursor gives streaming fetches without materializing the whole result, and the interview answer is the tradeoff, offset is stateless and positionally inconsistent under concurrent writes, keyset and cursors are stable and bounded. Jsonb in postgres is for queryable semi-structured data with GIN indexes, the inverted index over json paths, the sqlite counterpart is the json1 extension book 13 demonstrates on 3.53, and both books agree on the boundary: structured columns for what you filter and join, json for what you store and return.

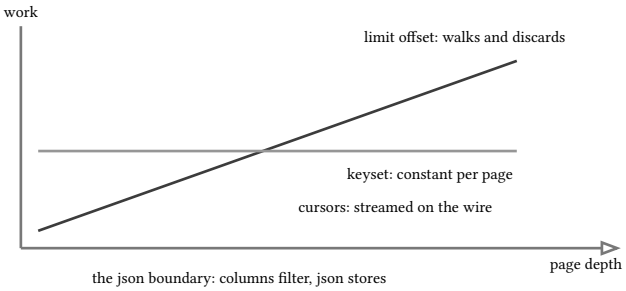


figure 79: offset cost grows with depth; keyset stays constant

17.7. mongo geo: the predicate underneath [TDD]

Point in polygon is ray casting: cast east from the point, count edge crossings, odd is inside, and points exactly on an edge count as inside because every geo library says so:

```
  Lon float64
  Lat float64
}

// Polygon is a ring of points, implicitly closed: the last vertex
// connects back to the first.
type Polygon []Point

// PointInPolygon is the ray casting predicate Mongo's $geoWithin
// with $polygon, PostGIS's ST_Contains, and every geo library run:
// cast a ray east from the point and count edge crossings. Odd means
// inside. Points exactly on an edge count as inside, which the test
// pins because it is the answer every follow-up asks about.
func PointInPolygon(p Point, poly Polygon) bool {
  inside := false
  n := len(poly)
  for i, j := 0, n-1; i < n; j, i = i, i+1 {
    a, b := poly[i], poly[j]
    if onSegment(p, a, b) {
      return true
    }
    if (a.Lat > p.Lat) != (b.Lat > p.Lat) {
      // longitude where the edge crosses the point's latitude
      cross := (b.Lon-a.Lon)*(p.Lat-a.Lat)/(b.Lat-a.Lat) + a.Lon
      if p.Lon < cross {
        inside = !inside
      }
    }
  }
  return inside
}

func onSegment(p, a, b Point) bool {
  cross := (p.Lon-a.Lon)*(b.Lat-a.Lat) - (p.Lat-a.Lat)*(b.Lon-a.Lon)
  if cross != 0 {
    return false
  }
  return p.Lon ≥ min(a.Lon, b.Lon) && p.Lon <= max(a.Lon, b.Lon) &&
    p.Lat ≥ min(a.Lat, b.Lat) && p.Lat <= max(a.Lat, b.Lat)
}

// InBBox is the cheap prefilter: most geo queries run a bounding box
// scan first and the expensive polygon test only on survivors.
func InBBox(p Point, poly Polygon) bool {
```

listing 90: ray casting with the on-edge rule, and the bbox prefilter (source: interview-repertoire/samples/ch17-go/geo.go, lines 5-48)

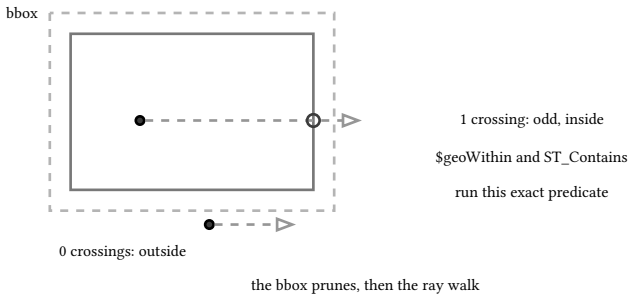


figure 80: ray casting east: odd crossings inside; the bbox prunes first

The tests cover the square, the concave L, and the boundary points, then the framing: mongo's `$geoWithin` with `$polygon` and postgis's `ST_Contains` run this predicate with spatial indexes, R-trees that prune the search with bounding boxes, which is exactly what `InBBox` models before the expensive walk.

sources: sqlite behavior verified on 3.53.x through modernc.org v1.58.0, `go test`, 9 tests in `ch17-go`. Postgres and mongo claims framed as such, floored to book 13, ch 9 (infrastructure: docker, databases, and testing, sqlite: the 3.53 feature set) and book 13, ch 13 (infrastructure: docker, databases, and testing, mongodb and aggregation pipelines) for the demonstrated versions.

18. network answers

The network round is layered: name the layers, say what each protocol guarantees, and explain tls without hand-waving the key exchange. The `ch18-go` module answers with running code, four echo servers, tcp, udp, http, and a websocket implemented over the stdlib alone, 9 tests under `make verify`.

18.1. the stack in one breath [DRILL]

Ip routes packets, no delivery guarantee. Tcp builds a reliable byte stream on top, ordered, retransmitted, flow and congestion controlled, connection-oriented. Udp keeps the packet shape, no ordering, no delivery promise, no connection, which is exactly why it is the substrate for dns, games, and real-time media. Http is a request-response protocol over tcp, ws upgrades an http connection into a framed bidirectional stream, and graphql is a query protocol riding http post, not a transport.

The contrast the echo servers make executable: the tcp echo is `io.Copy(c, c)` because tcp has no message boundaries, just bytes to return, while the udp echo reads one datagram and writes one datagram back, because boundaries are the one thing udp preserves:

```
}

func (rt *TCPRuntime) Done() <-chan struct{} { return rt.done }

// EchoUDP is the datagram counterpart: no connection, no goroutine
// per client, one loop reading a packet and writing it back to the
// sender's address. Message boundaries survive because UDP keeps
// them, where TCP never had them.
func EchoUDP(pc net.PacketConn) *UDPRuntime {
  rt := &UDPRuntime{pc: pc, done: make(chan struct{})}
  go rt.serve()
  return rt
}

type UDPRuntime struct {
  pc      net.PacketConn
  packets atomic.Int32
  done    chan struct{}
}

func (rt *UDPRuntime) serve() {
  buf := make([]byte, 2048)
  for {
    n, addr, err := rt.pc.ReadFrom(buf)
    if err != nil {
      close(rt.done)
```

listing 91: the udp loop: one packet in, the same packet back to its sender (source: interview-repertoire/samples/ch18-go/echo.go, lines 55-80)

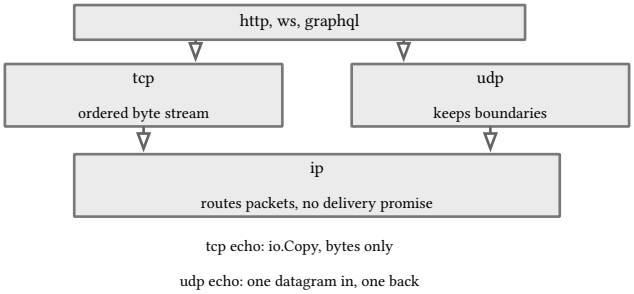


figure 81: ip routes, tcp streams, udp keeps boundaries

The suite pins the difference: three tcp writes on one connection come back as three reads the caller frames itself, one udp datagram comes back as exactly one datagram.

18.2. http, the protocol with semantics [EWC]

The http echo answers with the method it saw and the body it received, which is the whole protocol shape, request line, method, path, headers, body, response with status:

```
// EchoHTTP is the request/response protocol: methods, paths,
// headers, bodies. The echo answers with what it saw.
func EchoHTTP() http.Handler {
  mux := http.NewServeMux()
  mux.HandleFunc("/echo", func(w http.ResponseWriter, r *http.Request) {
    body, _ := io.ReadAll(r.Body)
    w.Header().Set("Echo-Method", r.Method)
    w.WriteHeader(http.StatusOK)
    _, _ = w.Write(body)
  })
  return mux
}
```

listing 92: the handler that reflects the protocol (source: interview-repertoire/samples/ch18-go/echo.go, lines 92-103)

Worth adding unprompted: http/1.1 keep-alive reuses connections, http/2 multiplexes streams over one tcp connection, http/3 moves to quic, a udp transport with tls built in, and the go http server speaks all three through the same handler interface.

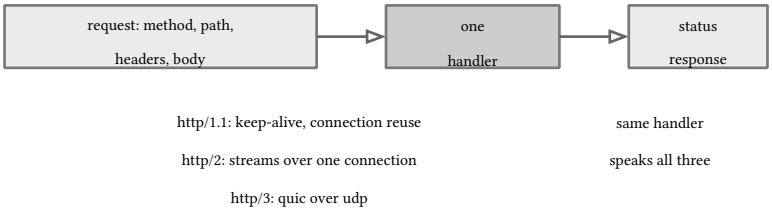


figure 82: request in, status out, one handler across versions

18.3. websockets from the metal [TDD]

A websocket is an http handshake followed by a framed protocol, and the module implements both from the stdlib. The handshake is the rfc 6455 arithmetic: base64 of sha1 over the client key plus a fixed guid, no negotiation, just proof:

```
// wsGUID is the magic string from RFC 6455: every handshake hashes
// the client key concatenated with exactly this.
const wsGUID = "258EAFA5-E914-47DA-95CA-C5AB0DC85B11"

// AcceptKey computes the Sec-WebSocket-Accept value the server must
// return: base64(sha1(key + GUID)), no negotiation, just proof both
// sides speak websocket.
func AcceptKey(clientKey string) string {
  sum := sha1.Sum([]byte(clientKey + wsGUID))
  return base64.StdEncoding.EncodeToString(sum[:])
}

// HandshakeWS upgrades one accepted connection by hand: read the
// http request, check the upgrade headers, write the 101 with the
// computed accept key. Stdlib only, which is the point of the demo.
func HandshakeWS(conn net.Conn) (*http.Request, error) {
  req, err := http.ReadRequest(bufio.NewReader(conn))
  if err ≠ nil {
    return nil, err
  }
  if !strings.EqualFold(req.Header.Get("Upgrade"), "websocket") ||
    !strings.Contains(strings.ToLower(req.Header.Get("Connection")), "upgrade") ||
    req.Header.Get("Sec-WebSocket-Key") == "" {
    fmt.Fprintf(conn, "HTTP/1.1 400 Bad Request\r\nContent-Length: 0\r\n\r\n")
    return nil, errors.New("not a websocket handshake")
  }
  key := req.Header.Get("Sec-WebSocket-Key")
  fmt.Fprintf(conn,
    "HTTP/1.1 101 Switching Protocols\r\n"+
      "Upgrade: websocket\r\nConnection: Upgrade\r\n"+
      "Sec-WebSocket-Accept: %s\r\n\r\n", AcceptKey(key))
  return req, nil
}
```

listing 93: the guid, the accept key, the upgrade check and the 101 (source: interview-repertoire/samples/ch18-go/ws.go, lines 16-48)

The frame codec handles the two header bytes, the extended length forms, the 4-byte mask on client frames, and the xor unmask, then the server echoes text and answers pings with pongs:

```
// ReadFrame decodes one frame from the reader: two header bytes,
// extended length for payloads over 125, the 4 mask bytes when the
// mask bit is set, then the payload unmasked in place.
func ReadFrame(r io.Reader) (Frame, error) {
  var hdr [2]byte
  if _, err := io.ReadFull(r, hdr[:]); err != nil {
    return Frame{}, err
  }
  f := Frame{
    Final:  hdr[0]&0x80 != 0,
    Opcode: hdr[0] & 0x0f,
    Masked: hdr[1]&0x80 != 0,
  }
  length := int64(hdr[1] & 0x7f)
  switch length {
  case 126:
    var ext [2]byte
    if _, err := io.ReadFull(r, ext[:]); err != nil {
      return Frame{}, err
    }
    length = int64(binary.BigEndian.Uint16(ext[:]))
  case 127:
    var ext [8]byte
    if _, err := io.ReadFull(r, ext[:]); err != nil {
      return Frame{}, err
    }
    length = int64(binary.BigEndian.Uint64(ext[:]))
  }
  if f.Masked {
    if _, err := io.ReadFull(r, f.Mask[:]); err != nil {
      return Frame{}, err
    }
  }
  payload := make([]byte, length)
  if _, err := io.ReadFull(r, payload); err != nil {
    return Frame{}, err
  }
  if f.Masked {
    for i := range payload {
      payload[i] ^= f.Mask[i%4]
    }
  }
  f.Payload = payload
  return f, nil
}
```

listing 94: frame decode with extended lengths and unmasking (source: interview-repertoire/samples/ch18-go/ws.go, lines 60-104)

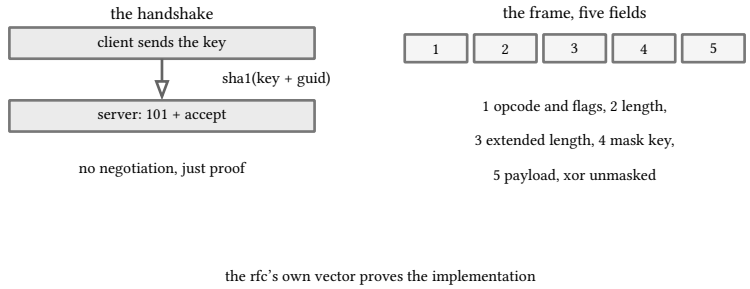


figure 83: handshake proof, then frames: opcodes, lengths, mask xor

The tests speak the client side over a real socket, masked payloads, a hand-built 300-byte extended-length frame, and the rfc's own handshake vector, `dGhlIHNhbXBsZSBub25jZQ==` hashing to `s3pPLMBiTxaQ9kYGzzhZRbK+xOo=`, which is the check that the implementation matches the spec rather than itself.

18.4. https, certificates, symmetric versus asymmetric [DRILL]

Https is http inside tls. The handshake: the client hello offers cipher suites, the server answers with its certificate, the certificate chains to a root the client already trusts, and that trust anchor is what a certificate authority sells, a signature over a domain identity the browser ships. Key agreement lands on a session secret, then everything else is symmetric encryption, aes-gcm typically, because symmetric is orders of magnitude cheaper than asymmetric.

That is the crisp version of the symmetric versus asymmetric answer: asymmetric, rsa and elliptic curve, solves key agreement and signatures at handshake scale, symmetric does the bulk traffic at wire speed, and tls uses each for exactly that. The follow-up about why the certificate works, the chain of trust, and the one about what a man in the middle actually breaks, the key exchange, since the attacker's certificate will not validate against a root both sides trust.

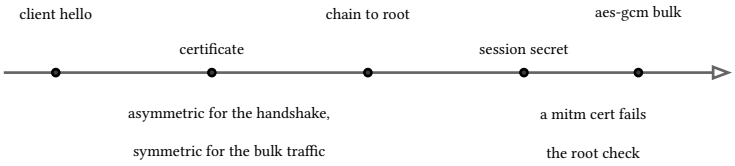


figure 84: handshake with asymmetric, bulk with symmetric

18.5. graphql versus rest, said precisely [DRILL]

Rest exposes resources with uniform verbs, graphql exposes one endpoint and a typed query language over it. The real tradeoffs: graphql shifts over-fetching and under-fetching from client guesswork to client choice, concentrates complexity on the server, and turns the n+1 problem into a resolver-layer problem, as book 17, ch 13 (the interview repertoire, react state: context, redux, graphql) demonstrated by counting queries. Rest keeps caching semantics from http itself, conditional gets, etags, cdn invalidation, that graphql's single post endpoint gives up by default. Neither is a transport, both ride http, and the honest close is that the choice is about where the query flexibility should live.

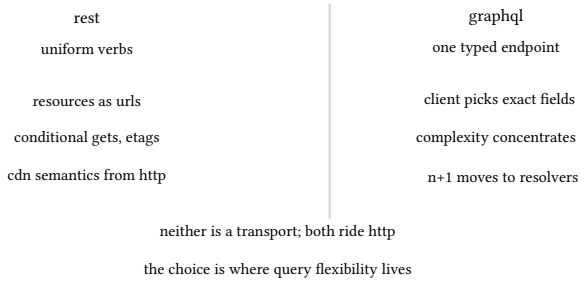


figure 85: rest against graphql: where the flexibility lives

sources: verified by `go vet` and `go test` through `make verify`, 9 tests in `ch18-go`, real sockets on loopback. Rfc 6455 guid and handshake vector cited from the websocket protocol, rfc editor, accessed 2026-09-10.

19. system design answers

The system design round rewards structure over volume: name the requirements, sketch the topology, state the bottleneck and the fix, and answer the follow-up with a tradeoff instead of a product. This chapter drills the spoken answers with the two topologies the inventory asks for by name, and every streaming claim floors to book 13, ch 15 (infrastructure: docker, databases, and testing, jetstream: streams, consumers, kv, kafka compared), whose capstone runs the real thing.

19.1. scaling with and without the cloud [DRILL]

Without the cloud the ladder is: one box, then a load balancer in front of two boxes, then a shared database behind them, then read replicas, then cache layers, then a queue to absorb writes, then sharding when the single writer saturates. Every rung is a purchase order and an operational liability.

With the cloud the ladder is the same and the purchase order disappears: managed load balancer, autoscaling groups, managed database with replicas one flag away, managed cache, managed queue. The honest close, the one interviewers wait for: the cloud changes the economics of the ladder, not the ladder. The scaling problems, the shared database, the thundering herd, the fan-out write storm, exist in both worlds, and the cloud rents you the rungs, it does not climb them for you.

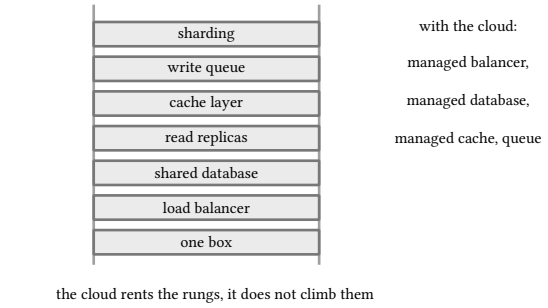


figure 86: the scaling ladder: every rung is the same, cloud or not

19.2. gateway, reverse proxy, kubernetes [DRILL]

An api gateway is the policy edge: authentication, rate limiting, routing, tls termination, request shaping. A reverse proxy is the mechanism underneath, nginx, envoy, caddy, forwarding inbound connections to upstream services. Kubernetes is the substrate: pods for units, deployments for replica sets with rolling updates, services for stable virtual ips, ingress for the http policy edge. Say the composition: an ingress or gateway product fronting services, services load balancing across deployment pods, and the gateway is where cross-cutting policy lives so the services do not each reimplement it.

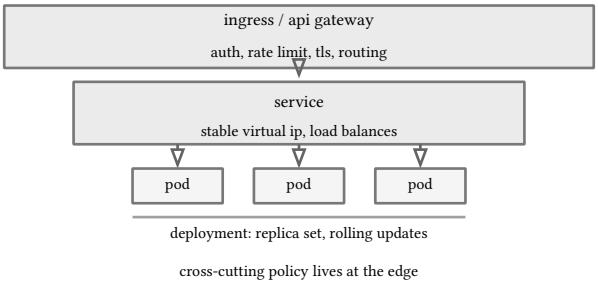


figure 87: ingress fronts services, services balance across pods

19.3. bottlenecks: tracing, logging, metrics [DRILL]

The bottleneck answer is method: measure before guessing, and the three observability legs carry the measurement. Tracing follows one request across services and names the slow hop. Metrics are aggregated time series, p99 latency, error rate, saturation, the red and use dashboards. Logs are the event detail for the hop the trace indicted. The workflow to narrate: the metric dashboard shows the symptom, the trace finds the hop, the log explains the line, and the fix lands behind a test that reproduces the original curve. The scoring sentence: a bottleneck nobody can reproduce is a fix nobody can verify.

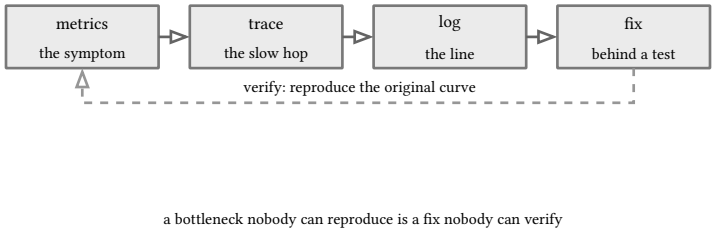


figure 88: symptom to hop to line to reproducing fix

19.4. cap, event-driven, orchestration [DRILL]

Cap: under a network partition a distributed store must pick between consistency and availability, and real systems pick per operation. Event-driven architecture is the availability-leaning shape: services emit facts, consumers react, and the system is eventually consistent because the facts land asynchronously. Orchestration versus choreography is the coordination question: an orchestrator, a workflow engine, calls the steps and owns retries, visible in one place but a center that can become a bottleneck, while choreography lets each service react to events, no center to fail but the flow exists only in the aggregate of subscriptions. The mature answer mixes them, orchestrating the transactions that must complete, choreographing the notifications that must not block.

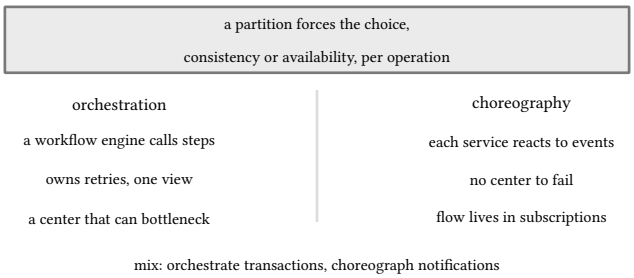


figure 89: cap picks per operation; center versus no center

19.5. the 300k concurrent users sketch [DRILL]

The numbers first, then the shape. 300k concurrent users at maybe one request per user per ten seconds is 30k requests per second, and if each holds a connection, the connections are the first wall. The topology that stands:

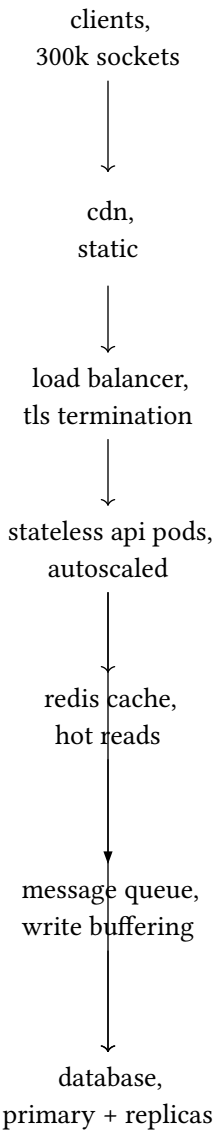


figure 90: 300k concurrent users, the shape that holds: stateless edges, shared cache, queue-absorbed writes, one replicated database

Walk it aloud: tls terminates at the edge, websocket or sse connections land on stateless pods that autoscale on connection count, reads hit a shared cache first, writes land on a queue and drain to one primary with read replicas carrying analytics. The follow-ups are arithmetic: 30k requests per second across pods sized at 2k requests per second is 15 pods plus headroom, cache hit ratio above 90 percent makes the database see only the tail, and the queue is what keeps a traffic spike from becoming a database outage. The bottleneck to volunteer: fan-out writes to 300k live connections, which is where the sse feed of book 17, ch 21 (the interview repertoire, go build part 2: storefront and live features) meets its limits and a dedicated pub/sub layer takes over.

19.6. queue before the database write [DRILL]

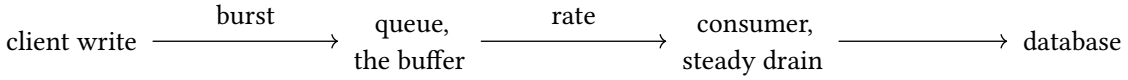


figure 91: writes go through the queue, the spike absorber

Accepting a write onto durable storage that is a queue, then draining to the database at its pace, converts a write spike from an outage into latency. The cost sentence the interviewer wants: the user's write is acknowledged before it is queryable, so the contract shifts from "saved and visible" to "accepted, visible within x", and that contract change is a product decision, not an engineering one. Offsets and redelivery semantics, the rest of that

conversation, are book 17, ch 22 (the interview repertoire, go streaming: jetstream batch pipelines) implemented against a real broker.

19.7. the remaining inventory, one paragraph each [DRILL]

Producer consumer: producers publish, consumers pull, the broker owns ordering and redelivery, and the offset is the consumer's bookmark, chapter 22 runs it live. S3 listing, the 10gb from postgres and the 10gb file in go: listing a prefix is paginated, `ListObjectsV2` pages of 1000, so count and stream instead of materializing. Ten gigabytes out of postgres is a cursor query streamed to the wire, never `SELECT` into memory, keyset paginated, the same discipline as db-answers. A 10gb file in go is `io.Copy` from file to response, streaming in chunks, since reading it into ram is the failure the question is checking for. Cloud cron: a scheduled function or a kubernetes cronjob hitting an idempotent endpoint, with a lock, because a cron that overlaps its own previous run is a distributed bug. Notification system: events onto a queue, a fan-out service resolving recipients, per channel senders with retries and a dead letter queue, and templates rendered per locale at the last step, the same shape the storefront feed of chapter 21 would grow into.

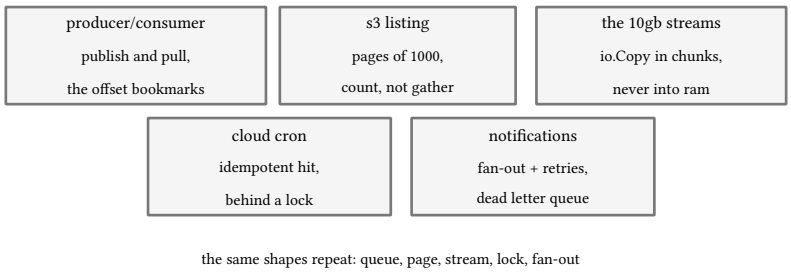


figure 92: the remaining inventory in one composite: queue, page, stream, lock, fan-out

sources: no online citations for the design patterns themselves, the drills are engineering practice floored by book 13, ch 19 (infrastructure: docker, databases, and testing, capstone part 1: services and topology) and book 13, ch 20 (infrastructure: docker, databases, and testing, capstone part 2: chat and notifications, one click) topologies, which run under `docker compose up -d`.

20. go build part 1: fetchers and rest apis

The go build round is "build this small service, live". The `ch20-go` module carries three builds: the parallel url fetcher with a bounded worker pool, a stdlib-only rest api over sqlite with caching and rate limiting, and a schema parser with a code generator. 15 tests under `make verify`, sqlite 3.53 through the pinned driver.

20.1. the parallel fetcher, five workers [TDD]

Fixed pool, one jobs channel, results in input order because each worker writes its own slot:

```
  Worker int
}

// FetchParallel fetches every url with a fixed pool of workers. The
// naive FetchSequential exists for the comparison the interview
// wants: total time of the pool is the slowest request, the sequence
// is the sum of all of them.
func FetchParallel(ctx context.Context, urls []string, workers int, get GetFunc) []FetchResult {
  if workers < 1 {
    workers = 1
  }
  results := make([]FetchResult, len(urls))
  jobs := make(chan int)
  var wg sync.WaitGroup
  for w := 1; w <= workers; w++ {
    wg.Add(1)
    go func(worker int) {
      defer wg.Done()
      for i := range jobs {
        body, err := get(ctx, urls[i])
        results[i] = FetchResult{URL: urls[i], Body: body, Err: err, Worker: worker}
      }
    }(w)
  }
feed:
  for i := range urls {
    select {
    case jobs <- i:
    case <-ctx.Done():
      break feed
    }
  }
  close(jobs)
  wg.Wait()
  return results
}

// FetchSequential is the one-at-a-time baseline.
```

listing 95: the pool, the ordered results, the sequential baseline (source: interview-repertoire/samples/ch20-go/fetcher.go, lines 14-51)

The concurrency gauge is the piece that turns the claim into a measurement: the get seam calls `Enter` and `Leave`, and the test asserts observed concurrency never exceeded the pool of five and never collapsed below two, against a real `httptest` server with 30 millisecond responses:

```
}

type ConcurrencyGauge struct {
  mu      sync.Mutex
  current int
  max     int
}

func (g *ConcurrencyGauge) Enter() {
  g.mu.Lock()
  defer g.mu.Unlock()
  g.current++
  if g.current > g.max {
    g.max = g.current
  }
}

func (g *ConcurrencyGauge) Leave() {
  g.mu.Lock()
  defer g.mu.Unlock()
  g.current--
}

func (g *ConcurrencyGauge) Max() int {
  g.mu.Lock()
  defer g.mu.Unlock()
  return g.max
}

func (g *ConcurrencyGauge) Reset() {
  g.mu.Lock()
  defer g.mu.Unlock()
  g.current = 0
  g.max = 0
}

// ErrShort is the error a get returns when the body is too short,
```

listing 96: the gauge, and the error sentinel (source: interview-repertoire/samples/ch20-go/fetcher.go, lines 72-108)

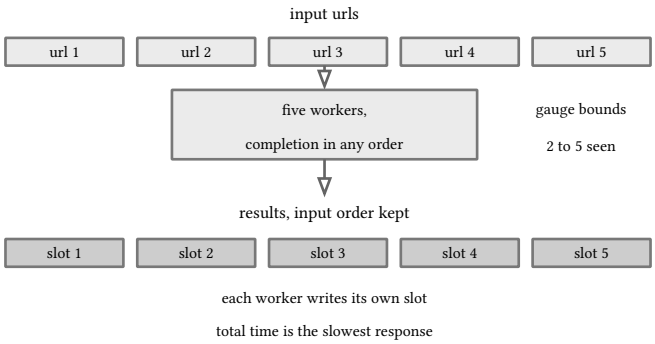


figure 93: indexed slots keep input order across any completion order

The two narrations that score: total time of the pool is the slowest request, not the sum, and errors flow through the same slots as data, so a failing url does not cancel the batch unless you decide it should, which the worker pool of book 17, ch 15 (the interview repertoire, go runtime: scheduler, gc, patterns) does with a context.

20.2. the rest api: stdlib, sqlite, cache, rate limit [TDD]

One `http.HandlerFunc`, three routes, and the two cross-cutting policies in front of them, the limiter and the cache:

```
  }, nil
}

func (s *Server) Close() error { return s.db.Close() }

func (s *Server) Handler() http.Handler {
  return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
    ip := clientIP(r)
    if !s.ips.Allow(ip) {
      w.Header().Set("Retry-After", "1")
      writeJSON(w, http.StatusTooManyRequests, map[string]string{"error": "rate limited"})
      return
    }
    // only GETs are cacheable: a cached POST would answer a
    // duplicate create with the first create's 201
    cacheable := r.Method == http.MethodGet
    key := r.Method + " " + r.URL.Path
    if cacheable {
      if cached, ok := s.cache.Get(key); ok {
        w.Header().Set("X-Cache", "hit")
        w.Header().Set("Content-Type", "application/json")
        w.WriteHeader(cached.Status)
        w.Write(cached.Body)
        return
      }
    }
    rec := &recorder{header: http.Header{}, status: http.StatusOK}
    s.route(rec, r)
    if cacheable {
      s.cache.Put(key, rec.snapshot())
    }
    if cacheable {
```

listing 97: rate limit per ip, cache GETs only, route, store the snapshot (source: interview-repertoire/samples/ch20-go/api.go, lines 57-88)

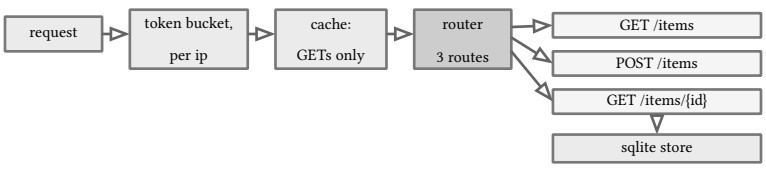


figure 94: limiter and cache in front, GETs only cached

The design lesson baked into that listing: only GETs are cached. The first draft cached every method and answered a duplicate create with the first create's 201, which the conflict test caught immediately, a real bug the tests still pin. The token bucket is the rate limiter to be able to draw:

```go
// IPLimiter is a token bucket per key: capacity tokens, one refill
// per interval, Allow takes a token or refuses.
type IPLimiter struct {
  mu       sync.Mutex
  capacity int
  interval time.Duration
  buckets  map[string]bucket
}

type bucket struct {
  tokens float64
  last   time.Time
}

func NewIPLimiter(capacity int, interval time.Duration) *IPLimiter {
  return &IPLimiter{capacity: capacity, interval: interval, buckets: map[string]bucket{}}
}

func (l *IPLimiter) Allow(key string) bool {
  l.mu.Lock()
  defer l.mu.Unlock()
  now := time.Now()
  b, ok := l.buckets[key]
  if !ok {
    b = bucket{tokens: float64(l.capacity), last: now}
  }
  elapsed := now.Sub(b.last)
  refill := float64(elapsed) / float64(l.interval)
  if refill > 0 {
    b.tokens = min(float64(l.capacity), b.tokens+refill)
    b.last = now
  }
  if b.tokens < 1 {
    l.buckets[key] = b
    return false
  }
  b.tokens--
  l.buckets[key] = b
  return true
}
```

listing 98: capacity, refill per interval, one bucket per ip (source: interview-repertoire/samples/ch20-go/api.go, lines 208-247)

The suite covers the crud cycle, the duplicate-sku 409, the cache hit and miss headers, invalidation, the fourth-burst 429 with `Retry-After`, and the bucket's refill over time, all over real http against a temp-file database that asserts its engine version at open.

20.3. the client-side limiter: a fake clock, no sleeps [TDD]

The server half of this module refuses the fourth burst with 429 and a `Retry-After`. The client half is the other side of that conversation: wait for your own token before the call, honor the server's `Retry-After` after a refusal, and cap the honoring so a hostile header cannot park the caller. The clock is the seam, constructor injection again, the testing chapter's whole thesis book 17, ch 16 (the interview repertoire, go testing: mocks and dependency injection), and it is what holds this suite at zero real sleeps where the server's own refill test sleeps 60 milliseconds:

```go
// fakeClock is the injected clock: Sleep advances the recorded now and
// appends the asked duration, so the suite asserts what a client would
// have waited without ever waiting it. Zero real sleeps anywhere in
// this file.
type fakeClock struct {
  now   time.Time
  slept []time.Duration
}

func (c *fakeClock) Now() time.Time { return c.now }
func (c *fakeClock) Sleep(d time.Duration) {
  c.now = c.now.Add(d)
  c.slept = append(c.slept, d)
}

func TestClientLimiterWaitsForItsOwnBucket(t *testing.T) {
  clock := &fakeClock{now: time.Unix(0, 0)}
  l := NewClientLimiter(1, time.Second, clock)
  ctx := context.Background()

  // the bucket is born full: the first wait costs nothing
  if err := l.Wait(ctx); err != nil {
    t.Fatal(err)
  }
  if len(clock.slept) != 0 {
    t.Fatalf("first wait slept %v", clock.slept)
  }
  // the second must wait one full interval for the refill
  if err := l.Wait(ctx); err != nil {
    t.Fatal(err)
  }
  if len(clock.slept) != 1 || clock.slept[0] != time.Second {
    t.Fatalf("second wait slept %v, want exactly [1s]", clock.slept)
  }
  // time passing on the clock refills the bucket without a sleep
  clock.now = clock.now.Add(2 * time.Second)
  if err := l.Wait(ctx); err != nil {
    t.Fatal(err)
  }
  if len(clock.slept) != 1 {
    t.Fatalf("refilled wait slept again: %v", clock.slept)
  }
}
```

listing 99: the fake clock: Sleep advances a recorded now, and the bucket test that goes red first (source: interview-repertoire/samples/ch20-go/clientlimit_test.go, lines 13-55)

Written test first: with no limiter in the module the suite's red is the missing constructor and the missing `Wait`, go's failing test being a failing build, and once it compiles the assertions are what bite, the first wait free, the second sleeping exactly one interval, the third free again because two seconds of recorded time passed on the fake. `fakeClock.Sleep` never blocks, it advances `now` and appends the duration, so the assertions read the waits a client would have taken rather than the wall clock's patience:

```go
// ClientClock is the seam: production passes the wall clock, the suite
// passes a fake whose Sleep advances a recorded now instead of
// blocking.
type ClientClock interface {
  Now() time.Time
  Sleep(d time.Duration)
}

type wallClock struct{}

func (wallClock) Now() time.Time        { return time.Now() }
func (wallClock) Sleep(d time.Duration) { time.Sleep(d) }

// MaxRetryAfter caps any server-imposed wait: the client sleeps the
// smaller of the header and 60 seconds, so a hostile or broken header
// cannot stall it indefinitely.
const MaxRetryAfter = 60 * time.Second

// clientAttempts bounds the retry loop: three refusals end in an
// error instead of a silent spin.
const clientAttempts = 3

// ErrStillLimited is what Do returns when every attempt met a 429.
var ErrStillLimited = errors.New("rate limited after every retry")

// LimitedResponse is the outcome the checked call reports: the status
// and the server's Retry-After, parsed by the caller that owns the
// http round trip.
type LimitedResponse struct {
  Status     int
  RetryAfter time.Duration
}
```

listing 100: the clock seam, the 60-second cap, the bounded attempts, the sentinel (source: interview-repertoire/samples/ch20-go/clientlimit.go, lines 18-49)

The declarations carry the policy. `MaxRetryAfter` is 60 seconds, so a `Retry-After` of any size collapses to one minute. `clientAttempts` bounds the loop at three refusals, and `ErrStillLimited` is the sentinel that ends a retry instead of a spin that hopes.

```
// Wait spends one token, sleeping toward the refill when the bucket is
// empty. The sleep happens outside the lock and every wake re-derives
// its credit from the clock, so concurrent waiters queue one interval
// apart instead of each cashing a fresh token on wake. The sleep is
// real on the wall clock and only advances the recorded now on the
// suite's fake.
func (l *ClientLimiter) Wait(ctx context.Context) error {
  if err := ctx.Err(); err ≠ nil {
    return err
  }
  for {
    l.mu.Lock()
    now := l.clock.Now()
    if elapsed := now.Sub(l.last); elapsed > 0 {
      l.tokens = min(l.capacity, l.tokens+float64(elapsed)/float64(l.interval))
      l.last = now
    }
    if l.tokens ≥ 1 {
      l.tokens--
      l.mu.Unlock()
      return nil
    }
    deficit := (1 - l.tokens) * float64(l.interval)
    l.mu.Unlock()
    l.clock.Sleep(time.Duration(deficit))
    if err := ctx.Err(); err ≠ nil {
      return err
    }
  }
}

// Do runs one limited call and retries a 429: each pass waits for its
// own token, a refusal makes the wait the server's Retry-After, capped
// at MaxRetryAfter, and exhausting clientAttempts returns
// ErrStillLimited rather than looping forever.
func (l *ClientLimiter) Do(ctx context.Context, call func(ctx context.Context) (LimitedResponse, error))
error {
  for range clientAttempts {
    if err := l.Wait(ctx); err ≠ nil {
      return err
    }
    resp, err := call(ctx)
    if err ≠ nil {
      return err
    }
    if resp.Status ≠ http.StatusTooManyRequests {
      return nil
    }
    if resp.RetryAfter > 0 {
      l.clock.Sleep(min(resp.RetryAfter, MaxRetryAfter))
    }
  }
  return ErrStillLimited
}
```

listing 101: Wait sleeps the exact deficit, Do honors the capped Retry-After (source: interview-repertoire/samples/ch20-go/clientlimit.go, lines 76-128)

`Wait` is the server bucket's arithmetic with the opposite refusal policy: refill by elapsed time, and when the bucket is empty compute the deficit, sleep exactly it, and let the sleep be the refill. `Do` spends a token, makes the call, and on a 429 sleeps the smaller of the header and the cap before the next pass. The second test scripts the three scenarios: a 120-second `Retry-After` collapses to exactly one recorded 60-second sleep and the retried call succeeds, a 2-second header is honored uncapped, and refusals that never end stop at three calls with the sentinel error, one honored wait per refusal.

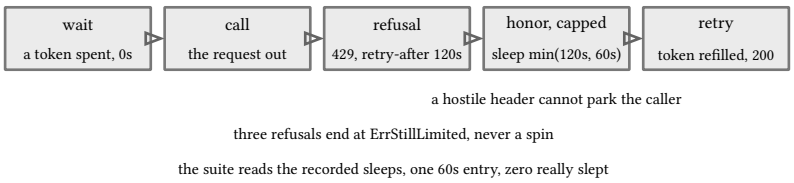


figure 95: the capped retry: one 429, one minute, the retry lands

20.4. the schema generator/parser [TDD]

Parse `CREATE TABLE` into a table model, generate go structs from it. The parser handles the sqlite core, names, types, not null, single-column primary keys, comma-safe parenthesized types, and rejects table-level constraints loudly instead of half understanding them:

```
    table, err := parseCreate(stmt)
    if err ≠ nil {
      return nil, err
    }
    tables = append(tables, table)
  }
  return tables, nil
}

func parseCreate(stmt string) (Table, error) {
  lowered := strings.ToLower(stmt)
  open := strings.Index(lowered, "(")
  if !strings.HasPrefix(lowered, "create table") || open < 0 {
    return Table{}, fmt.Errorf("not a create table statement: %.40s", stmt)
  }
  closeIdx := strings.LastIndex(stmt, ")")
  if closeIdx < open {
    return Table{}, fmt.Errorf("unbalanced parens: %.40s", stmt)
  }
  name := strings.TrimSpace(strings.TrimSpace(lowered[len("create table"):open]))
  if strings.HasPrefix(name, "if not exists") {
    name = strings.TrimSpace(name[len("if not exists"):])
  }
  body := stmt[open+1 : closeIdx]

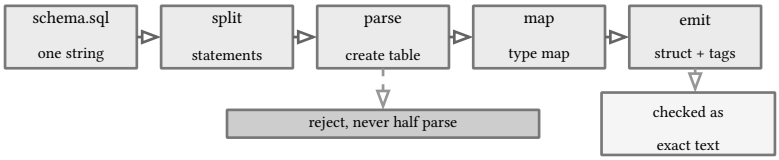
  var cols []Column
  primaryKey := ""
  for _, part := range splitTopLevel(body) {
    part = strings.TrimSpace(part)
    if part == "" {
      continue
```

listing 102: statement splitting, the create table shape, loud rejection (source: interview-repertoire/samples/ch20-go/schema.go, lines 32-62)

```
    return "float64"
  case strings.Contains(sqlType, "bool"):
    return "bool"
  default:
    return "string"
  }
}

// GenerateGoStructs renders parsed tables as go source, the schema
// generator half of the exercise.
func GenerateGoStructs(tables []Table) string {
  var sb strings.Builder
  for _, t := range tables {
    structName := PascalCase(t.Name)
    sb.WriteString(fmt.Sprintf("type %s struct {\n", structName))
    for _, c := range t.Columns {
      sb.WriteString(fmt.Sprintf("\t%s %s `json:%q`\n", PascalCase(c.Name), GoType(c.Type), c.Name))
    }
```

listing 103: type mapping and the struct emitter (source: interview-repertoire/samples/ch20-go/schema.go, lines 126-143)



the failure mode it guards against is silent misparse

figure 96: split, parse, map, emit, with loud rejection

The generated-struct test asserts exact source text, tags included, which is the discipline this repo's own listings enforce: generated output is checked like any other output. Say the boundary honestly: this is a teaching parser, the production version is a proper dialect grammar, and the failure mode it guards against is silent misparse.

sources: verified by `go vet` and `go test` through `make verify`, 15 tests in `ch20-go` over sqlite 3.53 through modernc.org/sqlite v1.58.0, http served by net/http/httptest, the client limiter exercised through an injected fake clock with zero real sleeps.

21. go build part 2: storefront and live features

Part 2 bolts the live half onto part 1's shapes: a storefront with transactional stock, a server-to-client event feed on the same service, and a deploy story that swaps versions without dropping requests. The `ch21-go` module, 8 tests under `make verify`.

21.1. the storefront and the reservation transaction [TDD]

Inventory in sqlite, and the operation that matters is reserve: one transaction checks the count and inserts the reservation, so two concurrent reserves cannot both take the last unit:

```
// Reserve takes stock atomically: one transaction checks the count
// and inserts the reservation, so two concurrent reserves can never
// both see the last unit. The check constraint is the backstop.
func (s *Storefront) Reserve(ctx context.Context, sku string, qty int) (int64, error) {
  tx, err := s.db.BeginTx(ctx, nil)
  if err ≠ nil {
    return 0, err
  }
  defer tx.Rollback()
  var have int
  if err := tx.QueryRowContext(ctx,
    "select qty from stock where sku = ?", sku).Scan(&have); err ≠ nil {
    return 0, err
  }
  if have < qty {
    return 0, ErrOutOfStock
  }
  if _, err := tx.ExecContext(ctx,
    "update stock set qty = qty - ? where sku = ?", qty, sku); err ≠ nil {
    return 0, err
  }
  res, err := tx.ExecContext(ctx,
    "insert into reservations (sku, qty, created_at) values (?, ?, ?)",
    sku, qty, time.Now().UTC().Format(time.RFC3339))
  if err ≠ nil {
    return 0, err
  }
  id, err := res.LastInsertId()
  if err ≠ nil {
    return 0, err
  }
  if err := tx.Commit(); err ≠ nil {
    return 0, err
  }
  remaining, _ := s.Qty(ctx, sku)
  if err := s.emit(ctx, "reserved", sku, remaining); err ≠ nil {
    return id, err
  }
  return id, nil
}
```

listing 104: the reserve transaction, check then decrement then record, then emit (source: interview-repertoire/samples/ch21-go/storefront.go, lines 105-144)

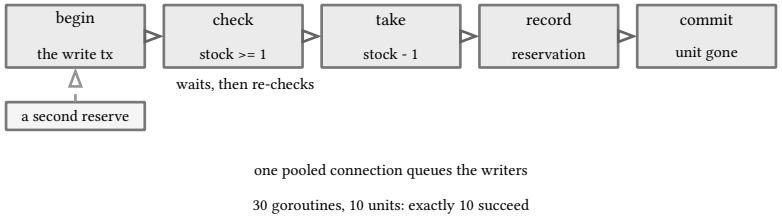


figure 97: check, take, record in one transaction: no double sell

The oversell test is the honest one: 30 goroutines reserve against a 10 unit stock and exactly 10 succeed, the stock lands at zero, and no reservation exists for a unit that was not there. The sqlite note to narrate: one pooled connection, `SetMaxOpenConns(1)`, makes concurrent writers queue on the single writer sqlite is, instead of racing into busy errors, the standard go-plus-sqlite answer this suite learned by failing first.

21.2. the feed, bolted on step by step [TDD]

The realtime feature is one endpoint on the same service, and the steps are worth narrating in order because the interview asks for the process. Step one, the storefront already emits every change through `emit`, stamped with a sequence number from the database. Step two, the feed holds a bounded history and a subscriber list:

```
// and a live subscriber list. Publish appends to history and hands
// the event to every subscriber without blocking on slow readers.
type Feed struct {
  mu   sync.Mutex
  hist []Event
  subs map[*Subscription]struct{}
}

func NewFeed() *Feed {
  return &Feed{subs: map[*Subscription]struct{}{}}
}

const feedHistory = 256

func (f *Feed) Publish(ev Event) {
  f.mu.Lock()
  defer f.mu.Unlock()
  f.hist = append(f.hist, ev)
  if len(f.hist) > feedHistory {
    f.hist = f.hist[len(f.hist)-feedHistory:]
  }
  for sub := range f.subs {
    sub.deliver(ev)
  }
}

// Subscribe returns a subscription that first replays every event
// after the given sequence, then tails live traffic. Closing the
// subscription detaches it from the feed.
```

listing 105: bounded history, publish to every subscriber without blocking (source: interview-repertoire/samples/ch21-go/feed.go, lines 6-34)

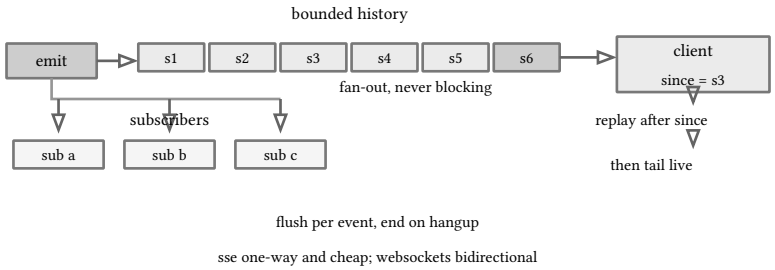


figure 98: bounded history plus subscribers: replay after since, then tail

Step three, the endpoint is `text/event-stream`, a replay of everything after the client's `since` parameter, then a live tail, flushing per event and ending when the client hangs up:

```
func (s *Storefront) ServeSSE(w http.ResponseWriter, r *http.Request) {
  flusher, ok := w.(http.Flusher)
  if !ok {
    http.Error(w, "streaming unsupported", 500)
    return
  }
  var since int64 = 0
  if raw := r.URL.Query().Get("since"); raw ≠ "" {
    since, _ = strconv.ParseInt(raw, 10, 64)
  }
  w.Header().Set("Content-Type", "text/event-stream")
  w.Header().Set("Cache-Control", "no-cache")
  sub := s.events.Subscribe(since)
  defer sub.Close()
  for {
    select {
    case <-r.Context().Done(): // the client hung up
      return
    case ev := <-sub.Ch():
      fmt.Fprintf(w, "id: %d\nevent: %s\ndata: %s\n\n",
        ev.Seq, ev.Kind, mustJSON(ev))
      flusher.Flush()
    }
  }
}
```

listing 106: the SSE endpoint: replay, tail, flush, hangup (source: interview-repertoire/samples/ch21-go/storefront.go, lines 223-247)

The stream test holds the connection open over real http, watches the replay arrive without any publish, then sees a live stock update flow down the open socket. The boundary to volunteer: sse is one-directional and cheap, websockets are bidirectional at protocol cost, chapter 18 implemented both ends of the latter, and the choice is whether the client ever needs to push on the same channel.

21.3. graceful shutdown and the live deploy [TDD]

`Shutdown` stops accepting and waits for in-flight requests, and the test proves the ordering: a slow request starts, shutdown begins and does not return, the request finishes with its body intact, shutdown completes, and new connections are refused:

```go
// DeployableServer is the graceful shutdown piece: one listener, a
// swappable handler, and Shutdown that drains in-flight work.
type DeployableServer struct {
  mu      sync.RWMutex
  handler http.Handler
}

func NewDeployable(h http.Handler) *DeployableServer {
  return &DeployableServer{handler: h}
}

func (d *DeployableServer) ServeHTTP(w http.ResponseWriter, r *http.Request) {
  d.mu.RLock()
  h := d.handler
  d.mu.RUnlock()
  h.ServeHTTP(w, r)
}

// Deploy swaps the handler under live traffic. Requests that already
// entered the old handler finish there; new requests see the new
// version immediately, which is the zero-downtime story without a
// second process.
func (d *DeployableServer) Deploy(h http.Handler) {
  d.mu.Lock()
  d.handler = h
  d.mu.Unlock()
}

// GracefulShutdown stops accepting, waits for in-flight requests up
// to the timeout, then closes everything still running.
func GracefulShutdown(srv *http.Server, timeout time.Duration) error {
  ctx, cancel := context.WithTimeout(context.Background(), timeout)
  defer cancel()
  return srv.Shutdown(ctx)
}
```

listing 107: the deployable front swaps handlers; GracefulShutdown drains with a timeout (source: interview-repertoire/samples/ch21-go/storefront.go, lines 254-288)

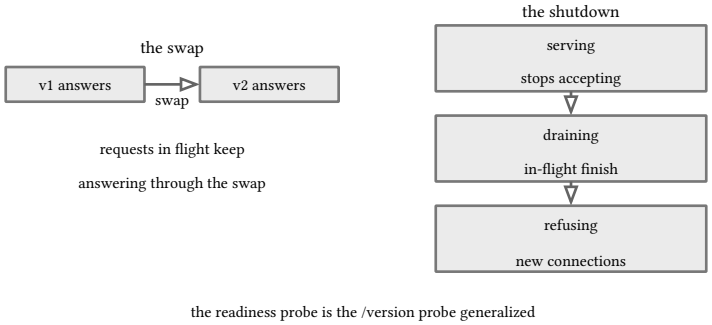


figure 99: swap the front under traffic; drain, then refuse

The deploy test runs one listener behind a swappable front handler, answers `/version` as v1, swaps to v2 under live traffic, and the next request sees v2 while storefront routes keep answering through the swap. The production sentence that finishes it: the same shape at process scale is the kubernetes rolling update, new pods up, old pods drained, and the readiness probe is the `/version` probe generalized.

sources: verified by `go vet` and `go test` through `make verify`, 8 tests in `ch21-go` over sqlite 3.53 through modernc.org/sqlite v1.58.0. Deploy and drain doctrine floors to book 13, ch 3 (infrastructure: docker, databases, and testing, containers: processes, signals, pid 1) for the signal and pid 1 mechanics.

22. go streaming: jetstream batch pipelines

The streaming round is durable messaging with redelivery semantics, and the only honest answer is a running one. The ch22-go module runs a real nats server in-process, jetstream file storage, a batch producer, a durable consumer, nak redelivery, and a restart test that kills and restarts the broker against the same store. 3 tests, each heavyweight, under `make verify`.

22.1. the embedded server, and two windows bugs [TDD]

The server is a library: options, `go srv.Start()`, wait for readiness. The listing is short because the lesson is in the comments, both learned from real red runs on this tree:

```
// storage rooted at dir, so a restart of the in-process server
// against the same dir replays the same durable state. The store
// dir must carry forward slashes: on windows a backslash path makes
// the file store init stall before the listener opens, a lesson
// this suite learned the hard way.
func StartServer(dir string) (string, *server.Server, error) {
  opts := &server.Options{
    JetStream:  true,
    StoreDir:   filepath.ToSlash(dir),
    ServerName: "irch22",
    Host:       "127.0.0.1",
    Port:       -1, // random free port

    NoSigs: true,
  }
  srv, err := server.NewServer(opts)
  if err ≠ nil {
    return "", nil, err
  }
  go srv.Start()
  // durations are nanoseconds: a bare 10_000 here is ten
  // microseconds, and the wait returns before the server starts
  if !srv.ReadyForConnections(10 * time.Second) {
    return "", nil, fmt.Errorf("nats server did not become ready")
  }
  // ClientURL already carries the nats:// scheme; prefixing another
  // one makes the client dial the host "nats"
  return srv.ClientURL(), srv, nil
}

// StopServer shuts the embedded server down cleanly so its file
```

listing 108: embedded server with file storage, forward slashes, real durations (source: interview-repertoire/samples/ch22-go/server.go, lines 14-44)

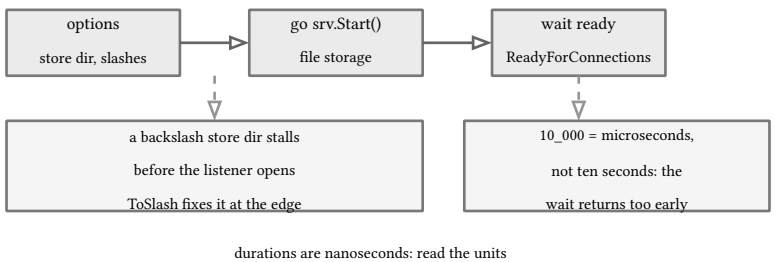


figure 100: options, start, wait ready, and the two windows bugs

Windows taught two lessons worth telling in an interview as bug stories. A backslash store dir made the jetstream file store stall before the listener opened, silent under `NoLog`, and `filepath.ToSlash` at the options boundary is the fix. And `srv.ReadyForConnections(10_000)` is not ten seconds, a `time.Duration` is nanoseconds, so it was ten microseconds and the wait returned before the server started. Unit-bearing types catch this at review time only if the reviewer reads the units.

22.2. batch produce, durable consume [TDD]

The producer stamps a `Nats-Msg-Id` for deduplication and publishes the batch. The worker binds a durable consumer, fetches in batches, acks successes, and naks failures straight back for redelivery:

```
// per batch element and a shared Nats-Msg-Id header for
// deduplication on redelivery attempts.
type Producer struct {
  js      nats.JetStreamContext
  subject string
}

func NewProducer(js nats.JetStreamContext, subject string) *Producer {
  return &Producer{js: js, subject: subject}
}

// PublishBatch sends every payload, stamping each with a dedupe id
// built from the batch name and index.
func (p *Producer) PublishBatch(name string, payloads [][]byte) (int, error) {
  for i, payload := range payloads {
    msg := nats.NewMsg(p.subject)
    msg.Data = payload
    msg.Header.Set("Nats-Msg-Id", fmt.Sprintf("%s-%d", name, i))
    if _, err := p.js.PublishMsg(msg); err != nil {
      return i, err
    }
  }
  return len(payloads), nil
}

// WorkerConfig tunes a durable consumer worker.
type WorkerConfig struct {
  Stream   string
  Subject  string
  Durable  string
```

listing 109: the batch producer with dedupe ids (source: interview-repertoire/samples/ch22-go/pipeline.go, lines 13-42)

```
  }
  defer nc.Close()
  js, err := nc.JetStream()
  if err ≠ nil {
    return 0, err
  }
  // upsert the durable: explicit config, same values every start,
  // so a restart binds instead of colliding on a mismatch
  consumer := &nats.ConsumerConfig{
    Durable:       cfg.Durable,
    FilterSubject: cfg.Subject,
    AckPolicy:     nats.AckExplicitPolicy,
    AckWait:       ackWait,
  }
  if _, err := js.AddConsumer(cfg.Stream, consumer); err ≠ nil {
    if !errors.Is(err, nats.ErrConsumerNameAlreadyInUse) {
      return 0, err
    }
    if _, err := js.UpdateConsumer(cfg.Stream, consumer); err ≠ nil {
      return 0, err
    }
  }
  sub, err := js.PullSubscribe(cfg.Subject, cfg.Durable,
    nats.BindStream(cfg.Stream))
  if err ≠ nil {
    return 0, err
  }
  // deliberately no Unsubscribe: unsubscribing a durable deletes
  // the consumer, and the whole point of durable is that it
  // outlives the connection that created it

  processed := 0
  for {
    select {
    case <-ctx.Done():
      return processed, nil
    default:
    }
    msgs, err := sub.Fetch(cfg.Batch, nats.MaxWait(250*time.Millisecond))
    if err ≠ nil {
      if err == nats.ErrTimeout {
        continue
      }
      return processed, err
    }
```

listing 110: the durable worker: upsert the consumer, fetch, ack, nak (source: interview-repertoire/samples/ch22-go/pipeline.go, lines 60-104)

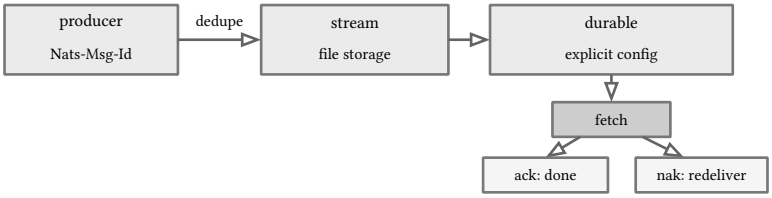


figure 101: stamp for dedupe, bind the durable, ack or nak

The consumer upsert is deliberate: the durable is created with explicit config once, and restarts bind to it, because binding with mismatched inline options is an error, another lesson from a red run. The nak test is observable behavior, a poison payload fails twice and succeeds on the third attempt, and the suite counts the attempts.

22.3. the offset is the ack floor [TDD]

Inspect reads the two numbers that explain durable semantics, the stream's message count and the consumer's pending and ack floor:

```
        // negative ack: redeliver now rather than after ack wait
        m.Nak()
        continue
      }
      m.Ack()
      processed++
    }
  }
}

// StreamState summarizes what the stream holds, the observable the
// restart test compares.
type StreamState struct {
  Messages uint64
  Consumer string
  Pending  uint64
  // AckFloor is the stream sequence the durable has acked through.
  AckFloor uint64
}

func Inspect(js nats.JetStreamContext, stream, durable string) (StreamState, error) {
  info, err := js.StreamInfo(stream)
  if err ≠ nil {
    return StreamState{}, err
  }
  ci, err := js.ConsumerInfo(stream, durable)
  if err ≠ nil {
```

listing 111: stream state, pending count, the ack floor (source: interview-repertoire/samples/ch22-go/pipeline.go, lines 107-133)

The restart test is the chapter's spine. Produce four, consume two, stop the worker mid-stream, shut the broker down completely, start a new broker against the same store directory. The stream still holds four messages, the durable still exists, the ack floor is still 2, and a fresh worker receives exactly the unacked tail. That is the "offset" of the kafka vocabulary, persisted by the broker per consumer, and the kafka comparison is the paragraph to have ready: kafka exposes the offset and lets the consumer commit it, jetstream tracks delivery per consumer with explicit acks, both redeliver on timeout or nack, and the choice between them is ecosystem and operations, topic partitioning and mirror making versus jetstream's simpler single-binary model. Book 13's book 13, ch 15

(infrastructure: docker, databases, and testing, jetstream: streams, consumers, kv, kafka compared) chapter carries the full comparison against its own running capstone.

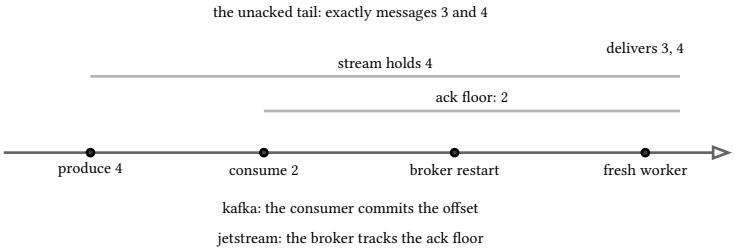


figure 102: kill the broker, restart: the floor survives, the tail is redelivered

sources: nats jetstream concepts, durable consumers, ack floors, redelivery, from docs.nats.io, accessed 2026-09-09 and pinned in the sources appendix. Verified by `go vet` and `go test` through `make verify` against an embedded nats-server v2.14.6 with nats.go v1.53.1, 3 tests in `ch22-go`.

23. algorithm patterns: window, pointers, search, greedy, bits

The pattern questions are the mid-loop currency: recognize the shape, write the loop, state the cost, move on. This chapter drills six recurring shapes, the sliding window, the inward pointer pair, binary search plain and rotated, two greedies with one sentence proofs, the backtrack decision tree, and the xor and popcount tricks, each written test first in the Go workspace `samples/ch23-go`, 13 tests run under `make verify` and again under `go test -race`. The cost vocabulary is floored by book 10, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking).

23.1. longest substring without repeating characters [TDD]

The sliding window earns its keep when the constraint is a run property, here the longest span of distinct runes. The window owns a left edge and a last-seen map, each rune admitted updates both, and the edge jumps past a repeat only when the previous copy still sits inside the window:

```go
// LongestUniqueSubstring returns the length of the longest run of
// distinct runes. The window slides once, left to right: each rune
// asks the last-seen map where its previous copy lived, and the
// left edge jumps past that copy only when it sits inside the
// current window. A jump to an index before the left edge would
// walk the window backward and is refused.
func LongestUniqueSubstring(s string) int {
  runes := []rune(s)
  last := map[rune]int{} // rune → most recent index seen
  left, best := 0, 0
  for i, r := range runes {
    if j, ok := last[r]; ok && j ≥ left {
      left = j + 1
    }
    last[r] = i
    if i-left+1 > best {
      best = i - left + 1
    }
  }
  return best
}
```

listing 112: one sweep, the last-seen map guards the left edge (source: interview-repertoire/samples/ch23-go/window.go, lines 3-23)

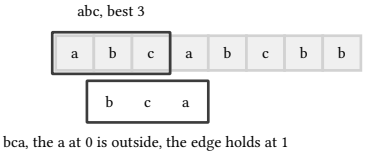


figure 103: the edge only ever moves right, a repeat outside the window cannot shrink it

The trap the `abba` fixture pins: the second a's previous copy lives at index 0, before the window the second b pushed to 1, so an unconditional `left = j + 1` reports 3. The `j ≥ left` guard refuses the jump, and the multibyte row `éaéö` holds the walk to runes, the lesson book 17, ch 5 (the interview repertoire, classics part 1: warmup, arrays, hashing) teaches with the reverser.

PITFALL: the window that resets to the start

> The failing variant restarts at the repeat's old index plus one without asking whether it sits behind the current edge, so `abba` reports 3. The fix is one comparison, `j ≥ left`.

23.2. two pointers: sorted pair sum, in-place dedupe [TDD]

Sorted input buys direction. The pair sum walks one pointer from each end, too small a sum moves the left pointer up, too large moves the right pointer down, and each step discards a value that provably belongs to no pair. The dedupe runs the same shape in place, a read index scans, a write index trails, a value moves forward only when it differs from the last written:

```go
// SortedPairSum walks two pointers inward over ascending input and
// returns the first pair of values adding to target. Values, not
// indices: the ch05 twin already owns the index contract, this one
// answers the variant interviewers ask with values. Too small a
// sum moves the left pointer, too large moves the right, and every
// step discards a value that can never belong to any pair.
func SortedPairSum(nums []int, target int) ([2]int, bool) {
  lo, hi := 0, len(nums)-1
  for lo < hi {
    sum := nums[lo] + nums[hi]
    switch {
    case sum == target:
      return [2]int{nums[lo], nums[hi]}, true
    case sum < target:
      lo++
    default:
      hi--
    }
  }
  return [2]int{}, false
}
```

listing 113: the inward walk over ascending input, values returned, not indices (source: interview-repertoire/samples/ch23-go/pointers.go, lines 3-23)

```go
// DedupeInPlace compacts a sorted slice onto its own prefix and
// returns the length of the unique run. The write index only ever
// trails the read index, so nothing is read after it is
// overwritten, and the caller keeps the tail beyond the returned
// length, untouched but stale.
func DedupeInPlace(nums []int) int {
  if len(nums) == 0 {
    return 0
  }
  write := 1
  for read := 1; read < len(nums); read++ {
    if nums[read] != nums[write-1] {
      nums[write] = nums[read]
      write++
    }
  }
  return write
}
```

listing 114: the write index only ever trails the read index (source: interview-repertoire/samples/ch23-go/pointers.go, lines 25-42)

The ch05 twin owns the index-returning two sum, book 17, ch 5 (the interview repertoire, classics part 1: warmup, arrays, hashing), this one answers the values variant. The dedupe contract to state unprompted: the returned k means the first k slots hold the uniques, the tail is stale but legal.

23.3. binary search and the rotated variant [TDD]

The plain loop keeps a half open interval, lo inclusive, hi exclusive, so the empty slice and the not-found case fall out of the loop without special handling. The midpoint is `lo` plus half the distance, the spelling that cannot overflow:

```
// BinarySearch returns the index of target in ascending input, or
// -1 when absent. The interval stays half open, lo inclusive and
// hi exclusive, so an empty slice needs no special case and the
// midpoint is computed as lo plus half the distance, the form that
// cannot overflow on a full slice.
func BinarySearch(nums []int, target int) int {
  lo, hi := 0, len(nums)
  for lo < hi {
    mid := lo + (hi-lo)/2
    switch {
    case nums[mid] == target:
      return mid
    case nums[mid] < target:
      lo = mid + 1
    default:
      hi = mid
    }
  }
  return -1
}
```

listing 115: half open interval, overflow safe midpoint (source: interview-repertoire/samples/ch23-go/search.go, lines 3-22)

The rotated variant leans on one fact: around any midpoint at least one of the two halves is sorted, and a sorted half answers in two comparisons whether the target lives inside it:

```go
// SearchRotated finds target in an ascending slice rotated by an
// unknown pivot, or -1. Around any midpoint one of the two halves
// is always sorted, and the sorted half answers in two comparisons
// whether target lives inside it: inside, recurse there, outside,
// it must be in the other half.
func SearchRotated(nums []int, target int) int {
  lo, hi := 0, len(nums)-1
  for lo <= hi {
    mid := lo + (hi-lo)/2
    switch {
    case nums[mid] == target:
      return mid
    case nums[lo] <= nums[mid]: // left half is sorted
      if nums[lo] <= target && target < nums[mid] {
        hi = mid - 1
      } else {
        lo = mid + 1
      }
    default: // right half is sorted
      if nums[mid] < target && target <= nums[hi] {
        lo = mid + 1
      } else {
        hi = mid - 1
      }
    }
  }
  return -1
}
```

listing 116: find the sorted half, ask it the two range questions (source: interview-repertoire/samples/ch23-go/search.go, lines 24-51)

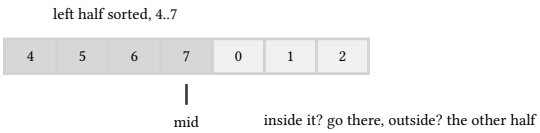


figure 104: mid splits the rotation, the sorted half answers, the other half inherits

The off-by-one that fails silently: `nums[lo]` $\leq$ `nums[mid]` must take the equal case as left-sorted, a single element half is sorted by definition, and dropping the equality breaks a two element rotation like `[3 1]`. Say the invariant aloud, target sits in the interval, one half around mid is sorted, both loops stay O(log n) only while it holds. The suite drives the rotated row, an unrotated row, absent targets, and the empty slice, floored by book 10, ch 14 (practical data structures and algorithms, searching and answer spaces).

23.4. greedy: jump game and interval scheduling [TDD]

A greedy answer lives or dies by its proof. The jump game's proof is reach: sweep once, carry the farthest index reachable so far, fail the moment the sweep passes it, since every index behind the reach is reachable too. Interval scheduling's is finish first: sort by end, take what starts at or after the last taken end, the earliest finisher never costs a later slot:

```go
// CanJump answers whether the index hop game reaches the last slot.
// One sweep carries the farthest reachable index, fails the moment
// the sweep index passes it, and succeeds otherwise. The reach is a
// sufficient certificate because every index behind it is reachable
// too, so tracking the maximum is not a guess, it is a proof.
func CanJump(jumps []int) bool {
  reach := 0
  for i, j := range jumps {
    if i > reach {
      return false
    }
    if i+j > reach {
      reach = i + j
    }
  }
  return true
}
```

listing 117: one sweep, the reach is the certificate (source: interview-repertoire/samples/ch23-go/greedy.go, lines 8-24)

```go
// MaxNonOverlapping counts the most intervals that never overlap,
// the finish-first greedy: sort by end, take each interval that
// starts at or after the end of the last one taken. Taking the
// earliest finisher can never cost a later slot, which is the one
// sentence proof the interviewer wants. The input is cloned because
// the sort belongs to the answer, not to the caller's slice.
func MaxNonOverlapping(intervals [][2]int) int {
  sorted := slices.Clone(intervals)
  slices.SortFunc(sorted, func(a, b [2]int) int { return cmp.Compare(a[1], b[1]) })
  taken, lastEnd := 0, 0
  for _, iv := range sorted {
    if taken == 0 || iv[0] ≥ lastEnd {
      taken++
      lastEnd = iv[1]
    }
  }
  return taken
}
```

listing 118: finish-first sort, take what fits, the clone keeps the caller's order (source: interview-repertoire/samples/ch23-go/greedy.go, lines 26-43)

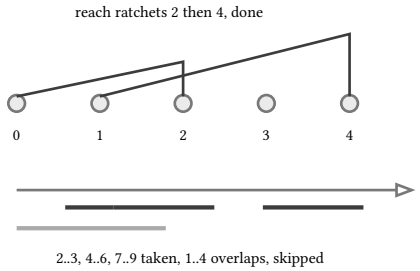


figure 105: reach ratchets up and never revisits, finish first takes three and skips one

The zero fixture `[3 2 1 0 4]` is the one worth naming aloud: every hop lands on the 0, the reach stops at 3, the last slot sits at 4. The clone is a contract, the test asserts the caller's slice keeps its order. The wider greedy canon, exchange arguments included, is floored by book 10, ch 18 (practical data structures and algorithms, greedy and backtracking).

23.5. backtracking: subsets and permutations [TDD]

Backtracking is a decision tree with an undo button. The subsets walk forks on skip or include at every element, the power set falls out at the leaves, and the path is copied out at emission so no emitted subset aliases the recursion's working slice. The permutations walk picks an unused element at each depth, the unmark and pop after each return is the backtrack itself:

```go
// Subsets enumerates the power set by skip-or-include: each element
// forks the walk twice, and the path is copied out at the leaf so
// no emitted subset aliases the recursion's working slice. Every
// append writes only at or beyond the current length, so the skip
// branch, fully explored first, can never observe the include
// branch's writes.
func Subsets(nums []int) [][]int {
  out := [][]int{}
  var walk func(i int, path []int)
  walk = func(i int, path []int) {
    if i == len(nums) {
      own := make([]int, len(path))
      copy(own, path)
      out = append(out, own)
      return
    }
    walk(i+1, path)
    walk(i+1, append(path, nums[i]))
  }
  walk(0, nil)
  return out
}
```

listing 119: skip or include, copy at the leaf (source: interview-repertoire/samples/ch23-go/backtrack.go, lines 3-24)

```
// Permutations enumerates every ordering by picking an unused
// element at each depth. The used flags make it a plain decision
// tree walked in try order, lexicographic over the input, which is
// what makes the output assertable, unlike an arbitrary emission
// order. The pair unmark and pop after the recursive call is the
// backtrack itself.
func Permutations(nums []int) [][]int {
  out := [][]int{}
  used := make([]bool, len(nums))
  path := make([]int, 0, len(nums))
  var walk func()
  walk = func() {
    if len(path) == len(nums) {
      own := make([]int, len(path))
      copy(own, path)
      out = append(out, own)
      return
    }
    for i, n := range nums {
      if used[i] {
        continue
      }
      used[i] = true
      path = append(path, n)
      walk()
      path = path[:len(path)-1]
      used[i] = false
    }
  }
  walk()
  return out
}
```

listing 120: pick, recurse, unmark and pop (source: interview-repertoire/samples/ch23-go/backtrack.go, lines 26-57)

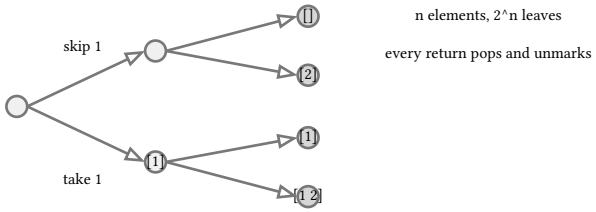


figure 106: two branches per element, the undo runs after each return

The count tests demand exactly 2^n subsets and n! orderings, all distinct, empty input included, which kills a walk that emits the empty set twice or drops it. Heap's single swap walk, book 17, ch 5 (the interview repertoire, classics part 1: warmup, arrays, hashing), is the cheaper generator when order does not matter, the memoized cousin is the dp ladder in book 17, ch 6 (the interview repertoire, classics part 2: dynamic programming and tries).

23.6. bit manipulation: single number, counting bits [TDD]

Xor is its own inverse and commutative, so folding a slice where every value appears twice except one cancels the pairs in any order and leaves the loner, one pass, no map, negatives included since two's complement xor is still bitwise. The popcount ladder fills the table in one linear pass: count[i] is the count of its parent i >> 1 plus the dropped last bit, so no row loops over bits:

```go
// SingleNumber isolates the value appearing once in a slice where
// every other value appears exactly twice. Xor is its own inverse
// and commutative, so the fold cancels each pair to zero whatever
// the order and keeps the loner. Negatives work unchanged, two's
// complement xor is still bitwise.
func SingleNumber(nums []int) int {
  acc := 0
  for _, n := range nums {
    acc ^= n
  }
  return acc
}
```

listing 121: the xor fold, pairs cancel, the loner survives (source: interview-repertoire/samples/ch23-go/bits.go, lines 3-14)

```go
// CountBits builds the popcount table from 0 to n inclusive, the
// ladder count[i] = count[i >> 1] + (i & 1): each row is its parent
// with the dropped last bit, so the table fills in one linear pass
// with no per-element bit loop. n must be 0 or more, the empty
// table for n = 0 is the single zero row.
func CountBits(n int) []int {
  counts := make([]int, n+1)
  for i := 1; i <= n; i++ {
    counts[i] = counts[i>>1] + (i & 1)
  }
  return counts
}
```

listing 122: count[i] = count[i >> 1] + (i & 1), parent first (source: interview-repertoire/samples/ch23-go/bits.go, lines 16-27)

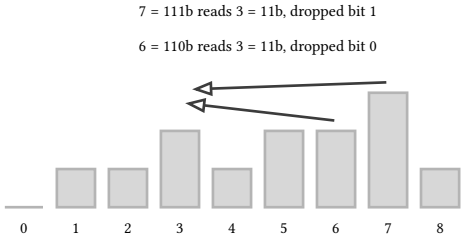


figure 107: the ladder's row is its parent plus the bit it dropped

The ladder test cross-checks every row against `math/bits` `OnesCount` from 0 to 16, two implementations of one function so neither can drift. The bit family, masks, clears, and the xor swap's honest limits, is floored by book 10, ch 16 (practical data structures and algorithms, number theory and modular arithmetic).

23.7. sorting internals, which sort when [DRILL]

The spoken answer, three sentences. Go's `sort` and `slices` packages run pdqsort, pattern defeating quicksort: it detects adversarial inputs, falls to insertion sort on small runs and heapsort when the recursion deepens, so average and worst case both sit at O(n log n). Comparison sorting cannot beat n log n, the decision tree has n! leaves and a log n! depth, so the answer is the library unless the keys carry structure: counting sort at O(n + k) and radix sort at O(n k) win when keys are small or fixed width, bucketing by value or digit instead of comparing. Mergesort stays the stability answer, Go's slice sort is not stable, `slices.SortStableFunc` is.

The follow-up to expect: why not always radix. Because k grows with the key width, a 64 bit key costs eight passes plus the bucket array, and pdqsort wins on comparables until n is large and the keys dense. The floor treatment is book 10, ch 13 (practical data structures and algorithms, sorting), and the Go surface to name, `slices.SortFunc` with `cmp.Compare` as this chapter's greedy does, is catalogued by book 11, ch 4 (the icpc world finals, the go toolbox).

family	cost	the honest when
pdqsort, the stdlib	n log n	comparables, no adversarial input
counting sort	n + k	small dense integer keys, k under log n
radix sort	n k	fixed width keys, large n, keys dense
stable mergesort	n log n	equal keys must keep input order

figure 108: the decision is structure first, comparisons only when the keys are opaque

sources: verified by `go vet`, `go test`, and `go test -race` in `samples/ch23-go`, 13 Go tests green, workspace `ch23-go`.

24. weighted graphs: dijkstra, union-find, mst

Weights are the second graph round. The traversals of book 17, ch 4 (the interview repertoire, graphs: dfs, bfs, topological sort) answered reachability and edge-count distance, and once edges carry costs the questions become cheapest path and cheapest connection. The answers here are dijkstra over a heap frontier, union-find as the cycle oracle, and the kruskal and prim pair over it. All of it runs in the `ch24-go` module under `make verify`, 12 tests, stdlib only, `container/heap` carrying both frontiers. The heap implemented from zero sits in book 17, ch 3 (the interview repertoire, data structures, implemented), and the depth floors to book 10, ch 11 (practical data structures and algorithms, shortest paths) and book 10, ch 12 (practical data structures and algorithms, minimum spanning trees and flows).

24.1. dijkstra, heap-driven [TDD]

Dijkstra is bfs with a relax loop. Each pop takes the cheapest candidate distance and offers every neighbor `popped + w`, the distance map doubles as the finalized set, and a node absent from the map is at infinity:

```go
// Dijkstra returns the distance map and the parent map from start.
// A node absent from dist is unreachable, absence is the infinity.
// The heap is lazily deleted: every relaxation pushes a new entry and
// the stale ones are skipped on pop, which buys decrease-key behavior
// without an indexed heap.
func Dijkstra(g *Graph, start int) (map[int]int, map[int]int) {
  dist := map[int]int{start: 0}
  parent := map[int]int{start: start}
  f := &frontier{{dist: 0, node: start}}
  for f.Len() > 0 {
    cur := heap.Pop(f).(hop)
    if cur.dist > dist[cur.node] {
      continue // stale entry, a shorter candidate was popped first
    }
    for _, e := range g.Neighbors(cur.node) {
      newDist := cur.dist + e.W
      if old, ok := dist[e.To]; !ok || newDist < old {
        dist[e.To] = newDist
        parent[e.To] = cur.node
        heap.Push(f, hop{dist: newDist, node: e.To})
      }
    }
  }
  return dist, parent
}

// ShortestPath reconstructs the minimum-weight path from start to
// target by walking the parents home, false when target is
// unreachable. Equal-cost ties keep the first parent found, so with
// neighbors sorted by id the returned path is the cheapest, then
// lowest-id, route.
func ShortestPath(g *Graph, start, target int) ([]int, bool) {
  dist, parent := Dijkstra(g, start)
  if _, ok := dist[target]; !ok {
    return nil, false
  }
  path := []int{target}
  for cur := target; cur ≠ start; {
    cur = parent[cur]
    path = append([]int{cur}, path...)
  }
  return path, true
}
```

listing 123: the distance map, the lazily-deleted frontier, the relax loop, the parent walk home (source: interview-repertoire/samples/ch24-go/dijkstra.go, lines 31-73)

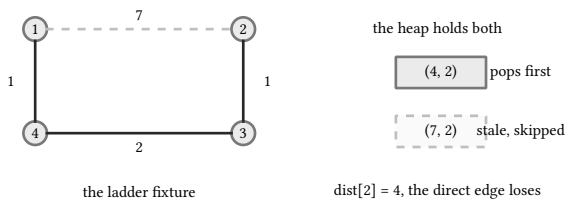


figure 109: one node, two entries: the shorter candidate wins the pop, the longer surfaces later and is skipped

The subtlety worth narrating is the stale entry. Go's `container/heap` has no decrease-key, so every relaxation pushes a new entry and the shorter one wins the pop, and when a longer candidate surfaces later the guard `cur.dist > dist[cur.node]` skips it. That is lazy deletion, and it is why the heap carries at most one entry per relaxation. The tie

rule is pinned too: relaxation is strict, neighbors are sorted by id, so equal-cost routes keep the lowest-id parent, and the square fixture of `TestDijkstraTieKeepsFirstParent` returns `[1 2 4]`, never `[1 3 4]`.

The non-negative rule is enforced, and the reason to say out loud is the pop itself: finalizing a node is the belief that nothing cheaper can arrive later, and a negative edge breaks the belief, the node's distance keeps improving after it left the frontier and the answer goes quietly wrong. The corpus makes it loud instead, `AddEdge` panics on a negative weight and `TestNegativeWeightPanicsOnAdd` pins it. What to run instead is the bellman-ford question, drilled at the bottom of this chapter.

NOTE: say the complexity with the heap named

One pop per surviving entry and one push per relaxation puts the loop at O(E log E), and since E stays under V^2 the exponent sits within a constant of log V, which is why the spoken O(E log V) is honest. The space follow-up is the distance and parent maps, both vertex-counted, the heap adds at most one entry per relaxation.

24.2. union-find with path compression [TDD]

The mst half rides on one small structure. Union-find tracks disjoint sets, `Find` returns the root of a set, and `Union` merges two sets and reports false when both endpoints already share a root, which is exactly the cycle test an undirected edge needs:

```
// Find returns the set root and compresses as it goes, two passes:
// one to locate the root, one to re-parent every node on the walked
// path directly onto it.
func (uf *UnionFind) Find(x int) int {
  root := x
  for uf.parent[root] ≠ root {
    root = uf.parent[root]
  }
  for uf.parent[x] ≠ root {
    uf.parent[x], x = root, uf.parent[x]
  }
  return root
}

// Union merges the two sets, attaching the lower-rank root under the
// higher and raising the rank only on a tie. It returns false when
// both endpoints already share a root, the cycle test Kruskal rides.
func (uf *UnionFind) Union(a, b int) bool {
  ra, rb := uf.Find(a), uf.Find(b)
  if ra == rb {
    return false
  }
  if uf.rank[ra] < uf.rank[rb] {
    ra, rb = rb, ra
  }
  uf.parent[rb] = ra
  if uf.rank[ra] == uf.rank[rb] {
    uf.rank[ra]++
  }
  return true
}
```

listing 124: two-pass find that re-parents the walked path, union by rank, the cycle refusal (source: interview-repertoire/samples/ch24-go/uf.go, lines 20-50)

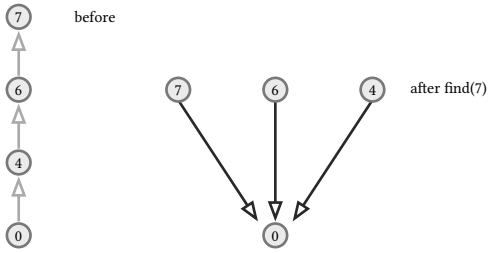


figure 110: find(7) walks three hops, then re-parents the whole path onto the root

`TestFindCompressesThePathItWalked` is white-box on purpose: it pairs equal-rank trees until node 7 hangs three hops off the root, calls `Find(7)` once, then asserts `parent[7]` and `parent[6]` both hold the root directly. Without compression those slots keep the old chain. `TestUnionByRankKeepsTheBiggerRoot` pins the attach direction, a rank-2 root absorbs a rank-1 tree with no rank raise, so trees stay shallow and the finds stay short. The refusal is the interface kruskal consumes, `TestUnionRefusesTheCycle` checks every shape of it, the repeat, the transitive repeat, the closing edge.

24.3. kruskal and prim, the mst pair [TDD]

Kruskal is a sorted edge list plus the cycle oracle: consume edges cheapest first, take each one whose endpoints union, skip the rest. The skip is the algorithm, an edge whose endpoints already share a root closes a cycle and can never sit in a tree:

```go
// Kruskal builds the minimum spanning tree by consuming edges sorted
// by weight and taking each one that joins two components of a
// union-find. ok is false when the graph is disconnected, no spanning
// tree exists, and the returned edges are the minimum forest instead.
func Kruskal(g *Graph) (tree []Edge, total int, ok bool) {
  ids := g.Nodes()
  idx := map[int]int{}
  for i, id := range ids {
    idx[id] = i
  }
  uf := NewUnionFind(len(ids))
  for _, e := range g.Edges() {
    if uf.Union(idx[e.From], idx[e.To]) {
      tree = append(tree, e)
      total += e.W
    }
  }
  return tree, total, len(tree) == len(ids)-1
}
```

listing 125: sorted edges consumed cheapest first, union as the take-or-skip test (source: interview-repertoire/samples/ch24-go/mst.go, lines 5-23)

Prim grows from a start node instead. The cut between the tree and the unseen nodes is a heap of crossing edges, pop the cheapest, add it, push its endpoint's edges, and the same lazy deletion runs here, a pop whose endpoint joined meanwhile is skipped:

```go
// Prim grows the tree from start: push every edge from the tree to an
// unseen node, pop the cheapest, skip pops whose endpoint joined the
// tree meanwhile, the same lazy deletion dijkstra uses. ok is false
// when start cannot reach every node.
func Prim(g *Graph, start int) (tree []Edge, total int, ok bool) {
  seen := map[int]bool{start: true}
  cut := &cutheap{}
  push := func(from int) {
    for _, e := range g.Neighbors(from) {
      if !seen[e.To] {
        heap.Push(cut, crossing{w: e.W, from: from, to: e.To})
      }
    }
  }
  push(start)
  for cut.Len() > 0 && len(tree) < len(g.Nodes())-1 {
    e := heap.Pop(cut).(crossing)
    if seen[e.to] {
      continue // stale entry, a cheaper crossing reached it first
    }
    seen[e.to] = true
    tree = append(tree, Edge{From: e.from, To: e.to, W: e.w})
    total += e.w
    push(e.to)
  }
  return tree, total, len(tree) == len(g.Nodes())-1
}
```

listing 126: the cut heap, pop the cheapest crossing, skip the stale pops (source: interview-repertoire/samples/ch24-go/mst.go, lines 45-71)

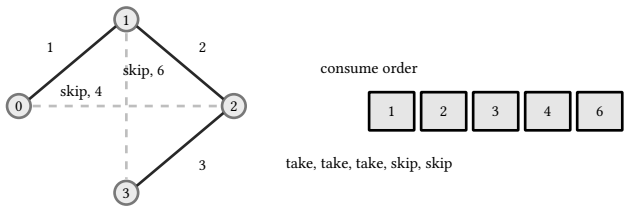


figure 111: the box fixture: the 1, 2, and 3 edges span, the 4 and the 6 would close cycles

When each wins is a representation question. Kruskal wants the edge list, sort once and stream, so it wins on sparse graphs and when edges arrive pre-sorted. Prim wants adjacency and a start node, so it wins on dense graphs and when the answer must hang off one component. The suite proves them interchangeable on totals, `TestPrimMatchesKruskalAndReportsDisconnection` asserts both return 6 on the box fixture, and both carry the same disconnection verdict.

PITFALL: a disconnected graph ships a forest unless you check

Both functions answer ok false when no spanning tree exists: kruskal returns the minimum forest across the components, prim from a start node returns the tree of the component it can reach. A caller that ignores the third return ships a forest labeled as a tree, and the disconnected fixtures in both test functions exist to keep that failure loud.

24.4. when bellman-ford and floyd-warshall [DRILL]

The spoken answer, one breath: dijkstra is single source over non-negative weights with a heap, O(E log V). Bellman-ford is single source with negative edges allowed, V-1 rounds of relaxing every edge, O(V E), and a V-th round that still improves something proves a negative cycle, so no shortest paths exist. Floyd-warshall is all pairs, dynamic programming over a V by V matrix, O(V^3), negative edges fine, negative cycles visible as a negative diagonal. The corpus implements both in the handbook, book 10, ch 11 (practical data structures and algorithms, shortest paths), not here: the loop asks for dijkstra and the mst pair as code and these two as a precise when, and this book keeps that split, the same division book 10, ch 10 (practical data structures and algorithms, graphs) draws for the unweighted family.

	scope	contract, cost
dijkstra	one source	non-negative, heap, O(E log V)
bellman-ford	one source	negatives ok, V-1 rounds, O(V E)
floyd-warshall	every pair	matrix dp, O(V^3), small V

a V-th improving round is the negative-cycle detector

figure 112: the when table, said before any code: scope and contract pick the algorithm

The follow-up to expect: why not always run floyd-warshall, one answer for every pair. Because V^3 is brutal past a few hundred nodes and the single-source question asks for a fraction of the work, one heap pass. The reverse follow-up: why does dijkstra fail on negative edges. The pop finalizes, finalization is a belief, and the belief only holds when every step forward adds cost, which is the same sentence as the non-negative contract this chapter enforces in `AddEdge`.

sources: verified by `go test` through `make verify`, 12 tests in `ch24-go`. Depth on the shortest-path family floors to book 10, ch 11 (practical data structures and algorithms, shortest paths), the mst and flows depth to book 10, ch 12 (practical data structures and algorithms, minimum spanning trees and flows), and the unweighted traversals this chapter builds on stay in book 17, ch 4 (the interview repertoire, graphs: dfs, bfs, topological sort).

25. distributed systems answers

The distributed round rewards the structure of the design round plus one move the other rounds do not test, the honest concession. Every answer floors to the patterns handbook, whose raft, ring, and messaging chapters carry the gated implementations, and the drills are written to survive “have you actually run this”, because one of them has to answer no out loud.

25.1. replication: leaders, followers, and lag [DRILL]

Single-leader replication is one node accepting writes and every other node applying the same log in order, so the leader holds the newest state and a follower read is a staleness decision, not a free win. Lag is the leader’s index minus the follower’s, one number, the per-follower matchIndex a raft leader keeps, and it is a budget, not a bug: leader reads pay the queue, follower reads accept staleness, read-your-writes pins the session to the node that took the write. Promotion is an election, not a switch: a follower that stops hearing heartbeats past a randomized timeout runs for leader, needs f plus 1 votes out of 2f plus 1 nodes, and the election restriction refuses the vote to a log that is behind, so a stale replica cannot win. What promotion costs, volunteered: the unreplicated tail is lost and survivors truncate diverged suffixes, the repair walk book 12, ch 11 (design patterns, concurrency, and distributed systems, replication and consensus: raft) implements. The follow-up is bounding the lag, answered with an alert on the gap and a read path that names its staleness contract.

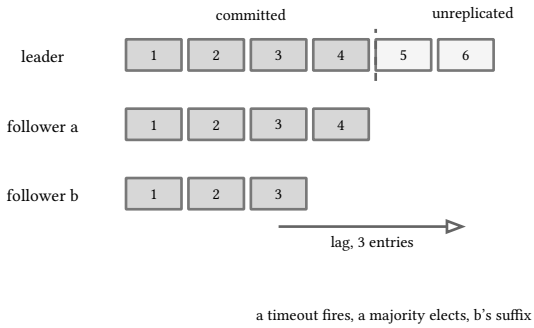


figure 113: the leader’s log, two followers behind it, the gap is the lag

25.2. sharding and partitioning strategies [DRILL]

Range partitioning hands each node a contiguous slice of the key space: range scans stay cheap, and load concentrates where keys cluster, the timestamp table whose newest range is one machine’s problem on every insert. Hash partitioning hashes the key first, spreads the space evenly, and gives up the scan, so the choice is locality against uniformity. Hot keys survive both, and the fixes are the triplet: a cache in front, salt on the one key, split the one burning range. Rebalancing is where the blast radius lives: key mod n moves almost every key the moment the modulus changes, while the ring with virtual nodes moves roughly 1/(n+1) on a join, all of it to the new node and none between survivors, book 12, ch 12 (design patterns, concurrency, and distributed systems, partitioning and consistent hashing). Both placements are valid mid-move, so a rebalance ships as backfill plus cutover behind a flag, and a botched one is an outage wearing a maintenance window. The follow-up is the hot key, and the triplet answers it.

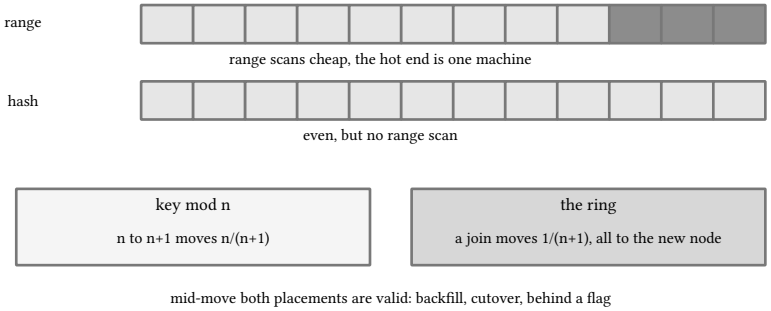


figure 114: range concentrates the hot end, hash spreads evenly, the join sets the blast radius

25.3. consensus and raft, honestly [DRILL]

Raft guarantees two things: at most one leader per term, each server voting once per term and persisting that vote, and a committed entry is never overwritten, commitment needing a majority of the current term, any two majorities of 2f plus 1 nodes sharing a node. The prior-term restriction, an old entry committing only when a current-term entry above it does, is the rule people forget. Then the concession, before anyone asks it: the corpus has no production raft. What exists is the patterns handbook's capstone, elections, log repair, the commit rules, persistence across restarts, over an in-memory channel transport with a drop predicate standing in for the network, book 12, ch 16 (design patterns, concurrency, and distributed systems, capstone: raft replicated configuration service). So "have you run it" answers no, not in production, and here is exactly where the pieces have run: the election restriction, the repair walk, and the commit counting are pure functions with pinned fixtures, book 12, ch 11 (design patterns, concurrency, and distributed systems, replication and consensus: raft), and the capstone wires them into nodes and kills the leader in its tests. Production adds what I would read first, real rpcs, snapshots, joint consensus, and naming the boundary beats claiming the stack. The follow-up is usually paxos, answered with raft's design point, decomposability, one head.

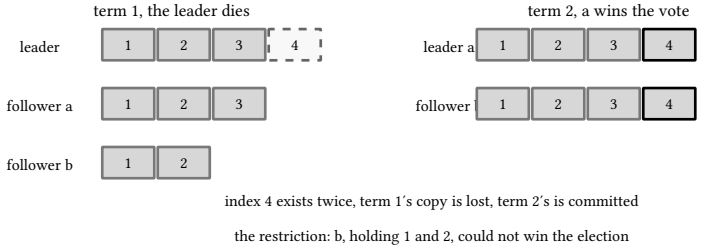


figure 115: one leader per term, the committed prefix survives the change, the tail does not

25.4. crdts versus distributed transactions [DRILL]

A crdt is a data type whose merge is associative, commutative, and idempotent, so replicas take writes in any order, take them twice, still converge, counters, sets, registers with a merge rule. What it survives is the concurrency the vector clock names, two writes with no causal order, and last-writer-wins by wall clock is the one merge rule to refuse, book 12, ch 10 (design patterns, concurrency, and distributed systems, distributed fundamentals). A distributed transaction is the opposite bet: a coordinator prepares every participant, locks are held, one round commits or aborts all, atomicity across shards bought with the round trips and the in-doubt window, a coordinator dying after prepare leaving everyone holding locks and guessing. The trade, said out loud: a crdt write is one local write, available through a partition, a two-phase commit pays the round trips, holds the locks, blocks on the coordinator. Crdt for mergeable state, counts, carts, presence, transactions for money or nobody's, the saga the pragmatic middle, compensations orchestrated the way the design chapter draws it, book 17, ch 19 (the interview repertoire, system design answers). Say the distinction first: raft is not two-phase commit, one log replicated to identical replicas against one commit across different participants. The follow-up is the failure question, answered with the window, the locks, and who is allowed to guess.

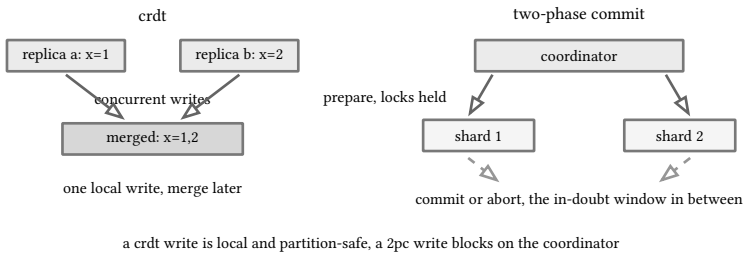


figure 116: merge converges with no coordinator, prepare and commit hold locks across round trips

25.5. delivery semantics and idempotency [DRILL]

At-most-once is fire and forget, right for telemetry where a lost sample is cheaper than a retry. At-least-once retries until acknowledged and duplicates, the sender unable to tell a delivery that never arrived from an ack that drowned. Exactly-once delivery does not exist on a real network, what the phrase names is exactly-once effect, at-least-once into a consumer that remembers what it applied, book 12, ch 13 (design patterns, concurrency, and distributed systems, messaging semantics and idempotency). The idempotency key is the design: minted once by the sender, stable across retries, never derived from the arrival, a fresh key per retry deduping nothing. The outbox closes the dual write: state change and message in one database transaction, a relay draining afterward, so a crash between saved and sent loses nothing. The running code behind the answer: the durable consumer of the streaming chapter binds by name, tracks its ack floor, naks failures back for redelivery, and after a broker restart receives exactly the unacked tail, book 17, ch 22 (the interview repertoire, go streaming: jetstream batch pipelines), the same contract book 13, ch 15 (infrastructure: docker, databases, and testing, jetstream: streams, consumers, kv, kafka compared) runs against a real broker. The follow-up is the double delivery, answered with the applied set, one membership test, the effect once.

PITFALL: exactly-once is an effect, not a delivery setting

No transport delivers exactly once across a real network, because the acknowledgment itself can be lost after the effect happened. What a broker's exactly-once means is at-least-once delivery into a consumer that deduplicates on a stable idempotency key, so the claim to defend in the interview is the dedupe, never the wire.

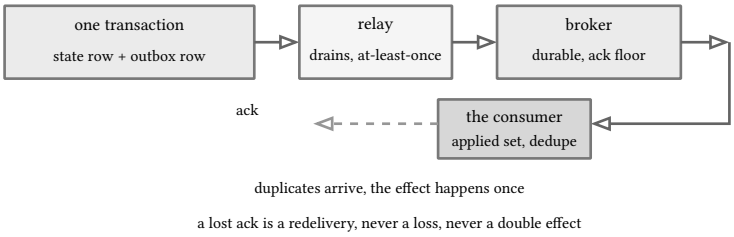


figure 117: at-least-once into an idempotent consumer, the outbox closing the dual write

sources: no measurements in this chapter, every number is the textbook value, quorums as 2f+1 and movement as 1/(n+1), and every drill floors to the patterns handbook, book 12, ch 10 (design patterns, concurrency, and distributed systems, distributed fundamentals) for quorum arithmetic and concurrency, book 12, ch 11 (design patterns, concurrency, and distributed systems, replication and consensus: raft) for the election and commit rules, book 12, ch 12 (design patterns, concurrency, and distributed systems, partitioning and consistent hashing) for the ring, book 12, ch 13 (design patterns, concurrency, and distributed systems, messaging semantics and idempotency) for the dedup contract and the outbox, and book 12, ch 16 (design patterns, concurrency, and distributed systems, capstone: raft replicated configuration service) for the raft that actually runs.

26. systems and runtime answers

The runtime round asks what the machine is doing under the code: where the bytes live, who translates an address, what a context switch pays, and how one thread talks to ten thousand sockets. The honest version of every answer here is the one you can say in three sentences and then defend, and the floor is the c manual, whose chapters run the real allocators, the real page tables, and a real socket server under gates. No new code rides with this chapter: it is the spoken layer, answer, follow-up, floor.

26.1. stack, heap, and allocators [DRILL]

What lives where: locals, arguments, and return addresses live on the stack, one frame per call, and the frame dies with the return. Anything whose lifetime must outlive the call that made it goes to the heap, and the heap knows nothing about calls.

Why the stack is fast and bounded: pushing a frame moves one pointer, the return frees it by moving the pointer back, the frames sit contiguous so the hot edge stays in cache, and there is nothing to search. The price of the speed is the bound, fixed at thread creation, and recursion past that bound is the stack overflow, the error to name before the interviewer does.

What the allocator does with a `malloc`: it walks its bins or free lists for a block that fits, splits off the loose tail, stamps the size and state into the block header, and returns a pointer to the interior. A `free` writes the block back into a bin and coalesces it with free neighbors. The follow-up is fragmentation, and it has two faces: external, where the free bytes exist but no contiguous run is large enough for the next request, and internal, where size-class rounding leaves slack inside every returned block. Both faces are the argument for arenas and pools, and the gated implementations floor to book 1, ch 14 (c23: the language, the machine, and the cloud, the heap and allocators) and book 1, ch 19 (c23: the language, the machine, and the cloud, allocator designs).

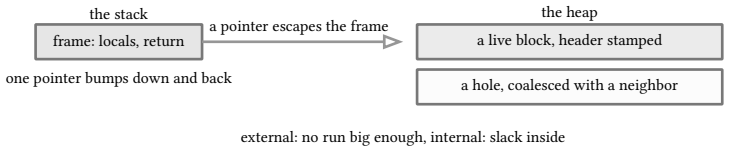


figure 118: the stack bumps and dies with the return, the heap waits for the free

26.2. virtual memory, pages, and the tlb [DRILL]

Every address a program touches is virtual, and the mmu translates it page by page, the textbook 4kb split of an address into a page number and an offset that never gets translated. The tlb is the cache of translations: a hit answers with no walk at all, a miss walks the page tables in memory, and that walk is the cost the tlb exists to skip, which is why locality of access pays twice, once in the cache and once in the tlb.

A page fault is not an error, and saying so plainly is the point of the question. The fault says the page is mapped but not resident: the kernel reads it in from the file or the swap, maps the zero page for fresh anonymous memory, or resolves a copy-on-write, then restarts the instruction that faulted, and the program never knows it happened. The error case is the fault on an address with no valid mapping behind it, that is the segfault. The follow-up to expect is copy-on-write: a `fork` shares the parent's frames read-only, and the write fault is what finally copies a frame, which is why fork can be cheap and the first write after it is not. The gated implementation floors to book 1, ch 12 (c23: the language, the machine, and the cloud, virtual memory and paging).

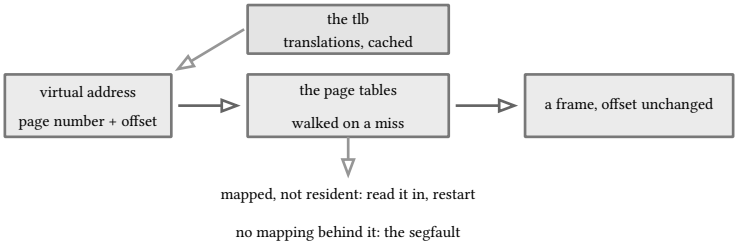


figure 119: a page number through the tables to a frame, the tlb skipping the walk

26.3. processes versus threads [DRILL]

A process owns an address space, the page tables behind it, and a file descriptor table, and the isolation is the point: one process crashing leaves its neighbors standing. A thread owns a stack and a register set and shares everything else with its siblings, the heap, the code, the descriptors, which is why threads are cheap to talk through and cheap to corrupt, and why locks exist at all.

What a scheduler actually switches: the general registers, the program counter, the stack pointer, and the floating point state, and that is the whole of a thread switch. A switch between processes additionally swaps the page tables, and that swap taxes the tlb, which is why a process switch costs more, while two threads of one process need no swap because they already share the tables. State the trade and stop: threads buy cheaper switches and pay in races, processes buy isolation and pay the page-table swap, and the everyday example of the second is the browser keeping each tab in its own process. The gated chapters are book 1, ch 11 (c23: the language, the machine, and the cloud, processes and program lifecycle), book 1, ch 15 (c23: the language, the machine, and the cloud, threads and synchronization), and book 1, ch 17 (c23: the language, the machine, and the cloud, cpu scheduling algorithms).

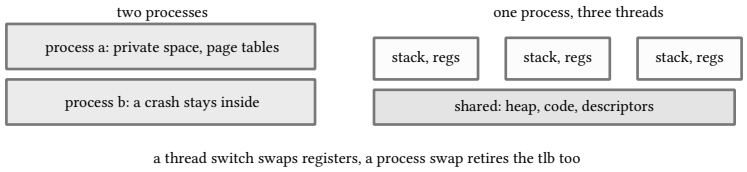


figure 120: two private address spaces beside one space with three stacks in it

26.4. epoll and the event loop [DRILL]

The readiness model: instead of parking one thread per socket, register every descriptor with the kernel once, then wait on the ready list. `epoll_wait` returns exactly the descriptors that can be read or written now, and the loop services those and waits again, so ten thousand open connections ride one thread and one stack.

Level versus edge triggered, the fork the follow-up probes: level reports a descriptor ready for as long as data remains, so a partial read is fine, the next wait reports it again. Edge reports the state change once, so the handler must drain until the read comes back empty or the remainder sits unread behind a notification already spent.

The contrast to draw on purpose is the javascript loop of book 17, ch 7 (the interview repertoire, javascript core: scope, closures, event loop), the same single-threaded discipline at a different layer. There the queue is the runtime's own, one call stack, the microtask queue drained to empty after every macrotask turn, timers waiting for a macrotask slot, and it schedules language jobs. Epoll schedules descriptors, and node's loop is the second wearing the first's clothes, the poll phase feeding the queues the language drains. The go answer hides the loop entirely, the netpoller parking a goroutine on the socket and the scheduler waking it, which is why book 17, ch 15 (the interview repertoire, go runtime: scheduler, gc, patterns) hands you blocking reads and still runs thousands of goroutines. The loop itself is gated in book 1, ch 18 (c23: the language, the machine, and the cloud, asynchronous patterns in c).

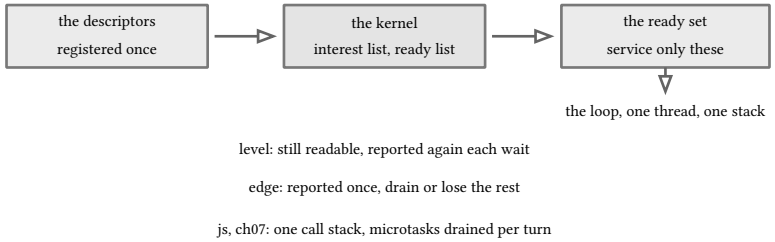


figure 121: descriptors registered once, the ready list returned, the loop drains it

PITFALL: the edge-triggered drain rule

Under edge-triggered epoll a descriptor reports its state change once. Read part of the buffer, return to the wait, and the rest of that buffer never reports again: no new state change occurs, so the connection hangs with bytes waiting in the kernel. The rule is to loop the read until it returns short or empty, and the same discipline covers writes, which block whenever the socket's send buffer fills. This is the lost-wakeup class, one notification spent on work only partially done.

26.5. c networking, sockets end to end [DRILL]

The ladder in call order: `socket` creates the endpoint and returns a descriptor, `bind` pins the local address and port, `listen` marks the endpoint passive and sizes the accept queue, `accept` pops the next completed connection as a new descriptor, and `connect` is the client's half of the same handshake. The descriptor `accept` returns is not the listener, it is one connection's private endpoint, and conflating the two is the first bug this round checks for.

What blocking means: a read on a socket with no data yet parks the calling thread until data arrives, so the naive server shape is one thread per connection, each thread mostly asleep while holding a whole stack. Scale the connections and the stacks and the scheduler sweeping them are the wall, long before the cpu is. The readiness loop breaks the shape: register the listener and every accepted socket, one `epoll_wait` returns the ready set, and the single loop accepts, reads, and writes only what is ready, so one stack serves every connection. Two gotchas worth volunteering before being asked: the send side blocks on a full socket buffer exactly as the receive side blocks on an empty one, and every descriptor `accept` returns must join the interest set before the next wait or its first bytes sit unwatched. The c manual runs both shapes under gates, book 1, ch 28 (c23: the language, the machine, and the cloud, http kernel) with a fixed thread pool and a stated reason for skipping the loop, and the event loop itself in book 1, ch 18 (c23: the language, the machine, and the cloud, asynchronous patterns in c).

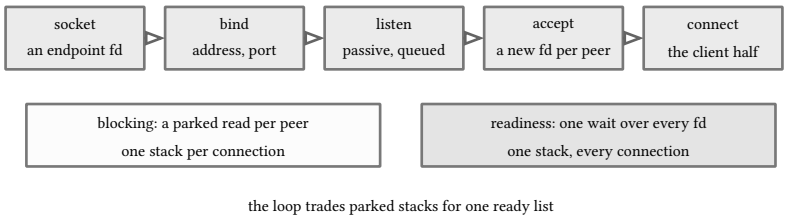


figure 122: socket, bind, listen, accept, connect, then the two shapes over the connections

floored to the gated implementations of the c manual: book 1, ch 14 (c23: the language, the machine, and the cloud, the heap and allocators) and book 1, ch 19 (c23: the language, the machine, and the cloud, allocator designs) for the stack and heap answers, book 1, ch 12 (c23: the language, the machine, and the cloud, virtual memory and paging) for paging and the tlb, book 1, ch 11 (c23: the language, the machine, and the cloud, processes and program lifecycle), book 1, ch 15 (c23: the language, the machine, and the cloud, threads and synchronization), and book 1, ch 17 (c23: the language, the machine, and the cloud, cpu scheduling algorithms) for the switch, and book 1, ch 18 (c23: the language, the machine, and the cloud, asynchronous patterns in c) with book 1, ch 28 (c23: the language, the machine, and the cloud, http kernel) for the loop and the sockets, all under `make verify`.

27. mathematics answers

The math round is computable or it is nothing: the questions below are answered with arithmetic you can do at the board, and every drill here is that arithmetic said out loud, the claim first, the computation second, the follow-up named before it arrives. The floors are the math handbook, proof, combinatorics, probability, statistics, plus the dsa analysis chapter for reading a cost claim. Drill numbers are textbook values, the workspace's measured.

27.1. asymptotics and solving recurrences [DRILL]

Big-O is a claim about growth, not about speed: $f(n) = O(g(n))$ says there exist witnesses $c > 0$ and n_0 with $f(n) \le cg(n)$ for every $n \ge n_0$, an upper bound on the rate the work grows at, carrying no constants, no cache effects, no lower-order terms. Big-Omega is the same claim from below, big-Theta both directions, and the honest answer says theta when it can prove both. The master theorem in one breath: for $T(n) = aT(n/b) + f(n)$ with $a \ge 1$ and $b > 1$, compare the root's work $f(n)$ against the leaves' work $n^{\log_b a}$. Leaves win by a polynomial and the answer is $\Theta\big(n^{\log_b a}\big)$, root wins by a polynomial with the regularity condition holding and it is $\Theta(f(n))$, tied within a log factor and the log multiplies in, $\Theta\big(n^{\log_b a} \log n\big)$. Mergesort, $2T(n/2) + n$, is the tie case at $n \log n$, and binary search and exponentiation by squaring, $T(n/2) + O(1)$ on the halved exponent, land at $\Theta(\log n)$, the shape the modular exponentiation loop actually runs, book 10, ch 16 (practical data structures and algorithms, number theory and modular arithmetic), and Karatsuba's $3T(n/2) + n$ is the classic $n^{\log_2 3}$, about $n^{1.585}$. The unclean cases, uneven splits like $T(n) = T(n/2) + T(n/3) + n$ or root work of $n \log n$, do not fit the theorem, and the recursion tree does: draw the splits, sum each level, count the levels, and the tree is the proof the theorem only shortcuts. The follow-up is whether $n \log n$ beats n^2 in practice, answered from the definition: past some n_0 it wins, below it the quadratic insertion sort's constants win, and reading a cost claim against a benchmark honestly is a chapter of its own, book 10, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking).

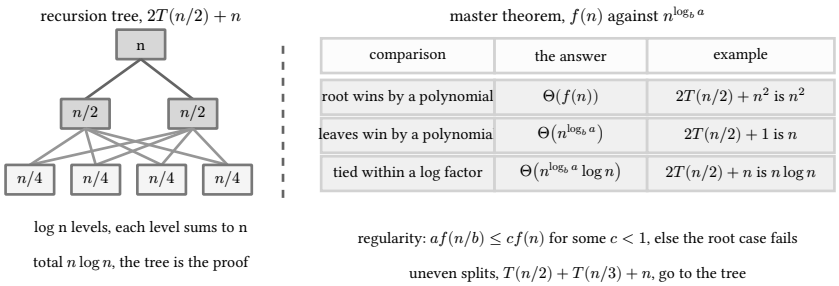


figure 123: the recursion tree sums level by level, the theorem only compares the root against the leaves

27.2. probability you can compute at the board [DRILL]

Start from the sample space: with equally likely outcomes the probability is favorable over total, and board questions die on the total, not the favorable. The complement is the first move whenever the words "at least one" appear, because a union of many events is messy and "none" is one intersection: P(at least one) is 1 minus P(none). The birthday staple: the chance that two of n people share a birthday is $1 - \prod_{i=1}^{n-1}(365 - i)/365$, about 0.12 at 10 people, 0.51 at 23, 0.97 at 50, the coin flip sitting at 23 people, and the same arithmetic prices hash collisions, a 64-bit space reaching a coin-flip collision near 2^{32} keys. Expectations are linearity first: $E[X + Y] = E[X] + E[Y]$ always, with independence nowhere required, so write the quantity as a sum of indicators and add. The indicator sum answers the fixed points of a random permutation: each of the n positions is fixed with probability $1/n$, so the expected count is $n \cdot 1/n = 1$, for every n, the derangement ladder underneath being the combinatorics chapter's, book 2, ch 14 (mathematics for programmers, combinatorics). The coupon collector is the other staple: n coupon types, each draw uniform, the expected draws to see them all is nH_n over the harmonic number H_n, about $n \ln n + 0.577n$, a 52-card deck costing about 236 draws. The follow-up is the variance of a sum, answered honestly: variances add for independent terms only, $V(X + Y) = V(X) + V(Y)$, covariance otherwise, and the

conditioning machinery is the probability chapter's, book 2, ch 18 (mathematics for programmers, probability and distributions).

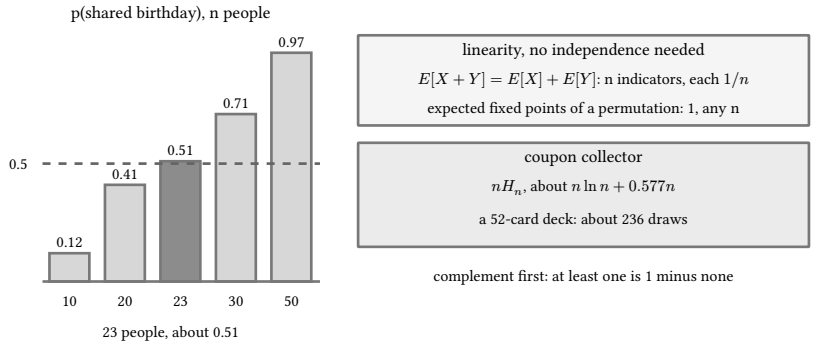


figure 124: the birthday ladder crosses one half at 23 people, linearity turns sums into additions

27.3. counting and combinatorics [DRILL]

Two rules carry everything: stages in sequence multiply, disjoint alternatives add, and every counting answer is one of those two or the repair of a miscount. Permutations against combinations: drawing k from n where order matters gives $n(n-1)...(n-k+1)$ ordered tuples, $4 \times 3 = 12$ ordered pairs from four, and order irrelevant means the binomial coefficient $\binom{n}{k} = n!/(k!(n-k)!)$ divides out the $k!$ orderings that do not matter, $\binom{4}{2} = 6$ subsets. Stars and bars is the identical-items move: k identical candies into n distinct bins is a string of k stars and $n-1$ bars, and choosing the bar positions among the $n+k-1$ slots gives $\binom{n+k-1}{k}$, four candies into three bins being $\binom{6}{2} = 15$. Inclusion-exclusion is the overcount repair: summing set sizes counts each overlap twice, so $|A \cup B| = |A| + |B| - |A \cap B|$, three sets subtract the pairs and add the triple back, and the derangement count $!n$ is its purest form, walking to $n!/e$ as n grows, every identity here pinned in exact arithmetic, book 2, ch 14 (mathematics for programmers, combinatorics). The pigeonhole principle is the zero-arithmetic bound, $n+1$ items into n boxes forces a shared box, the proof chapter's functions section, book 2, ch 13 (mathematics for programmers, logic, sets, and proof). The follow-up is add or multiply: stages multiply, alternatives add, and the miscount worth confessing is the one where the stages were not actually independent.

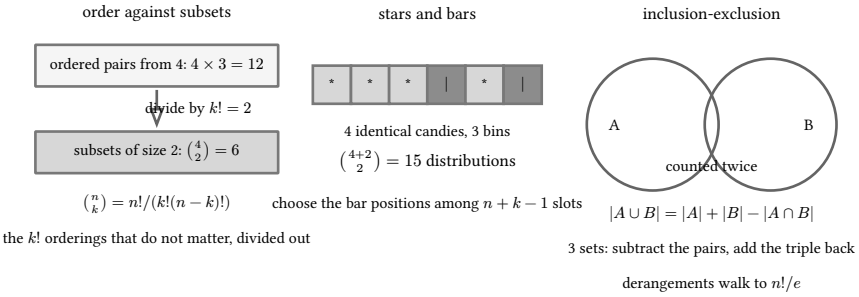


figure 125: order divides out, bars make bins identical, the overlap is added back once

27.4. reading statistics: baselines, p-values, overfitting [DRILL]

A baseline owns the claim because a raw score has no referent: 90 percent accuracy on a 90/10 class split is the majority-class baseline wearing a model, and the honest claim is the lift, model score minus trivial predictor, regression judged against predict-the-mean under squared error. Name the baseline, then the number, book 2, ch 19 (mathematics for programmers, statistics and inference). The p-value is the probability, under the null hypothesis, of an outcome at least as extreme as the one observed: a number about the test and the data, not the probability that the null is true, not one minus the probability the effect is real, and not alpha, which is the size $P(\text{reject} \mid H_0)$ chosen before the test, the power being $1-\beta$. A p under alpha rejects the null and confirms nothing else. Multiple testing is where naive reading breaks: 20 independent tests at $\alpha = 0.05$ give at least one false rejection with probability $1 - 0.95^{20}$, about 0.64, the complement trick from the probability section turned against

you, and the repair is a stricter per-test threshold, Bonferroni's alpha divided by the number of tests. Overfitting is memorization with a smooth face: training error falls as capacity grows while held-out error falls, turns, and rises, a degree $n - 1$ polynomial passing through any n points exactly, interpolation being the memorization limit, and the cure is capacity, or data, or both, the bias-variance framing, high bias underfitting and high variance overfitting, being the estimators chapter's vocabulary, book 2, ch 19 (mathematics for programmers, statistics and inference). The follow-up is how you would know, answered with a held-out set touched once, at the end.

PITFALL: the p-value is not the probability the null is true

The definition runs one way: it is $P(\text{data this extreme} \mid H_0)$, the probability of the outcome given the null, never $P(H_0 \mid \text{data})$, the probability of the null given the outcome. Swapping the conditionals is the most common statistics answer to get wrong at the board, and the swap moves the number in both directions.

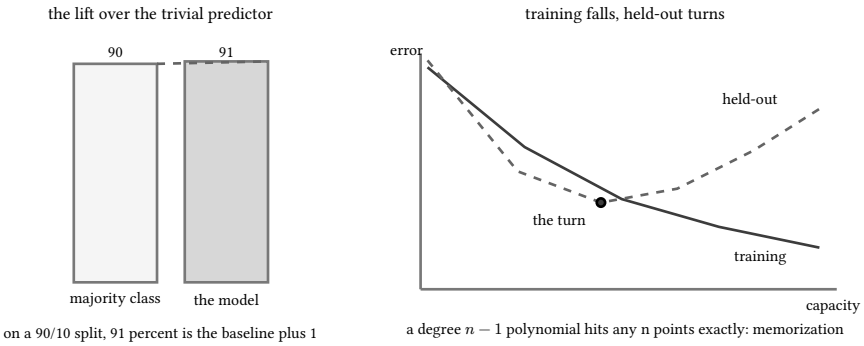


figure 126: the baseline owns the number, the gap opens where the fit memorizes

27.5. the statistics workspace: exact binomial, auc, bootstrap, verdicts [TDD]

The drill above reads statistics at the board; this section computes them, a workspace in `ch27-go` a candidate can speak from and run, 26 test functions under `make verify-go`, stdlib only, every number below measured off the suite's own output. Four answers, one file each. The exact two-sided binomial first, because exact is the claim worth defending: no normal approximation anywhere, the p-value summed over the cells themselves, and the arithmetic in log space so n in the thousands never forms a factorial that overflows or a product of tiny cells that underflows, the central cell of a 10k-flip fair coin coming back 0.0079786 where a product-of-cells path returns plain 0:

```
func BinomialPMF(k, n int, p float64) float64 {
  if k < 0 || n < 0 || k > n || p < 0 || p > 1 {
    return math.NaN()
  }
  if p == 0 {
    if k == 0 {
      return 1
    }
    return 0
  }
  if p == 1 {
    if k == n {
      return 1
    }
    return 0
  }
  lg := func(v int) float64 { l, _ := math.Lgamma(float64(v + 1)); return l }
  l := lg(n) - lg(k) - lg(n-k) + float64(k)*math.Log(p) + float64(n-k)*math.Log1p(-p)
```

```
  return math.Exp(l)
}

// BinomialTestTwoSided returns the exact two-sided p-value by the point
// probability method: every cell whose probability is at most the
// observed cell's, within tieTol, joins the region, and the p-value is
// the sum over that region. Doubling the smaller tail is the shortcut
// this replaces, and the shortcut has two failure modes worth naming at
// a board: it can pass 1, and on an asymmetric p it disagrees with the
// cell sum about which cells count as extreme. The cells are visited by
// ratio, ln P(k+1) = ln P(k) + ln((n-k)/(k+1)) + ln(p/(1-p)), one
// arithmetic path for the observed cell and the region alike: past the
// validity check the walk evaluates no factorials, only ratios. Invalid
// arguments propagate the PMF's NaN.
func BinomialTestTwoSided(k, n int, p float64) float64 {
  if math.IsNaN(BinomialPMF(k, n, p)) {
    return math.NaN()
  }
  if p <= 0 || p ≥ 1 {
    // a point mass puts every probability on one cell: there the
    // observed cell is the whole region and the answer is 1, and
    // every other cell is impossible and the answer is 0. The ratio
    // walk cannot be trusted here: at p = 1 the k = 0 cell is -Inf
    // and the ratio +Inf, and -Inf + Inf is the NaN that sinks the
    // bar and empties the region.
    if (p <= 0 && k == 0) || (p ≥ 1 && k == n) {
      return 1
    }
    return 0
  }
  lpm0 := float64(n) * math.Log1p(-p) // the k = 0 cell: C(n,0) = 1
  ratio := math.Log(p / (1 - p))
  obs := lpm0
  for i := range k {
    obs += math.Log(float64(n-i)/float64(i+1)) + ratio
  }
  bar := math.Exp(obs) * (1 + tieTol)
  var sum float64
  lpm := lpm0
  for i := 0; i <= n; i++ {
    if pk := math.Exp(lpm); pk <= bar {
      sum += pk
    }
    if i < n {
      lpm += math.Log(float64(n-i)/float64(i+1)) + ratio
    }
  }
  return min(sum, 1)
}
```

listing 127: the pmf through lgamma, then the two-sided region visited by ratio, never by factorial (source: interview-repertoire/samples/ch27-go/binom.go, lines 19-85)

Two choices inside are the spoken answer. The point-probability method, not tail doubling: the region is every cell whose probability is at most the observed cell's, while doubling the smaller tail is the shortcut that can pass 1 and that on an asymmetric p disagrees with the cell sum about which cells count as extreme. And the cells are visited by ratio, $\ln P(k+1) = \ln P(k) + \ln((n-k)/(k+1)) + \ln(p/(1-p))$, so past the validity check the walk evaluates no factorials at all, only ratios, whatever the n. Measured: k 3 of n 10 fair flips sums its eight

symmetric tail cells to 0.34375, the center k 5 admits every cell and returns exactly 1, the pmf of 2 heads in 5 flips is 0.3125, and invalid arguments hand back NaN rather than a zero that reads as certainty.

The auc is one sort, never the $O(n^2)$ pair loop the definition suggests: midranks once, every tie sharing the average of the ranks it spans so a tie is worth exactly half a win, then $U = R^{+} - n_1(n_1+1)/2$ over $n_1 n_0$, the Mann-Whitney identity:

```
func Midranks(xs []float64) []float64 {
  order := make([]int, len(xs))
  for i := range order {
    order[i] = i
  }
  slices.SortFunc(order, func(a, b int) int { return cmp.Compare(xs[a], xs[b]) })
  ranks := make([]float64, len(xs))
  for i := 0; i < len(order); {
    j := i
    for j+1 < len(order) && xs[order[j+1]] == xs[order[i]] {
      j++
    }
    avg := float64(i+j+2) / 2 // ranks are 1-based: positions i..j average to (i+1 + j+1)/2
    for _, at := range order[i : j+1] {
      ranks[at] = avg
    }
    i = j + 1
  }
  return ranks
}

// AUC returns the area under the ROC curve of scores against binary
// labels: the share of positive-negative pairs where the positive
// outscores the negative, a tie worth 1/2. It is computed as the
// Mann-Whitney statistic, never the O(n^2) pair loop: midranks once,
// sum the ranks of the positives, U = R+ - n1(n1+1)/2 over n1*n0, one
// sort end to end. A sample holding only one class has no cross pair,
// the concordance is undefined, and the 0.5 chance line is the only
// honest number to say out loud; the empty sample is that same case.
// Slices of different lengths are a caller bug and panic.
func AUC(scores []float64, labels []bool) float64 {
  if len(scores) != len(labels) {
    panic(fmt.Sprintf("AUC: %d scores against %d labels", len(scores), len(labels)))
  }
  n1 := 0
  for _, pos := range labels {
    if pos {
      n1++
    }
  }
  n0 := len(labels) - n1
  if n1 == 0 || n0 == 0 {
    return 0.5
  }
  ranks := Midranks(scores)
  rplus := 0.0
  for i, pos := range labels {
    if pos {
      rplus += ranks[i]
    }
  }
  u := rplus - float64(n1)*float64(n1+1)/2
  return u / float64(n1*n0)
}
```

listing 128: ties share their average rank, the positives' rank sum becomes the auc in one line (source: interview-repertoire/samples/ch27-go/ranks.go, lines 13-66)

The toy the tests pin: positives 0.2 and 0.4 against negatives 0.1 and 0.3, three of four cross pairs concordant, auc 0.75. A sample holding one class only has no cross pair, the concordance is undefined, and 0.5 is the only honest number to say out loud. The property test is the punchline the interview wants: 300 seeded cases with quantized scores forcing heavy ties, the midrank auc matching the brute-force pair loop to the last bit, book 14, ch 18 (knowledge discovery, roc and auc) for the curve the number summarizes.

The bootstrap is deterministic or it cannot be pinned: a PCG stream seeded (seed, seed), a callback metric over row indices, one buffer reused across replicates, nearest-rank percentiles:

```
func PercentileCI(metric func(idx []int) float64, n, resamples int, seed uint64, loPct, hiPct float64) (lo,
hi float64) {
  if n < 1 || resamples < 1 {
    return math.NaN(), math.NaN()
  }
  rng := rand.New(rand.NewPCG(seed, seed))
  idx := make([]int, n)
  vals := make([]float64, resamples)
  for r := range resamples {
    for j := range idx {
      idx[j] = rng.IntN(n)
    }
    vals[r] = metric(idx)
  }
  slices.Sort(vals)
  return nearestRank(vals, loPct), nearestRank(vals, hiPct)
}

// nearestRank picks the ceil(pct% of m)-th smallest of ascending vals,
// clamped into [1, m]. The product pct*m is formed before the /100 so
// the arithmetic stays exact through the multiplication and only the
// one division can round.
func nearestRank(sorted []float64, pct float64) float64 {
  rank := int(math.Ceil(pct * float64(len(sorted)) / 100))
  rank = min(max(rank, 1), len(sorted))
  return sorted[rank-1]
}
```

listing 129: the resample loop, the reused index buffer, the nearest-rank readout (source: interview-repertoire/samples/ch27-go/boot.go, lines 21-46)

Pinned from a real run: 4000 resamples of a 12-row sample summing to 56, mean 4.67, seed 7, return the 95 percent interval [3, 6.833], and the same seed replays it byte-identical forever, which is the only reason a test may pin an interval at all. The reused buffer carries the contract a range variable carries: look, do not retain.

The verdicts, holm's step-down against benjamini-hochberg's step-up:

```
func Holm(ps []float64, alpha float64) []bool {
  validateFamily(ps, alpha)
  m := len(ps)
  order := ascending(ps)
  reject := make([]bool, m)
  for i, k := 0, 1; i < m; i, k = i+1, k+1 {
    if ps[order[i]] > alpha/float64(m-k+1) {
      break
    }
    reject[order[i]] = true
  }
  return reject
}

// BH runs the step-up false-discovery procedure of Benjamini and
// Hochberg (1995) at rate q: sort the p-values ascending, walk k = 1..m
// against the ascending bars (k/m)*q, keep the largest k whose p(k)
// clears its bar, and reject exactly the first k hypotheses in sorted
// order. The verdict slice preserves the input order. Panics if any p
// or the rate falls outside [0,1].
func BH(ps []float64, q float64) []bool {
  validateFamily(ps, q)
  m := len(ps)
  order := ascending(ps)
  reject := make([]bool, m)
  kstar := 0
  for i, k := 0, 1; i < m; i, k = i+1, k+1 {
    if ps[order[i]] <= float64(k)/float64(m)*q {
      kstar = k
    }
  }
  for i := range kstar {
    reject[order[i]] = true
  }
  return reject
}
```

listing 130: descending bars that stop the family at the first miss, ascending bars that keep the largest k (source: interview-repertoire/samples/ch27-go/multiple.go, lines 18-53)

Measured on the triple (0.01, 0.03, 0.04) at level 0.05: holm rejects one, the first bar alpha over m clearing 0.01 and the second alpha over 2 stopping the family at 0.03, while bh rejects all three, 0.04 clearing the last bar q itself. Same family, opposite verdict counts, and the contrast is the answer to which one to use, family-wise control for proof, false discovery for triage, book 14, ch 40 (knowledge discovery, multiple testing: bonferroni and fdr). A p-value outside [0,1] or NaN panics both walks on sight: an input shaped like a probability that is not one deserves the loud failure now, not a quietly wrong verdict set later. The production version of this whole workspace runs the user's mined dota-helper engine, book 4, ch 31 (go 1.27, numerical statistics in go).

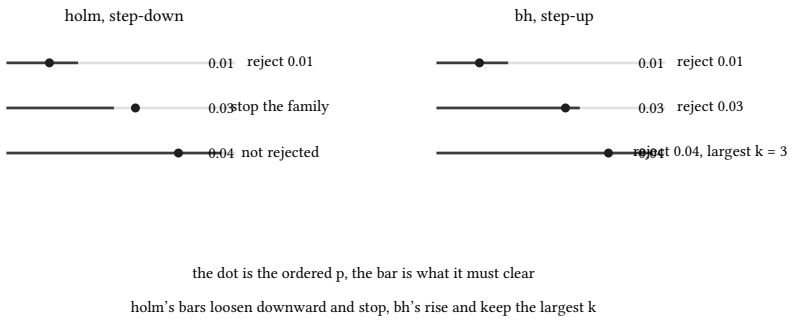


figure 127: descending holm bars stop the family at the first miss, ascending bh bars clear all three

27.6. backtesting without lookahead [DRILL]

The claim a backtest makes is that the score knew the outcome before the outcome existed, and every leakage bug is some version of the score reading the future. The discipline is one sentence: at event i the model may read the state built from strictly earlier events, and nothing at or after i. The dota-helper engine's replay walker mines this exactly, book 4, ch 32 (go 1.27, fitting, backtesting, and guarded promotion): each draft is walked in pick order, the state before pick i splits into same-side allies, opposing enemies, and the taken list covering picks and bans, all of it built from the events before i and nothing else, and each pick is then scored against the alternatives that state left available. The future of the draft is exactly the part of the file the walker must never open.

Why shuffle-split leaks on temporal data, said plainly: a random split scatters tomorrow's rows into the training set, so the model is fitted on the very distribution it is graded against, and adjacent drafts share players, metas, and patches, so row-level randomness hands the test set a near twin of every train row. K-fold is the same leak repeated k times, every row serving as both teacher and exam. The repairs, in the order the question wants them: split by time, fit strictly on the past and score strictly on the future, one pass; when entities repeat, split by entity, never by row; and when edges or thresholds are fitted, fit them on train and freeze them before the holdout is touched, the engine's calibration being decile edges fitted on train scores with the holdout bins expected to form a monotone ladder.

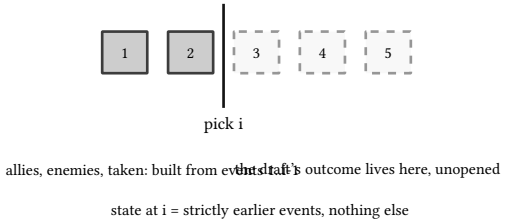


figure 128: the cursor at pick i reads everything strictly earlier, the file below it stays closed

The follow-up traps, named before they arrive. "The holdout improved on the second look" is the holdout spent: each consult leaks one decision back into the fitting, and the honest protocol touches the holdout once, at the end. "We tuned on the backtest data" is the first leak with a dashboard on it. "Shuffle the drafts and the scores barely move" is the confession, because shuffling destroys exactly the temporal structure deployment will respect, and a score indifferent to it was never reading time at all. The promotion guard in the next section is what the honest numbers knock on.

27.7. guarded promotion and multiple testing in a game tool [DRILL]

The scene: a tool screens hero cells by the hundred, each promising a win-rate lift, and every promise wants to rewrite the live config. The guard the dota-helper engine runs, book 4, ch 32 (go 1.27, fitting, backtesting, and guarded promotion), is four doors, and every door is checked before any file is opened for writing. Door one, the proposal carries a sane finite weight vector, NaN and both infinities refused on sight because a NaN fails every comparison silently. Door two, the baseline the gain was measured against still equals the live weights, a config that moved since making the numbers stale and sending the candidate back to refit. Door three, the holdout gain

clears the floor. Door four, the bootstrap interval's low end sits strictly above 0, a mean gain that could be zero being no gain at all, the same interval the workspace above pins. Any refusal leaves the config bytes identical, and the proof is a byte comparison, not a promise.

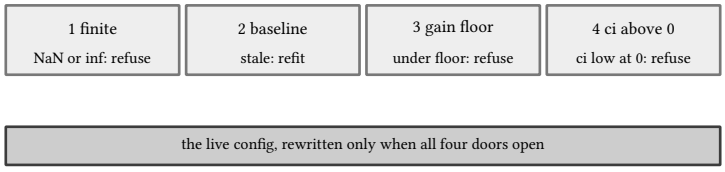


figure 129: four doors, every check before the write, any refusal leaves the bytes identical

Multiple testing is why the knock list is short. Screen 200 cells at 0.05 and the all-null expectation alone hands you 10 false discoveries, m times alpha, the scenario the statistics drill carries at 20 tests through the complement, 1 - 0.95^20. Family-wise control, bonferroni and holm, prices the whole family's false-rejection risk down to 0.05 and prices true discoveries out of the market too: the workspace's measured triple keeps only 1 of its 3 bh rejections under holm. False-discovery control keeps the rejections and bounds their false share instead, the right currency when the screen is a triage and every survivor gets a human look, book 14, ch 40 (knowledge discovery, multiple testing: bonferroni and fdr). The composition the question is fishing for: fdr trims the candidate list, the backtest prices the survivor, and the four doors decide whether the config opens at all.

floored to the math handbook and its exact-arithmetic gates: book 2, ch 13 (mathematics for programmers, logic, sets, and proof) for the pigeonhole and induction habits, book 2, ch 14 (mathematics for programmers, combinatorics) for the sum and product rules, the binomial ladder, and inclusion-exclusion with the derangements, book 2, ch 18 (mathematics for programmers, probability and distributions) for spaces, conditioning, and expectations, book 2, ch 19 (mathematics for programmers, statistics and inference) for hypothesis tests, estimators, and baselines, with book 10, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking) owning the cost-claim reading and book 10, ch 16 (practical data structures and algorithms, number theory and modular arithmetic) the halving recurrence behind exponentiation by squaring, and the computed half floored further to book 14, ch 39 (knowledge discovery, hypothesis testing and p-values) and book 14, ch 18 (knowledge discovery, roc and auc) behind the workspace's exact test and auc, book 14, ch 40 (knowledge discovery, multiple testing: bonferroni and fdr) behind its verdicts, and book 4, ch 31 (go 1.27, numerical statistics in go) where the same machinery runs mined engine data. Drill numbers are textbook values, the workspace's own numbers measured.

28. data mining answers

The mining round asks three kinds of question: name the measure and defend the choice, name the family and defend the parameters, and place the algorithm inside the discovery process it serves. Every drill floors to the kdd handbook, and one answer, pagerank, has to concede out loud what the corpus does not ship.

28.1. similarity and distance measures [DRILL]

Euclidean distance is the length of the difference vector, the minkowski family at p equals 2, holding manhattan at p equals 1 and shrinking strictly as p grows, book 14, ch 5 (knowledge discovery, distance metrics). It pays attention to magnitude, right when the coordinates are the signal, wrong when one loud axis dwarfs the rest. Cosine is the dot product over the two lengths, the angle alone, scaling either vector by any constant changes nothing, the right attitude for counts and ratings, book 14, ch 6 (knowledge discovery, similarity measures). Jaccard is for sets, intersection over union, a fraction from 0 to 1 needing no order, no magnitudes, no dimensions, the default for baskets and tags. Normalization is where the answer changes, and saying so is the answer: z-scoring each axis hands euclidean a new metric where the loud coordinate stops dominating, centering turns cosine into the pearson correlation, so pure offset stops counting and only co-movement does. The chooser, one breath: coordinates take euclidean, counts take cosine, sets take jaccard, normalization says which magnitudes count.

PITFALL: normalization is a modeling decision, not a preprocessing detail

Cosine is blind to magnitude, so on raw ratings it calls a user who rated everything 5 identical to one who rated everything 1. Z-scoring each axis rewrites euclidean, centering turns cosine into correlation, either move changes the neighbor ranking, so the spoken answer names the measure and the normalization together, never the measure alone.

coordinates	euclidean	the magnitudes are the signal
counts, documents	cosine	direction only, scaling changes nothing
baskets, tags	jaccard	intersection over union, no magnitudes

z-scoring changes euclidean, centering turns cosine into correlation

cosine on raw ratings calls all-5s identical to all-1s

figure 130: the representation picks the measure, normalization decides which magnitudes count

28.2. clustering: k-means, hierarchical, dbscan [DRILL]

Three families, one sentence each. K-means is the centroid family: lloyd alternates two moves, assign every point to the nearest centroid by squared euclidean, then move every centroid to the mean of what it holds, the objective never increases, the seeds pick the basin, nothing promises the global optimum, book 14, ch 31 (knowledge discovery, k-means). K is an input, and who picks it is the real question: the elbow, read off a table of sse by k, the table worth reading being the optimum per k, which the kdd chapter computes by exhaustive enumeration. Never from thin air. Hierarchical is the connectivity family: agglomerative, every point its own cluster, merge the closest pair until one remains, the distance matrix the only input, the linkage rule extending point distance to cluster distance, single taking the minimum cross pair and chaining, complete taking the maximum and dragging, the dendrogram cut where the task needs it and no k chosen up front, book 14, ch 32 (knowledge discovery, hierarchical clustering). Dbscan is the density family: a radius and a core threshold and never a k, a point is core when its neighborhood holds the threshold count of others, clusters grow breadth-first from cores, borders absorb the label without expanding, what nothing reaches is labeled noise, the label the other families cannot produce, the cluster count an output, book 14, ch 33 (knowledge discovery, dbscan). Density beats centroids when the clusters are not spheres, the sizes are uneven, and the noise must be named rather than force-assigned to the nearest mean.

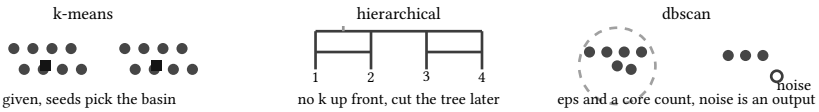


figure 131: centroids with k given, a dendrogram cut later, density that labels its own noise

28.3. itemsets: apriori and fp-growth [DRILL]

Support first, defined out loud: the support of an itemset is the count or fraction of transactions whose basket contains it. The confidence of a rule is the support of the union over the support of the antecedent, a conditional frequency, and confusing the two is the classic screen, a pair can be confident and useless because the consequent alone is near universal. Neither is interestingness: lift divides confidence by what independence would give and reads 1 when the rule says nothing. Apriori is the law named a priori: every subset of a frequent itemset is frequent, each transaction witnessing the superset witnessing every subset at once, and the algorithm runs the contrapositive before counting, one infrequent subset kills the candidate uncounted, the anti-monotone prune. Levelwise, that is join the frequent k minus 1 itemsets sharing a prefix, prune on subsets, count survivors, one database pass per level, book 14, ch 25 (knowledge discovery, levelwise apriori). Fp-growth reads the database exactly twice: pass one counts singles into the frequency list, pass two rewrites every transaction in that order into a prefix tree with counted edges, then mining walks conditional pattern bases per item, no candidate generation at all, book 14, ch 28 (knowledge discovery, fp-growth). The trade, said plainly: apriori spends passes and keeps the database on disk, fp-growth spends memory on the compressed tree and stops paying for passes. The counting pass underneath both is a group by over baskets, the database answers chapter's home turf, book 17, ch 17 (the interview repertoire, database answers).

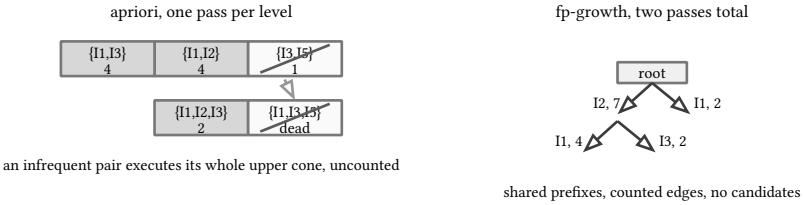


figure 132: one dead pair kills its whole upper cone, the tree shares prefixes and counts edges

28.4. cosine neighbors and smoothed bayes [TDD]

The chooser above says counts take cosine; this workspace computes it, `ch28-go`, 22 test functions under `make verify-go`, stdlib only, every number below the suite's measured output. The representation move comes first and is half the answer: baskets and draft picks are one-hot sets, so the cosine of two vectors that carry no magnitudes at all is the shared id count over the sqrt of the two sizes, computed on sorted ints in one linear merge, no hash map and no float feature vector anywhere:

```
func SetOf(ids []int) Set {
  s := slices.Clone(ids)
  slices.Sort(s)
  out := s[:0]
  for i, v := range s {
    if i == 0 || v != s[i-1] {
      out = append(out, v)
    }
  }
  return Set{ids: out}
}

// Len is the set's popcount, the sqrt of its one-hot norm squared.
func (s Set) Len() int { return len(s.ids) }

// IDs exposes the sorted ids for counting passes; callers must not
// mutate the slice.
func (s Set) IDs() []int { return s.ids }

// Cosine is the cosine similarity of two sets' one-hot vectors: the
// shared id count over sqrt(|a| * |b|). Scaling either vector by any
// constant changes nothing here because presence has no magnitude, and
// the intersection is one linear merge of the sorted slices, the whole
// reason the ids stay ints.
func Cosine(a, b Set) float64 {
  if a.Len() == 0 || b.Len() == 0 {
    return 0
  }
  return float64(shared(a.ids, b.ids)) / math.Sqrt(float64(a.Len())*float64(b.Len()))
}

// shared counts the intersection of two ascending slices in one merge,
// the same walk merge sort finishes with.
func shared(a, b []int) int {
  n, i, j := 0, 0, 0
  for i < len(a) && j < len(b) {
    switch {
    case a[i] < b[j]:
      i++
    case a[i] > b[j]:
      j++
    default:
      n++
      i, j = i+1, j+1
    }
  }
  return n
}
```

listing 131: presence is the only fact: sorted ids, one merge for the intersection, cosine as a count over two sqrts (source: interview-repertoire/samples/ch28-go/sets.go, lines 20-67)

Measured: {1,2,3} against {2,3,4} shares 2 of sqrt(3 **3) and scores 0.6667, {1,2} against {2} scores 1 over sqrt(2** 1), 0.7071, and a duplicated item in the basket changes nothing because presence is the only fact the set keeps. kNN on top of that similarity is a sort and a weighted vote:

```
func (kn *KNN) Score(q Set) float64 {
  type neighbor struct {
    sim float64
    id  int64
    won bool
  }
  ns := make([]neighbor, len(kn.ids))
  for i, tf := range kn.sets {
    ns[i] = neighbor{sim: Cosine(q, Set{ids: tf}), id: kn.ids[i], won: kn.won[i]}
  }
  sort.Slice(ns, func(a, b int) bool {
    if ns[a].sim != ns[b].sim {
      return ns[a].sim > ns[b].sim
    }
    return ns[a].id < ns[b].id
  })
  if kn.k < len(ns) {
    ns = ns[:kn.k]
  }
  var vote, weight float64
  for _, n := range ns {
    y := -1.0
    if n.won {
      y = 1.0
    }
    vote += n.sim * y
    weight += n.sim
  }
  if weight == 0 {
    return 0
  }
  return vote / weight
}
```

listing 132: similarity-weighted votes, ties by id, zero-similarity neighbors carry no weight (source: interview-repertoire/samples/ch28-go/knn.go, lines 48-80)

The tie the follow-up asks about is pinned: the query {1,3} sits at cosine 0.5 from a winner {1,2} and at 0.5 from a loser {3,4}, and the two weighted votes cancel to exactly 0, the chance line. A zero-similarity neighbor carries no weight at all, so it cannot drag a real vote toward 0, only real neighbors speak, and similarity ties break by train id ascending so neighbor selection never depends on the order drafts were handed to the trainer.

Bernoulli naive bayes is the other comparator on the same sets, the prior log-odds plus one smoothed log ratio per present item:

```
// Score is the win log-odds of the query set: the prior log-odds plus,
// per present item, the log ratio of its class conditionals
// P(f|win)/P(f|loss), each cell Laplace-smoothed against the binary
// denominator Nclass + 2a, the 2 standing in for the two states a
// Bernoulli cell takes. Positive favors a win, 0 is exactly the
// decision boundary, and large alpha flattens every item term toward 0
// until only the prior speaks. A train split holding only one label
// carries no discrimination and scores a flat 0, the chance-line
// convention the knn carries too.
func (nb *NaiveBayes) Score(s Set) float64 {
  if nb.nWin == 0 || nb.nLoss == 0 {
    return 0
  }
  score := math.Log(float64(nb.nWin) / float64(nb.nLoss))
  for _, ft := range s.ids {
    w, l := nb.winCnt[ft], nb.lossCnt[ft]
    if w == 0 && l == 0 {
      continue
    }
    pw := (float64(w) + nb.alpha) / (float64(nb.nWin) + 2*nb.alpha)
    pl := (float64(l) + nb.alpha) / (float64(nb.nLoss) + 2*nb.alpha)
    score += math.Log(pw / pl)
  }
  return score
}
```

listing 133: the prior log-odds, laplace-smoothed conditionals, out-of-vocabulary items skipped (source: interview-repertoire/samples/ch28-go/naivebayes.go, lines 43-67)

The hand-computable fixture is the spoken answer: two winners carrying item 1 and one loser without it, alpha 1, and the score of {1} is ln 2 + ln(9/4) = ln 4.5, measured 1.5041. Laplace is the zero-cell rescue, without it one absent count makes the ratio infinite and a single unseen cell dominates every score; an item counted zero in both classes is out of vocabulary and skipped, because scoring a stranger would add another copy of the prior per stranger and drift every query toward the rarer class. Both comparators floor to the kdd handbook, book 14, ch 6 (knowledge discovery, similarity measures) for the measure, book 14, ch 20 (knowledge discovery, nearest neighbor) and book 14, ch 19 (knowledge discovery, naive bayes) for the families, and the same pair runs report-only beside the mined dota-helper engine's live linear scorer, book 4, ch 32 (go 1.27, fitting, backtesting, and guarded promotion).

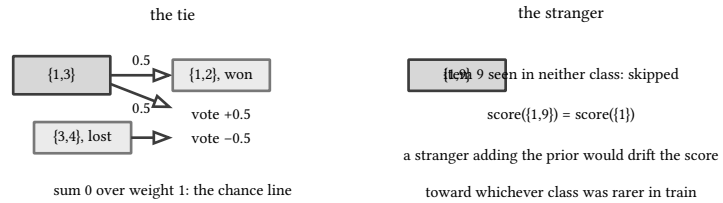


figure 133: the tie query cancels its two equal votes, the stranger item adds nothing

28.5. the levelwise pass, counted and pruned [TDD]

The itemsets drill above draws the lattice on nine baskets; this workspace runs the same baskets through the levelwise pass:

```
func FreqItemsets(txs []Set, minCount int) []Itemset {
  if minCount < 1 || len(txs) == 0 {
    return nil
  }
  single := map[int]int{}
  for _, tx := range txs {
    for _, id := range tx.ids {
      single[id]++
    }
  }
  var freq []Itemset
  var level [][]int
  for _, id := range slices.Sorted(maps.Keys(single)) {
    if single[id] ≥ minCount {
      level = append(level, []int{id})
      freq = append(freq, Itemset{Items: []int{id}, Count: single[id]})
    }
  }
  seen := map[string]bool{}
  for _, s := range level {
    seen[key(s)] = true
  }
  for len(level) > 0 {
    var next [][]int
    for i := 0; i < len(level); i++ {
      for j := i + 1; j < len(level); j++ {
        a, b := level[i], level[j]
        if !slices.Equal(a[:len(a)-1], b[:len(b)-1]) {
          continue
        }
        cand := append(slices.Clone(a), b[len(b)-1])
        if !allSubsetsFrequent(cand, seen) {
          continue
        }
        if c := countSupport(txs, cand); c ≥ minCount {
          next = append(next, cand)
          freq = append(freq, Itemset{Items: cand, Count: c})
        }
      }
    }
    for _, s := range next {
      seen[key(s)] = true
    }
    level = next
  }
  return freq
}
```

listing 134: singles in one sweep, join on the shared prefix, prune every subset before counting, one database pass per level (source: interview-repertoire/samples/ch28-go/apriori.go, lines 29-75)

Measured at support 2: the singles count 6, 7, 6, 2, 2, all five frequent; six pairs survive; {3,5} dies at count 1 and its whole upper cone, {1,3,5} included, is never counted at all, the anti-monotone law doing its work before the database pass; two triples survive at count 2 each, {1,2,3} and {1,2,5}; 13 itemsets in all, and the brute-force cross-check enumerates all 31 subsets of the five singles and agrees with the levelwise output on every one. Then the rule arithmetic, counted straight from the database:

```
func Assess(txs []Set, ant, con []int) Rule {
  a, c := slices.Clone(ant), slices.Clone(con)
  slices.Sort(a)
  slices.Sort(c)
  // duplicated needles would fail containsAll against the deduped
  // basket sets, so the same normalization SetOf applies happens here
  a, c = slices.Compact(a), slices.Compact(c)
  if shared(a, c) > 0 {
    panic(fmt.Sprintf("assess: antecedent %v overlaps consequent %v", ant, con))
  }
  union := append(a, c...)
  slices.Sort(union)
  sU, sA, sC := countSupport(txs, union), countSupport(txs, a), countSupport(txs, c)
  if len(txs) == 0 || sA == 0 {
    return Rule{Support: math.NaN(), Confidence: math.NaN(), Lift: math.NaN()}
  }
  fU, fA, fC := float64(sU), float64(sA), float64(sC)
  conf := fU / fA
  return Rule{
    Support:    fU / float64(len(txs)),
    Confidence: conf,
    Lift:       conf / (fC / float64(len(txs))),
  }
}
```

listing 135: support, confidence, and lift as three support counts and two divisions (source: interview-repertoire/samples/ch28-go/apriori.go, lines 94-117)

The pair of rules the trap lives in, measured: {1,2} implies 3 and {1,2} implies 5 both carry confidence 0.5 and mean opposite things, lift 0.75 against 2.25, because item 3 shows in 6 of 9 baskets while item 5 shows in 2, and a near-universal consequent hands confidence out for free. On the independence fixture, baskets {1}, {2}, {1,2}, and the empty basket, the rule {1} implies 2 reads confidence 0.5 and lift exactly 1, the number that says the rule says nothing. Floors: book 14, ch 25 (knowledge discovery, levelwise apriori) for the levelwise law, book 14, ch 29 (knowledge discovery, interestingness, multilevel, and quantitative rules) for lift as the interestingness divide.

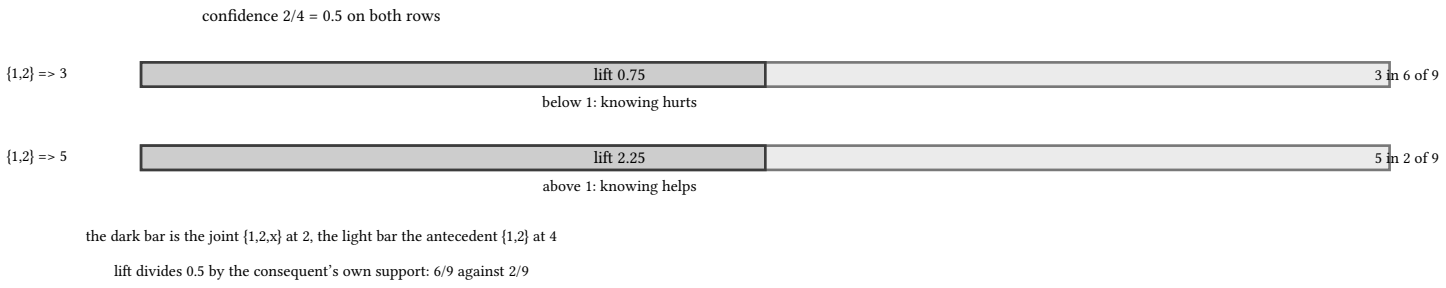


figure 134: same confidence 0.5, opposite lift: the consequent's own support decides

28.6. pagerank and the pipeline view [DRILL]

The random surfer: a walker sits on a page and at each step either follows a random out link or teleports to a random page, and the rank of a page is the share of walkers it holds at equilibrium, the stationary distribution of the walk. The arithmetic is one line, a page's rank is the sum over its in-links of each giver's rank divided by that giver's out degree, a page with one out link hands its whole rank forward, a page with a hundred hands each a hundredth. Damping, the textbook 0.85, earns its place twice: a sink page with no out links would trap the walker forever, and the teleport term makes the walk ergodic, so the fixed point exists and plain iteration converges to it. Then the concession, before anyone asks: the corpus ships no pagerank. What the corpus runs is the pipeline the question rides in, and that pipeline is the frame for every mining question: selection, preprocessing,

transformation, mining, evaluation, iterated until the patterns count as knowledge, mining stage 4 of 5, book 14, ch 1 (knowledge discovery, the knowledge discovery process). The capstone walks it end to end, a generated corpus, an embedded store whose cleaning views fill planted holes, pure go miners over views, and a planted-truth evaluation scoring what came back against what was planted, book 14, ch 48 (knowledge discovery, the capstone: a kdd pipeline on duckdb, with coverage appendices). So have you run pagerank answers no, and how would you mine a graph answers with the loop, not the name: what is selected, how cleaned, how transformed, what mined, how evaluated.

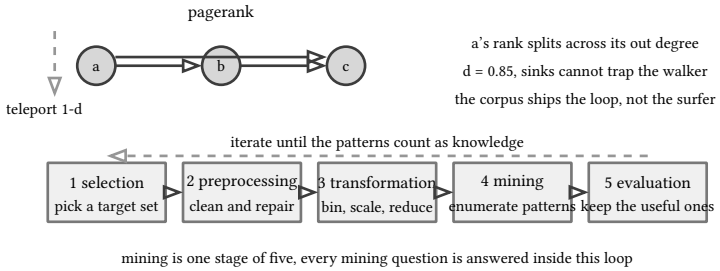


figure 135: the surfer with damping on top, the five-stage loop underneath, mining is stage 4

floored to: the kdd handbook chapters carrying the gated implementations, book 14, ch 5 (knowledge discovery, distance metrics) and book 14, ch 6 (knowledge discovery, similarity measures) for the measures, book 14, ch 31 (knowledge discovery, k-means), book 14, ch 32 (knowledge discovery, hierarchical clustering), and book 14, ch 33 (knowledge discovery, dbscan) for the families, book 14, ch 25 (knowledge discovery, levelwise apriori) and book 14, ch 28 (knowledge discovery, fp-growth) for the itemsets, with the workspace's own floors at book 14, ch 20 (knowledge discovery, nearest neighbor) and book 14, ch 19 (knowledge discovery, naive bayes) for the comparators and book 14, ch 29 (knowledge discovery, interestingness, multilevel, and quantitative rules) for the lift divide, book 14, ch 1 (knowledge discovery, the knowledge discovery process) for the loop, book 14, ch 48 (knowledge discovery, the capstone: a kdd pipeline on duckdb, with coverage appendices) for the pipeline that runs, and book 4, ch 32 (go 1.27, fitting, backtesting, and guarded promotion) where the same cosine and bayes run report-only on mined engine data.

29. python answers

The python round asks four questions that separate the person who has run the interpreter from the person who has only read about it: what the gil actually locks, how the data model dispatches, why closures surprise, and which concurrency lane the work belongs in. Every answer is spoken, drill-only, and floored on the python manual.

29.1. the gil and what it actually locks [DRILL]

The Global Interpreter Lock is one mutex inside cpython that a thread must hold to run python bytecode. It locks bytecodes, not bytes: it guards the interpreter's own state, and it is neither a memory model nor a thread-safety guarantee for your objects, book 8, ch 17 (python 3.14, threads and the gil). What releases it is the half the question fishes for: any blocking system call, file and network io, sleep, and c extensions that release it around their compute, so waiting overlaps even when computing cannot. The handoff: the interpreter sells a switch interval, the 0.005 second default, and which thread runs at the end of an interval is the operating system's decision, the interpreter runs no scheduler of its own. So cpu-bound threads do not parallelize, the bytecode cannot overlap, and parallel cpu work means processes, one interpreter and one gil per process. The aside, stated carefully: free-threaded builds exist, 3.13 shipped the build available but explicitly experimental, later releases move it toward supported while it stays a separate binary that is not the default, and the corpus pins the standard build. The follow-up is "so is python thread safe": no, the callout is the whole answer.

PITFALL: the gil serializes bytecode, not your updates

One thread's counter increment is three steps, read, add, store, and the gil can be handed off between them, so two threads can both read zero and both store one. The lock protects the interpreter's internals, never your invariants, and shared state still takes a lock.

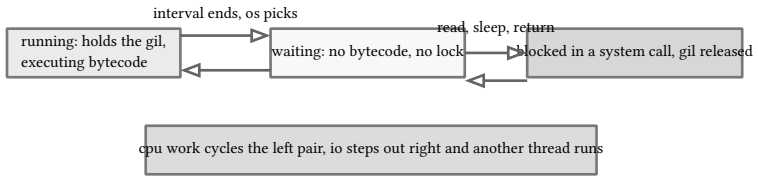


figure 136: one thread through the gil: the two ways out of running, only one of them overlaps

29.2. the data model: dunders and protocols [DRILL]

Everything a python program touches at runtime is an object, and every name is only a binding to one, book 8, ch 3 (python 3.14, the object model). Dunder methods are the operator table of the language, and the runtime, not your code, decides when to dial one: define the len hook and the len call dials it, define iter and for loops take the object, define getitem and indexing, slicing, iteration, and membership all go through it, define the enter and exit pair and the with statement runs them, enter's return binding the as name. The dispatch rule the question is really asking: implicit lookups go to the type, walking the class dicts and bypassing the instance's own namespace, so a hook parked on the instance is dead weight the builtin never dials. That is what makes objects first-class, the protocol on the type means every builtin trusts the object, sorted dials lt and nothing else, and defining eq alone makes the class unhashable, because equal objects must hash equal. Dataclasses are the same surface generated: a decorator writing init, repr, and eq from the declared field list, the dunders by machine instead of by hand. The follow-up is "why the type": one lookup site per protocol, so the dispatch cannot be spoofed per instance.

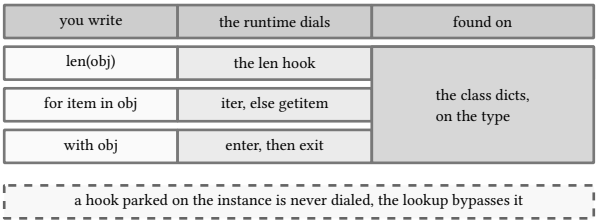


figure 137: implicit calls dial hooks on the type, the instance's namespace is bypassed

29.3. decorators, closures, scopes [DRILL]

Name resolution is a walk up four namespaces, local, enclosing, global, builtin, and a closure is the enclosing layer's machinery: the inner function keeps a reference to the cell holding the enclosing name, and the cell outlives the call that created it, book 8, ch 6 (python 3.14, functions, scopes, and decorators). The gotcha the round fishes for: closures capture cells, not values. A lambda written in a loop shares the loop variable's one cell, the cell holds the last value once the loop ends, and a list of three such lambdas answers with that last value three times. The two fixes, named out loud: bind at def time through a default argument, or call a factory that builds a fresh cell per invocation. A decorator is the same machinery wearing syntax: a function that takes a function and returns a function, the at spelling rebinding the name at def time to whatever the decorator hands back, which is why stacked decorators apply bottom up. functools.wraps is the apology the wrapper owes: without it the wrapper's name and doc replace the wrapped function's, and help and tracebacks lie about who is running. The follow-up is the gotcha asked as code, answered with the cell, then the factory.

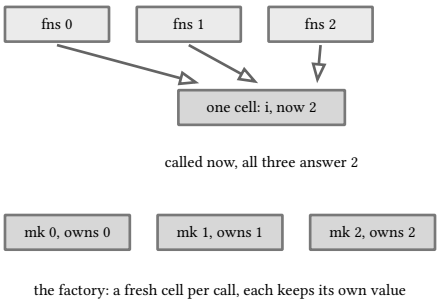


figure 138: three lambdas share the loop's one cell and all answer 2, the factory mints a cell per call

29.4. asyncio versus threads [DRILL]

Asyncio is cooperative multitasking: one thread, one event loop, and the concurrency comes from suspension, not from more threads, book 8, ch 21 (python 3.14, asyncio). The loop yields exactly where the code says await, and only there: a coroutine that never awaits starves everything sharing the loop, awaiting a coroutine runs it like a subroutine, and the body only overlaps once it is wrapped in a task the loop schedules. Cancellation holds the same contract, it lands at the next await, never mid-statement. The two-lanes answer, said as one sentence: io-bound work goes to async when the stack is already async, or to threads when the calls block, the gil releasing around the wait either way, and cpu-bound work goes to processes, because no lane inside one interpreter runs bytecode in parallel. The contrast the strong candidate volunteers: go multiplexes many goroutines onto few os threads with work stealing, a blocking goroutine parks and its thread runs another, book 17, ch 15 (the interview repertoire, go runtime: scheduler, gc, patterns), javascript is asyncio's family, one call stack, run to completion, the microtask queue drained between turns, book 17, ch 7 (the interview repertoire, javascript core: scope, closures, event loop), and python threads are real os threads the os switches at the interval, the difference from async being who decides to switch. The follow-up is "why not threads for everything": they overlap the waiting and still lose the cpu lane.

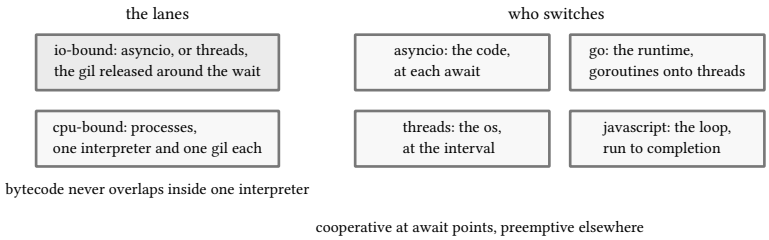


figure 139: io waits in either lane, cpu leaves the interpreter, and the models differ in who switches

floored to the python manual: book 8, ch 17 (python 3.14, threads and the gil) for the gil states and the lock toolbox, book 8, ch 3 (python 3.14, the object model) for the data model, book 8, ch 6 (python 3.14, functions, scopes, and decorators) for scopes, cells, and decorators, book 8, ch 21 (python 3.14, asyncio) for the loop and its tasks, and book 8, ch 1 (python 3.14, toolchain: cpython 3.14 on windows) for the interpreter build the corpus pins.

30. lua answers

The lua round is a contrast round. The questions arrive addressed to a javascript practitioner, and the answer that lands names what lua does differently instead of translating a habit across: metatables against prototypes, stackful coroutines against the event loop, `_ENV` against the scope chain. Three drills, spoken, every claim floored to the lua manual whose gated samples carry the depth, and the javascript side of each contrast floored to this book's own book 17, ch 7 (the interview repertoire, javascript core: scope, closures, event loop).

30.1. metatables and metamethods [DRILL]

A metatable is a table of fallbacks keyed by event name, and lua's object system is the protocol that consults it, because there is no class keyword anywhere in the language. Any value may carry one: tables get theirs from `setmetatable`, every other type shares one per type, and only C or the debug library can change those, which is how the string library gives every string an `__index` pointing at the string table so `("abc"):len()` works. Read and write are separate events. `__index` fires only when the access is not a table or the key is absent, the metavalue can be a function, a table, or any value with its own metavalue, and because the follow-up access is regular, misses chain, and that chain is inheritance, `setmetatable(child, {__index = parent})` being the whole mechanism. `__newindex` is the write twin and the trap: an assignment routed through it never performs the primitive store, and `rawset` and `rawget` exit the protocol entirely, the escape hatch the follow-up probes for. The operator events are orthodox: arithmetic searches the first operand then the second and passes the operands in original order, `__eq` runs only between two tables or two full userdata that are not primitively equal, `__lt` and `__le` take any mixed pair, and `__call` prepends the callee to the argument list. So give the conclusion, not the tour: oop in lua is dispatch by agreement, one table asking another what to do about a miss, a constructor an ordinary function returning a fresh table plus a metatable, methods living in the `__index` table, `self` the colon sugar, the full event inventory at book 9, ch 3 (lua 5.5, values, metatables, metamethods).

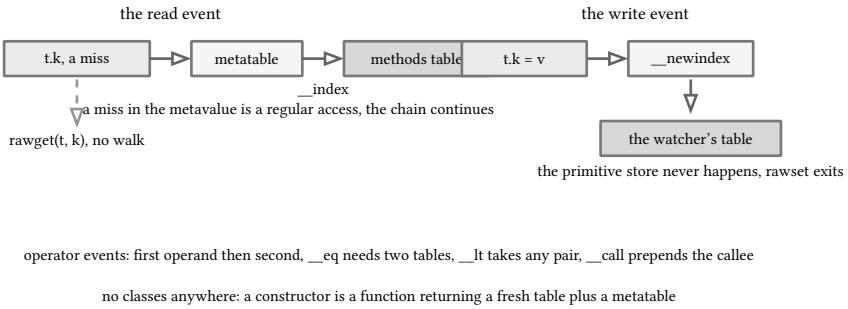


figure 140: the read miss walks the index chain, the write detours, the raw pair exits both

30.2. coroutines versus javascript async [DRILL]

A lua coroutine is a value of type `thread`, a collaborative execution context with its own stack, and the stack is the whole answer. `coroutine.create` only builds it, `resume` runs it, `yield` suspends it, status walks suspended, running, normal, dead, and because the entire stack parks, a function at any nesting depth can yield, the 5.4 guarantee holding in 5.5 that yields cross `pcall` bodies and metamethods, book 9, ch 7 (lua 5.5, coroutines). Control is asymmetric: `yield` returns to whoever resumed and `resume` runs the body on, values flowing both ways, resume arguments entering the body and each `yield`, yield arguments coming back out of the matching resume. Now the contrast, book 17, ch 7 (the interview repertoire, javascript core: scope, closures, event loop): javascript runs one call stack, suspension happens only at an `await`, and `await` exists only inside async functions, so functions are colored, one async callee forcing async on every caller up the chain, the parked continuations riding a microtask queue the event loop drains to empty after every macrotask. Lua has neither colors nor a scheduler: the caller of a yielding function cannot tell, no syntax marks the suspension point, nothing runs until some `resume` calls it, so a coroutine is a value you store and pass, `wrap` handing back the plain function the generic `for` wants and trading

`resume`'s error tuple for exceptions. The boundary, volunteered before it is asked: one thread of control at a time, cooperative only, no parallelism, and an unprotected error kills the coroutine without unwinding its stack so the debug library can read the corpse.

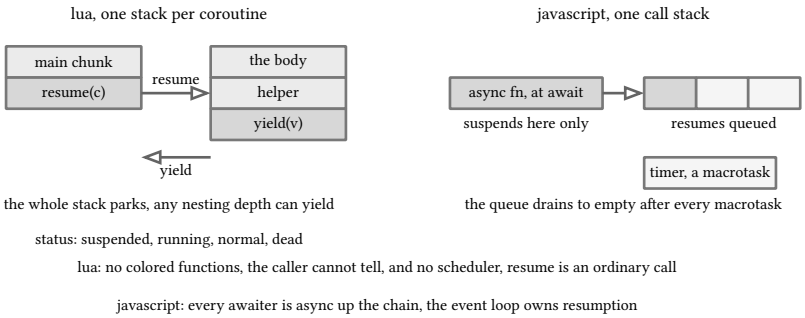


figure 141: lua parks whole stacks and owns no scheduler, javascript parks one frame and pays the queue

30.3. closures and environments against javascript [DRILL]

Closures match until you name the unit of sharing. A lua closure captures upvalues, and the box is created when the block executes the local declaration: two closures from one factory invocation share one upvalue box, two invocations build independent boxes, private state in five lines, book 9, ch 6 (lua 5.5, functions). The javascript gotcha is the mirror, book 17, ch 7 (the interview repertoire, javascript core: scope, closures, event loop): a `var` loop shares one binding and every callback prints 3, `let` gives per-iteration bindings and they print 0, 1, 2, and lua lands on the `let` side by construction, every executed declaration a fresh box, no hoisting to share. Then the part javascript has no answer for: there is no global scope in lua, only a variable named `_ENV`. Every free name compiles to `_ENV.var`, every chunk compiles inside an external local named `_ENV`, and every function captures whichever `_ENV` was visible where it was defined, so the global namespace is one ordinary upvalue, book 9, ch 8 (lua 5.5, environments, load, gc). Shadow `_ENV` with a local and globals change, `load`'s fourth argument swaps it per chunk, and `_G` is not the mechanism, just an ordinary global pointing at the usual table. The line to say out loud: javascript resolves free names up a scope chain that ends at a real global object, so a sandbox is a realm or a vm boundary, while lua's globals are a captured variable, so a sandbox is `load(src, name, "t", env)` and the untrusted chunk never sees `io`.

PITFALL: _ENV is a captured variable, not a registry

Globals do not resolve through a hidden table the runtime owns. They compile to field reads on `_ENV`, an upvalue like any other, so a chunk's global namespace is swapped by handing it a different `_ENV` and nothing else. Assigning `_G = {}` rebinds one name and changes nothing about how any free name resolves.

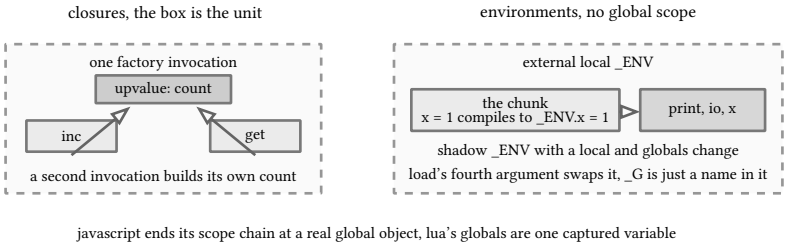


figure 142: one upvalue box per executed declaration, the chunk's globals one captured `_ENV`

sources: no measurements in this chapter, every claim the manual's own semantics, and every drill floors to the lua manual, book 9, book 9, ch 3 (lua 5.5, values, metatables, metamethods) for the event tables and the raw access, book 9, ch 6 (lua 5.5, functions) for the upvalue box, book 9, ch 7 (lua 5.5, coroutines) for the stack, the resume yield channel, and the un-unwound corpse, and book 9, ch 8 (lua 5.5, environments, load, gc) for `_ENV`, `load`, and the sandbox.

31. c sharp answers

The c sharp round rewards the same move as the go round, naming the mechanism under the keyword. Every answer here floors to the csharp-net manual: async owns the state machine, typesystem owns boxing, memory and allocation own the heap, linq owns the deferred plan, and the code side of these questions already runs gated in this book's c sharp workspaces, chapters 2, 3, 5, and 6. These four sections are the spoken layer over that floor: the answer to say out loud, the follow-up to expect, and the concession to volunteer first.

31.1. async await machinery: state machines and context [DRILL]

Await does not wait. The compiler rewrites an async method into a state machine: the locals become fields, each await is a suspension point where the machine records its place, hands the unfinished task back to the caller, and registers the rest of the method as a continuation, book 6, ch 9 (c# 15, f# 11, and .net 11, async). The thread is released at the suspension, which is the entire win, outstanding work with no thread parked against it. What resumes, and where, is the context question. By default the continuation is posted back to the captured SynchronizationContext, the ui thread in a desktop app, the request context in classic server code, and ConfigureAwait(false) skips the capture and resumes on the thread pool, which is why library code calls it everywhere and application code at the top usually does not. Blocking against suspending, said precisely: awaiting an incomplete task suspends the machine and frees the thread, reading Result or calling Wait on that same task parks a thread that does nothing. The classic deadlock is sync over async on a context, the ui thread calls Result, the continuation must post to the very context the Result call is blocking, and neither side moves. Async void is the twin trap, nothing can await it and its exceptions take down the process, so it stays reserved for event handlers. The follow-up is ValueTask, answered as the allocation-avoiding refinement for paths that usually complete synchronously.

PITFALL: the deadlock is the context, not the task

Result, Wait, and GetAwaiter().GetResult on an incomplete task burn a thread unconditionally, and where a SynchronizationContext exists they deadlock outright: the continuation is queued to the very context the blocking call holds. Await instead, all the way up, and let library code ConfigureAwait(false) its way out of the capture.

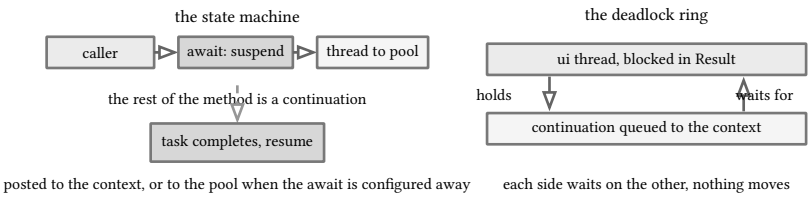


figure 143: suspension frees the thread, a blocked context waits on itself

31.2. linq as an embedded dsl [DRILL]

Linq is the functional core of c sharp worn as syntax. Query syntax, from where select, is sugar the compiler translates into exactly the method calls, Where OrderBy Select chained as extension methods, one compiler output however it is spelled, book 6, ch 14 (c# 15, f# 11, and .net 11, linq). Under the whole thing sit iterators: a query expression builds a plan, nothing runs, and each operator wraps its source in another enumerable that hands out one element at a time on demand. That is deferred execution, and it cuts both ways. The good edge is composition, filter and project and group cheaply, materialize once at the end. The bad edge is multiple enumeration: every enumeration re-runs the plan against the current data, so a query enumerated twice over a list that changed in between gives two different answers, and a query over a store re-issues the work per pull. The operators that return something other than IEnumerable, ToList, ToArray, ToDictionary, First, Count, Any, Sum, execute immediately and materialize, which is the lever: keep the query deferred while composing, snapshot it once when the answer must be stable, and never hand a live query to code that expects a snapshot. The follow-up is the paradigm

question, answered by owning it: this is the same map filter reduce trio the paradigms chapter drills, book 17, ch 2 (the interview repertoire, paradigms: oop, fp, mvc, mvvm, ecs), closure over a captured local included, with the purity test being no stage writing into its input.

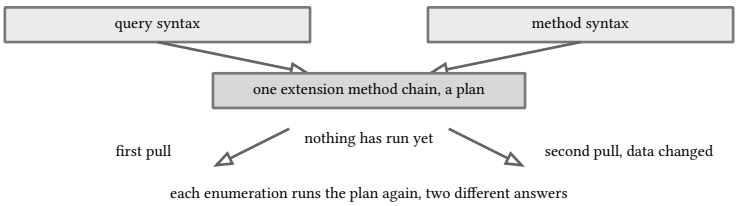


figure 144: two spellings collapse into one chain, and every enumeration runs the plan again

31.3. value versus reference, boxing [DRILL]

A value type variable holds its data, a reference type variable holds a reference to an object on the heap, and assignment is where the difference surfaces: assigning a struct copies the whole value so the copies diverge, assigning a class copies the reference so both names watch one object mutate, book 6, ch 3 (c# 15, f# 11, and .net 11, the type system). Boxing is the seam between the two worlds: casting a value type to object or to an interface it implements allocates a box on the heap and copies the value in, the box is separate storage so mutations after the copy never reach it, and each box is an allocation the collector must later track. It hides in plain sight, non generic collections, interface-typed calls inside numeric loops, interpolation in older shapes. Generics are the first cure and the reason they exist: `List<int>` stores ints unboxed, and variance conversions are reference conversions, so a sequence of int cannot be read as a sequence of object without a wrapper, the compiler refusing the free version precisely because it would box. The escape hatch below that is the stack-only family: stackalloc reserves a buffer on the stack, a span over it is a window onto that memory with no heap allocation, and the cost is the discipline, a stack-only type cannot be boxed, cannot be stored on the heap, cannot survive an await, book 6, ch 10 (c# 15, f# 11, and .net 11, memory and runtime). When the question turns to hot paths the answer is the ladder: keep values in generic containers, span over stackalloc or pooled arrays for buffers, book 6, ch 11 (c# 15, f# 11, and .net 11, allocation: the managed heap, pools, and arenas), and treat every object-typed boundary as a box minted per call.

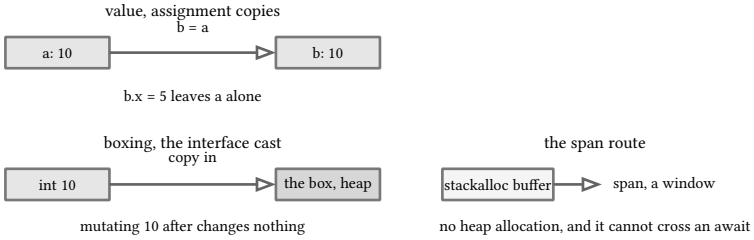


figure 145: assignment copies the value, an interface cast copies it into a heap box, spans stay off the heap

31.4. the gc generations against go's [DRILL]

The dotnet collector is generational and compacting, the go collector is concurrent and non compacting, and the honest answer contrasts the designs rather than ranking them. Dotnet: new objects allocate into generation 0 by bumping a pointer in a reserved segment, collections fire most often there because most objects die young, survivors are promoted to generation 1 and then 2, and each collection marks the live objects, relocates them, and compacts the freed space, which is what keeps the allocation pointer a bump and survivors local to each other, book 6, ch 10 (c# 15, f# 11, and .net 11, memory and runtime). Objects of 85,000 bytes or more allocate directly on the large object heap, not compacted by default because copying that much is its own cost problem, book 6, ch 11 (c# 15, f# 11, and .net 11, allocation: the managed heap, pools, and arenas). Go: concurrent tri-color mark and sweep, non generational, non compacting, with sub-millisecond stop-the-world pauses at both ends of the cycle and pacing by GOGC and GOMEMLIMIT rather than by generations, book 17, ch 15 (the interview repertoire, go runtime: scheduler, gc, patterns). What each buys, said out loud: compaction buys locality and a permanent bump allocator but pays pauses proportional to the surviving set it moves, and dotnet hides those behind background

collection of generation 2 plus the workstation against server configurations; non compacting buys go the tiny pause and a collector that runs beside the mutator, and pays fragmentation, free memory no sweep ever makes contiguous again, with escape analysis as the compiler's part, keeping what it can on the stack so the heap never sees it. The follow-up is always which is faster, refused as asked: the two collectors optimize different costs, and the honest comparison names the workload, allocation-heavy and short-lived against long-lived and large.

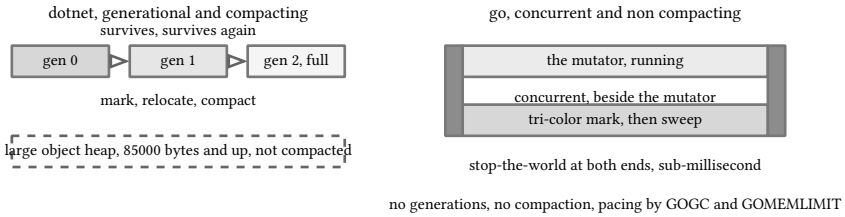


figure 146: dotnet compacts survivors up the generations, go sweeps concurrently and moves nothing

floored to the csharp-net manual: book 6, ch 9 (c# 15, f# 11, and .net 11, async) for the state machine and the rules that keep async honest, book 6, ch 14 (c# 15, f# 11, and .net 11, linq) for the two spellings and the deferred plan, book 6, ch 3 (c# 15, f# 11, and .net 11, the type system) for value against reference and boxing, book 6, ch 10 (c# 15, f# 11, and .net 11, memory and runtime) for spans, stackalloc, and the generational heap, book 6, ch 11 (c# 15, f# 11, and .net 11, allocation: the managed heap, pools, and arenas) for the large object heap and the pools, with go's side carried by book 17, ch 15 (the interview repertoire, go runtime: scheduler, gc, patterns).

32. css answers: box model, cascade, layout

The css round asks three questions in different clothes: how big is the box, which declaration wins, and where does the content land. This chapter answers each with the smallest model that stays true, the box model as arithmetic, the cascade as a sortable key, flex and grid as free-space distribution, media queries as predicates over a viewport. The `ch32-js` workspace runs the models under `node --test`, 14 tests, zero dependencies, and the three stylesheets are real files, listed verbatim, never executed by the suite, the same honesty the dom chapter set.

32.1. the box model and the two box sizes [EWC]

A box is four nested rectangles, content, padding, border, margin, and everything turns on which of them the `width` property names. Under `content-box`, the default, width names the content edge and padding and border add on. Under `border-box`, width names the border edge and the content area shrinks to pay for what it contains. Margin sits outside both, it separates siblings, it is never part of the box. The stylesheet holds both sizes side by side:

```
/* The reset answer: border-box makes the width property name the
   border edge, so a width: 100% child with padding stops
   overflowing its parent. */
*,
*::before,
*::after {
  box-sizing: border-box;
}

/* content-box, the default: width names the content edge and
   padding and border add on. The 120px declaration puts 144px on
   the line, 176px once the 16px margins separate it. */
.card-content {
  box-sizing: content-box;
  width: 120px;
  padding: 8px;
  border: 4px solid #333;
  margin: 16px;
}

/* border-box: the same declarations, the width now names the
   border edge, the content area shrinks to 96px to pay for the
   padding and border it contains. */
.card-border {
  box-sizing: border-box;
  width: 120px;
  padding: 8px;
  border: 4px solid #333;
  margin: 16px;
}
```

listing 136: the reset, then the same card measured under content-box and border-box (source: interview-repertoire/samples/ch32-js/src/box-model.css, lines 5-34)

The arithmetic the listing implies is one function pair:

```
// The outer width, border edge to border edge. content-box adds
// padding and border to the declared width, border-box already
// contains them. Margin sits outside both: it separates siblings,
// it is not part of the box.
export function outerWidth(width, padding, border, boxSizing = "content-box") {
  if (boxSizing === "border-box") return width;
  return width + 2 * (padding + border);
}

// The content width, what the children get to lay out in. The same
// pair of numbers, read the other way.
export function contentWidth(width, padding, border, boxSizing = "content-box") {
  if (boxSizing === "content-box") return width;
  return width - 2 * (padding + border);
}
```

listing 137: outerWidth and contentWidth, the same numbers read both ways (source: interview-repertoire/samples/ch32-js/src/boxsize.mjs, lines 5-19)

The spoken answer with numbers: a `width: 120px` card with `8px` padding and a `4px` border occupies 144px under content-box and 120px under border-box, the content area 120px and 96px respectively, and the `16px` margins push neighbors 176px and 152px away. That arithmetic is why the universal reset is `box-sizing: border-box`: sizing by the border edge makes a `width: 100%` child with padding stop overflowing its parent, and every framework ships exactly that reset.

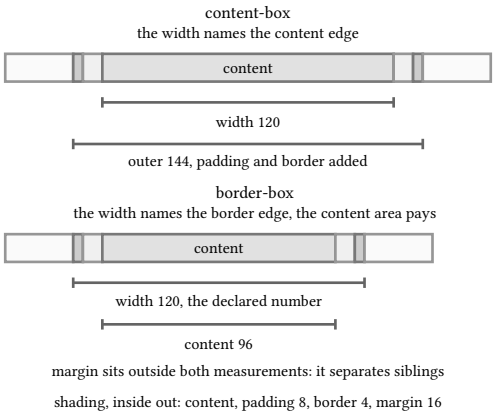


figure 147: one element measured both ways, the width property names a different edge

32.2. the cascade, specificity, and selector matching [TDD]

The cascade answers “which declaration wins” with a sort, not a vote, and matching runs first. A selector either describes a node or it does not, and the engine below is the whole grammar the interview needs, type, class, id, and the descendant and child combinators, walked over the plain tree book 17, ch 8 (the interview repertoire, javascript and the dom) builds:

```
// One compound like "li.card#first": a type, any number of
// classes, an optional id, all of which must sit on the node.
export function matchesCompound(node, compound) {
  const type = /^[a-z][\w-]*/i.exec(compound)?.[0];
  const id = /#([\w-]+)/.exec(compound)?.[1];
  const classes = [...compound.matchAll(/\.([\w-]+)/g)].map((m) => m[1]);
  if (type && node.tag !== type) return false;
  if (id && node.attributes?.id !== id) return false;
  const own = (node.attributes?.class ?? "").split(/\s+/).filter(Boolean);
  return classes.every((c) => own.includes(c));
}

// "ul.list > li.card" becomes units with a combinator each: the
// first unit has none, ">" is child, a bare space is descendant.
export function parseSelector(selector) {
  const units = [];
  let combinator = "none";
  for (const token of selector.split(/\s+/).filter(Boolean)) {
    if (token === ">") {
      combinator = "child";
      continue;
    }
    units.push({ compound: token, combinator });
    combinator = "descendant";
  }
  return units;
}

// Can `node` be the subject, the rightmost unit, with every unit
// to its left satisfied against `ancestors` (nearest last)?
function chainFrom(node, units, i, ancestors) {
  if (!matchesCompound(node, units[i].compound)) return false;
  if (i === 0) return true;
  const parent = ancestors[ancestors.length - 1];
  if (units[i].combinator === "child") {
    return parent !== undefined && chainFrom(parent, units, i - 1, ancestors.slice(0, -1));
  }
  for (let k = ancestors.length - 1; k >= 0; k--) {
    if (chainFrom(ancestors[k], units, i - 1, ancestors.slice(0, k))) return true;
  }
  return false;
}
```

listing 138: compound matching, the two combinators, and the rightmost-first chain walk (source: interview-repertoire/samples/ch32-js/src/match.mjs, lines 6-47)

The subject of a selector is its rightmost compound, so the walk tests every node as the subject and looks leftward through its ancestors:

```
// Every node in tree order that the selector's subject matches.
// The browser's engine differs by using indexes, not by being a
// different idea.
export function select(root, selector) {
  const units = parseSelector(selector);
  const hits = [];
  const walk = (node, ancestors) => {
    if (units.length > 0 && chainFrom(node, units, units.length - 1, ancestors)) {
      hits.push(node);
    }
    for (const child of node.children ?? []) walk(child, [...ancestors, node]);
  };
  walk(root, []);
  return hits;
}
```

listing 139: select: every node in tree order the subject matches (source: interview-repertoire/samples/ch32-js/src/match.mjs, lines 49-63)

Conflict resolution is a sortable key with four fields, compared in order, importance, origin, specificity, source order, and specificity itself is a triple (a, b, c), ids, classes, types, compared lexicographically, so one id beats any pile of classes and one class beats any pile of types:

```
// Specificity as (a, b, c): ids, classes, types, summed across
// every compound of the selector. Combinators carry none.
export function specificity(selector) {
  const ids = (selector.match(/#[\w-]+/g) ?? []).length;
  const classes = (selector.match(/\.[\w-]+/g) ?? []).length;
  const types = (selector.match(/(?:^|[\s>])[a-z][\w-]*/gi) ?? []).length;
  return [ids, classes, types];
}

// Lexicographic compare: -1 when a loses, 0 on a tie, 1 when a
// wins. (1, 0, 0) beats (0, 99, 99), which is the whole point.
export function compareSpecificity(a, b) {
  for (let i = 0; i < 3; i++) {
    if (a[i] !== b[i]) return a[i] - b[i];
  }
  return 0;
}
```

listing 140: specificity as a triple, compared lexicographically (source: interview-repertoire/samples/ch32-js/src/cascade.mjs, lines 6-22)

```
function beats(a, b) {
  if (a.important !== b.important) return a.important;
  const aInline = a.origin === "inline";
  const bInline = b.origin === "inline";
  if (aInline !== bInline) return aInline;
  const byClass = compareSpecificity(specificityOf(a), specificityOf(b));
  if (byClass !== 0) return byClass > 0;
  return a.order > b.order;
}

// The winning declaration of a conflict. Rules carry either a
// selector or an explicit specificity, an important flag, an order,
// and optionally origin "inline" for the style attribute.
export function winner(rules) {
  let best = null;
  for (const rule of rules) {
    if (best === null || beats(rule, best)) best = rule;
  }
  return best;
}
```

listing 141: the tiebreaker chain and the winning declaration (source: interview-repertoire/samples/ch32-js/src/cascade.mjs, lines 28-47)

An inline style enters with an origin above every selector, which is the "inline beats id" rule, and `!important` flips the importance field, which is why an author's important declaration beats a plain style attribute and the attribute's own important beats it back. The tests pin all three inversions.

PITFALL: !important is a debugger, not a layer

Importance outranks every rung below it, including declarations you have not written yet, so each `!important` silently raises the stakes for the next conflict and the sheet converges on two kinds of important. The cascade already has a tiebreaker for "this rule should win", specificity and order. Reach for important to settle an argument with a third-party stylesheet you cannot reorder, never inside your own.

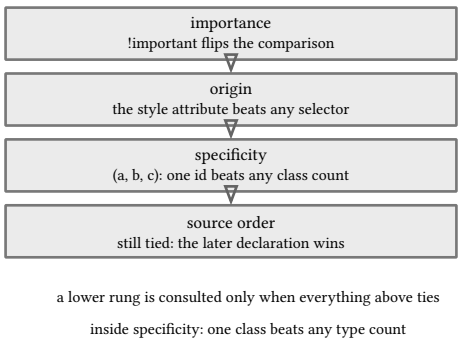


figure 148: the cascade ladder, one rung per tiebreaker, the first difference decides

32.3. flex and grid, the layout answers [EWC]

Flex and grid answer "where does the content land" by giving the space a different owner. In flex the items own a basis and the container's leftover space is distributed into the items, in proportion to their grow factors. In grid the tracks own the space, the fixed tracks are paid first and the remainder splits by the fr ratio, and the items land wherever the tracks resolved. The stylesheet holds the three layouts the inventory names, a nav bar, a card grid, a holy-grail skeleton:

```
/* The nav bar: one flex line, the brand left and the links right,
   gap spacing them instead of margins. */
.nav {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
}

.nav ul {
  display: flex;
  gap: 1rem;
  list-style: none;
}

/* The card grid: tracks sized by minmax, the grid owns the column
   count and no flex free-space walk ever runs. */
.card-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(180px, 1fr));
  gap: 1rem;
}

/* The holy grail: header and footer rows, three columns where the
   sides stay fixed and the article takes the remaining fr. */
.holy {
  display: grid;
  min-height: 100vh;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header header header"
    "nav article aside"
    "footer footer footer";
  grid-template-columns: 180px 1fr 220px;
}

.holy > header { grid-area: header; }
.holy > nav { grid-area: nav; }
.holy > article { grid-area: article; }
.holy > aside { grid-area: aside; }
.holy > footer { grid-area: footer; }
```

listing 142: the flex nav bar, the auto-fill card grid, the grid holy grail (source: interview-repertoire/samples/ch32-js/src/layout.css, lines 5-45)

The arithmetic each implies:

```
// Flex free-space distribution: every item keeps its basis, the
// leftover space (container minus the bases) splits in proportion
// to each grow factor. Grow 0 takes none of it.
export function distributeFlexGrow(items, containerWidth) {
  const totalGrow = items.reduce((sum, it) => sum + (it.grow ?? 0), 0);
  const free = containerWidth - items.reduce((sum, it) => sum + it.basis, 0);
  if (totalGrow === 0) return items.map((it) => it.basis);
  return items.map((it) => it.basis + (free * (it.grow ?? 0)) / totalGrow);
}

// Grid fr resolution: a track is a number of pixels (fixed) or
// { fr: n }. The fixed tracks take their length first, the
// remainder splits by the fr ratio, so 1fr 3fr over 800px is
// 200px and 600px.
export function resolveFr(tracks, space) {
  let fixed = 0;
  let frUnits = 0;
  for (const track of tracks) {
    if (typeof track === "number") fixed += track;
    else frUnits += track.fr;
  }
  const share = frUnits > 0 ? (space - fixed) / frUnits : 0;
  return tracks.map((track) => (typeof track === "number" ? track : track.fr * share));
}
```

listing 143: free space by grow ratio, fr tracks against what the fixed tracks leave (source: interview-repertoire/samples/ch32-js/src/layout.mjs, lines 5-28)

The numbers to narrate: a 300px line holding two 100px items with grow 1 and 3 has 100px free, splits it 25 and 75, and lands at 125 and 175. A 1000px grid with a 200px fixed track and 1fr 3fr remaining pays the 200 first and splits the remaining 800 into 200 and 600. Centering is its own spoken answer with its own precedent, flex, grid, and absolute plus translate, floored to book 17, ch 8 (the interview repertoire, javascript and the dom), and nothing here revises it.

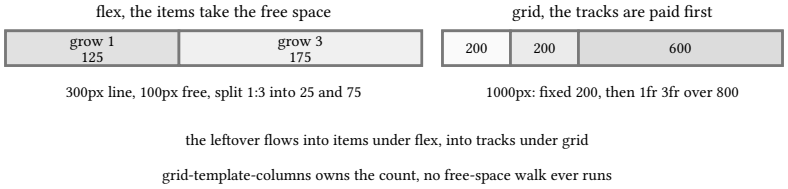


figure 149: flex distributes leftover space into items, grid distributes it into tracks first

32.4. responsive and media queries [TDD]

A media query is a predicate over the viewport, and the css it guards applies while the predicate holds. `min-width` and `max-width` are both inclusive at their boundary, `and` demands every condition, and the engine is deliberately small:

```
export function mediaApplies(query, viewport) {
  return query.split(/\s+and\s+/).every((condition) => holds(condition, viewport));
}

function holds(condition, viewport) {
  const m = /\((min|max)-width:\s*([\d.]+)px\)/.exec(condition);
  if (!m) throw new Error(`unsupported media condition: ${condition}`);
  const bound = Number(m[2]);
  return m[1] === "min" ? viewport.width ≥ bound : viewport.width ≤ bound;
}

// The mobile-first pick: the base block is the styles outside any
// query, and each min-width block overrides while it matches. The
// applicable block that answers is the last one in source order.
export function pickBlock(blocks, viewport) {
  let chosen = null;
  for (const block of blocks) {
    if (mediaApplies(block.query, viewport)) chosen = block;
  }
  return chosen;
}
```

listing 144: applicability as a predicate, and the mobile-first pick (source: interview-repertoire/samples/ch32-js/src/media.mjs, lines 5-25)

Mobile-first is the discipline that falls out: write the small-screen styles as the base, add `min-width` blocks that only add, and each block overrides while its query matches, so the answer for any viewport is the last applicable block in source order. The spoken rule the interviewer is checking: breakpoints sit where the content breaks, not where a device table says, and the file below never unsays anything the base declared:

```css
/* Mobile-first: the base styles are the small-screen styles, and
   every breakpoint below sits where the content breaks, not where
   a device table says it should. */

.grid {
  display: grid;
  gap: 1rem;
}

/* One card per line is the base. Around 600px the cards can pair
   up, so the first breakpoint pairs them. */
@media (min-width: 600px) {
  .grid {
    grid-template-columns: repeat(2, 1fr);
  }
}

/* Three across once the line can hold them, and the nav toggle
   retires in favor of the full nav. */
@media (min-width: 900px) {
  .grid {
    grid-template-columns: repeat(3, 1fr);
  }

  .nav-toggle {
    display: none;
  }
}

/* The next breakpoint is not a number yet: it arrives when the
   content, watched at real sizes, breaks. */
```

listing 145: the base and two breakpoints, whole file (source: interview-repertoire/samples/ch32-js/src/responsive.css, lines 1-31)

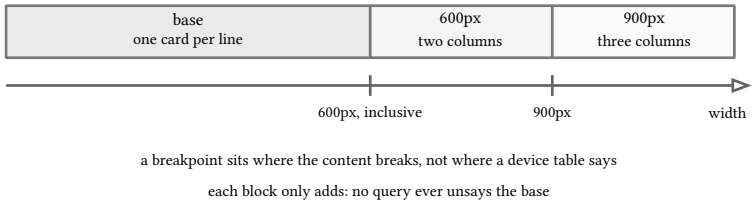


figure 150: the breakpoint ladder: the base is always on, each block adds from its threshold rightward

32.5. the spoken css answers [DRILL]

z-index and stacking contexts: z-index orders elements inside a stacking context, and the trap is that it only takes effect on positioned elements, anything but static, and on children of flex and grid containers. A new context is born from opacity below 1, a transform, a filter, fixed or sticky positioning, and a child inside a context can never win against one outside it, so the element with z-index 9999 losing to a sibling is almost always two contexts, not a number too small.

em versus rem: em multiplies the font size it inherits, so nested ems compound down the tree, 1.2em inside 1.2em on a 16px root is 23px, and the third level compounds again. rem performs one multiplication against the root size and never compounds. The rule to say out loud: type in rem when it should track the page default, spacing in em when it should track the element's own type, and know which one the value sits under.

display none, visibility hidden, opacity 0: display none removes the box, nothing is laid out, painted, or announced. visibility hidden keeps the box and its space, invisible but present. opacity 0 keeps box and pixels, transparent

but still there, still clickable unless pointer-events says no. The distinction that carries into the next chapter: only display none removes the element from the accessibility tree, the other two leave it in, which is why visually hidden utilities use clip patterns instead of opacity when the content should stay announceable.

The position values: static is the flow. relative offsets from the flow slot and leaves the slot behind. absolute positions against the nearest positioned ancestor and leaves the flow. fixed positions against the viewport. sticky is relative until a scroll threshold, then held inside its containing block, and the trap: sticky needs a scroll container and a threshold, a sticky child of a non-scrolling parent does nothing.

vh versus dvh: vh is measured against the large viewport, the mobile browser with its url bar hidden, so a 100vh hero overflows the visible screen while the bar is shown. dvh tracks the dynamic viewport as the bar shows and hides, svh and lvh name the small and large ends. The honest trade: dvh resizes while the user scrolls, which can shift layout during the scroll, so heroes take dvh and stable chrome takes svh.

Why border-box is the reset default: it makes width name the border edge, so padding stops breaking 100 percent widths, the arithmetic in the first section, and it is the one line every framework agrees on.

sources: verified by `npm run verify` under node 26.3.0, 14 tests in `ch32-js`, zero dependencies. The stylesheets are real files, listed verbatim, not executed by the suite. The tree the selector engine walks and the centering precedent both belong to book 17, ch 8 (the interview repertoire, javascript and the dom), the invalidation consumer of these styles is the virtual dom, book 17, ch 11 (the interview repertoire, react core: components, props, virtual dom), and an id inside a selector is a hash lookup in the browser's implementation, the kinship to book 10, ch 6 (practical data structures and algorithms, hashing).

33. the rendering pipeline and accessibility

Chapter 8 built the dom as a tree, chapter 32 built the stylesheets as models, and this chapter walks what the browser does with both: two trees parsed, intersected into a render tree, laid out, painted, composited, and beside the pixels a second published tree, the accessibility tree, derived from the dom for assistive technology. The `ch33-js` workspace classifies a change by the stages it invalidates, reduces the dom to the accessibility tree, and computes the WCAG contrast ratios, 10 tests under `node --test`, zero dependencies.

33.1. parse to dom and cssom [EWC]

Html parsing builds the dom, one node per tag, the same tree book 17, ch 8 (the interview repertoire, javascript and the dom) builds and walks. Css parsing builds the cssom, and it is a tree for the same reason the dom is: rules nest, and inheritance flows down it, a resolved font-size on the body is the inherited starting point of every descendant, and every value in it has already survived the cascade book 17, ch 32 (the interview repertoire, css answers: box model, cascade, layout) sorts. The two trees never merge. What merges is their intersection, the render tree of the next section, and everything downstream of parsing consumes that intersection. The browser derives one more tree from the dom alone, the accessibility tree of the fourth section, which css touches only through the hiding properties, display none and visibility, never through the cascade.

33.2. layout, paint, composite [DRILL]

Layout, called reflow when it reruns, walks the render tree and gives every node a box, position and size, geometry. Paint walks the boxes and emits paint lists, the order to fill, stroke, and place text in. Compositing hands layers to the gpu and combines them, the stage where transform and opacity do their work without touching the two above it. Each stage consumes the previous one's output, which is why the cost of a change is the earliest stage it invalidates plus everything downstream, the question the classifier in the next section answers.

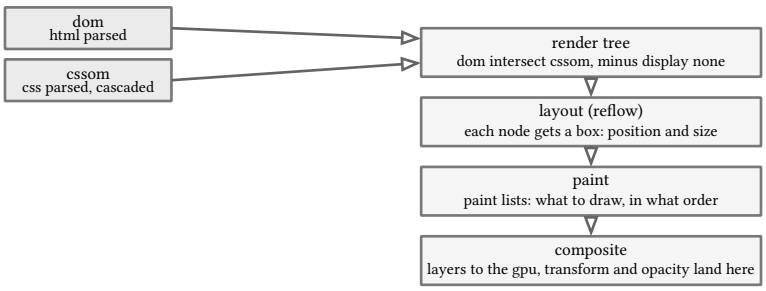


figure 151: two trees in, a render tree out, then the three stages, each producing its own artifact

A visibility hidden node joins the render tree, laid out and painted as transparent. A display none node never joins it, no box, no paint, and no accessibility entry, the property the fourth section inherits.

33.3. reflow versus repaint versus compositing [TDD]

The classifier is a table walked by a function, the change names its class and the class names its stages:

```
// width/height/top force layout and everything downstream;
// color/background only invalidate paint; transform/opacity ride
// the compositor alone; a text content change re-lays-out.
const LAYOUT_CHANGES = new Set(["width", "height", "top", "left", "font-size", "textContent"]);
const PAINT_CHANGES = new Set(["color", "background", "border-color", "box-shadow"]);
const COMPOSITE_CHANGES = new Set(["transform", "opacity"]);

export function classifyChange(change) {
  if (LAYOUT_CHANGES.has(change)) return ["layout", "paint", "composite"];
  if (PAINT_CHANGES.has(change)) return ["paint", "composite"];
  if (COMPOSITE_CHANGES.has(change)) return ["composite"];
  throw new Error(`unmodeled change: ${change}`);
}

// A class change costs whatever the delta declares: pass the
// properties that actually differ between the two classes, the
// union of their stages answers, in pipeline order.
export function classifyClassChange(differing) {
  const stages = new Set();
  for (const change of differing) {
    for (const stage of classifyChange(change)) stages.add(stage);
  }
  return ["layout", "paint", "composite"].filter((stage) => stages.has(stage));
}
```

listing 146: stage classification, and a class change paid by the union of its delta (source: interview-repertoire/samples/ch33-js/src/pipeline.mjs, lines 6-29)

A class change is answered by its delta: diff the declared properties of the two classes and pay the union of what differs, a swap that only moves color repaints, one that moves width relayouts. The honest framing to volunteer before anyone asks: this is the spec-level truth, what each property invalidates by definition. Browsers optimize underneath it, a width change that lands on an unchanged subtree may relayout nothing it does not touch, and a transform-only animation never enters layout at all. The classifier is the guarantee, the optimizations are why unmeasured intuitions surprise you.

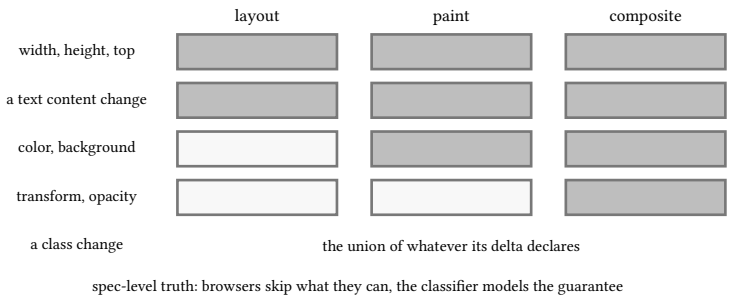


figure 152: the trigger table: one row per change class, a filled cell per stage it pays

33.4. the accessibility tree and semantic html [TDD]

The accessibility tree is the dom as assistive technology receives it, derived, never authored: roles come from tags, names from text and attributes, structure from nesting. The reducer is honest about its one hard rule, display none, spelled by the hidden attribute in this model, and aria-hidden prune a node together with its whole subtree:

```
// display:none is the only hiding that removes the node from the
// accessibility tree; hidden and aria-hidden="true" prune the same
// way in this model.
function pruned(node) {
  const attrs = node.attributes ?? {};
  return attrs.hidden !== undefined || attrs["aria-hidden"] === "true";
}

// An anchor is only a link while it carries an href, which is the
// trap: a bare <a> degrades to a generic node.
export function implicitRole(node) {
  if (node.tag === "a") return node.attributes?.href ? "link" : "generic";
  if (/^h[1-6]$/.test(node.tag)) return "heading";
  return IMPLICIT_ROLES[node.tag] ?? "generic";
}

// aria-label wins, alt names an image, the node's own text names
// the rest.
export function accessibleName(node) {
  const attrs = node.attributes ?? {};
  if (attrs["aria-label"]) return attrs["aria-label"];
  if (node.tag === "img") return attrs.alt ?? null;
  return node.text ?? null;
}

export function accessibilityTree(node) {
  if (pruned(node)) return null;
  const attrs = node.attributes ?? {};
  const role = attrs.role ?? implicitRole(node);
  const entry = { role, name: accessibleName(node), children: [] };
  if (role === "heading") entry.level = Number(node.tag[1]);
  for (const child of node.children ?? []) {
    const branch = accessibilityTree(child);
    if (branch) entry.children.push(branch);
  }
  return entry;
}
```

listing 147: pruning, implicit roles, naming, and the reduction itself (source: interview-repertoire/samples/ch33-js/src/a11y.mjs, lines 16-52)

```
// The tab-order walk: naturally focusable elements and tabindex 0
// in document order, positive tabindex first in numeric order (the
// anti-pattern that scrambles it), tabindex -1 removed entirely.
export function tabOrder(root) {
  const entries = [];
  const walk = (node) => {
    if (pruned(node)) return; // a pruned node is not focusable either
    const attrs = node.attributes ?? {};
    const tab = attrs.tabindex =/= undefined ? Number(attrs.tabindex) : null;
    const focusable = tab =/= null || isNaturallyFocusable(node);
    if (focusable && tab =/= -1) {
      entries.push({ role: attrs.role ?? implicitRole(node), name: accessibleName(node), tabindex: tab ??
0 });
    }
    for (const child of node.children ?? []) walk(child);
  };
  walk(root);
  const positives = entries.filter((e) => e.tabindex > 0).sort((a, b) => a.tabindex - b.tabindex);
  const rest = entries.filter((e) => e.tabindex ≤ 0);
  return [...positives, ...rest];
}

function isNaturallyFocusable(node) {
  if (node.tag === "a") return Boolean(node.attributes?.href);
  return ["button", "input", "select", "textarea"].includes(node.tag);
}
```

listing 148: the tab-order walk, positives first, tabindex −1 dropped (source: interview-repertoire/samples/ch33-js/src/a11y.mjs, lines 54-78)

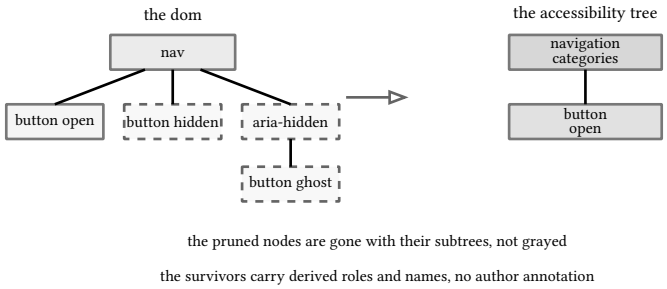


figure 153: the nav subtree before and after the reduction: hidden prunes, roles and names survive

Semantic html is the answer to “how do you make it accessible”: the native tags already carry the roles, the h1 through h6 outline, the landmarks nav and main, and a button that is focusable and announceable with no attributes at all. The trap the model pins: an anchor is only a link while it carries an href, a bare anchor degrades to a generic node, no role, no focus, because the role is conditional on the attribute.

33.5. aria, contrast, and keyboard paths [TDD]

Contrast is arithmetic, and the arithmetic is the WCAG 2.2 definition: relative luminance linearizes each srgb channel and combines them with the rec 709 weights, the ratio is the lighter luminance plus 0.05 over the darker plus 0.05, and the thresholds are 4.5:1 for body text, 3:1 for large text and for non-text ui components, criteria 1.4.3 and 1.4.11:

```
export function linearChannel(value) {
  const c = value / 255;
  return c ≤ 0.04045 ? c / 12.92 : ((c + 0.055) / 1.055) ** 2.4;
}

export function relativeLuminance(rgb) {
  const [r, g, b] = rgb.map(linearChannel);
  return 0.2126 * r + 0.7152 * g + 0.0722 * b;
}

// (lighter + 0.05) / (darker + 0.05), direction-agnostic.
export function contrastRatio(fg, bg) {
  const l1 = relativeLuminance(fg);
  const l2 = relativeLuminance(bg);
  return (Math.max(l1, l2) + 0.05) / (Math.min(l1, l2) + 0.05);
}

// The WCAG 2.2 thresholds: 1.4.3 holds text to 4.5:1 and large
// text to 3:1, 1.4.11 holds non-text ui components to 3:1.
export const CONTRAST_THRESHOLDS = { text: 4.5, largeText: 3, ui: 3 };

export function passesContrast(ratio, threshold = CONTRAST_THRESHOLDS.text) {
  return ratio ≥ threshold;
}
```

listing 149: luminance, the ratio, and the three thresholds (source: interview-repertoire/samples/ch33-js/src/contrast.mjs, lines 6-29)

The known numbers the tests pin: black on white is exactly 21:1 either direction, #767676 on white is 4.54:1, the lightest gray that still passes body text, and #888888 is 3.55:1, enough for large text and ui edges, not enough for a paragraph. The tab walk beside it: naturally focusable elements and tabindex 0 follow document order, positive tabindex jumps the queue in numeric order, tabindex −1 leaves the walk while staying programmatically focusable. The first rule of aria, prefer the native element: a button is a state machine someone else maintains, a div with role button is one you maintain, and the role attribute carries none of the keyboard behavior for free.

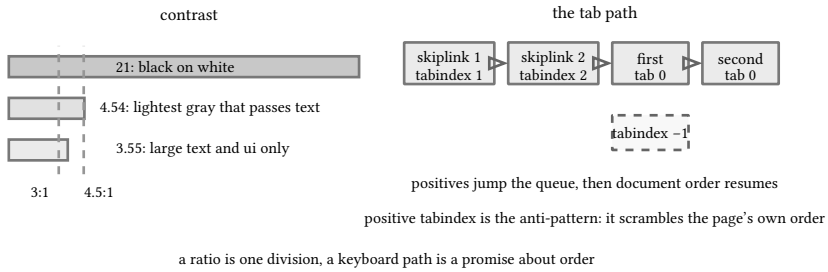


figure 154: the contrast ladder against its two thresholds, and the keyboard path the tab walk takes

33.6. the spoken rendering answers [DRILL]

What forces reflow: writing geometry then reading it. A style write marks layout dirty, the next `offsetHeight` or `getBoundingClientRect` read forces the relayout synchronously, and interleaving writes and reads inside a loop pays a full layout per iteration, the layout thrash. The fix is batching, every write, then every read.

Why animations jank: an animation that animates width pays layout on every frame, one that animates transform and opacity pays compositing only, the gpu lane, and the frame budget is roughly 16ms at 60hz. The spoken rule: animate transform and opacity, keep width and top out of the hot path, and measure when a transform-only animation still stutters, because the compositor has its own ceiling.

How the virtual dom helps, floored to book 17, ch 11 (the interview repertoire, react core: components, props, virtual dom): the diff batches declarative changes into one mutation pass, so a component updating ten nodes

pays one layout for the batch instead of ten interleaved ones, and reconciliation is the pipeline's biggest customer, the reason a framework's update model is a rendering answer, not only a state one.

What the accessibility tree is: the dom rewritten as roles, names, and structure, pruned by display none and aria-hidden, published to assistive technology, and the reason semantic tags are the cheapest accessibility there is, they populate the tree with no author effort at all.

Semantic landmarks and heading order: nav, main, header, footer, aside name the regions a screen reader jumps between, and h1 through h6 outline the page one level at a time, a path the reader walks, which is why skipping from h2 to h5 is an audit finding even when the type size looks right.

floored to the corpus: the dom tree itself and the tree walk idiom are book 17, ch 8 (the interview repertoire, javascript and the dom), the scheduling that decides when any of this runs is the event loop of book 17, ch 7 (the interview repertoire, javascript core: scope, closures, event loop), the server-side contrast to one process juggling many connections is epoll in book 17, ch 26 (the interview repertoire, systems and runtime answers), and the batching consumer is the virtual dom of book 17, ch 11 (the interview repertoire, react core: components, props, virtual dom). The contrast thresholds are WCAG 2.2's own rows, 1.4.3 and 1.4.11, w3.org/TR/WCAG22, accessed 2026-09-29. Verified by `npm run verify` under node 26.3.0, 10 tests in `ch33-js`, zero dependencies.

34. security answers: the named taxonomy

The security round is a naming round. Handed a broken endpoint or a leaky page, the scoring move is to name the family first, because the family names the fix and the fix names the test that proves it. This chapter owns the web families the six service lanes floor elsewhere and the rehearsal book had no chapter for: xss, csrf, cors, the injection grammar, and the response headers. The `ch34-js` workspace runs the models under `node --test`, 14 tests, and the models are honest about being hand-modeled spec shapes, not a browser and not a database.

34.1. the taxonomy, owasp-shaped [DRILL]

The OWASP Top 10 is the shared vocabulary and the 2025 edition moved the entries around, so quote it by name and year (OWASP, "Top 10:2025", fetched 2026-09-29): broken access control still holds the top spot, security misconfiguration sits second, supply chain failures entered third, cryptographic failures fourth, and injection fell from third in 2021 to fifth. The numbers drift, the families do not, and the interview answer rides the family. When handed a vulnerability, name the class, then the fix, then the floor in this corpus: broken access control means authorization was checked at the route and not at the object, floored by the six authz chapters, book 4, ch 22 (go 1.27, authz and rbac) first among them. Injection means input reached a parser as grammar, floored by the parameterized store chapters of the six service parts. Cryptographic failures mean a hash, a comparison, or a key choice was wrong, floored by the authn lanes this chapter re-quotes below. Xss, csrf, and cors are this chapter's own. Ssrf, the server fetching a url an attacker controls, gets named here and floored on the kernel and middleware chapters of the six service parts, where the honest defense lives: allowlist the egress destinations, refuse to fetch user-supplied urls, pin the scheme.

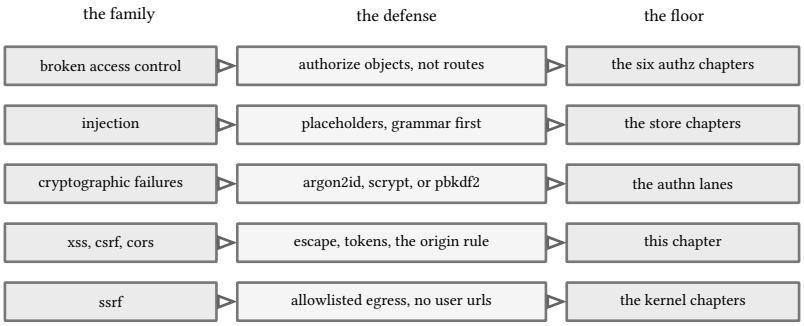


figure 155: name the family, the family names the fix, the fix names the floor

34.2. xss and csrf [TDD]

Xss is a context problem, which is why one universal sanitizer is the wrong answer. The same payload string is inert in a text node, a breakout attempt inside a quoted attribute, and a scheme execution in a url sink, and each surface has its own escape. The workspace walks all three:

```
export function escapeText(s) {
  let out = String(s);
  for (const [from, to] of TEXT_ENTITIES) out = out.replaceAll(from, to);
  return out;
}

// an attribute value sits inside quotes, so the quotes themselves are
// the boundary: leaving either one live lets the payload close the
// attribute and open a new one (onmouseover and friends)
export function escapeAttr(s) {
  let out = escapeText(s);
  out = out.replaceAll('"', "&quot;");
  out = out.replaceAll("'", "&#39;");
  return out;
}

// url sinks execute schemes, so the check strips the whitespace and
// control bytes browsers strip before reading a scheme, then keeps
// only the schemes on the allowlist. deny by default, not by denylist
const ALLOWED_SCHEMES = new Set(["http:", "https:", "mailto:"]);
const STRIPPED = new Set([" ", "\t", "\n", "\r", "\0"]);

export function safeUrl(u) {
  const trimmed = Array.from(String(u))
    .filter((c) => !STRIPPED.has(c))
    .join("");
  const colon = trimmed.indexOf(":");
  if (colon === -1) return { ok: true, url: trimmed };
  const scheme = trimmed.slice(0, colon + 1).toLowerCase();
  if (ALLOWED_SCHEMES.has(scheme)) return { ok: true, url: trimmed };
  return { ok: false, scheme, reason: "scheme refused" };
}
```

listing 150: text entities, the attribute quotes that close the boundary, and the url scheme allowlist with the whitespace browsers strip (source: interview-repertoire/samples/ch34-js/src/escape.mjs, lines 11-42)

The spoken taxonomy: stored xss persists the payload in the database and fires for every viewer, reflected xss rides the request back into the page, dom-based xss never leaves the browser, a sink like innerHTML writing a value the server never saw. The fix tracks the sink, not the source, and the honest concession is that the model escapes strings, it does not run a parser.

Csrf exploits the browser, not the user: the attacker's page cannot read your cookies across origins, but it can make the browser attach them to a forged request. The double-submit defense demands something an attacker's page cannot supply, an echo of a token it was never allowed to read:

```
export function timingSafeEqual(a, b) {
  const x = Buffer.from(String(a), "utf8");
  const y = Buffer.from(String(b), "utf8");
  if (x.length !== y.length || x.length === 0) return { equal: false, reads: 0 };
  let diff = 0;
  let reads = 0;
  for (let i = 0; i < x.length; i++) {
    diff |= x[i] ^ y[i];
    reads += 1;
  }
  return { equal: diff === 0, reads };
}

export function verifyDoubleSubmit(cookieToken, headerToken) {
  if (!cookieToken || !headerToken) {
    return { ok: false, reason: "missing token half" };
  }
  return { ok: timingSafeEqual(cookieToken, headerToken).equal };
}
```

listing 151: the timing-safe compare, every byte read into one accumulator, the read count riding the return so the test proves no early exit (source: interview-repertoire/samples/ch34-js/src/csrf.mjs, lines 13-31)

The compare must be timing-safe or the defense leaks a matching prefix through response time, one byte of the token confirmed per forged request. The test drives a first-byte mismatch and asserts the model still read every byte. SameSite=Lax is the modern first line, the browser declines to attach the cookie to cross-site posts, and it kills most csrf with no code at all. The trap it leaves open:

PITFALL: Lax still rides top-level navigation

Lax sends the cookie on top-level navigation with safe methods, so an endpoint that mutates state on GET keeps the csrf hole open even with Lax everywhere. State changes belong behind non-safe methods, and the token pair stays as the second lock. And the classic trap question, can CORS prevent csrf: no, CORS is a read relaxation, the browser sends the forged request regardless and CORS only decides whether the attacker's page may read the response, and simple requests fly without a preflight at all.

34.3. cors and the same-origin rule [TDD]

The same-origin policy is the browser's default isolation: a script on one origin may not read a response from another. Cors is the relaxation mechanism, and the whole subject reduces to one triple and one decision. The triple:

```
export function originOf(url) {
  const m = String(url).match(/^([a-z]+):\/\/([^/?#]+)/i);
  if (!m) return null;
  let host = m[2];
  let port = null;
  const colon = host.lastIndexOf(":");
  if (colon !== -1 && /^\d+$/.test(host.slice(colon + 1))) {
    port = Number(host.slice(colon + 1));
    host = host.slice(0, colon);
  }
  const scheme = m[1].toLowerCase();
  return { scheme, host: host.toLowerCase(), port: port ?? DEFAULT_PORTS[scheme] ?? null };
}

export function sameOrigin(a, b) {
  const x = originOf(a);
  const y = originOf(b);
  if (!x || !y) return false;
  return x.scheme === y.scheme && x.host === y.host && x.port === y.port;
}
```

listing 152: the origin triple with the https 443 and http 80 defaults filled in before any comparison (source: interview-repertoire/samples/ch34-js/src/origin.mjs, lines 8-27)

The traps the triple sets: scheme differs means origin differs, so https on 443 and http on 80 are two origins on one hostname, and subdomains are different origins outright, `app.example.com` is not `example.com`. The decision is which requests preflight. Per MDN (“Cross-Origin Resource Sharing”, fetched 2026-09-29), a request flies without preflight only when the method is GET, HEAD, or POST, every header is on the safelist, accept, accept-language, content-language, content-type, range, and the content type is one of the three form values. Everything else sends an OPTIONS first:

```
// which requests preflight: the safelisted methods fly without asking,
// the safelisted headers stay unread, and the content type must stay a
// form value. everything else sends an OPTIONS first
const SAFE_METHODS = new Set(["GET", "HEAD", "POST"]);
const SAFE_HEADERS = new Set(["accept", "accept-language", "content-language", "content-type", "range"]);
const SIMPLE_CONTENT_TYPES = new Set([
  "application/x-www-form-urlencoded",
  "multipart/form-data",
  "text/plain",
]);

export function needsPreflight({ method, headers = [], contentType = null }) {
  if (!SAFE_METHODS.has(method)) return true;
  if (headers.some((h) => !SAFE_HEADERS.has(h.toLowerCase()))) return true;
  if (contentType !== null && !SIMPLE_CONTENT_TYPES.has(contentType.split(";")[0].trim())) return true;
  return false;
}

// the response decision: a same-origin request never consults cors, an
// allowed cross-origin request gets its origin echoed back, the
// wildcard is for anonymous reads only, and the wildcard with
// credentials is the error row, the browser refuses the combination
export function acao({ origin, allowed, credentials = false, wildcard = false }) {
  if (!allowed) return { header: null, decision: "omit, no advice for this origin" };
  if (wildcard && credentials) {
    return { header: null, decision: "refuse: the wildcard cannot ride credentials" };
  }
  if (wildcard) return { header: "*", decision: "wildcard, anonymous reads" };
  return { header: origin, decision: "echo, exact origin" };
}
```

listing 153: the preflight triggers, then the response decision: exact-origin echo, the anonymous wildcard, and the credentials error (source: interview-repertoire/samples/ch34-js/src/origin.mjs, lines 29-58)

The response table the interviewer wants: an allowed origin gets its exact origin echoed in ACAO, the wildcard serves anonymous reads only, and the wildcard with credentials is the error row, the browser refuses the combination outright, so credentialed cross-origin reads need the echo plus `Access-Control-Allow-Credentials: true`.

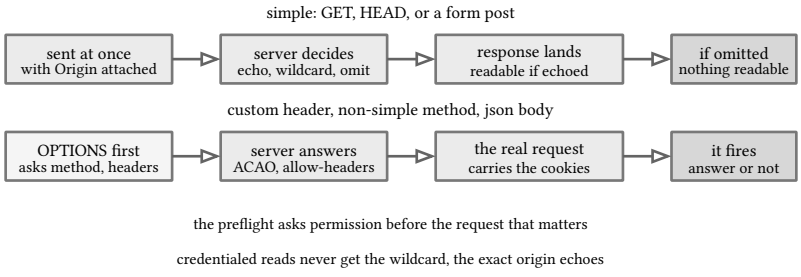


figure 156: the two lanes: simple requests fly, everything else asks first

34.4. sql injection [TDD]

The one-sentence version interviewers keep scoring: concatenation lets input change the grammar of the query, placeholders decide the grammar before any input exists. The workspace models the taint, and the classic payload rides inside the listing where it belongs:

```
export function concatenated(base, userInput) {
  return { sql: base + userInput, tainted: true, params: [] };
}

export function parameterized(base, params) {
  return { sql: base, tainted: false, params };
}

// does the assembled text carry the classic tautology walk: a quoted
// exit, an always-true OR, or a comment swallowing the tail
export function carriesTautology(assembled) {
  return /'\s*or\s+1\s*=\s*1|--|\/\*/i.test(assembled.sql);
}
```

listing 154: the taint model: concatenation tainted by construction, the placeholder version with the sql text fixed (source: interview-repertoire/samples/ch34-js/src/query.mjs, lines 7-19)

```
const BASE = "SELECT id, name FROM users WHERE name = '";
const PAYLOAD = "x' OR 1=1 --";

test("concatenation lets the payload rewrite the grammar", () => {
  const got = concatenated(BASE, PAYLOAD);
  assert.equal(got.sql, BASE + PAYLOAD);
  assert.equal(got.tainted, true);
  assert.equal(carriesTautology(got), true);
  assert.ok(got.sql.includes("OR 1=1"));
  // the closing quote is commented out, the predicate is true for every row
  assert.ok(got.sql.endsWith("--"));
```

listing 155: the payload closing the quote, splicing the always-true OR, and commenting out the tail (source: interview-repertoire/samples/ch34-js/src/query.test.mjs, lines 5-15)

Read the defeat aloud: the payload closes the string literal the template opened, splices an OR over a tautology, and the comment swallows the closing quote, so the predicate is true for every row. The parameterized version is indifferent to the same bytes, the sql text never changes, and the payload arrives at the engine as an ordinary string value that matches no name in the table. The follow-up is second-order injection: a payload stored safely through placeholders can still detonate later when a different code path concatenates it back out of the database, which is why the fix is parameterization everywhere rather than sanitizing input at the door. The floors are real: the store chapters of the six service parts run prepared statements against real engines, and the query side of the rehearsal is book 17, ch 17 (the interview repertoire, database answers). The concession stands, this model flags strings, it does not parse sql.

34.5. security headers: csp, hsts, and friends [TDD]

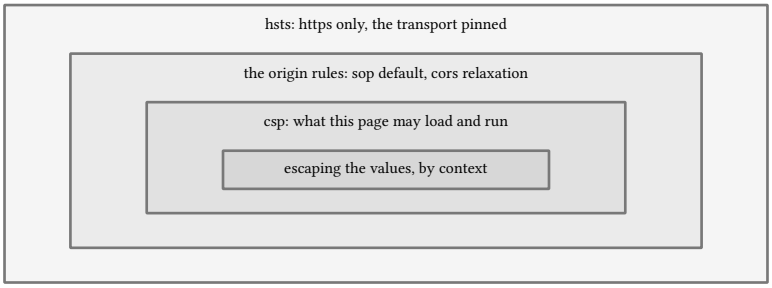
Headers are the defense you can read in a curl. Csp restricts what a page may load and execute, hsts pins the transport, and the workspace parses both as data:

```
export function parseCsp(header) {
  const directives = {};
  for (const part of String(header).split(";")) {
    const tokens = part.trim().split(/\s+/).filter(Boolean);
    if (tokens.length === 0) continue;
    directives[tokens[0]] = tokens.slice(1);
  }
  const script = directives["script-src"] ?? directives["default-src"] ?? [];
  const hasNonceOrHash = script.some((s) => s.startsWith("'nonce-") || s.startsWith("'sha"));
  return {
    directives,
    scriptSources: script,
    unsafeInline: script.includes("'unsafe-inline'"),
    unsafeInlineIgnored: script.includes("'unsafe-inline'") && hasNonceOrHash,
  };
}

// hsts as data: max-age in seconds, includeSubDomains extends the
// policy down the host, and preload signals eligibility for the
// browser-built lists, which demand one year plus subdomains
export function parseHsts(header) {
  const parts = String(header).split(";").map((p) => p.trim().toLowerCase());
  let maxAge = null;
  for (const p of parts) {
    const m = p.match(/^max-age=(\d+)$/);
    if (m) maxAge = Number(m[1]);
  }
  const includeSubDomains = parts.includes("includesubdomains");
  const preload = parts.includes("preload");
  return {
    maxAge,
    includeSubDomains,
    preload,
    preloadEligible: preload && includeSubDomains && maxAge ≠ null && maxAge ≥ 31536000,
  };
}
```

listing 156: csp directives with the script-src reading and its default-src fallback, the nonce-ignores-unsafe-inline case, and the hsts parse with the preload bar (source: interview-repertoire/samples/ch34-js/src/headers.mjs, lines 7-42)

Two facts carry the Csp questions. `unsafe-inline` defeats the script policy, MDN is blunt about it (“Content-Security-Policy”, fetched 2026-09-29: developers should avoid `'unsafe-inline'` because it defeats much of the purpose of having a CSP). And the quieter trap: a nonce or hash beside `unsafe-inline` makes the browser ignore the keyword, so a policy carrying both has silently changed meaning rather than failed. Hsts parses to three knobs, max-age in seconds, includeSubDomains extending the policy down the host, and preload, whose browser lists demand one year of max-age plus subdomains (MDN, “Strict-Transport-Security”, fetched 2026-09-29). The spoken ladder the round actually tests: same-origin policy, CORS, and CSP are three different things. SOP is the browser’s default isolation, CORS relaxes who may read across origins, CSP restricts what a page may load inside its own origin. Confusing any two is the fail marker.



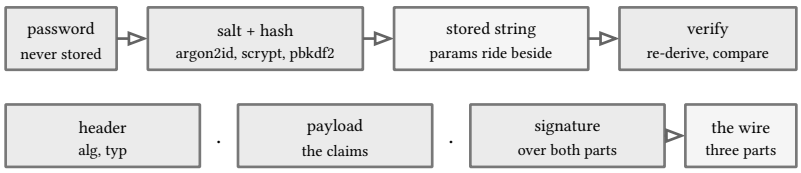
no https and the origin rules are spoofable, no csp and one injected tag runs

figure 157: the layers are concentric: each one assumes the layer outside it held

34.6. authn, authz, tokens: oauth and oidc at interview altitude [DRILL]

The six lanes each built the same spine and chose their hash on platform honesty, so name the lane and its choice: go runs argon2id with EdDSA tokens, book 4, ch 21 (go 1.27, authn). C sharp runs `Rfc2898DeriveBytes.Pbkdf2` and ES256, its reader pinning the algorithm before any payload byte is trusted so the classic algorithm-confusion attack dies at the door, book 6, ch 25 (c# 15, f# 11, and .net 11, authn). Javascript runs scrypt at `N=2^17`, 128 MiB of memory against gpu farms, with ed25519 for signatures. Python runs `hashlib.pbkdf2_hmac`, salted sha256 iterated 210000 times. Lua builds sha-256 by hand and chains hmac and pbkdf2 over it. The c lane walks the whole ladder, sha-256, then hmac, then pbkdf2, each floor built before the next composes it. Bcrypt gets named honestly on the OWASP row (Password Storage Cheat Sheet, fetched 2026-09-29): argon2id first, minimum 19 MiB of memory at two iterations, scrypt the fallback, bcrypt for legacy only, work factor at least 10, and its 72-byte input cap is the reason a bcrypt password field maxes there, the pedigree and the cap both worth saying out loud. Memory-hardness is why argon2id won the recommendation, it prices gpu attack the way bcrypt could not.

The authz half is one set of answers across all eight lanes. 401 versus 403: unauthenticated versus authenticated and still refused. IDOR, insecure direct object reference, is authorization missing at the object level, the route checked the session but never asked whether that session owns the id in the path. Deny by default means the route audit exists and fails when a new route appears ungated, and the rbac matrix is a test, not a wiki page, book 4, ch 22 (go 1.27, authz and rbac). Token format: JWT versus PASETO is floored on book 16, ch 22 (interview defense cookbook, order-demo deep dive), whose answer is the pinned algorithm and the encrypted payload. Oauth 2.0 is delegated authorization, not authentication, RFC 6749′s four roles (fetched 2026-09-29) are resource owner, client, authorization server, resource server, and none of them assert who the user is. OIDC is the authentication layer on top, the id token “represents the outcome of an authentication process” (OpenID Foundation, “How OpenID Connect Works”, fetched 2026-09-29). The spoken flow is authorization code plus PKCE: the code is short-lived, single-use, delivered through the browser channel, and exchanged server-side, while PKCE’s verifier and challenge close the code-interception hole for clients that cannot hold a secret. Key exchange belongs to the math: book 2, ch 17 (mathematics for programmers, group actions and structure) builds diffie-hellman in the open and ends with the sentence this book repeats, never build crypto from a textbook toy.



the signature is over the bytes, dots included, padding stripped

one stored secret, per-request proof, refresh rotation on reuse

figure 158: the hash lane proves who you are once, the token lane carries the proof per request

34.7. attack rounds [DRILL]

The strongest security answer is a test someone ran. This corpus proved its own lanes three ways worth quoting by name. Go holds forgery resistance as a fuzz target, `FuzzJWTParse` feeding mutated tokens to the verifier and holding the alg pin through all of them, book 4, ch 30 (go 1.27, the test suite). C sharp runs the ETag duel over the wire, two racing writers and a conditional request referee deciding which byte wins, book 6, ch 28 (c# 15, f# 11, and .net 11, concurrency control). Lua replays the four vector scenarios, the happy walk, the login limits, the refresh reuse attack, and the roles story, each on its own fresh fixture because the fourth acts with the session the third revoked, book 9, ch 31 (lua 5.5, the test suite). Rate limiting belongs in the same breath, the bucket and the 429 contract floored by book 4, ch 26 (go 1.27, rate limiting). And the meta-answer for any find-the-vulnerability prompt, the three sentences that end the round: name the class, name the fix, name the test that proves it. The first sentence earns the vocabulary, the second earns the engineering, the third earns the job.

sources: verified by `npm run verify` under node 26.3.0, 14 tests in `ch34-js`, and the models are hand-modeled spec shapes, not a browser and not a database. The lanes floor the spine: go's argon2id and EdDSA chapters at book 4, ch 21 (go 1.27, authn) with its authz and limit chapters beside them, c sharp's Rfc2898 and ES256 lane at book 6, ch 25 (c# 15, f# 11, and .net 11, authn), javascript's scrypt and ed25519 lane, python's 210000-iteration pbkdf2 lane, lua's hand-built hmac and pbkdf2 lane, and the c lane's sha256, hmac, pbkdf2 ladder. Https floors to book 17, ch 18 (the interview repertoire, network answers), PASETO versus JWT to book 16, ch 22 (interview defense cookbook, order-demo deep dive), key exchange and the never-build-toys sentence to book 2, ch 17 (mathematics for programmers, group actions and structure). Online claims verified 2026-09-29: MDN's CORS, CSP, and HSTS pages, OWASP's Top 10:2025 and Password Storage Cheat Sheet, RFC 6749, and the OpenID Foundation's OIDC overview, each cited where it lands above with its access date.

35. back of the envelope, said aloud

Estimation rounds are won before any number appears: the interviewer is listening for whether the candidate knows what to estimate, states assumptions out loud, and lands inside a factor. This chapter drills the spoken method, the number ladder every round expects, and five timed boards, all of it arithmetic on textbook values, none of it a measurement. The floors are real: the latency ladder belongs to the patterns handbook, and the posture the estimates get checked against, mean versus p99, was measured in the six service books, not guessed at a whiteboard.

35.1. the method [DRILL]

Four moves, in order, every time. First, restate the quantity being estimated as a formula in words, users times requests per user per day over seconds in a day, because the formula is the answer and the number is only its evaluation. Second, name the per-unit size, the bytes per row, the requests per second per pod, and say the assumption out loud so the interviewer can correct it, a corrected assumption is a free point, a silent one is a hidden fault. Third, do the arithmetic aloud in powers of ten, rounding as you go, because the listener is grading the arithmetic, not the product. Fourth, sanity-band the result against something known, another system, a published number, the same estimate cut a different way, and name the band: within 2x is the claim, a tighter claim needs a measurement.

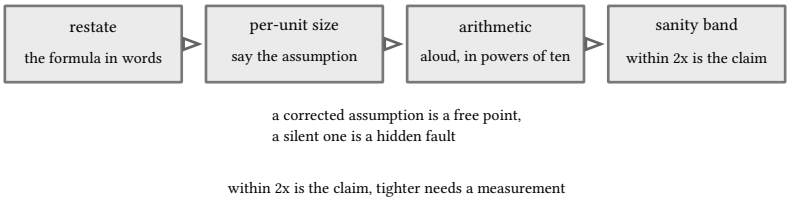


figure 159: the four moves, in order, every estimate

35.2. the numbers every round expects [DRILL]

Powers of two first: 2^10 is a thousand, 2^20 a million, 2^30 a billion, 2^40 a trillion, every ten powers of two buying three powers of ten. The drift is the detail that lands: each step runs 2.4 percent heavy, compounding to about 10 percent at 2^40, so say “roughly” once and stop apologizing. Time units in seconds: a day is 86,400, call it 10^5 when coarse, a year is about 3.15 x 10^7, which is pi times ten to the seventh and the one mnemonic worth keeping, a million seconds is 11.6 days, a billion is 32 years.

Then the latency ladder, restated exactly as the patterns handbook pins it: a round trip within a datacenter is around 0.5 milliseconds, across a continent around 30, and reading an nvme drive is around 100 microseconds. A protocol that adds 10 sequential cross-continent round trips costs 300 milliseconds before any work happens, book 12, ch 10 (design patterns, concurrency, and distributed systems, distributed fundamentals), whose source row pins norvig’s numbers and the berkeley interactive guide. The ladder earns its keep as a budget: 0.5 milliseconds a hop says ten service hops inside one datacenter cost 5 milliseconds, and one continental hop costs sixty datacenter hops, which is why locality is a design decision and not a tuning knob.

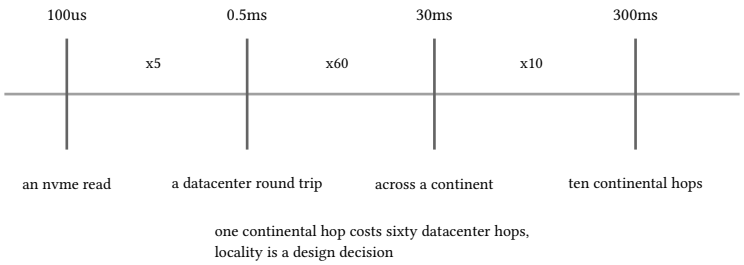


figure 160: the ladder as a budget: each tick costs the one before it many times over

35.3. storage and memory, worked [DRILL]

The template board, said aloud. 100 million rows at one kilobyte each is 10^8 times 10^3, which is 10^11 bytes, which is 100 gigabytes, decimal convention, stated, and kept for the rest of the board. The index doubles it: a b-tree index over the key plus row metadata runs to about half the payload, and the write amplification of maintaining it is why the honest number is 2x, so 200 gigabytes all in. Row versus page arithmetic: a 4 kilobyte page holds four of these rows, so the table is roughly 25 million pages, and the number to say is that scans read pages, not rows, so a query touching one column of every row still pays the full 100 gigabytes unless an index or a columnar store changes what a read means.

Sizing a cache from the hot set: the traffic rule of thumb is that 90 percent of requests touch 10 percent of keys, so the hot set is 10 million keys at a kilobyte, 10 gigabytes, plus per-entry overhead that a redis-style store charges at roughly 2x the payload, call it 20 gigabytes. That number decides the architecture, a hot set that fits one cache node is one node, one that does not is a cluster or a tiered design, and the 90 percent hit ratio that keeps the database seeing only the tail is the same sentence the 300k sketch of book 17, ch 19 (the interview repertoire, system design answers) leans on.

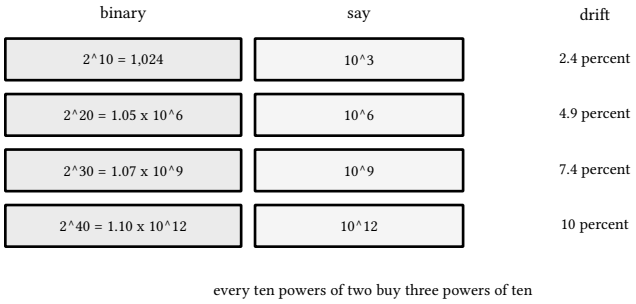


figure 161: powers of two against powers of ten, with the drift named

35.4. qps, fanout, headroom, worked [DRILL]

Split reads from writes before any other number, because every capacity decision downstream depends on the ratio, and most systems run 10 to 1 read heavy, some 100 to 1. Then say what the average hides: mean and p99 are different systems, a 30 millisecond mean with a 200 millisecond p99 is a service that is fast for most callers and broken for a profitable few, and the p99 is the number capacity is bought against. That posture is measured, not estimated, in this corpus: the six service books each carry a load chapter that drives real traffic through the service and reports the percentiles, one representative being book 4, ch 28 (go 1.27, load and p99), and the estimation drill's job is to predict the order of those curves before they are measured.

The 30k template, restated from the design chapter as the pattern to reuse: 300k concurrent users at one request per user per ten seconds is 30k requests per second, pods sized at 2k requests per second make 15 pods by arithmetic, and the queue and the cache absorb what the arithmetic cannot see. Fanout multiplies it: a write that must reach 300k live connections is not one write, it is a fanout of 300k deliveries, and the write storm is a different system from the write that produced it. Headroom, said as a rule: size for twice the estimate, because the estimate is arithmetic on guesses, the spike is real, and half a quiet cluster is cheap while one saturated cluster is an outage. The rule buys the rolling deploy too, half the fleet can be down for a release precisely when the estimate was doubled.

35.5. five boards in ten minutes [DRILL]

Five timed boards, two minutes each, the arithmetic written the way it should be said.

Videos served a day. Assume a billion users, one video each per day, so 10^9 views over 86,400 seconds is about 12,000 views a second average, peak two to three times that on a shared evening timezone. Egress owns the design: 10^9 views at 30 megabytes is 30 petabytes a day, no origin serves that, so the cdn is the system and the origin is its fill tier, the arithmetic decides the architecture before any box is drawn.

Cache or database for a lookup. The store answers a point lookup in about a millisecond, memory answers in the microseconds, and at 30k requests a second the store column is 30 concurrent in-flight queries before any spike. The deciding number is the hit ratio: 90 percent hits cut store load tenfold, and the hot set arithmetic from the storage board is what says whether that ratio is buyable, so the answer is a hit-ratio question, never a preference.

Letters in the mail store. Assume 200 million letters a day, metadata at a kilobyte, so 200 gigabytes a day, about 73 terabytes a year, one ordinary database per year of metadata. A scanned image at 200 kilobytes is 40 terabytes a day, 14.6 petabytes a year, so scanning everything is a policy decision the arithmetic exposes, not a default a storage team drifts into.

One server or ten. A box carries about 2k requests a second, the estimate says 12k, arithmetic buys 6 boxes, practice buys 10: the extra four are headroom plus the ability to take half the fleet down for a rolling deploy. One server stays right until it is a single failure domain, a deploy outage, and a queue for every restart at once, and the tenth box is cheaper than the incident that proves the point.

Shard or replicate. Read heavy at 10 to 1 with 30k requests a second means about 27k reads: replicas carrying 2k reads each make 14 replicas, and the one writer at 3k writes a second is still comfortable, so replicate first, it is the cheaper rung and it buys read scale without moving any data. Sharding is the answer when the single writer saturates or the store outgrows one machine, it buys write scale at the price of cross-shard queries and rebalancing, and saying “replicate first, shard last” in that order is the answer the round listens for.

floored to: every number in this chapter is arithmetic on textbook values, none is a measurement. The latency ladder and its pinned source, norvig’s numbers and the berkeley interactive guide, belong to book 12, ch 10 (design patterns, concurrency, and distributed systems, distributed fundamentals). The 300k arithmetic restated as the qps template belongs to book 17, ch 19 (the interview repertoire, system design answers). The measured posture the estimates are checked against, mean versus p99 under real load, lives in the six service books’ load chapters, book 4, ch 28 (go 1.27, load and p99) as one representative of the lanes the service corpus closed.

36. system design prompts, four worked

The design round eventually stops asking about shapes and hands over a prompt: design a url shortener, a rate limiter, a news feed, a chat service. This chapter works all four with the same protocol, the 300k sketch of book 17, ch 19 (the interview repertoire, system design answers) promoted from one answer to a method, and one prompt, the shortener, ships test-driven go code because its two interesting pieces, a base62 codec and a windowed counter allocator, are exactly interview-sized. The other three are drills floored on systems this corpus actually runs, and the delivery ladder at the close settles the long-polling question the network chapter leaves open.

36.1. how to run a design prompt [DRILL]

The protocol, six moves on a clock. Requirements first, functional and non-functional, and the number you are told, users, requests, retention, is a requirement, write it down and use it. Estimation is the opening move, not garnish: run the four-move method of book 17, ch 35 (the interview repertoire, back of the envelope, said aloud) on the given number before any box is drawn, because the arithmetic picks the topology more often than preference does. Then the topology, drawn, and the walk-aloud over it, naming what each hop does and what it costs. Volunteer the bottleneck before being asked, the interviewer is grading whether you can see it, and meet every follow-up with a tradeoff, never a defense: the question is rarely “is your design wrong”, it is “what does it cost to be right a different way”.

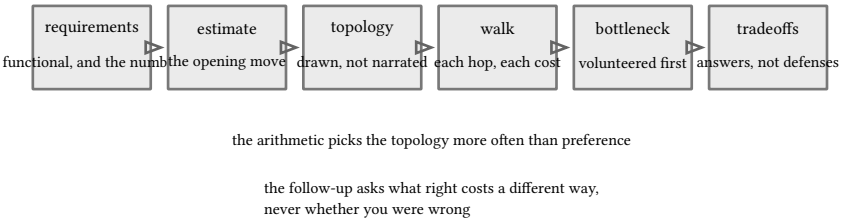


figure 162: the prompt clock: six moves, the estimate second, never last

36.2. url shortener [TDD]

The arithmetic first, from a stated assumption of 100 million new links a year. That is 3 writes a second steady, 30 at a 10x peak, and a 10 to 1 read ratio makes reads 300 a second at peak. The five-year store is 500 million links at about 150 bytes a row, 75 gigabytes of payload, twice that with the index, so 150 gigabytes before replicas, call the honest band 150 to 250. The keyspace arithmetic is the part worth saying slowly: 62^7 is about 3.5 trillion codes, seven characters cover 500 million links roughly seven-thousand-fold, so length is never the constraint and growth buys decades, not a migration.

Code assignment is the one real decision, and the workspace ships it test-driven. A hash of the url needs a collision check on every insert, a database round trip on the hot path, while a counter hands out integers that are unique by construction and encodes them in base62. The codec, with the overflow guard checked before the multiply because a wrapped int64 is silent:

```
// Encode renders n in base 62. Negative n panics: the allocator never
// mints one, and inventing a spelling the decoder would refuse hides a
// bug rather than reporting it.
func Encode(n int64) string {
  if n < 0 {
    panic("base62: negative code")
  }
  if n == 0 {
    return "0"
  }
  // 62^11 overflows int64, so 11 characters is the ceiling and the
  // buffer never grows
  var buf [11]byte
  i := len(buf)
  for n > 0 {
    i--
    buf[i] = alphabet[n%62]
    n /= 62
  }
  return string(buf[i:])
}

// Decode parses s back to its integer. Both failures are errors, never
// zero-and-hope: a byte outside the alphabet, and a value past int64,
// checked before the multiply because a wrapped int64 is silent.
func Decode(s string) (int64, error) {
  if s == "" {
    return 0, ErrSyntax
  }
  const max = int64(1<<63 - 1)
  var n int64
  for i := 0; i < len(s); i++ {
    d := digit(s[i])
    if d < 0 {
      return 0, ErrSyntax
    }
    if n > (max-int64(d))/62 {
      return 0, ErrOverflow
    }
    n = n*62 + int64(d)
  }
  return n, nil
}
```

listing 157: encode by repeated division, decode with the pre-multiply overflow check (source: interview-repertoire/samples/ch36-go/base62.go, lines 19-61)

The allocator is the piece interviewers probe: two api instances must never mint the same code. The answer is windows, each instance reserves a range from one persistent counter and serves it from local arithmetic, so the hot path coordinates nothing:

```go
// Counter is the one coordination point: reserve size consecutive
// codes and return the first. A deployment backs this with a single
// counter row updated inside one transaction, so instances never talk
// to each other on the hot path, each talks to the counter once per
// window and serves size codes from local arithmetic alone.
type Counter interface {
  Reserve(size int64) (start int64, err error)
}

// WindowAllocator hands out consecutive codes from the window it
// holds, refreshing from the counter only when the window runs dry.
type WindowAllocator struct {
  counter Counter
  size    int64
  next    int64 // the next code to hand out
  end     int64 // the first code outside the window
}

// New claims the first window immediately, so a broken counter fails
// at construction instead of on the first request.
func New(c Counter, size int64) (*WindowAllocator, error) {
  if size <= 0 {
    return nil, errors.New("allocator: window size must be positive")
  }
  a := &WindowAllocator{counter: c, size: size}
  if err := a.refill(); err ≠ nil {
    return nil, err
  }
  return a, nil
}

// Next returns one code. Calls inside a window touch nothing but the
// allocator's own fields, and exhaustion crosses the boundary as an
// error the caller can map to an http 503 rather than a wrong answer.
func (a *WindowAllocator) Next() (int64, error) {
  if a.next ≥ a.end {
    if err := a.refill(); err ≠ nil {
      return 0, err
    }
  }
  code := a.next
  a.next++
  return code, nil
}

func (a *WindowAllocator) refill() error {
  start, err := a.counter.Reserve(a.size)
  if err ≠ nil {
    return err
  }
  a.next, a.end = start, start+a.size
  return nil
}
```

listing 158: the counter is the one coordination point, once per window (source: interview-repertoire/samples/ch36-go/allocator.go, lines 8-60)

The suite pins the properties the design claims: every one- and two-character code round-trips, 3,906 of them, max int64 round-trips at exactly eleven characters, non-alphabet input and past-int64 values are errors, two allocators over one counter hand out disjoint codes, and a drained pool returns the exhaustion error rather than a repeated code, eleven tests under `go vet`, `go test`, and `go test -race`.

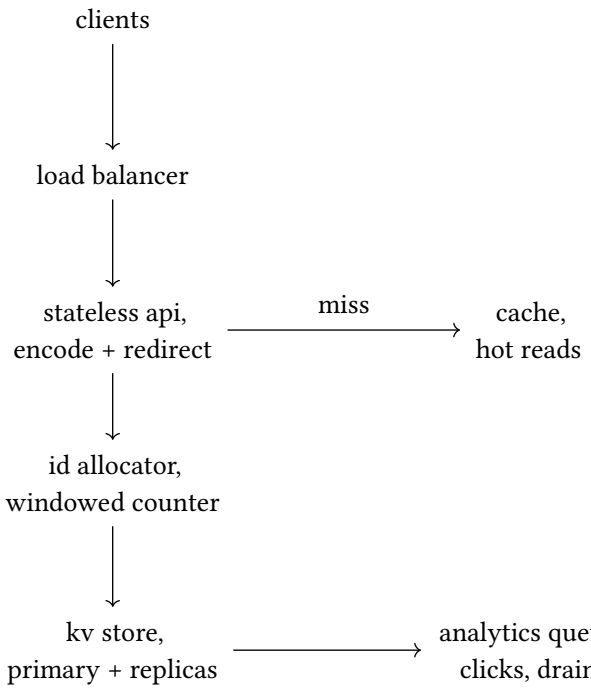


figure 163: the shortener: stateless edges, an allocator per pod, one store, cache beside

The punchline to volunteer before being asked: at a 10 to 1 read ratio and 300 reads a second, the shortener is a cache problem wearing a design crown, hot codes live in memory with a ttl near the code's lifetime, cold codes in a kv store one lookup deep, and the redirect is one line of handler. The traps, one line each. Hash with collision check against counter assignment, counter wins, the collision check is a store hit on every write. Custom aliases, a unique index and a clean rejection, never a retry loop. 301 against 302: 301 is cached by browsers and stops the analytics, 302 keeps every click visible and pays the redirect, say which you are choosing and why. Expiry is a lazy delete, a ttl checked on read plus a nightly sweep, the store never blocks on it. Rate limiting the creator, not the reader, is the abuse surface, the bucket shape and its 429 contract are book 4, ch 26 (go 1.27, rate limiting).

36.3. rate limiter [DRILL]

The arithmetic from the stated load: 100k requests a second peak, 10 million active users, a limit of 100 requests per minute per user. The bucket per user refills at 100 over 60, about 1.67 tokens a second, with burst 100. State is the number that decides the shape: 64 bytes a user is 640 megabytes for everyone, about a gigabyte all in, so the full state fits one machine and the question becomes where the check runs. The trade, quantified with the datacenter round trip: a shared-store check costs about 0.5 milliseconds per request, which at 100k a second is 50 seconds of added latency across the fleet every second, while a local bucket adds nothing but n instances can allow n times the limit until they converge. Converge by asynchronous sync, buckets ship their deltas to the shared store every second and load their neighbors' state on startup, and say the approximation out loud: the limit is soft between syncs, worst case the burst times the sync interval, and a soft limit is the correct product answer for a rate limiter, an exact one is for money.

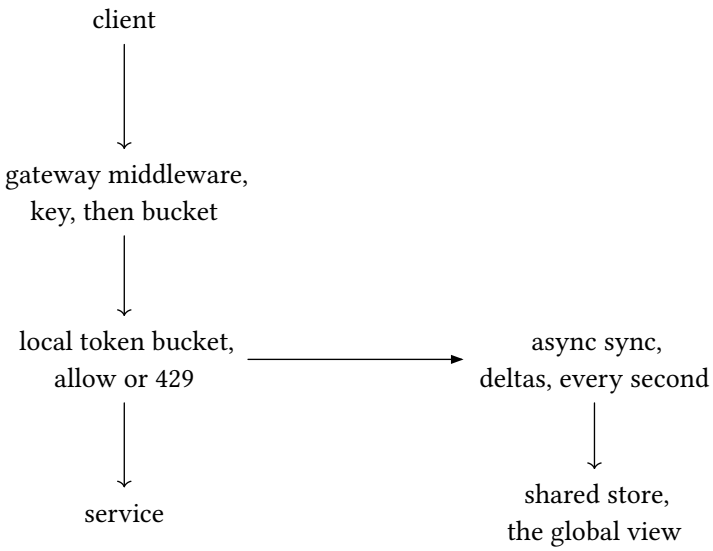


figure 164: local buckets on the hot path, the shared store off it, sync in between

The floors are the six limit chapters the service corpus shipped, one per language, the token bucket from first principles and its 429 contract in book 4, ch 26 (go 1.27, rate limiting), the platform contrast where the same bucket rides middleware in book 6, ch 30 (c# 15, f# 11, and .net 11, rate limiting), and the limiter as a resilience pattern beside retries and breakers in book 12, ch 15 (design patterns, concurrency, and distributed systems, resilience). The traps: 429 carries `Retry-After`, a client that cannot learn when to come back will hammer anyway. Burst against sustained, a limit of 100 a minute is not 100 at every instant, the bucket shape is the difference. Fairness across keys, one hot tenant must not eat the shared capacity, key the buckets and cap per key. What a paying tier buys is a higher refill, which is the same bucket with different constants, say that and the product conversation ends.

36.4. news feed [DRILL]

The arithmetic: 10 million users, 200 follows each, 2 posts a day each, so 20 million posts a day. Fanout-on-write multiplies each post by its audience: 20 million times 200 is 4 billion timeline writes a day, about 46k a second sustained, with the morning spike a multiple of that, and that is the write storm. Fanout-on-read inverts it: 200 merges per view, every reader pays the fanout, the store stays small and the read path is a distributed query. The celebrity number to say aloud: one post by a 1M-follower user is 1 million fanout writes in pure push, one user's Tuesday is more than twenty seconds of the entire fleet's write budget. The resolution is hybrid push-pull: push for ordinary users, pull at read time for celebrity follows, the threshold a tuned product number, and the same logic pulls for inactive users whose pushed timelines are never read. The timeline store is trimmed to 300 to 500 items per user, about 300 gigabytes at 100 bytes an item for 10 million users, and the trim is stated as a product contract change, the feed promises a window not a history, the same move as the queue-first contract shift in book 17, ch 19 (the interview repertoire, system design answers).

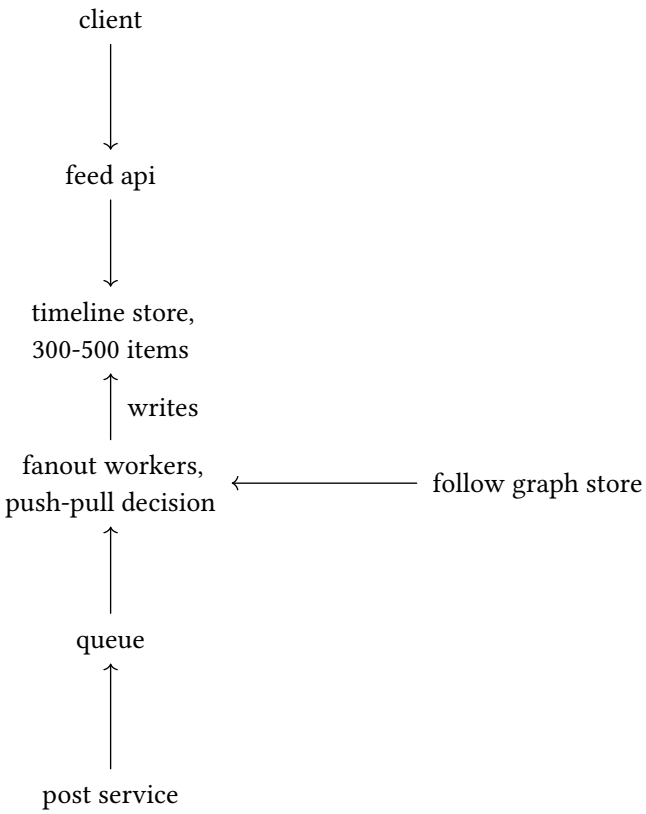


figure 165: posts through a queue to fanout workers, readers hit the timeline

The floors: per-key ordering and the outbox that keeps a post and its fanout message atomic are book 12, ch 13 (design patterns, concurrency, and distributed systems, messaging semantics and idempotency), and the feed that actually runs in this book, bounded history, replay after a sequence number, live tail, is book 17, ch 21 (the interview repertoire, go build part 2: storefront and live features), the same shape with one publisher and no fanout. The traps: deletion propagates as tombstones, a deleted post must leave every timeline it already entered. Ranking against chronological is a product decision that changes the store, ranked feeds precompute scores and the timeline becomes a cache of a model. Pagination consistency, a cursor into a moving timeline, the cursor must be a post id snapshot, never an offset. The user who follows 1 million is the celebrity problem seen from below, pull for them too, their view is a live merge the API assembles.

36.5. chat [DRILL]

The arithmetic: 1 million concurrent, 10 messages per user-hour, so 10 million messages an hour, about 2.8k a second, at 200 bytes each roughly 0.55 megabytes a second. Bandwidth is trivial, and saying so is the point: connections are the wall. A gateway pod holds about 100k sockets, the epoll economics of book 17, ch 26 (the interview repertoire, systems and runtime answers), so 1 million concurrent is 10 pods before headroom. Route by consistent hash on the conversation id so both participants land on the same pod, the partitioning logic of book 12, ch 12 (design patterns, concurrency, and distributed systems, partitioning and consistent hashing), and an in-memory conversation becomes possible: both sockets in one process, delivery is a channel send. Presence is ttl heartbeats, 1 million over 30 seconds is about 33k writes a second, coalesced into batches, and a missed heartbeat expires into “last seen”, never “gone”. Offline is ch19′s notification shape: an inbox queue per offline user, a push notification on arrival, a dead letter queue for the push that cannot land, delivery on reconnect. Receipts are at-least-once with client message-id dedupe, the exactly-once-effect contract of book 17, ch 25 (the interview repertoire, distributed systems answers).

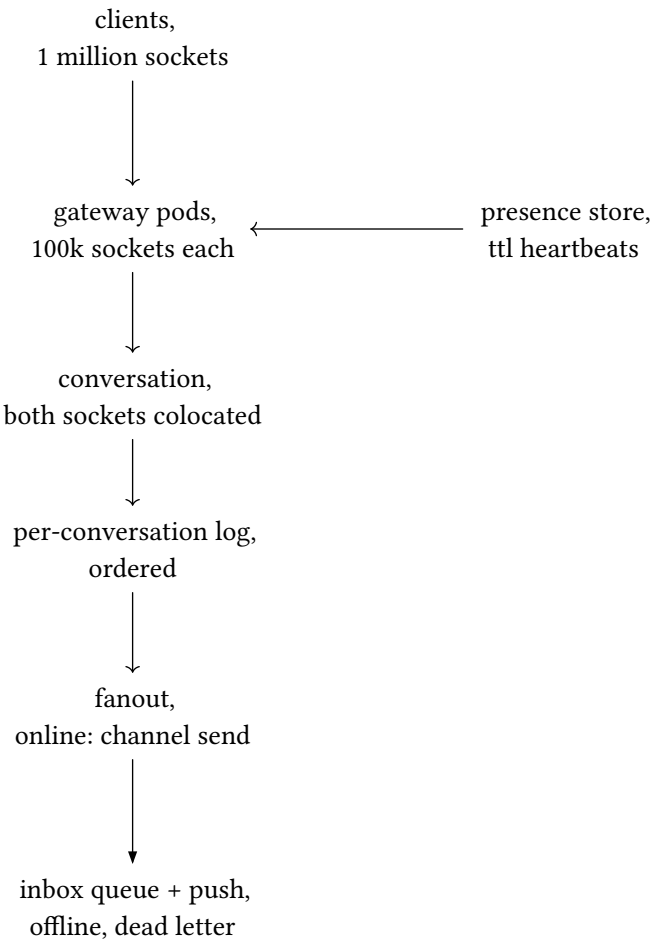


figure 166: hash by conversation, both sockets on one pod, inbox for the offline

The floors: the chat that actually runs under `docker compose up -d`, nats subjects, jetstream durability, htmx over sse, is book 13, ch 20 (infrastructure: docker, databases, and testing, capstone part 2: chat and notifications, one click), the durable consumer and its ack floor are book 13, ch 15 (infrastructure: docker, databases, and testing, jetstream: streams, consumers, kv, kafka compared), and the frames underneath, the websocket handshake arithmetic and the frame codec, are book 17, ch 18 (the interview repertoire, network answers). The traps: per-conversation ordering is the contract, global ordering is a non-goal that costs a single writer, say the scope before drawing. Offline sync is the inbox replay plus cursor, the same dedupe keys guarding the double delivery. Group fanout is the news feed's problem at conversation scale, a 500-member room is 500 fanout writes per message and the same hybrid logic applies. End-to-end encryption is the honest concession: the server can route and store ciphertext, key exchange and ratchets are a different discipline, book 2, ch 17 (mathematics for programmers, group actions and structure) for the structures, and a design round answer that claims e2ee and server-side search has not thought about either.

36.6. long polling, sse, websockets: the delivery ladder [DRILL]

Three mechanisms, by cost and semantics. Long polling is a held request: the client asks, the server holds the response open until data arrives or a timeout, the client re-asks. It costs one request per delivery, works through every proxy and firewall, and needs nothing from the server but a slow handler. Sse is a server-to-client stream over plain http, `text/event-stream`, one response held open with events flushed down it: one-way, cheap, reconnects built into the browser, the mechanism the feed of book 17, ch 21 (the interview repertoire, go build part 2: storefront and live features) uses. Websockets are bidirectional frames after an http upgrade handshake, both directions on one connection, the highest capability and the highest cost, the handshake arithmetic and the frame codec the network chapter implements from the metal. Each wins somewhere: polling when the network is hostile and the client is dumb, sse when the flow is server-to-client, which most feeds, tickers, and notifications are, websockets when the client pushes on the same channel, chat, games, collaborative editing. The fallback ladder is part of the

design: try websockets, degrade to sse when a proxy strips the upgrade, degrade to long polling when even the stream is refused, the client names its mechanism, the server speaks all three, and the long-polling rung closes the gap this book had, it was the one delivery mechanism named in prompts and never worked.

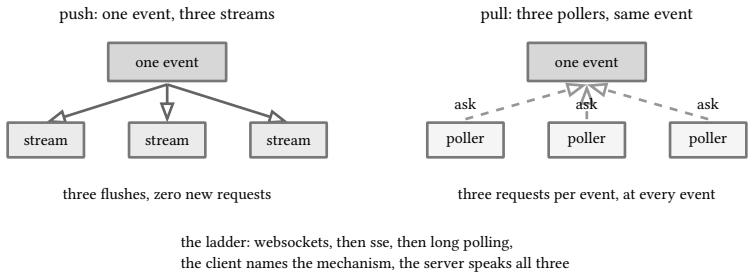


figure 167: push against pull: the same fanout through both delivery shapes

floored to: no measurements in this chapter, every number is arithmetic, the running systems it leans on are real. The estimation method is book 17, ch 35 (the interview repertoire, back of the envelope, said aloud). The shortener's keyspace arithmetic floors on book 10, ch 6 (practical data structures and algorithms, hashing) for the hash map behavior it declines and book 12, ch 12 (design patterns, concurrency, and distributed systems, partitioning and consistent hashing) for the consistent hash it borrows. Ordering, the outbox, and per-key lanes are book 12, ch 13 (design patterns, concurrency, and distributed systems, messaging semantics and idempotency). The queue and the chat that run are book 13, ch 15 (infrastructure: docker, databases, and testing, jetstream: streams, consumers, kv, kafka compared), book 13, ch 19 (infrastructure: docker, databases, and testing, capstone part 1: services and topology), and book 13, ch 20 (infrastructure: docker, databases, and testing, capstone part 2: chat and notifications, one click) under `docker compose up -d`. The wire underneath the delivery ladder is book 17, ch 18 (the interview repertoire, network answers), and the bucket behind the rate limiter is the six limit chapters, one per language of the service corpus.

37. behavioral answers, from the register

The behavioral round is the one round the corpus answers outright. Every story in this chapter is assembled from register rows, resume lines, and the defense cookbook chapters that already drill them, and the governing rule is the register's own: no invented employers, no invented outcomes, no number the record cannot show. Where the record is thin the card says what is checkable and stops, because the point of a behavioral answer is surviving the depth probe that follows it, and the depth probe here is a short one: open the register and show me. The question bank closes the chapter, sixteen questions, every one routed to a card, a script, or a chapter that already exists.

37.1. star, but honest [DRILL]

STAR is situation, task, action, result, and the house version states its discipline before its shape. The register's no-metrics row and the analysis metrics chapter mean the R is a scoped claim tied to an inspectable artifact, never an impact percentage. "The behavioral requirements held, the harness surfaced the undocumented behavior", "the client brought me back twice", "the archive shows the path, repo 9 stars, gist 0" are results, each one checkable today. "Improved performance by 40 percent" is a number no engagement under NDA can source, the metrics chapter bans it in writing, and the pivot to rehearse is that chapter's own prepared line: I do not put numbers on my resume I cannot prove in this room, so let me show you the ones I can, volume 0, ch 8 (resume defense analysis, the no metrics policy).

Then the loop. Situation earns one sentence, the employer and the constraint. Task earns one sentence, the scope owned, phrased in the verbs the resume already carries, designed, built, migrated, wrote. Action is the mechanism and gets the most air, because it is the only state in the story nobody else can claim, and it is where the depth probe lands. Result is the scoped claim plus the artifact behind it, and the guard sits on the result state: a claim with no inspectable artifact does not get softened, it gets routed back, tied to something checkable or dropped. A story that ends on an uncheckable number ends on the one thing this resume never carries.

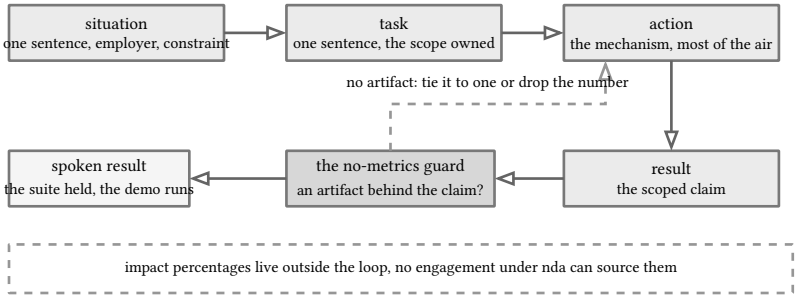


figure 168: the star loop, the guard on result, a claim with no artifact routes back, not softer

37.2. the story cards [DRILL]

Ten cards, one paragraph each, three slots per card: the question it answers, the register rows that back it, and the mechanism with its scoped result. A card with no row behind it does not exist, and a card whose row is thin says so out loud.

Card one, migration equivalence, answers "a project you owned end to end". The row is `emp-dik-migration`, legacy Java and .NET services moved to Go and NodeJS with behavioral equivalence as the contract. The mechanism: capture the legacy service's outputs on recorded requests as golden masters, the harness pattern walked line by line in book 17, ch 16 (the interview repertoire, go testing: mocks and dependency injection), port, run both and diff, decide every diff explicitly with the client, cut over with the old service kept warm for rollback. The scoped result: the behavioral requirements held, and the diffs the harness surfaced were mostly undocumented behavior, which is what the capture step exists to find, book 16, ch 5 (interview defense cookbook, employment defense).

Card two, the southbound edge mesh, answers “a system you designed”. Rows `emp-dik-mesh` and `repo-mesh-stack`: agents collect at construction sites, buffer locally through uplink loss, reconcile on reconnect with idempotent writes, sites with no direct uplink relay through peers, southbound the leg toward the devices, northbound toward the cloud. The scoped result is a mechanism claim, a day of outage costs latency, not data, and the demo stack behind it, edgex compose, mosquitto, telegraf, influxdb, grafana, is public.

Card three, the orders pipeline, answers “a service you built end to end”. Rows `emp-ntq-orders` and `repo-order-stack`: gRPC endpoints on Protobuf contracts over PostgreSQL, field numbers reserved, never renumbered, never reused, order creation made retry-safe by client-generated idempotency keys under a uniqueness constraint, the state machine enforcing transitions in one place. The scoped result: the order processing services ran for the quick-commerce app, and the demo stack, gin, sqlc, postgres migrations, paseto auth, is public.

Card four, iso-matched tests under a strict external spec, answers “working to a spec you did not write”. Rows `emp-fpt-header` and `emp-fpt-avro`: one developer inside a large team on the E470 toll program, five months, logic refactored out of private methods, unit tests written to match the ISO standard the resume cites and the Detailed Design Docs, Avro schemas evolving compatibly behind a registry. The scoped result: the tests matched the external spec, and the scope is conceded before it is attacked, to spec, five months, not the vision system.

Card five, legacy exchange ownership, answers “a time you owned something”. Rows `emp-pal` and `repo-flower-stack`: whitelabel exchange platforms where the money path was the whole game, decimal columns never float, balances derived from journaled entries, a wrong balance investigated by replaying the journal to find where the divergence entered, audit logs not optional. The scoped result: fixed and built on platforms where that discipline is the product, with FlowerShop as the public artifact of the frontend side.

Card six, the coverage hole, answers “tell me about a failure”, and it is told as the honest one. Rows `repo-llm-coverage` and `contrib-llm`: the 100 percent figure held on the pre-python_service tree the line was written against, then the python service landed, 17 files, 2,328 code lines, no tests, and the headline outlived the tree it measured. The mechanism of the catch: the measurement pass, tokei for the size, an independent atomic-mode run for coverage. The scoped result: 99.6 percent Go statements measured, the badge at 99.5, the 100 percent claim retired to historical in the register, book 16, ch 20 (interview defense cookbook, llm-tournament deep dive).

Card seven, a decision to redo, answers “a time you were wrong”. Rows `contrib-swe` and `repo-swe-archive`: the library lived on a gist with 100+ stars, and when the project was archived to GitHub in January 2025 the gist was deleted. The mechanism: freeze a finished thing rather than pretend it needs maintenance, right, and delete the only starred copy while doing it, the mistake. The scoped result, measured: repo 9 stars, gist 0, no other copy exists to check, so the redo keeps the gist standing as a pointer. Stars were the only proof the work had readers.

Card eight, the live gap as a built period, answers “what have you been doing since December”. Rows `gap-now`, `edu-degree`, and the contribution rows: a planned period, the accredited BSc runs to 2027, and the output is inspectable, the tournament repo at 46,545 code lines held at 99.6 percent, the handbook corpus building under its own make gates. The scoped result is the gaps chapter’s own line, the period was production, not absence, delivered in three sentences and stopped, book 16, ch 6 (interview defense cookbook, gap and chronology narratives).

Card nine, conflict, is taught as a shape, because the register carries no recorded quarrel and the specifics sit under client NDAs. The row is `emp-dik-craft`, the recurring disagreement of client work, test discipline against delivery pressure, and the shape comes from the stances chapter: steelman the other side, concede frames freely, concede facts never, book 16, ch 14 (interview defense cookbook, four stances, steelmanned and attacked). The mechanism sentence: the argument ends as an artifact, a golden master harness, an explicit middleware chain, not as a win. The scoped result: engagements ended on schedule and the client returned, three engagements across two modes. What is checkable is the shape and the returning client, and the card stops there.

Card ten, the frontend accessibility line, answers “frontend work you owned”. The backing is the resume line itself, Angular, React, and Svelte frontends with a11y and i11n for client projects, plus the two chapters that teach the mechanisms, book 17, ch 32 (the interview repertoire, css answers: box model, cascade, layout) and book 17, ch 33 (the interview repertoire, the rendering pipeline and accessibility). The register keeps no dedicated frontend

row, so the card says that: the mechanisms are the chapters' to teach, the claim is the resume line, and the public artifact of the frontend period is FlowerShop's Angular storefront, `repo-flower-stack`.

NOTE: client confidential, once, calmly

The standing rule for every thin card: say "client confidential" once, calmly, then pivot to what is public, the problem class, the tech decisions, the demo repo. A card that invents specifics to feel complete converts a scope limit into a credibility risk, book 16, ch 5 (interview defense cookbook, employment defense) owns the rule.

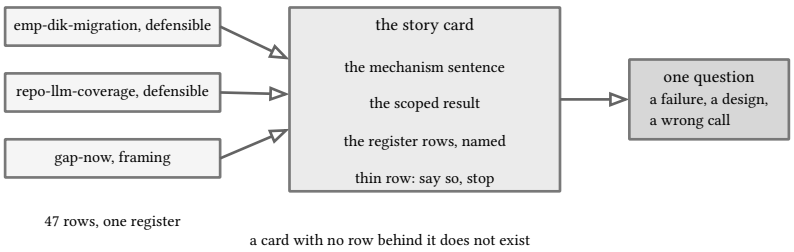


figure 169: each card is a chain, register rows into a mechanism and a scoped result, answering one question

37.3. tell me about yourself [DRILL]

The ninety-second arc is assembled, not improvised, and its parts are the register's: the identity line, the tags, the four-employer arc, and the gap as a built period with inspectable output. Spoken:

I am Raeliosol, and the header carries no title by design. Eight years since the first job, four employers, C, Go, Java, C#, JavaScript, Python, and Lua API work, SQLite and DuckDB where a database server would be overkill, distributed systems where the wire matters, and test driven development as the default rather than the tag. The arc: a first job fixing money paths on whitelabel exchange platforms, five months inside a large team on the E470 toll system writing tests against an ISO spec, then three engagements at Dikshatek across five years, client APIs, a Java and .NET to Go migration proven by behavioral equivalence, and a southbound edge mesh I designed for construction sites, with ten months at NTQ between the Dikshatek runs building the order processing services, which is also where a previous client rehired me. Since December 2025 I have been on a planned build period: the BSc runs to 2027, the tournament repo holds 46,545 code lines at 99.6 percent Go coverage, and the handbook corpus behind this resume builds under its own make gates. I am returning deliberately, and this conversation is the start of that.

Every clause routes somewhere: the identity and tags to the header rows, the arc to `emp-pal` through `emp-dik-header`, the rehire to `gap-2022`, the build period to `gap-now` and `edu-degree`, the numbers to `repo-llm-sloc` and `repo-llm-coverage`. Ninety seconds is the budget because the tenth second past it buys nothing, the silence rule owns the close, book 16, ch 17 (interview defense cookbook, coming out ahead in every exchange).

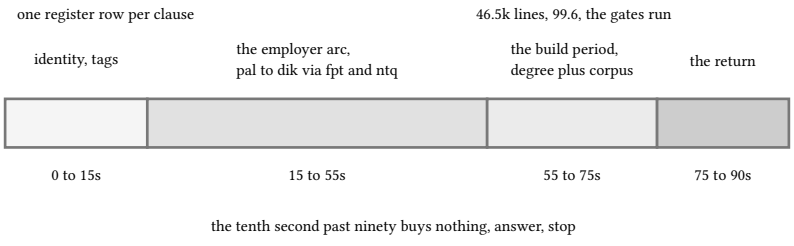


figure 170: the ninety second arc, identity and tags, the employer arc, the build period, the return

37.4. conflict, failure, ownership [DRILL]

Three full scripts, the cards with the most follow-up surface, each with its three traps rehearsed: what would the other person say, what did you lose, what changed afterwards.

Conflict, spoken: the honest opening is that the register carries no recorded quarrel and the specifics sit under client NDAs, so here is the shape and the nearest true material. The recurring disagreement of client work is test discipline against delivery pressure, the row is tests, handlers, middlewares, business logic. The shape: steelman the other side first, the deadline is the contract and unverified code is a liability shipped to the client, concede the frame freely, yes, not everything deserves a test, hold the fact, the merge and cutover paths do. Then make it mechanical instead of personal: on the migrations the argument ended as a golden-master harness, the pattern walked line by line in chapter 16 of this book, on the API work as an explicit middleware chain, logging, recovery, auth, handler, testable end to end. Traps: the other person would say the deadline was the job and my discipline was a cost, and they would be partly right. What I lost was my own time, not the client's calendar, the engagements ended on schedule. What changed: I stopped arguing discipline in the abstract and started demonstrating it, the harness is the argument.

Failure, spoken: the failure is mine and it is measured. The resume line said 100 percent coverage, true against the tree it was written for. Then the python service landed, 17 files, 2,328 code lines, no tests, and the headline number outlived the tree it measured. My own measurement pass caught it: 46,545 code lines of 62,058 total, 99.6 percent Go statements in an independent atomic-mode run, the badge reading 99.5. The failure was not the gap, untested code happens, it was letting a claim drift without re-measuring, on the one repo whose whole point was the discipline. Traps: the other person would say the badge was decoration and nobody would have checked, granted, which is why the check has to be mine. What I lost is the 100 percent claim, permanently, it is retired to historical in the register. What changed: every number I speak now arrives with its measurement and its command.

Ownership, spoken: the first job. Whitelabel exchange platforms, C#, Java, Spring, MySQL, AWS, where I fixed more than I designed, which is what a first job is for. What I owned was the money path: decimal columns, never float, balances derived from journaled entries, so a client reporting a wrong balance was not a mystery but a reconstruction job, replay the journal, find where the divergence entered, and audit logs were not optional in that domain. Traps: the other person, the platform's original architects, would say the journal was their design and my work was maintenance, and they would be right, the ownership was of fixes and incidents, not the platform. What I lost, nothing dramatic is on the record, the honest cost is the domain knowledge stayed under NDA, and the public residue of that period is FlowerShop. What changed: the shape stayed with me, append-only records, derived state, replay as repair, the same skeleton the outbox carries in the patterns handbook.

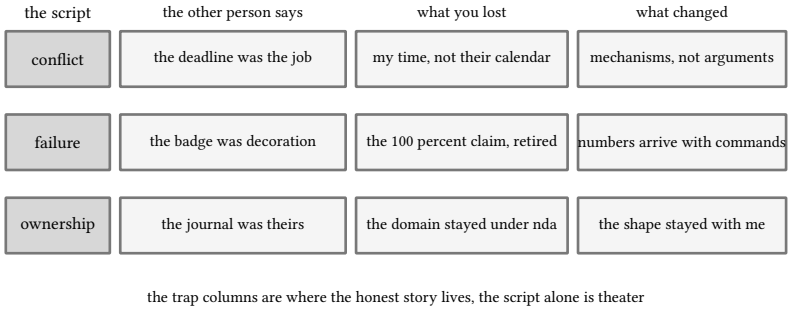


figure 171: three scripts, three traps each, the trap answers rehearsed with the script, not after it

37.5. why leaving, why us, weakness [DRILL]

Why leaving has a strongest version and the register holds it: the NTQ engagement ran its course in October 2022, and Dikshatek, where I had already contracted in 2020 and 2021, brought me back in December for a two-year run, `gap-2022`, rehired by a previous client. Every exit in the register is an engagement ending, the 2025 one ended as planned into the deliberate build period, and the sentence to speak is the gaps chapter's: the pattern to read is that previous clients rehired me, twice. No exit in the record is a walkout, and none needs a softer telling.

Why us is the preparation question inverted. The playbook's closer asks the team one question that uses the preparation, what does the team's test discipline look like, and where does it hurt, book 16, ch 9 (interview defense cookbook, hostile cross examination playbook). Why-us runs the same discipline in the other direction: one true,

specific fact about them, drawn from the preparation, tied to one row of the record, the mechanism I would bring stated as the bridge. Two slots, both checkable, and the generic praise sentence fills neither.

Weakness and why-should-we-not-hire-you are the endgame pair, reused in interview voice. The weakness: distribution over promotion, I build things that are testable and documented, including for myself, and underinvest in making noise about them, eight stars on 46.5 thousand code lines is the trait rendered as a number. The not-hire: if you need a specialist, one stack at scale, one domain for years, I am the wrong shape, I am broad by design, four languages and the systems underneath them, and the depth is proven where it counts, the handbooks and the repos. Both cards are prepared because the questions arrive, and the games chapter prices the volunteered weakness as a commitment device that retires the attack, book 16, ch 15 (interview defense cookbook, game theory and the evolutionary drivers), which is the one place volunteering pays.

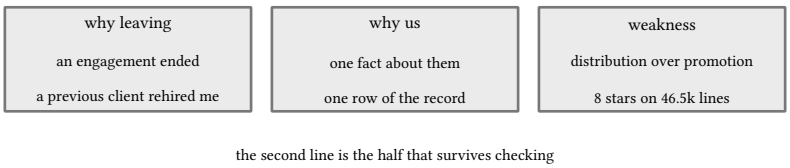


figure 172: three cards, one slot each from the record, prepared before the question lands

37.6. the drill and the traps [DRILL]

The question bank, sixteen questions, each with its route. The openers: tell me about yourself, the arc above. Walk me through the resume, the timeline drill, eight years, four employers, one of them bringing me back twice, every transition measured in weeks, book 16, ch 6 (interview defense cookbook, gap and chronology narratives). The motivation four: why leaving, the returning client. Why us, the two-slot preparation. Weakness and why should we not hire you, the endgame pair. The story six: conflict with a teammate, the shape card. Conflict with a stakeholder, the provider material, third-party sandboxes drift from production, the resolution was pinning contracts with tests and treating sandbox pass as necessary, not sufficient. Failure, the coverage card. Proudest work, the mesh, the resume's one designed-and-built row. Ownership, the exchange card. A time you were wrong, the deleted gist. Where in five years, the degree runs to 2027 and the return is deliberate, and nothing more is invented past that. Why no titles on the resume, contractor and outsourcing titles varied per engagement and carried no standard meaning, so the bullets do the signaling and seniority is answered with scope, `no-titles`, book 16, ch 8 (interview defense cookbook, no titles and no metrics policy answers). The gap question, card eight, three sentences maximum. Salary expectation in an early screen, one sentence, a range anchored in your own record, and the full treatment is the negotiation chapter's, book 17, ch 39 (the interview repertoire, offers and negotiation).

The traps, four. The anchor first: the interviewer names a number or a frame before you do, and the defense is the fallacies chapter's row, restate the number from your own record before negotiating inside theirs, with the communication chapter's cheaper half, frame first and there is nothing to restate against, book 16, ch 13 (interview defense cookbook, fallacies, biases, and appeals). The silence after an answer: a move, not a vacancy, whoever fills it pays in truth or status, so answer, stop, and let the pause belong to the person who created it, book 16, ch 17 (interview defense cookbook, coming out ahead in every exchange). Loaded and presupposition questions: both answers convict, so split the presupposition and answer the true half, the counter table's own row. And the unprompted weakness: the cookbook's rule is that every sentence past the question is a fresh claim under new risk, the prepared weakness exists for the question, not for the air, and conceding scope under attack is the opposite of opening unforced.

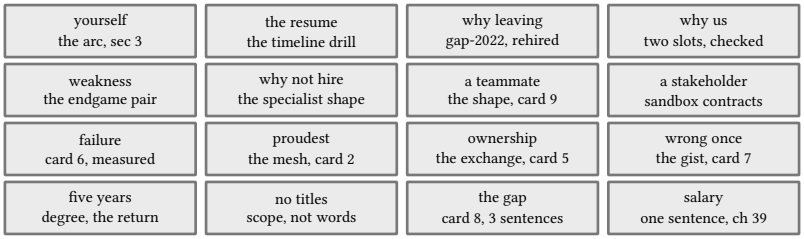


figure 173: the sixteen questions as chips, every one routing to a card, a script, or a chapter

floored to the register and the books that defend it: the stories are the register's, none is invented, none carries a number the register cannot show, book 16, ch 5 (interview defense cookbook, employment defense) for the per-employer material, book 16, ch 6 (interview defense cookbook, gap and chronology narratives) for the timeline and the gap cards, book 16, ch 9 (interview defense cookbook, hostile cross examination playbook) for the rules under pressure and the endgame pair, book 16, ch 14 (interview defense cookbook, four stances, steelmanned and attacked) for the concession rule the conflict card runs, book 16, ch 17 (interview defense cookbook, coming out ahead in every exchange) for silence, framing, and the stop, with volume 0, ch 4 (resume defense analysis, employment claims) and volume 0, ch 3 (resume defense analysis, contributions) holding the rows behind the cards and volume 0, ch 8 (resume defense analysis, the no metrics policy) owning the no-metrics discipline the result state carries.

38. the observed round, take-homes, debugging

The observed round is problem solving with someone watching, and the 2026 loop made the watching the point: unassisted code proves less than it did, so the graded channel is what you say while you work. This chapter drills that channel in order, the five minutes before the first line, the narration while writing, the recovery from the blank, the rules for the assistant one keystroke away, the timed mocks this book already contains, the take-home posture, and the debugging round with its proof rule. The pinned numbers come from the landscape chapter, the delivery mechanics from the cookbook.

38.1. the first five minutes [DRILL]

Four moves, in order, before any code. Restate the problem in your own words and get a yes, “here is what I understand, the input is this, the output is that, is that right”, because the yes converts the next twenty minutes from possibly-solving-the-wrong-problem into solving. Name the input shape and the constraints, sizes, ranges, sorted or not, duplicates allowed, what the empty input returns, the questions the code cannot answer later. Name the solution shape before writing: brute force first with its complexity stated, then the improvement with its mechanism and its complexity, the two-shape sentence, the naive double loop is quadratic, the hash map buys linear time for linear space. Agree on the definition of done, working code, which edge cases, tested how, so the finish line exists before the pen moves. The five minutes are not overhead, they are the cheapest minutes of the round, and an interviewer who watches them sees a process rather than a race.

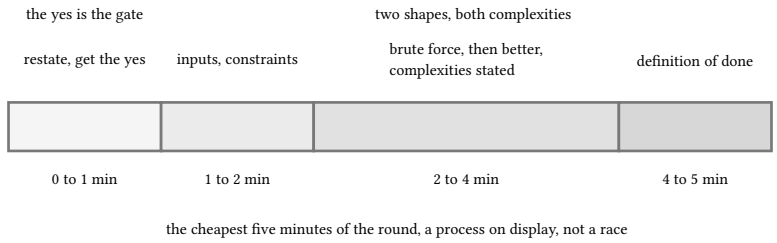


figure 174: the five minute clock, restate and yes, inputs and constraints, both shapes, done

38.2. narration while writing [DRILL]

Narrate decisions, not keystrokes. The channel carries why this and not that, the tradeoff at the turn, the reason the map is a map. Every skip is flagged aloud, “I will pretend this helper exists and test it after”, so the observer can price the skip instead of discovering it. Uncertainty is annotated in the same breath as the line it touches, “this is n log n from the sort, the inner scan is still naive, I will come back to it”, because a stated doubt is a plan and an unstated one is a hole.

The evidence for spending the air this way is pinned in the landscape chapter: the interviewing.io 100k analysis found code and problem solving scores dominate pass decisions and communication is the smallest rubric factor, and the same narration is how a problem solving score gets observed at all, book 17, ch 1 (the interview repertoire, the 2026 interview landscape). Both facts hold at once. The rubric pays least for talking and the score it pays for cannot be seen without it, so the narration is not performance, it is the observable half of the work, which is exactly what a loop grades now that unassisted code alone proves so much less.

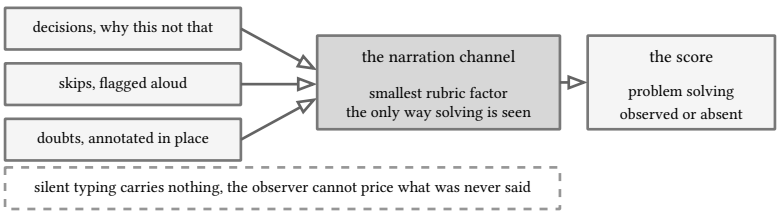


figure 175: narration is the observable channel, decisions, skips, doubts, carried on the air

38.3. blanking recovery [DRILL]

The blank is a state, not a verdict, and it has a loop. Name it aloud, “I have lost the thread, let me restate what I have”, which converts the silence into a move the observer can follow. Shrink to a smaller problem, solve for one element, for the empty input, for n equal to 2, because the small case re-derives the invariant the blank lost. Write the naive version and label it, the labeled naive is a plan with a complexity attached, the unlabeled one reads as the ceiling. Annotate the uncertainty and keep moving, write the line you believe in, mark the doubt, continue, a wrong line marked beats a blank page every round. The loop exits when the thread is back, and running it twice in one round costs less than hiding the blank once, the hidden blank is the only unrecoverable version.

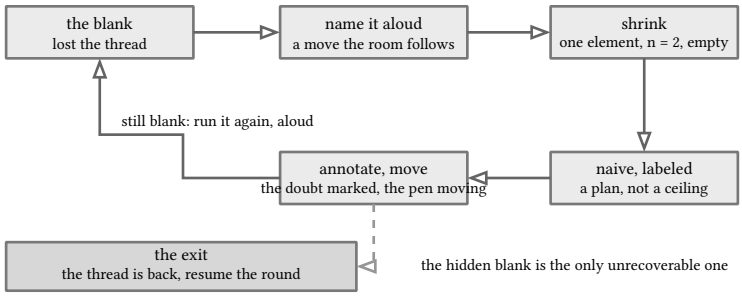


figure 176: the blank has a loop, name it, shrink it, label the naive, annotate, move

38.4. ai-assist norms [DRILL]

The numbers first, pinned in the landscape chapter from Karat’s 2026 trends report, 400 engineering leaders: 62 percent of organizations still ban AI in interviews while estimating over half of candidates use one anyway, live technical interviews still run at 79 percent in the US, 71 percent say AI makes technical skills harder to assess, and take-homes and automated code tests degrade fastest as signal, book 17, ch 1 (the interview repertoire, the 2026 interview landscape). Three norms follow. Disclosure first: state what you intend to use before you use it, the rule is theirs to give and asking for it is free, 62 percent of the rooms ban it. Scope, named: autocomplete and syntax recall against generation and solution shape are different concessions, and saying which one you are making is the difference between use and smuggling. Concede cleanly: if the round bans the assistant, closing it is the whole answer, no negotiation, and the honest posture underneath all three is that the loop already assumes the assistant exists, which is why the graded channel is the narration and the follow-ups rather than the keystrokes.

PITFALL: the undisclosed assist

The one unrecoverable move in this round is the assist the interviewer can see but you have not named, the paste that lands too fast, the API call the connection log shows. Disclosure costs a sentence, the undisclosed assist costs the trust every later answer rides on, the repeated-game price, book 16, ch 15 (interview defense cookbook, game theory and the evolutionary drivers) states it.

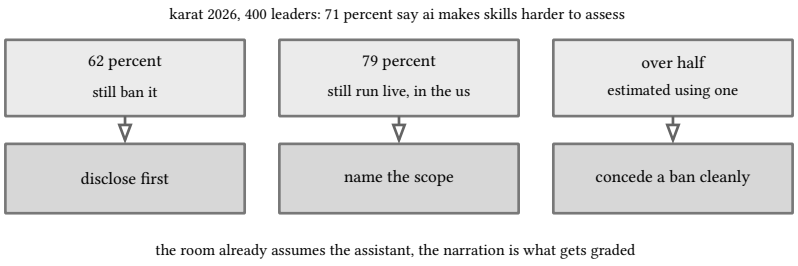


figure 177: the 2026 numbers pinned, 62 ban, 79 still live, half use one anyway

38.5. timed mocks from this book [DRILL]

The mock deck is already written, it is this book, and no external question set is needed. The 25-minute set: any section of the two classics chapters or the algorithm patterns chapter, book 17, ch 5 (the interview repertoire,

classics part 1: warmup, arrays, hashing), book 17, ch 6 (the interview repertoire, classics part 2: dynamic programming and tries), book 17, ch 23 (the interview repertoire, algorithm patterns: window, pointers, search, greedy, bits), one problem, spoken start to finish. The 40-minute set: the four design prompts of the design prompts chapter, book 17, ch 36 (the interview repertoire, system design prompts, four worked), topology drawn aloud, the tradeoffs owed out loud. The 15-minute set: the emitter or the promise subset from the javascript from scratch chapter, book 17, ch 9 (the interview repertoire, javascript from scratch), mechanism over polish. The debugging set: delete a line from any go sample in this book and hand it over, reproduce, isolate, fix, prove, the section below owns the loop.

The scoring rubric is the book's own three labels, asked as questions: does the code run, the TDD question. Is the mechanism named, the EWC question. Are the follow-ups caught, the DRILL question. Two of three is a fail, and the rubric needs no external judge because the labels already define what finished means in this corpus. Partner protocol: the partner holds the chapter file, fires the DRILL traps written in it, and scores only the three labels, one line each, no coaching mid-mock, the feedback lands after the clock.

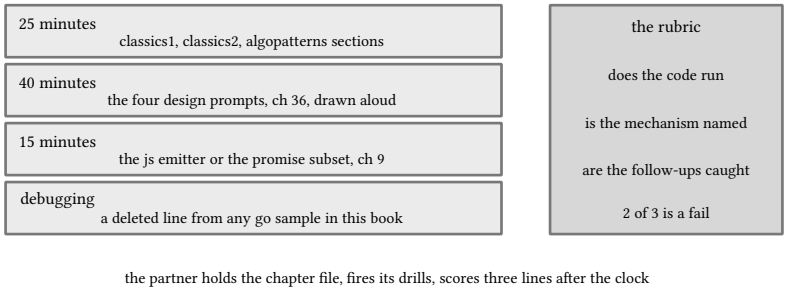


figure 178: the mock deck map, four lanes, one rubric, the partner holds the file

38.6. take-home discipline [DRILL]

Take-homes degrade fastest as signal, the Karat row says so, so the strategy is over-transparency. The scope contract comes before any code: what is in, what is out, what done grades, asked and answered before building, because the unasked scope question is how a weekend becomes two. The method is the beats of the react build chapter, book 17, ch 14 (the interview repertoire, react build: a listing page, test driven), the template this corpus already drills: the test file alone first, the honest red, the minimal green, then the extension, commit order carrying the story. The readme tells the grader how to run it, prerequisites, the one command, the one test command, and what was cut and why, written for a stranger. And the handoff is one command, the corpus's own make verify posture, the same gate every sample in this book runs under: the grader's first action is the readme, the second is the command, and anything that needs a live explanation inside the repo has already failed.

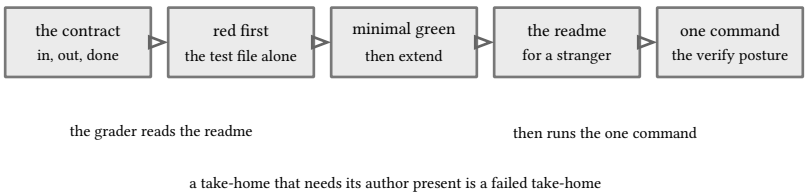


figure 179: the take-home ladder, contract, red, green, readme, one command

38.7. the debugging round [DRILL]

Four moves, and the last one is the rule. Reproduce: the bug that cannot be reproduced cannot be proved fixed, so the smallest input that shows it comes first, printed, deterministic where possible. Isolate: bisect the path, and the signals have an order, the metric names the symptom, the trace names the hop, the log explains the line, book 12, ch 14 (design patterns, concurrency, and distributed systems, observability) owns all three, and the test layers name the seam, store, service, handler, notifier, each with its own suite, book 17, ch 16 (the interview repertoire, go testing: mocks and dependency injection). Fix: the smallest change that removes the cause, not the symptom, and if

the fix is not small, the isolation was not done. Prove: the failing test comes first, written against the reproduction, red, then the fix, then the same test green and the suite green, because a fix without a prior red test is a story, and the debugging round grades the proof, not the anecdote.

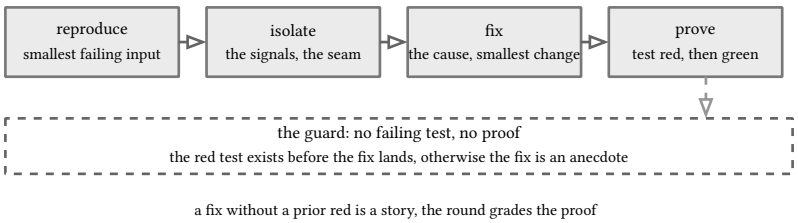


figure 180: reproduce, isolate, fix, prove, the failing test arrives before the fix

floored to the pinned and the drilled: book 17, ch 1 (the interview repertoire, the 2026 interview landscape) for the round facts this chapter leans on, the 100k analysis reading and the Karat numbers with their dates, book 16, ch 17 (interview defense cookbook, coming out ahead in every exchange) for the delivery mechanics, the stop, the frame, the silence, book 16, ch 9 (interview defense cookbook, hostile cross examination playbook) for the hostile follow-ups these answers must survive, and the debugging round's two floors, book 17, ch 16 (the interview repertoire, go testing: mocks and dependency injection) for the test layers that name the seam and book 12, ch 14 (design patterns, concurrency, and distributed systems, observability) for the signals that say where to look first.

39. offers and negotiation

The offer conversation is the one round most candidates rehearse least, and it rewards the same discipline as every other round: know what is being asked, answer from a record, price the exchange honestly. This chapter drills the offer as information, the anchor as a bias with a counter, the timeline as a repeated game, and the scripts, word for word, with no invented numbers anywhere: the record this book defends carries no metrics by policy, and the negotiation module keeps that discipline, every number in a script is a script variable, your number, their number, the walk-away.

39.1. reading an offer [DRILL]

Read the offer as a grid, not a number. Base is the only component everything else compounds on: the bonus is a percentage of it, the next negotiation prices off it, relocation and level both key on it, so a base concession is never a single-year concession. Bonus is the discretionary discount: ask for the realized range for the level, the target is a forecast and the payout history is the fact. Equity demands the illiquidity honesty: options at a strike are a bet priced by a vesting clock, a preference stack, and a tax event, and the four questions to ask are strike, preference, vesting schedule, and the last round's price. Value equity at zero for the rent-paying decision, let it be upside, and never trade base for equity at face value, that trade prices a lottery ticket at par.

Benefits are read by price, not by count: insurance with a real coverage number costs the company real money, a gym discount costs almost nothing, and the grid cell is what the benefit costs them, not how it photographs. Scope is the quietest cell and the most expensive: what the offer buys, the team, the on-call rotation, the manager, the systems owned, because scope sets the record you bring to the next negotiation. Team signal is the last cell: how they negotiated is how they manage, a rushed deadline on a sloppy agreement is data about the working environment, and the offer is the only period where the company negotiates with you while trying to impress you.

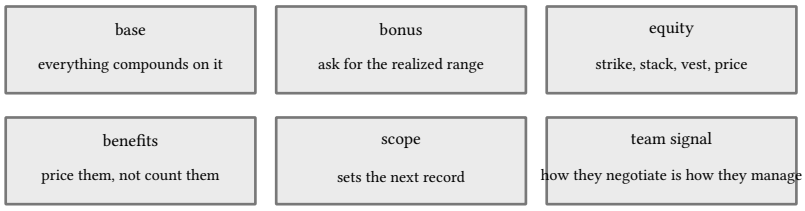


figure 181: the offer grid: six cells, one reading per cell

39.2. anchoring from the record [DRILL]

The anchoring material is already owned: the first number spoken frames every number after it, the bias is theirs most of the time, and the counter is to restate the number from your own record before negotiating inside theirs, because measurement is the anchor you control, book 16, ch 13 (interview defense cookbook, fallacies, biases, and appeals). The cheaper half of the same move is to frame first: your definition and your criterion on the table before their number arrives, a frame stated first reads as ground, the identical frame stated second reads as resistance, book 16, ch 17 (interview defense cookbook, coming out ahead in every exchange). In the offer conversation the frame is the criterion: pay against the record, the checkable artifacts, not self-reported impact.

Here the no-metrics policy stops being a defense problem and becomes an asset: a record with no invented impact numbers has nothing to walk back and nothing to discount, every claim resolves to something openable, book 16, ch 8 (interview defense cookbook, no titles and no metrics policy answers). The negotiation use is direct: their number anchors on levels and surveys, yours anchors on checkable work, and when the two disagree the honest ask is that the record win in both directions, the same sentence the cookbook drills for the interview room. A number backed by an artifact cannot be argued away, only declined, and declining is a decision you can see.

39.3. competing timelines [DRILL]

The exploding offer is manufactured urgency, the marker is “decide today, the window closes tonight”, and the counter is architectural: open channels, written trails, agreements restated, the deadline asked for in writing, book 16, ch 16 (interview defense cookbook, the deception taxonomy). A real deadline survives being written down, a manufactured one often softens the moment it is restated in an email, and the restatement costs nothing to request.

Honest sequencing is the move that keeps the repeated game intact: tell each process its true state, in process elsewhere, one offer in hand, your process is the one I want to finish, and never invent a competing offer. A fabricated offer is inflation, and the interview is move one of a repeated game where every later move gets repriced: the panel talks to the next panel, the industry is a village, and a word caught discounting follows you into the renegotiation, book 16, ch 15 (interview defense cookbook, game theory and the evolutionary drivers). The acceleration ask is the honest instrument: tell the second company that company A’s deadline is real, name the date, ask to accelerate, accept no for an answer, because a declined acceleration is information, it prices your process at their real speed. Commitment is reputation repricing in both directions: accepting and reneging is the defection the village prices forever, and a stated deadline held to, even at a cost, is a commitment device, the payoff matrix changed before the argument started.

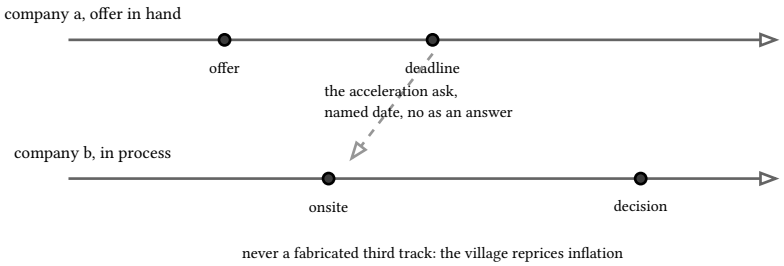


figure 182: two processes, one honest acceleration window, no invented third

39.4. the scripts [DRILL]

Five scripts, drilled word for word, every number a variable.

The early-screen range answer, for when the recruiter asks expectations before anyone has met you: “for this scope in this market I am looking in a range, my number to anchor it is your number for the level, and I will weigh it against my record.” Give a range if pressed, never a single number, the range’s top is the ask and its floor is the walk-away plus margin, and say the basis, a range with no basis invites theirs.

The offer call: gratitude, then silence, and never acceptance on the call. “Thank you, I am glad this came together, I want to read the whole package in writing, when do you need my answer by”, then stop. Silence after an answer is a move, whoever fills it pays in concessions, and the pause belongs to the person who created it, book 16, ch 17 (interview defense cookbook, coming out ahead in every exchange). Accepting on the call throws away every lever at the one moment they are all live, and a verbal offer is memory, a written offer is an offer.

The counter: one number, anchored to scope and the record, delivered once. “Everything in the package works as written except the base, the scope as described sits above the number, here is the number that closes it today, and if the number cannot move I want to talk about what else can.” One ask prices clean, two asks dilute each other and read as appetite, and the counter that concedes everything else up front is the commitment device that makes the single ask believable.

Accept: short, warm, same-day, and the agreed terms restated in writing the same day, agreements restated in the room is the whole architectural move. Decline: short, warm, the door open, one true sentence of reason if a reason is owed, no counteroffer theater and no silence, because the declined company is still in the repeated game, panels forward names and remember them. Status asks are priced in patience, spend them rarely: ask once, with a date attached, and take the no. Concessions split by what they spend, concede scope freely, it retires attacks before they arrive, concede truth never, truth once conceded cannot be rebought, book 16, ch 17 (interview defense cookbook,

coming out ahead in every exchange). Repair works mid-negotiation too: a number misspoken is corrected before they check it, a self-caught error is a process working, an externally caught one is a story about you.

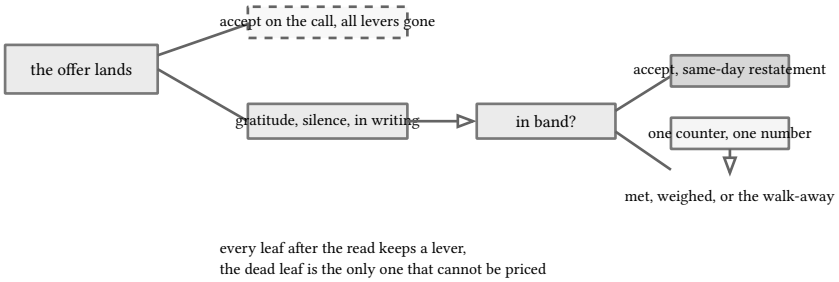


figure 183: the offer call as a game tree: accept now is the one dead leaf

39.5. closing the loop after a no [DRILL]

The post-rejection note goes out same day, two sentences: thanks, one specific thing the process taught you, the door open, and a request that the note be kept with the file. No paragraph of rebuttal, no case for reconsideration unless they ask, the note is a move in the repeated game, not an appeal of the verdict, and the specific detail is what makes it a person's note rather than a template.

The register discipline carries over: keep the pipeline the way the corpus keeps its claims, dates, states, what was asked and what was offered, in the private copy, so the next negotiation anchors on a record instead of memory, the same method the repo evidence chapter drills, book 16, ch 11 (interview defense cookbook, the repo evidence method). Memory flatters and drifts, a written table does not, and the walk-away from last cycle is the floor of this cycle's range.

Re-approach timing is priced by change, not by calendar: go back when something checkable is different, a shipped artifact, a new chapter of work, a role that matches the specific reason given at the no, and reference that reason in the first line. A re-approach with nothing new reads as a rerun, a re-approach with a new artifact reads as growth, and the difference is the whole sentence.

floored to: no numbers were invented for this chapter, every script carries script variables, and the negotiation theory is not new here. Anchoring as a bias and its counter belong to book 16, ch 13 (interview defense cookbook, fallacies, biases, and appeals), framing first, silence, question control, and the two concessions belong to book 16, ch 17 (interview defense cookbook, coming out ahead in every exchange), the repeated game, repricing, and commitment devices belong to book 16, ch 15 (interview defense cookbook, game theory and the evolutionary drivers), manufactured urgency and the architectural counter belong to book 16, ch 16 (interview defense cookbook, the deception taxonomy), the no-metrics posture as an asset belongs to book 16, ch 8 (interview defense cookbook, no titles and no metrics policy answers), and the register discipline belongs to book 16, ch 11 (interview defense cookbook, the repo evidence method).

40. the transfer answer: java and rust shops

Every shop interview eventually asks it: our stack is java, or rust, and your record is go and c. The wrong answer apologizes for the record, the right one runs a method: name the nearest language you already own, map mechanisms instead of syntax, scope the ramp by artifacts, and floor every mapping on work this corpus actually contains. This chapter works both languages, java from the employment record, rust from the c manual's metal, with the honest concession up front where a record does not exist.

40.1. the transfer method [DRILL]

First move, name the nearest corpus language, because transfer is not symmetric and the bridge is specific: java's nearest is c-sharp, both run on a vm with a runtime gc, both carry generics, a big standard library, and an embedded query dsl, and the c-sharp answers chapter already owns most of that ground. Rust's nearest is c, both put memory layout and lifetimes in the programmer's hands, and the c manual already teaches the exact bug class rust was built to retire.

What transfers is mechanisms, never syntax: the memory model, who frees and when, the concurrency model, threads, green threads, or event loops, the error channel, exceptions, values, or result types, the io shape, blocking, async, or reactors, and the debugging instincts, where profiling attaches and where locks deadlock. A mechanism understood once is portable, a syntax memorized is worth one language. What does not transfer is everything mechanical: the idioms, the toolchain muscle memory, the ecosystem names, the library lore, and the honest sentence is that the non-transferring layer is weeks of comfort, not years of understanding.

The ramp is scoped by artifacts, not by weeks: read one service end to end, then fix small bugs in it, then land one small feature behind its tests, then take one design review, and each artifact is checkable by the team the way every claim in this corpus is checkable by a reader. Saying the ramp as artifacts is the same discipline as the resume: no invented metrics, a named deliverable instead of a promise.

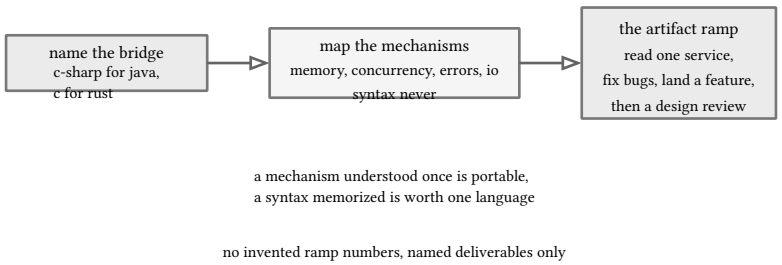


figure 184: name the bridge, map the mechanisms, ramp by artifacts

40.2. java, from the record [DRILL]

The record first, because the answer opens there: the FPT stint is Spring Boot with Avro over PostgreSQL and Kafka on GCP behind ISO-matched tests, the PALTech work is whitelabel exchange platforms whose FlowerShop artifact is spring boot JPA on MySQL with an Angular storefront, and the Dikshatek migration moved legacy Java and .NET services to Go and NodeJS under behavioral equivalence, which is the transfer question answered in the harder direction, reading java well enough to reproduce its behavior in another runtime. That is the sentence to lead with, then the four mappings.

The gc mapping: the jvm's collectors are generational, betting the weak generational hypothesis that most objects die young, so a copying nursery collects short-lived objects cheaply and survivors graduate to an old generation, with g1 the region-based modern shape. Go's collector is the opposite bet, one heap, concurrent tri-color mark and sweep, non-generational, paying sub-millisecond stops instead of a generational layout, book 17, ch 15 (the interview repertoire, go runtime: scheduler, gc, patterns). The interview sentence: both trade throughput against pause, the jvm bets on death rates, go bets on cheap concurrent marking.

The concurrency mapping: java 21 shipped virtual threads, jep 444, runtime-scheduled onto carrier threads the way goroutines schedule onto os threads, with one honest caveat the jep itself pins: a virtual thread inside a `synchronized` block or a native call pins its carrier, and the scheduler does not compensate by expanding parallelism, so hot paths migrate to `ReentrantLock`. Go's scheduler grew up with handoff as the design: a goroutine blocking in a syscall drops its processor to another thread instead of pinning it, book 17, ch 15 (the interview repertoire, go runtime: scheduler, gc, patterns). Say both and the follow-up writes itself, the pinning trap is the jep's own text.

The query dsl mapping: java streams are the same embedded language as linq, deferred until a terminal operation, chained methods over a sequence, the same traps, re-enumerating an expensive pipeline and smuggling side effects into a lambda, and the whole answer including the floor transfers from book 17, ch 31 (the interview repertoire, c sharp answers). The error mapping: checked exceptions put the failure surface in the signature, which is a cousin of result types, and the honest stance is two-sided, they encode what callers must face, and they are also why so much java wraps them into runtime exceptions and loses the encoding, the same trade go resolved by returning error values, book 17, ch 15 (the interview repertoire, go runtime: scheduler, gc, patterns). And spring's container is not magic to unlearn, it is constructor injection with the wiring moved into a runtime container, the dependency injection book 17, ch 16 (the interview repertoire, go testing: mocks and dependency injection) already drills, so the transfer is learning where the wiring lives, not what wiring is.

40.3. rust, from the metal [DRILL]

The concession first, out loud, because a scope concession retires the attack before it arrives: the register carries no rust employment, nothing in the record claims it, and what follows is the mapping, not a resume line. Rust's nearest corpus language is c, and the c manual is the honest floor: ownership is the compile-time answer to the exact bug class the manual teaches, use-after-free, double free, the dangling pointer, book 1, ch 14 (c23: the language, the machine, and the cloud, the heap and allocators) and book 1, ch 19 (c23: the language, the machine, and the cloud, allocator designs) for the manual's own treatment of who frees.

The ownership rules are three, and the rust book states them plainly: each value has an owner, one owner at a time, the value is dropped when the owner leaves scope. The consequence is move semantics, `let s2 = s1` invalidates `s1`, because two owners would free twice, and passing to a function moves or copies exactly like assignment. Borrowing is the second half: a reference reads without owning, many shared borrows or one mutable borrow, never both, and the checker enforcing that is the type system, compile time, zero runtime cost. Option and result replace two silences at once, null and the unchecked throw, and the whole shape, the two carriers and the railway they compose into, is already built in c in this corpus, book 2, ch 27 (mathematics for programmers, option, result, and the railway).

No gc is a design consequence, not a feature bullet: making freeing a static fact is what removes the runtime collector, and the price is that lifetimes become part of function signatures, the borrow checker part of the build. Go chose the other payment, a concurrent collector and the freedom to hand aliases around, book 17, ch 15 (the interview repertoire, go runtime: scheduler, gc, patterns). The honest answer to "would you write rust" is scoped: yes at a security boundary, in wasm, on embedded, anywhere the c manual's bug class is the incident report and the pause budget is microseconds, and no for the service shape, an api over a store, which this corpus shipped six times over in six runtimes, book 16, ch 3 (interview defense cookbook, specialization tag answers), none of whose requirements price rust's compile-time rent.

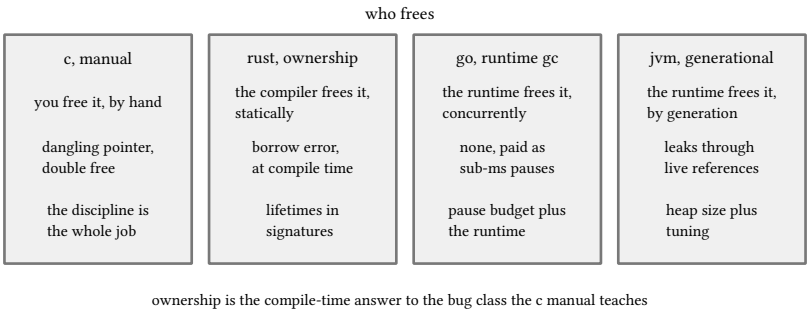


figure 185: four ways to answer who frees this memory

40.4. the traps [DRILL]

Four traps, one honest line each. “Green threads” as if java invented them in 21: java carried a green-threads experiment decades ago, dropped it for native threads, and virtual threads in 21 are the return with a real scheduler and the pinning caveat above, so the answer that lands names jep 444 and its two pinning cases instead of the folklore name. “Final means immutable”: final locks the reference, not the object, the instance behind a final field is still mutable, and the confusion is the same one the value versus reference drill retires for readonly in c-sharp, book 17, ch 31 (the interview repertoire, c sharp answers). “The borrow checker slows it down at runtime”: no, the checks are type system facts settled at compile time, which is the entire design point, the runtime pays nothing for the borrow it already proved. “Rust for everything”: no, gc is fine for the service shape, the six-lane corpus is the existence proof, and rust prices in where the boundary is security or the pause budget, not where the work is an api over a store, saying when not rust is part of the transfer answer, the same concession discipline the whole book drills.

floored to: no employment facts were invented, the record rows are quoted from the claim register, FPT’s spring boot with Avro, PostgreSQL, Kafka, ISO-matched tests, PALTech’s whitelabel exchange platforms and the spring boot JPA FlowerShop, Dikshatek’s legacy Java and .NET to Go and NodeJS migration under behavioral equivalence, and the rust concession is the register’s own silence. The mechanisms floor on book 17, ch 15 (the interview repertoire, go runtime: scheduler, gc, patterns) for the gc and scheduler contrasts, book 17, ch 31 (the interview repertoire, c sharp answers) for the linq and value-against-reference bridges, book 2, ch 27 (mathematics for programmers, option, result, and the railway) for option and result, book 1, ch 14 (c23: the language, the machine, and the cloud, the heap and allocators) and book 1, ch 19 (c23: the language, the machine, and the cloud, allocator designs) for the manual memory rust answers, and book 16, ch 3 (interview defense cookbook, specialization tag answers) for the six-lane service corpus the when-not-rust trap leans on. The two online pins, jep 444 for virtual threads and the rust book’s ownership chapter, carry their urls and access dates in the book’s source table.

41. the one billion row challenge

One problem, nine lanes, one line of output. The 1brc challenge aggregates a billion `station;temperature` rows into a single sorted map of minimum, mean, and maximum per station, and this chapter runs it in c, rust, go, java, c sharp, javascript, python, lua, and sqlite, stdlib only in every lane, 24 variants in all, with identical output bytes on every artifact any of them is pinned against. Every claim elsewhere in this book was proven with a test. Here the test still comes first, a byte diff against a pinned oracle for every variant, and only then a stopwatch explains what correctness already bought.

The order is the order the work demands. The contract and its arithmetic come first, because one wrong rounding rule costs more than any speedup. The sqlite lane follows, where the aggregate lives in a query instead of a loop. Then the eight program lanes in the corpus canonical order, each walking its own decision ladder with the measured ratios attached. The full-scale table closes, where the memory shape of each road decides who finishes at all.

41.1. the challenge and its rules [DRILL]

The contest lives at 1brc.dev with the repository at github.com/gunnarmorling/1brc, both accessed 2026-10-08 and cited in the sources footer. The input is a text file of rows, one per line, `name;value`, where the value carries exactly one fractional digit and lands in [−99.9, 99.9], and the name is UTF-8, 1 to 100 bytes long, containing no `;` and no newline, with at most 10k distinct names. The output is exactly one line, here the committed fixture's own opening:

{Abha=-0.8/16.9/28.9, Abidjan=-3.6/24.0/42.0, …}

Stations sort by name, every number carries one decimal digit, and `, ` separates entries. The contest scored wall time on a fixed box. This chapter scores something else first: byte-exactness against the official output contract, then wall time as commentary.

The official artifacts are pinned verbatim under `tools/1brc/ref/`, fetched from the repository at main with sha256 records: `CreateMeasurements.java`, the data generator, 413 hardcoded `WeatherStation(name, mean)` records with the station drawn uniformly per row and the value `round(gaussian(mean, 10) * 10) / 10` from ThreadLocalRandom, which cannot be seeded. Then `CalculateAverage_baseline.java`, the reference solution whose rounding shape pins the output contract, plus the 12 official sample pairs under `ref/samples/` that this corpus validates against. The repository also keeps an earlier pre-fix rounding variant of the reference solution, explicitly history, not the spec.

Our generator mirror is `tools/1brc/create_measurements.py`. It keeps the official semantics, uniform station draw, gaussian centered on the station mean with standard deviation 10, rounded to tenths with the same half-up rule, but drives them from the corpus 64-bit Knuth LCG `x' = 6364136223846793005 * x + 1442695040888963407 mod 2^64` with Box-Muller for the gaussian, so every run is byte-identical, which ThreadLocalRandom cannot offer. Two divergences are stated rather than hidden: the LCG plus Box-Muller replaces `nextGaussian`, and the station pick takes the full state mod 413, a bias below 2^-52. The 413-station corpus is transcribed once into `stations.csv`.

The tier ladder keeps honest scale discipline. The committed gate fixture is 10k rows over the 413-station corpus, `ch41-fixture/measurements-10k.txt` with its byte-exact expected twin, plus the edge battery `edges.txt`, 37 rows over 11 stations, pinning the exact-half asymmetry, minus zero, both range rails, BMP sort order, prefix families, and the solo station. Two more batteries closed the blind attack round at the end of the wave: `edges-astral.txt`, 5 rows, separates supplementary-plane names from a private-use name so codepoint order and UTF-16 unit order disagree and the gate sees which one each lane actually sorts by, and `edges-malformed.txt`, one semicolon-less row among valid ones, which every lane must reject with a nonzero exit and empty stdout. The 10^7 tier is generated on demand under the gitignored data dir and never regenerated mid-wave. The 10^9 input, 13795438436 bytes,

about 13.8 GB, is built once on measurement day, verified, and deleted. Gates never touch it, the sizing discipline of book 17, ch 35 (the interview repertoire, back of the envelope, said aloud) applied to a test ladder.

Every lane gates the same way: byte diff against the committed fixtures, the comparison trimming the line terminator on both sides because text-mode stdio on windows ends the line with `\r\n` in ten of the variants, a split the divergence paragraph below pins. The c, rust, java, lua, and sqlite variants ride `tools/run-1brc.ps1`, which compiles or invokes each lane, diffs stdout against all three fixtures, and adds two must-fail legs: every runner variant has to reject `edges-malformed.txt` with a nonzero exit and empty stdout, and the two fixed-window tuned roads, c and java, have to reject a generated row longer than their 1 MiB window instead of silently folding it and dropping the rest of the file. The go, c sharp, and javascript lanes ride the book's wildcard suites with real test files, 15 go test functions under vet and the race detector, 29 xunit facts expanding to 33 cases, 35 node –test cases, and the python lane carries 60 plain asserts in `test_onebrc.py` executed by the runner's py leg.

41.2. the exact arithmetic [TDD]

The official reference solution is a double pipeline. Values parse with `Double.parseDouble`, min, max, and sum accumulate as doubles, the mean is `sum / count`, and printing applies `round(x) = Math.round(x * 10.0) / 10.0`, where `Math.round` is `floor(v + 0.5)`, half-up toward plus infinity. A float pipeline over money would be a defect, and the money-is-cents law of book 17, ch 17 (the interview repertoire, database answers) says why. Here the same shape is exact, and the proof is the chapter's first discovery.

Every input is `k/10` for an integer `k`, so the running double sum tracks the exact rational `sum_tenths / 10`. The exact mean is a rational whose denominator is the station's row count, so it either sits exactly on a half integer or keeps a distance of at least `1 / (2 * count)` from one. At the official sample scale the accumulated noise sits orders of magnitude below that gap. At the full `10^9` scale the honest statement is narrower: per-station sums reach the 1e8 range, where the worst-case bound for a pathological summation order tightens to the gap's own order, while the random arrival order a real stream produces keeps the drift about four orders below it, and the official evaluation produced its own expected output through this same double pipeline, so no artifact either side can check separates the two roads. `round(sum * 10.0)` therefore lands on the exact integer tenths sum on every input a fixture can pin, and the printed mean is always `floor(sum_tenths / count + 1/2)`. The double pipeline is exact rational arithmetic in disguise at any checkable scale, so every lane computes it in integers and removes the question entirely:

```
mean_tenths = (2 * sum_tenths + count) // (2 * count)
```

Floor division, exact, and order-independent, since integer sums commute and merge order cannot move a byte of output. The oracle `tools/1brc/reference_average.py` implements exactly this and validates byte-exact against all 12 official sample pairs, re-verified 12 of 12 while writing this chapter. The oracle is binary end to end for a reason the wave itself hit: its first text-mode version crashed on a cp1252 console that refused the station names and wrote `\r\n` under a windows redirect, so it now reads bytes and writes through `sys.stdout.buffer`, byte-exact by construction on any console and any redirect, which is what lets the 12-of-12 claim say byte and mean it.

The floor is asymmetric on purpose. `floor(x + 0.5)` rounds halves toward plus infinity: a mean of +0.05 prints 0.1 while −0.05 prints 0.0. The edge battery pins it twice over, station `a` with rows 0.0 and 0.1 printing 0.1 against station `aa` with rows −0.1 and 0.0 printing 0.0, and the six-row mix pair doing the same at three tenths above and below zero. This is neither banker's rounding nor half away from zero, and the sqlite lane below shows a database whose `round()` does the latter, which is why the REAL column road is banned there.

Minus zero needs one sanitation rule. A `-0.0` row parses to the integer 0, integer tenths carry no negative zero, and 0 formats as `0.0`, never `-0.0`. In the reference solution the same effect arrives for free because `Math.round` returns a long, and a long has no negative zero either.

Floor division is load-bearing in languages that truncate. C, java, c sharp, and javascript divide integers toward zero, so each lane bumps the quotient when the remainder is negative, `Math.floorDiv` in java, a hand-written

`floor_div` in c. The `neg` station in the edge battery is the pin: rows 1.2, −3.4, and 0.1 sum to −21 tenths over 3 rows, the numerator is −39 over 6, truncation says −0.6, the floor says −0.7, and the pinned output is −0.7.

Sorting is byte order in disguise. Stations sort by codepoint order, which equals UTF-8 byte order on well-formed input, which is what `memcmp`, go and python byte comparison, and sqlite BINARY collation all already do. The trap belongs to three languages whose native string comparison walks UTF-16 code units: java, c sharp, and javascript all equal codepoint order for BMP names and split from it for supplementary characters, where a surrogate pair sorts below U+E000. The first cuts of this chapter's java naive, c sharp naive, and both javascript variants sorted that way, caught by the closing attack round: on supplementary names they disagreed with the byte lanes, and java and c sharp even disagreed with their own tuned variants. Every variant now ends in a byte or codepoint comparison, the tuned lanes over raw UTF-8, java naive through `Arrays.compareUnsigned` on the encoded bytes, c sharp naive through a UTF-8 byte comparer, javascript through an allocation-free `codePointAt` walk. The official corpus and the fixture are BMP-only, and `measurements-20` among the official samples carries supplementary characters, emoji and a racing car appended to 19 station names, but as an identical suffix on every name, a shape where UTF-16 unit order still coincides with codepoint order, so no official artifact distinguishes the two orders. The committed astral battery does, and the gate runs it on every variant. The edge battery that pins the rest of this is 37 lines:

```
a;0.0
a;0.1
aa;-0.1
aa;0.0
St. Louis;99.9
St. Louis;-99.9
Ångström;-0.0
Ångström;0.3
Ångström;-0.3
Zürich;12.3
Zürich;12.3
Zürich;12.3
日本;50.0
日本;-50.0
日本;0.0
日本;0.1
solo;-42.1
mix;0.1
mix;0.1
mix;0.1
mix;0.0
mix;0.0
mix;0.0
mixn;-0.1
mixn;-0.1
mixn;-0.1
mixn;0.0
mixn;0.0
mixn;0.0
big;-99.9
big;-99.9
big;-99.9
big;-99.9
big;99.9
neg;1.2
neg;-3.4
neg;0.1
```

listing 159: the edge battery: the half asymmetry in a against aa and mix against mixn, minus zero, both rails, bmp sort, prefix families, the solo station (source: interview-repertoire/samples/ch41-fixture/edges.txt, lines 1-37)

41.3. the sqlite lane [TDD]

The aggregate can live in a query. Both scripts import the text file through the official 3.53.4 shell and print the one-line map, byte-diffed by the runner against both fixtures. The import shape is shared: `.mode csv` with `.separator ';'` splits each row, UTF-8 names pass through as raw bytes, and the import is wrapped in one `BEGIN`/`COMMIT` because per-row autocommit would fsync per row, the file-level locking story of book 13, ch 7 (infrastructure: docker, databases, and testing, sqlite: transactions and isolation). Both schemas also declare their columns `NOT NULL`, which makes this the loud lane on malformed rows: a line with no `;` imports a NULL temperature, the constraint rejects it, and `.bail on` aborts the throwaway build with a nonzero exit and empty stdout instead of aggregating NULL tenths into silent garbage.

The textbook road is banned before it starts. A REAL temperature column finished with `round(avg(temp), 1)` is wrong twice: sqlite's `round()` is half-away-from-zero, so +0.05 prints 0.1 while −0.05 prints −0.1 against the contract's 0.1 and 0.0, and `avg()` sums binary floats, which the aggregate contract forbids. Storing REAL and re-deriving tenths is no escape either, since `CAST(temp * 10 AS INTEGER)` truncates a product that can sit an ulp below

the integer. So both scripts keep the value as TEXT and parse it with `CAST(digit-text AS INTEGER)`, an exact decimal-text-to-integer conversion, plus `unicode(digit) - 48` for the single fractional digit.

The parse carries a sign trap. `-99.9` is `-(99*10 + 9) = -999`, not `-99*10 + 9 = -981`, so the whole tenths value is negated, never the integral part alone. The `-0.0` row hides the bug, 0 either way, and the `-99.9` row exposes it. Both ride the edge battery. Integer division is the second trap, probed on the pinned shell: `/` on two integers truncates toward zero and `%` takes the dividend's sign, so the mean is `quot - (rem < 0)`, the same bump every other lane writes. The naive script parses at query time:

```
WITH tenths AS (
  SELECT station,
         CASE WHEN substr(temp, 1, 1) = '-'
           THEN -(CAST(substr(temp, 2, instr(temp, '.') - 2) AS INTEGER) * 10
                 + unicode(substr(temp, length(temp), 1)) - 48)
           ELSE CAST(substr(temp, 1, instr(temp, '.') - 1) AS INTEGER) * 10
                + unicode(substr(temp, length(temp), 1)) - 48
         END AS tenths
  FROM measurements
),
agg AS (
  SELECT station, min(tenths) AS mn, max(tenths) AS mx,
         sum(tenths) AS total, count(*) AS cnt
  FROM tenths GROUP BY station
),
floored AS (
  SELECT station, mn, mx,
         (2 * total + cnt) / (2 * cnt) AS quot,
         (2 * total + cnt) % (2 * cnt) AS rem
  FROM agg
),
means AS (
  SELECT station, mn, mx,
         CASE WHEN rem < 0 THEN quot - 1 ELSE quot END AS mean
  FROM floored
)
SELECT '{' || group_concat(
         station || '='
         || printf('%s%d.%d', CASE WHEN mn < 0 THEN '-' ELSE '' END,
                   abs(mn) / 10, abs(mn) % 10) || '/'
         || printf('%s%d.%d', CASE WHEN mean < 0 THEN '-' ELSE '' END,
                   abs(mean) / 10, abs(mean) % 10) || '/'
         || printf('%s%d.%d', CASE WHEN mx < 0 THEN '-' ELSE '' END,
                   abs(mx) / 10, abs(mx) % 10),
         ', ' ORDER BY station) || '}'
FROM means;
```

listing 160: query-time tenths from text, the floored mean with the remainder bump, group_concat in station order (source: interview-repertoire/samples/ch41-sqlite/naive.sql, lines 66-101)

The tuned script's junction is where the tenths parse runs. A `GENERATED ALWAYS AS ... STORED` column declared in the schema makes `.import` itself produce the integer, computed once per row at insert, so the query becomes a pure integer scan and the database artifact is worth keeping. The two alternatives were measured at the 10^7 tier and rejected: a staging pass into a `WITHOUT ROWID` table lands the same pure scan but pays a second full write pass and leaves the dropped staging table's pages at the high-water mark, a 422 MB file against the shipped shape's 232 MB, and a covering index on `(station, tenths)` makes the GROUP BY stream in index order but costs more to build than the sort it saves, 27.82 s total against the shipped shape. `temp_store = MEMORY` was screened too, 21.94 s, 61 percent over the shipped total, rejected because the full-scale sort spill would then sit in RAM.

```
.bail on
PRAGMA page_size = 8192;
PRAGMA cache_size = -262144;
PRAGMA synchronous = OFF;
.mode csv
.separator ';'
BEGIN;
CREATE TABLE measurements(
  station TEXT NOT NULL,
  temp TEXT NOT NULL,
  tenths INTEGER GENERATED ALWAYS AS (
    CASE WHEN substr(temp, 1, 1) = '-'
      THEN -(CAST(substr(temp, 2, instr(temp, '.') - 2) AS INTEGER) * 10
            + unicode(substr(temp, length(temp), 1)) - 48)
      ELSE CAST(substr(temp, 1, instr(temp, '.') - 1) AS INTEGER) * 10
           + unicode(substr(temp, length(temp), 1)) - 48
    END) STORED);
.import tools/1brc/data/measurements.txt measurements
COMMIT;
.mode list
```

listing 161: the pragma ladder, the stored tenths column, one transaction around the import (source: interview-repertoire/samples/ch41-sqlite/tuned.sql, lines 62-81)

The pragma ladder is three lines and every one is load-bearing. `page_size = 8192` before the first table trims about one level off the b-tree depth for a wide append, `cache_size = -262144` holds 256 MiB of pages, and `synchronous = OFF` drops the per-commit fsync a throwaway, rebuildable database does not need. `journal_mode` and `locking_mode` are skipped for a stricter reason than speed: they are row-returning pragmas, they print result rows, and `journal_mode = off` also refuses to engage in-script on this shell build, printing `delete`, so emitting either would corrupt the one-line stdout contract.

Measured at 10^7 on the re-measured box: naive 13.9 s total against tuned 14.5 s, the two totals inside box variance of each other at this tier, though the stream-day box had tuned ahead, 13.67 against 16.03. The phases still tell the tuned story: run as separate import and query halves, naive spends 5.39 s importing and 9.20 s querying while tuned spends 7.24 s importing and 4.13 s querying, the stored column moving about 1.9 s of parsing from query to import and cutting the query to less than half, the same trade a materialized view makes. The tuned halves sum under the one-process total because a standalone query rides the page cache the import just warmed, one more reason the full-scale run, one process per variant, is the number that decides: 1923 s against 2400 s there, 0.80x, with database files of 23.6 GB against 21.1 GB, the stored column's price.

41.4. c: the window and the table [EWC]

C has no standard hash map, and that absence is the lane's story. The naive road, 100 sloc, is `fopen` plus `fgets` one line at a time, `sscanf` for the value, a flat station array walked by `strcmp` on every row, `qsort` at the end: 5.473 s at 10^7 and 808.4 s at 10^9, match both times, exact arithmetic throughout, since the contract forbids binary floats even on the slow road.

The tuned ladder, 186 sloc, makes three decisions. Reading: one 1 MiB `fread` window swept in place, `memchr` to each newline and semicolon, the partial tail carried to the front on refill so a name is never split across window edges. Parsing: character arithmetic into integer tenths, no float, no libc call. The dictionary: an open-addressing table with FNV-1a over the raw name bytes and linear probing, built insert-or-find only, because names repeat all day and a station is never deleted, so there are no tombstones and no delete path at all:

```
// insert-or-find: returns the slot for this name, creating it empty on a
// miss. length equal plus memcmp decides the hit; the probe steps by one.
static size_t find_slot(const unsigned char *name, size_t length,
                        uint64_t hash) {
  size_t slot = (size_t)hash & (TABLE_SLOTS - 1);
  for (;;) {
    if (slot_offset[slot] == SLOT_EMPTY) {
      if (population * 4 ≥ TABLE_SLOTS * 3)
        die("hash table past 75 percent load");
      if (arena_fill + length > ARENA_BYTES)
        die("name arena exhausted");
      // slot_length is uint16_t, so a longer name would silently truncate
      if (length > UINT16_MAX)
        die("station name too long");
      slot_offset[slot] = (uint32_t)arena_fill;
      slot_length[slot] = (uint16_t)length;
      memcpy(arena + arena_fill, name, length);
      arena_fill += length;
      population++;
      return slot;
    }
    if (slot_length[slot] == length &&
        memcmp(arena + slot_offset[slot], name, length) == 0)
      return slot;
    slot = (slot + 1) & (TABLE_SLOTS - 1);
  }
```

listing 162: insert-or-find with linear probing, the name copied into the arena once on first sight, the 65k length cap dying loudly (source: interview-repertoire/samples/ch41-c/onetuned.c, lines 49-74)

Each name is copied into a 4 MiB byte arena exactly once, on first sight, and the slot arrays are parallel, offset and length apart from the accumulators, so a probe walk touches cache lines that hold nothing but keys. The window sweep that feeds it:

```
  size_t fill = 0;
  size_t pos = 0;
  for (;;) {
    // find the next complete line in the window; on a miss carry the
    // partial tail to the front and refill, so a name is never split
    unsigned char *newline =
        memchr(window + pos, '\n', fill - pos);
    if (!newline) {
      size_t carry = fill - pos;
      memmove(window, window + pos, carry);
      fill = carry + fread(window + carry, 1, WINDOW_BYTES - carry, file);
      pos = 0;
      if (fill == carry) {
        // a zero-byte read is only EOF when the window had room; a full
        // window is one row bigger than WINDOW_BYTES, and folding it as
        // a final line would silently discard the rest of the file
        if (carry == WINDOW_BYTES) {
          int next = fgetc(file);
          if (next != EOF) {
            ungetc(next, file);
            die("row longer than the read window");
          }
        }
        // end of file: leftover bytes are one last line without a newline
        if (carry) {
          unsigned char *separator = memchr(window, ';', carry);
          if (!separator)
            die("line without ';'");
          size_t name_length = (size_t)(separator - window);
          update_slot(find_slot(window, name_length,
                                hash_name(window, name_length)),
                      parse_tenths(separator + 1, window + carry));
        }
        break;
      }
      continue;
    }

    unsigned char *separator =
        memchr(window + pos, ';', (size_t)(newline - (window + pos)));
    if (!separator)
      die("line without ';'");
    size_t name_length = (size_t)(separator - (window + pos));
    update_slot(find_slot(window + pos, name_length,
                          hash_name(window + pos, name_length)),
                parse_tenths(separator + 1, newline));
    pos = (size_t)(newline - window) + 1;
  }
```

listing 163: the fread window: memchr to the newline, the partial tail carried to the front, the one-byte probe past a full window, the final line without its newline (source: interview-repertoire/samples/ch41-c/onetuned.c, lines 157-204)

The window guards its own edges, a defect class the closing attack round exposed in three lanes: when a full window holds no newline, a one-byte probe past it decides between a true end of file, which folds the window as one final line, and more file, which dies as a row longer than the window instead of silently discarding everything after it. The same round found the arena's 16-bit slot length silently truncating names past 65535 bytes, which now dies too, far outside the contract's 100-byte bound.

`floor_div` is load-bearing exactly as the arithmetic section pinned, c's / truncates toward zero and the `neg` station needs the bump. `_CRT_SECURE_NO_WARNINGS` silences the ucrt deprecation warnings the clang plus msvc-header lane emits. The ladder stops at the serial tuned road: 0.423 s at 10^7, 12.9x over naive, and 58.6 s at 10^9. The window-plus-carry shape below is the one the whole chapter shares, the pattern the c sharp and javascript lanes would adopt mid-wave when their whole-file roads hit the wall:

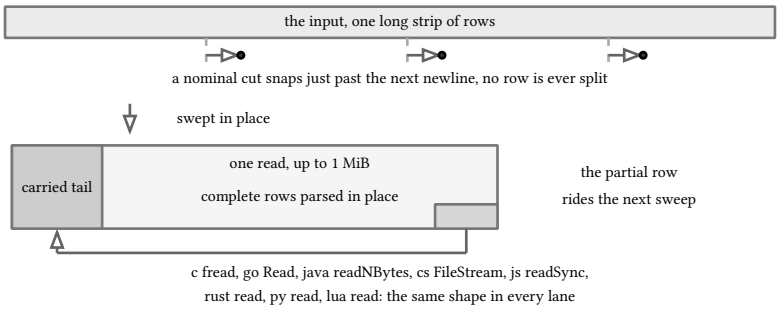


figure 186: one window swept in place, the tail carried forward, cuts snapped past newlines

41.5. rust: byte walks and an even table [EWC]

The rust naive road, 109 sloc, is `fs::read_to_string` plus `lines()` plus `split_once` into a `HashMap<&str, _>` folded through the entry api, the whole file in one `String`, the java naive memory shape: 0.599 s at 10^7. The arithmetic is integer tenths on the slow road too, and the value parser is strict from the first cut, digits, one dot, one fractional digit, then the row end, anything else dies loudly.

The tuned road, 337 sloc, is the chapter's shared window in rust form, and the window carries the lane's own language cost: std exposes no memchr, so the newline and the semicolon walks are plain byte loops over the window slice, a scan c, go, c sharp, and javascript ride a library call for while rust and java hand-roll it, neither standard library shipping a byte search:

```
fn handle_line(line: &[u8], table: &mut Table) {
    let Some(semi) = line.iter().position(|&byte| byte == b';') else {
        die("line without ';'");
    };
    let tenths = parse_tenths(&line[semi + 1..]);
    table.observe(&line[..semi], tenths);
}

fn scan<R: Read + ?Sized>(reader: &mut R, table: &mut Table) -> io::Result<()> {
    let mut buffer = vec![0u8; SCAN_START];
    let mut carry = 0usize;
    loop {
        if carry == buffer.len() {
            buffer.resize(buffer.len() * 2, 0);
        }
        let read = reader.read(&mut buffer[carry..])?;
        if read > 0 {
            let data = carry + read;
            let mut pos = 0usize;
            while pos < data {
                let Some(nl) = buffer[pos..data].iter().position(|&byte| byte == b'\n') else {
                    break;
                };
                handle_line(&buffer[pos..pos + nl], table);
                pos += nl + 1;
            }
            buffer.copy_within(pos..data, 0);
            carry = data - pos;
        } else {
            if carry > 0 {
                handle_line(&buffer[..carry], table);
            }
            return Ok(());
        }
    }
}
```

listing 164: the window walk: position loops for the newline and the semicolon, copy_within carrying the tail, the doubling escape for a line longer than the window, the final line without its newline (source: interview-repertoire/samples/ch41-rust/onetuned.rs, lines 159-194)

The dictionary is the chapter's open-addressing table, FNV-1a over the raw name bytes with `wrapping_mul`, linear probing, an append-only name arena, insert-or-find only because a station is never deleted, growth at 70 percent load by rebuilding the slot array over entries whose arena spans never move:

```
    fn find(&mut self, name: &[u8]) -> usize {
        let mask = (self.slots.len() - 1) as u64;
        let mut slot = fnv1a(name) & mask;
        loop {
            let occupied = self.slots[slot as usize];
            if occupied == 0 {
                let start = self.names.len() as u32;
                self.names.extend_from_slice(name);
                self.entries.push(Entry {
                    name_start: start,
                    name_end: self.names.len() as u32,
                    min: 0,
                    max: 0,
                    sum: 0,
                    count: 0,
                });
                self.slots[slot as usize] = self.entries.len() as u32;
                if self.entries.len() * 10 ≥ self.slots.len() * 7 {
                    self.grow();
                }
                return self.entries.len() - 1;
            }
            let entry = &self.entries[(occupied - 1) as usize];
            if &self.names[entry.name_start as usize..entry.name_end as usize] == name {
                return (occupied - 1) as usize;
            }
            slot = (slot + 1) & mask;
        }
    }

    fn grow(&mut self) {
        self.slots = vec![0; self.slots.len() * 2];
        let mask = (self.slots.len() - 1) as u64;
        for (index, entry) in self.entries.iter().enumerate() {
            let name = &self.names[entry.name_start as usize..entry.name_end as usize];
            let mut slot = fnv1a(name) & mask;
            while self.slots[slot as usize] ≠ 0 {
                slot = (slot + 1) & mask;
            }
            self.slots[slot as usize] = (index + 1) as u32;
        }
    }
```

listing 165: insert-or-find with fnv probing and linear steps, the name appended to the arena once on first sight, grow rebuilding the slots at 70 percent load (source: interview-repertoire/samples/ch41-rust/onetuned.rs, lines 33-74)

The dictionary junction was measured both ways, the go lane's discipline. The scan and parse swap bought 11 percent, 0.599 s down to 0.532 s over the std map keyed by the raw name bytes under the default SipHash hasher, and the FNV table landed at 0.528 s, inside box noise of the map. At 10k stations a probe is one touch either way and the byte walks dominate, so the dictionary choice is not where rust's time goes at this tier, the same conclusion the go lane drew from its two orderings: measure, do not assert.

Two mechanism notes the other lanes hand-roll. Rust's / truncates toward zero like c's, and the mean folds through `i64::div_euclid`, floor division shipped in std, the `neg` station's bump without hand-writing it. And both sorts are byte comparisons by construction: std's `Ord for str` is `as_bytes().cmp(other.as_bytes())` in the pinned 1.99.0 source, the tuned lane sorts the raw name slices the same way, and utf-8 byte order is codepoint order, the astral battery's gate proof.

The final junction is `--parallel`. Nominal cuts snap forward past newlines with 4 KiB probes, one `thread::scope` worker per `available_parallelism` slot opens its own `File`, seeks to its start, and feeds the same `scan` through a `take(end - start)` reader, so the shard road reuses the serial scan unchanged, and the master absorbs shard tables by exact integer sums so arrival order cannot move a byte. Measured: 0.088 s wall at 10^7, 6.8x over naive, spending 0.906 s of cpu, wall bought with cpu like every parallel row. Both variants write raw bytes, `println!` and `write_all` do no windows newline translation, so the lane ends `}\n` top to bottom.

41.6. go: map against table, measured [EWC]

The go ladder keeps both dictionary roads and measures the gap instead of asserting it. The naive road, 110 sloc, is `bufio.Scanner` handing one allocated string per line to `strings.Cut` and a stdlib map, 0.670 s at 10^7 and 91.6 s at 10^9, match both times. The tuned scan is the chapter's shared window in go form, one 1 MiB raw `Read` walked with `bytes.IndexByte`, the carry `copy` moving the partial tail to the front, and the buffer doubling whenever a pathological single line fills it without a newline:

```
func scanLines(r io.Reader, observe func(name []byte, tenths int)) error {
  handle := func(line []byte) error {
    semi := bytes.IndexByte(line, ';')
    if semi < 0 {
      return fmt.Errorf("line without ';': %q", line)
    }
    t, err := parseTenthsBytes(line[semi+1:])
    if err ≠ nil {
      return err
    }
    observe(line[:semi], t)
    return nil
  }
  buf := make([]byte, scanWindow)
  carry := 0
  for {
    if carry == len(buf) {
      grown := make([]byte, len(buf)*2)
      copy(grown, buf[:carry])
      buf = grown
    }
    n, rerr := r.Read(buf[carry:])
    if n > 0 {
      data := buf[:carry+n]
      pos := 0
      for pos < len(data) {
        nl := bytes.IndexByte(data[pos:], '\n')
        if nl < 0 {
          break // partial line stays in the buffer
        }
        if err := handle(data[pos : pos+nl]); err ≠ nil {
          return err
        }
        pos += nl + 1
      }
      carry = copy(buf, data[pos:]) // memmove the partial tail to the front
    }
    if rerr ≠ nil {
      if errors.Is(rerr, io.EOF) {
        break
      }
      return rerr
    }
  }
  if carry > 0 {
    return handle(buf[:carry])
  }
  return nil
}
```

listing 166: one raw window walked with indexByte, the carry memmove, the doubling escape for a line longer than the window (source: interview-repertoire/samples/ch41-go/onetuned.go, lines 39-87)

`scanLines` takes an `io.Reader` and consumes it with plain `Read`, which is what lets the sharded road hand it an `io.SectionReader` over the same file with zero changes. The value folds into integer tenths by byte arithmetic at the scan site, zero allocation per line. The dictionary junction: `OnetunedMap` keeps the stdlib map, probing with `agg[string(name)]`, the one compiler-blessed allocation-free bytes-to-string conversion, while `Onetuned` swaps in a slice-backed open-addressing table with FNV-1a, linear probing, and an append-only name arena:

```
// find returns the entry for name, inserting it on first sight. The
// returned pointer stays valid until the next find: grow only rebuilds
// slots, and appends happen before the pointer is taken.
func (t *stationTable) find(name []byte) *tableEntry {
  mask := uint64(len(t.slots) - 1)
  i := fnv1a(name) & mask
  for {
    slot := t.slots[i]
    if slot == 0 {
      start := int32(len(t.names))
      t.names = append(t.names, name...)
      t.entries = append(t.entries, tableEntry{
        nameStart: start,
        nameEnd:   int32(len(t.names)),
      })
      t.slots[i] = uint32(len(t.entries))
      if len(t.entries)*10 ≥ len(t.slots)*7 {
        t.grow()
      }
      return &t.entries[len(t.entries)-1]
    }
    e := &t.entries[slot-1]
    if int(e.nameEnd-e.nameStart) == len(name) &&
      bytes.Equal(t.names[e.nameStart:e.nameEnd], name) {
      return e
    }
    i = (i + 1) & mask
  }
}
```

listing 167: the slice-backed table: fnv probe, arena span, grow on load, the entry pointer stable across appends (source: interview-repertoire/samples/ch41-go/onetuned.go, lines 148-176)

Measured at 10^7 on the re-measured quiet box, the scan and parse swap bought 51 percent, naive 0.670 s down to the map road's 0.326 s, and the table swap lost ground right there, 0.378 s, 16 percent behind the map. The stream-day box, loaded by a foreign six-core process, had flipped the tier the other way, table 0.519 s ahead of the map's 0.587 s, and full scale put the map back ahead, 42.1 s against 53.3 s on the same 413-station input. Two boxes, two orderings at the tier: dictionary micro-tuning is load- and tier-sensitive, so measure at the scale and on the box you ship. Both variants stay gated, which is why the flips were seen at all.

The final junction is `OnetunedSharded`, 325 sloc total for the tuned file. Nominal cuts at `size * i / shards` snap forward past the next newline with 4 KiB probes, one scanner goroutine per GOMAXPROCS slot scans its `io.SectionReader` shard through the same `scanLines`, and `mergeFrom` folds shard tables by exact integer sums, race-clean under `go test -race`, the goroutine discipline of book 17, ch 15 (the interview repertoire, go runtime: scheduler, gc, patterns). 0.057 s at 10^7, 11.8x over naive, and 7.7 s at 10^9.

41.7. java: byte windows and a pinned charset [EWC]

The java naive road, 63 sloc, is `Files.readString` plus `split` plus a `HashMap<String, long[]>`: 1.557 s at 10^7, then dead in 0.3 s at 10^9 with `Required array size too large`, the 2^31 cap on a single array. The tuned road, 236 sloc, reads through a `FileChannel` behind a 1 MiB window, probes stations by raw name bytes with FNV-1a and parallel slot arrays, and parses tenths by byte arithmetic.

Output is a charset decision before it is a speed decision. `System.out` encodes with the platform codepage when redirected, so the naive variant pins a UTF-8 `PrintStream` over `FileDescriptor.out` and the tuned variant writes raw bytes straight through `System.out.write`, untouched by any charset. The fixture stations include non-ASCII names,

so this pin is not cosmetic: the same class of failure surfaced while re-verifying the oracle for this chapter, a cp1252 console refusing the station names until the interpreter ran UTF-8.

The parallel junction splits the file into byte ranges, one `scanRange` per worker on an `ExecutorService` fixed pool above a 4 MiB threshold. The align rule is the subtlety: a worker with a nonzero start must land on a line boundary first, and reading from `start - 1` makes a boundary already sitting on a line start eat one byte instead of one line:

```
// scan every complete line whose first byte lies in [start, limit).
// a caller with a nonzero start must land on a line boundary first.
static StationTable scanRange(Path path, long start, long limit)
    throws IOException {
  StationTable table = new StationTable();
  try (FileChannel channel = FileChannel.open(path, StandardOpenOption.READ)) {
    if (start > 0) {
      // the worker before us owns the line straddling start, so discard
      // through the newline that ends it. reading from start - 1 makes
      // the newline-at-boundary case eat one byte instead of one line
      channel.position(start - 1);
      InputStream align = Channels.newInputStream(channel);
      int letter = align.read();
      while (letter != '\n' && letter != -1)
        letter = align.read();
      if (letter == -1)
        return table;
    }

    InputStream input = Channels.newInputStream(channel);
    byte[] window = new byte[WINDOW_BYTES];
    long windowBase = channel.position(); // file offset of window[0]
    int fill = 0;
    int pos = 0;
    for (;;) {
      if (windowBase + pos ≥ limit)
        break; // the next line belongs to the next worker
```

listing 168: the align rule: read from start minus one, discard through the newline, the window carried like the c lane's (source: interview-repertoire/samples/ch41-java/Onetuned.java, lines 138-164)

Merging absorbs each shard table by exact integer sums, so shard arrival order cannot move the output:

```
    // the merge junction: fold a finished shard in by exact integer
    // sums, so the result never depends on shard arrival order
    void absorb(StationTable shard) {
      for (int slot = 0; slot < TABLE_SLOTS; slot++) {
        if (shard.names[slot] == null)
          continue;
        int into = probeArray(shard.names[slot]);
        if (counts[into] == 0) {
          minimums[into] = shard.minimums[slot];
          maximums[into] = shard.maximums[slot];
          sums[into] = shard.sums[slot];
          counts[into] = shard.counts[slot];
        } else {
          minimums[into] = Math.min(minimums[into], shard.minimums[slot]);
          maximums[into] = Math.max(maximums[into], shard.maximums[slot]);
          sums[into] += shard.sums[slot];
          counts[into] += shard.counts[slot];
        }
      }
    }
```

listing 169: absorb folds a finished shard by exact integer sums, min and max widened, sums and counts added (source: interview-repertoire/samples/ch41-java/Onetuned.java, lines 41-60)

The window carries the same guard the c lane learned: a full window with no newline probes one byte past it through the channel, and a byte there means a row longer than the window, an `IOException`, never a silent fold that drops the rest of the file.

Measured: 0.159 s wall at 10^7 with the 20-thread junction on, 9.8x over naive, spending 1.484 s of cpu, roughly three and a half times the c lane's cpu for two fifths of its wall, the junction's whole story. At 10^9 on a loaded box, 7 s wall against 84 s cpu.

41.8. c sharp: byte keys and snapped cuts [EWC]

The c sharp naive road streams: `File.ReadLines` yields per-line strings into a `Dictionary`, 0.921 s at 10^7 and 69.7 s at 10^9, match both times, because `ReadLines` never holds the file. The tuned road, 288 sloc, opens a `FileStream` with `bufferSize: 1`, so the window reads go straight to the OS without a second buffering layer, sweeps a 1 MiB window with span `IndexOf` for the newline and semicolon, and folds tenths by byte arithmetic.

The dictionary is byte-keyed: FNV-1a over the raw UTF-8 name bytes, linear probing, 16384 slots for the 10k-station contract, load factor near 0.61 before any growth. Each slot keeps the canonical `byte[]` and its decoded `string` as a pair, both allocated on first sight and never per row, and the final sort compares the UTF-8 bytes directly, which is codepoint order, no detour needed:

```
    // Positive result: the slot holding this name. Negative result: the
    // empty slot at ~result, ready for insertion.
    private static int Probe(Table table, ReadOnlySpan<byte> name)
    {
        int mask = table.Names.Length - 1;
        int slot = (int)(Fnv1a(name) & (ulong)mask);
        while (table.Texts[slot] != null)
        {
            if (table.Names[slot].AsSpan().SequenceEqual(name)) return slot;
            slot = (slot + 1) & mask;
        }
        return ~slot;
    }

    private static ulong Fnv1a(ReadOnlySpan<byte> bytes)
    {
        ulong hash = 14695981039346656037;
        foreach (byte current in bytes) hash = (hash ^ current) * 1099511628211;
        return hash;
    }
```

listing 170: the byte-keyed probe and fnv-1a over utf-8 name bytes, the pair claimed on the empty slot (source: interview-repertoire/samples/ch41-cs/src/Onetuned.cs, lines 282-301)

The final junction snaps and merges. Nominal cuts walk forward to the byte after a newline through small positional reads, rows are short so one 4 KiB probe is almost always enough and the loop only exists for honesty, then `Parallel.For` scans each part through its own window and table, and the calling thread merges with exact integer arithmetic, `Console.Out.Write` all the way, never `WriteLine`:

```
    // The final junction: line-aligned file ranges scanned in parallel,
    // each part through its own window, then merged on the calling thread
    // with exact integer arithmetic. Deterministic output regardless of
    // scheduling, and no whole-file buffer anywhere: the cuts snap
    // forward with one small positional read per boundary.
    public static string RunParallel(string path, int windowBytes = DefaultWindowBytes)
    {
        long length;
        using (var probe = OpenRead(path, FileOptions.RandomAccess))
        {
            length = probe.Length;
        }
        int parts = (int)Math.Clamp(length, 1L, Environment.ProcessorCount);
        long[] bounds = LineBounds(path, length, parts);
        var tables = new Table[parts];
        Parallel.For(0, parts, i =>
        {
            var local = new Table(InitialSlots);
            using (var stream = OpenRead(path, FileOptions.SequentialScan))
            {
                ScanStream(stream, bounds[i], bounds[i + 1], local, windowBytes);
            }
            tables[i] = local;
        });
        Table merged = tables[0];
        for (int i = 1; i < parts; i++)
        {
            for (int slot = 0; slot < tables[i].Texts.Length; slot++)
            {
                if (tables[i].Texts[slot] != null)
                {
                    AddEntry(merged, tables[i].Names[slot], tables[i].Texts[slot]!,
                        tables[i].Min[slot], tables[i].Max[slot], tables[i].Sum[slot],
tables[i].Count[slot]);
                }
            }
        }
        return Finish(merged);
    }
```

listing 171: snapped line bounds, parallel.for over per-part windows and tables, merge on the calling thread (source: interview-repertoire/samples/ch41-cs/src/Onetuned.cs, lines 85-122)

This lane paid for the chapter's sharpest lesson. Its first tuned road, like javascript's, read the whole file, and the 2 GB cap on a single array only speaks up at 10^9: the fix rebuilt both lanes on the c lane's windowed chunks with carry regressions added to the suites, told with the full-scale table below. Measured after the fix, Release builds: 0.363 s tuned at 10^7, 2.5x over naive, 0.114 s parallel, 8.1x, and at 10^9, 26.7 s tuned, 2.7 s parallel.

41.9. javascript: the map that stays [EWC]

The javascript naive road, 84 sloc, is `readFileSync` as one utf8 string, `split` per line and per field, a `Map` keyed by station. At 10^9 it does not die quickly, it dies slowly, killed at the commit cap mid grind with 14.9 GB resident, the slow-death version of the read-all wall. The tuned road, 213 sloc, keeps the same `Map` and changes everything around it: positional `readSync` into a 1 MiB `Buffer` window, `Buffer.indexOf` riding memchr for the newline and the semicolon, tenths off the raw byte codes, and the name decoded to a string once per row as the map key:

```
// codes with the validation folded into the same byte loads (optional
// minus, digits, the dot, one fraction digit, then exactly the row end
// at `end`: a dot-chasing scan runs past the row into stale bytes and
// never stops on its own), Map update keyed by the decoded name. No
// arrays, no line strings, no parseFloat.
function sampleRow(window, pos, semi, end, stations) {
  let cursor = pos + semi + 1;
  let negative = window[cursor] === MINUS;
  if (negative) cursor += 1;
  let tenths = 0;
  let digits = 0;
  let code;
  while (cursor < end && (code = window[cursor]) ≥ DIGIT_ZERO && code ≤ DIGIT_ZERO + 9) {
    tenths = tenths * 10 + (code - DIGIT_ZERO);
    digits += 1;
    cursor += 1;
  }
  const dot = cursor;
  if (
    digits === 0 ||
    dot + 2 =/= end ||
    window[dot] =/= DOT ||
    window[dot + 1] < DIGIT_ZERO ||
    window[dot + 1] > DIGIT_ZERO + 9
  ) {
    throw new Error(`malformed row: ${JSON.stringify(window.toString("utf8", pos, end))}`);
  }
  tenths = tenths * 10 + (window[dot + 1] - DIGIT_ZERO);
  if (negative) tenths = -tenths;

  const name = window.toString("utf8", pos, pos + semi);
  const station = stations.get(name);
  if (station === undefined) {
    stations.set(name, [tenths, tenths, tenths, 1]);
  } else {
    if (tenths < station[0]) station[0] = tenths;
    if (tenths > station[1]) station[1] = tenths;
    station[2] += tenths;
    station[3] += 1;
  }
}
```

listing 172: one row: the semicolon bounded inside the row, the value shape checked in the fold, tenths off byte codes, the map keyed by the decoded name (source: interview-repertoire/samples/ch41-js/src/onetuned.mjs, lines 139-179)

Both roads validate the row shape as they parse, a hardening the closing attack round forced. The first tuned cut searched for the semicolon without bounding it to the row, so a semicolon-less line made it invent station names containing newlines and exit 0, and the naive cut spun forever on a value with no dot, `charCodeAt` past the end returning NaN forever. The semicolon now has to land inside the row, the value has to be an optional minus, digits, one dot, one digit, then the row end, and anything else dies loudly with a nonzero exit.

The per-row decode is deliberate, and it is the lane's measured negative result: v8 hashes a string once and caches the hash on the canonical instance, so the built-in `Map` answers each probe without rehashing, and a javascript-level open-addressing table re-implements interning the runtime already provides, loses, and costs sloc. The other lanes' table builds buy real cache locality on parallel slot arrays, and this lane's honest tuned:naive is 1.05x, 2.005 s against 2.098 s at 10^7, v8's string machinery already at the floor: the serial tuned road pays for its window

with the same per-row decode the naive road gets nearly for free from `split`, and the serial ladder buys essentially nothing until the worker junction.

The final junction is `worker_threads`: bounds snapped forward past newlines by the parent with one small `readSync` window per cut, each worker reading its isolated range into its own buffer, the parent merging exact integer sums, `process.stdout.write`, never `console.log`. Workers are capped at 8. Measured: 0.374 s at 10^7, 5.6x over naive, and 38.1 s wall against 301.4 s cpu at 10^9.

41.10. python: bytes windows, processes at the end [EWC]

The python naive road, 65 sloc, is text-mode `readlines`, one `str` per line into a dict, exact integer tenths all the way. 9.035 s at 10^7, then `MemoryError` at 99.7 s at 10^9. The tuned road, 173 sloc, reads 1 MiB bytes chunks, finds `;` and `\n` with `bytes.find`, and folds the value into tenths straight from the bounds, never materializing the value as a slice. The dict is keyed by the bytes name slice, hashed as bytes, skipping the UTF-8 decode for all but the 10k first sights, and decoded once per station at render:

```
def absorb(agg, data, start, stop):
    """fold the line data[start:stop] (no newline) into agg."""
    semi = data.find(b";", start, stop)
    if semi < 0:
        raise ValueError(f"line without ';': {bytes(data[start:stop])!r}")
    value = tenths(data, semi + 1, stop)
    name = data[start:semi]
    cur = agg.get(name)
    if cur is None:
        agg[name] = [value, value, value, 1]
        return
    if value < cur[0]:
        cur[0] = value
    if value > cur[1]:
        cur[1] = value
    cur[2] += value
    cur[3] += 1


def scan(handle, agg, limit=None):
    """walk every complete line of the binary handle into agg; limit caps
    the number of bytes consumed (shards read exact spans). the final
    line may lack its newline, which the caller decides to keep."""
    remaining = limit
    carry = b""
    while remaining is None or remaining > 0:
        want = CHUNK if remaining is None else min(CHUNK, remaining)
        chunk = handle.read(want)
        if not chunk:
            break
        if remaining is not None:
            remaining -= len(chunk)
        data = carry + chunk if carry else chunk
        start = 0
        while True:
            stop = data.find(b"\n", start)
            if stop < 0:
                break
            absorb(agg, data, start, stop)
            start = stop + 1
        carry = data[start:]
    return carry
```

listing 173: absorb folds a line by bounds with no value slice, scan walks chunks with find and a carry (source: interview-repertoire/samples/ch41-py/onetuned.py, lines 49-90)

The serial ladder buys 1.16x, 7.771 s at 10^7. That is the honest ceiling of pure-python parsing, the interpreter is the floor, and the measurement says so instead of hiding it. The win is the final junction: `--parallel` snaps spans past newlines with a 4096-byte probe, one process per span on a `ProcessPoolExecutor`, the gil forcing the process boundary the other lanes do not need, book 17, ch 29 (the interview repertoire, python answers). 1.381 s at 10^7, 6.5x wall, while cpu inflates to 14.4 s across 20 workers, wall time bought with cpu. At 10^9: 138 s parallel against 1222.6 s serial.

41.11. lua: interning is the hash map [EWC]

Lua's lane, 44 and 88 sloc, is the smallest, and its decisions are mostly about what not to build. The naive road is `io.lines`, one string per row behind the stdio buffer, `string.match` for the split, `tonumber` on digit text as an exact

text-to-integer conversion, a plain table, `table.sort` at the end: 8.589 s at 10^7 and 1271.9 s at 10^9, match both times, because `io.lines` streams.

The tuned road reads 1 MiB blocks with `file:read(CHUNK)` and scans them with plain-mode `string.find`, which runs at C speed over the block while the interpreter pays per row only for the handful of bytes that matter. The value parses from `string.byte` arithmetic into lua 5.5 integers, no substring, no `tonumber`, no float. The map is a plain table keyed by the station string, and that is the junction: lua interns short strings, up to 40 bytes, and every corpus name fits, so the per-row `sub` either finds the interned copy or interns it once, and the stats lookup rides an already-hashed key. A custom open-addressing map keyed by the raw byte range would save one `sub` per row and re-implement the intern table the language already gives, measured not worth it in stdlib lua:

```
local function scan(block)
  local pos = 1
  while true do
    local stop = find(block, "\n", pos, true)
    if not stop then
      return sub(block, pos)
    end
    local semi = find(block, ";", pos, true)
    if not semi or semi ≥ stop then
      error("row without ';'")
    end
    local name = sub(block, pos, semi - 1)
    -- the value is exactly [-]digits '.' digit between semi+1 and stop-1:
    -- parse it from bytes, no substring, no float, no tonumber, and
    -- validate the shape before folding, because the fold reads the
    -- byte after the integral run and must never reach past the row's
    -- end into the next row's first byte
    local i = semi + 1
    local negative = byte(block, i) == 45 -- '-'
    if negative then
      i = i + 1
    end
    local digits_from = i
    local value = 0
    local c
    while true do
      c = byte(block, i)
      if not c or c < 48 or c > 57 then
        break
      end
      value = value * 10 + c - 48
      i = i + 1
    end
    -- strict shape: at least one integral digit, then '.', then
    -- exactly one fraction digit as the row's last byte. c is the
    -- byte that stopped the digit run, so no second fetch for it
    local frac = byte(block, i + 1)
    if i == digits_from or c ~= 46 or not frac or frac < 48 or frac > 57 or i + 1 ~= stop - 1 then
      error("malformed value")
    end
    local tenths = value * 10 + frac - 48
    if negative then
      tenths = -tenths
    end
    local st = stats[name]
    if st then
```

```
      if tenths < st[1] then
        st[1] = tenths
      end
      if tenths > st[2] then
        st[2] = tenths
      end
      st[3] = st[3] + tenths
      st[4] = st[4] + 1
    else
      stats[name] = { tenths, tenths, tenths, 1 }
    end
    pos = stop + 1
  end
end
```

listing 174: plain-mode find over the block, string.byte tenths validated in the fold, the interned table key, the carry returned as the tail (source: interview-repertoire/samples/ch41-lua/onetuned.lua, lines 56-115)

Formatting is integer math end to end, `magnitude // 10` and `magnitude % 10`, never a float, and the sort rides `string <`, which is strcmp byte order in every lua, locale never involved, verified by file probe on this box. Two divergences are stated: argv with non-ASCII bytes is mojibake here, so the fixture is referenced by path only, and the parallel road is honestly absent. Stdlib lua 5.5 has no threads and no process spawning, and coroutines are cooperative on one OS thread, so they cannot shorten wall time. The tuned fold validates the value shape now, the closing attack round's fix for a fold that once read the next row's first byte as the fraction digit, and the strictness costs about 0.19 s at this tier, measured against the pre-validation binary on the same box. The ladder tops out at 3.202 s at 10^7, 2.7x over naive, and 422.1 s at 10^9, serial. That absence is the lane's own lesson: the final junction belongs to the language, and lua's stdlib simply does not carry one.

41.12. the full-scale table [TDD]

Two tables, two disciplines. The 10^7 tier is best of 3, re-measured wholesale on a quiet box the day the closing hardening round ended, so every cell shares one day and one box, the number a reader can reproduce on any machine with the generator; the streams' first pass had run on a box loaded by a foreign six-core process, where cells ran up to nearly 80 percent slower, the memory-bound lanes most. The 10^9 measurement day is one run per variant on that loaded box, the number that tells the truth about memory shape. The input is 13795438436 bytes, the expected line from the reference aggregator. Wall seconds, sloc is non-comment non-blank:

lane	naive s	tuned s	tuned:naive	parallel s	parallel:naive	sloc n/t
c	5.473	0.423	12.9x	-	-	100/186
rust	0.599	0.528	1.1x	0.088	6.8x	109/337
go	0.670	0.378	1.8x	0.057	11.8x	110/325
java	1.557	0.159	9.8x	(same build)	(same)	63/236
c sharp	0.921	0.363	2.5x	0.114	8.1x	88/288
javascript	2.098	2.005	1.05x	0.374	5.6x	84/213
python	9.035	7.771	1.16x	1.381	6.5x	65/173
lua	8.589	3.202	2.7x	absent	-	44/88
sqlite	13.9	14.5	0.96x	-	-	44/46

Four rows carry footnotes. The java tuned figure is the 20-thread junction already on, the 10^7 file clears the 4 MiB threshold. The go tuned figure is the table variant, with the map intermediate at 0.326 s. The sqlite figures are one-process totals, with the import and query halves measured separately at 5.39 plus 9.20 for naive and 7.24 plus 4.13 for tuned, the query alone at 0.45x. The rust row is the one block not sharing the re-measure day: the

lane joined at the rust wave, measured on the wave 4 box with the same warmup plus best-of-3 protocol, std-map intermediate 0.532 s. C sharp rows are Release builds, the Debug numbers on the stream-day box for the record are 117.8, 104.5, and 9.8.

The full-scale table, one run per variant, wall and cpu seconds, tree cpu for the parallel lanes:

lane	variant	wall s	cpu s	outcome
c	onenaive	808.4	803.3	match
c	onetuned	58.6	58.2	match
java	Onenaive	0.3	-	out of memory: Files.readString, required array size too large
java	Onetuned	7	84	match, 20-thread junction
go	onenaive	91.6	94.1	match
go	onetuned-map	42.1	41.5	match
go	onetuned-table	53.3	53	match, map wins at full scale
go	onetuned-sharded	7.7	71	match
c sharp	onenaive	69.7	68.8	match, streaming ReadLines, Release
c sharp	onetuned	26.7	26	match, windowed chunks, Release
c sharp	onetuned-parallel	2.7	43.6	match, snapped Parallel.For cuts, Release
javascript	onenaive	824	818.5	killed at the commit cap, 14.9 GB resident mid read-all grind
javascript	onetuned	194.2	191.4	match, windowed chunks
javascript	onetuned-workers	38.1	301.4	match, windowed worker shards
python	onenaive	99.7	94.4	MemoryError at readlines
python	onetuned	1222.6	1213.6	match
python	onetuned-parallel	138	1968.4	match
lua	onenaive	1271.9	1265	match
lua	onetuned	422.1	419.9	match
sqlite	naive	2400	2209.8	match, db 21.1 GB
sqlite	tuned	1923	1895	match, db 23.6 GB, 0.80x total

The read-all wall is this table's own lesson. Java dies in 0.3 s on the 2^31 array cap, javascript grinds at 14.9 GB resident until killed, python hits `MemoryError` at 99.7 s, while the streaming naive roads, c `fgets`, go scanner, c sharp `ReadLines`, lua `io.lines`, and every streaming import finish. The challenge is a memory-shape problem before it is a speed problem.

One dating note: these rows are the measurement-day binaries, taken before the closing attack round's parse validation, window guards, and sort comparators landed. The 10^7 table above carries the hardened binaries, where lua's added validation costs about 6 percent, isolated against the pre-validation binary in back-to-back runs on the same box, and the c and java guards run once per file and the comparators once per station, below noise. The rust lane joined after measurement day and carries no 10^9 row here.

PITFALL: the tier a gate never sees

The tuned c sharp and javascript roads originally read the whole file too. Both passed every gate, both measured clean at the 10^7 tier, and both were wrong about the only tier that counts: the 2 GB caps on a single array and a single Buffer only speak up at 10^9. The fix, commits `9bc2b19f` and `c386646e`, rebuilt both on the c lane's windowed chunks and added small-window carry regressions to the suites, 28 of 28 and 25 of 25 then, 33 of 33 and 35 of 35 after the closing round's astral and malformed cases, so the failure class now fails small. A fixture gate cannot see what only the full-scale input exposes, which is why the measurement day exists and why these rows name their dead honestly.

41.13. the divergences that stay [TDD]

Malformed input is out of contract, and the fleet's answer to it is pinned rather than assumed. Structurally broken rows, a missing semicolon, a value with no dot, a row longer than a fixed window, die loudly everywhere: the runner's must-fail leg holds a nonzero exit and empty stdout for all thirteen variants it gates, and the c sharp and javascript suites carry the same cases. Value text that violates only the shape splits the fleet, and the split is the lesson: the oracle, python, go, and rust reject `1.20` outright, javascript and lua tuned reject it after the hardening, while the c, java, and c sharp tuned roads fold it to 1.2 and sqlite reads only the final digit, so the one-fractional-digit rule is enforced by the lanes that parse text strictly and merely survived by the rest. Blank lines are rejected by the oracle and silently skipped by the naive java and javascript roads, one more reason the contract, not the lane, is the arbiter.

Two splits ride the input's bytes rather than its rows. A `\r` inside a station name is legal by the official letter and divides text-mode line readers, where c, c sharp, and python naive, plus the oracle's first text-mode cut, died, from the byte readers that keep it as data. A leading BOM is stripped by c sharp naive's `File.ReadLines` while every byte lane keeps it in the first name. So the chapter pins LF-terminated, BOM-free files as the input shape it teaches. The final newline splits by stdout mode: ten variants write through windows text-mode streams and end `}\r\n` when redirected, c both, java naive, python all three, lua both, and sqlite both, while the byte-writing lanes, tuned java, go, c sharp, javascript, and rust, end `}\n` like every official sample. That is why the gate's diff trims the terminator on both sides and why the byte claim in this chapter's opening lives on the payload.

sources: the challenge rules, the reference solution, the data generator, the shell wrappers, and all 12 sample pairs are pinned verbatim from 1brc.dev and github.com/gunnarmorling/1brc at main, both accessed 2026-10-08, kept under `tools/1brc/ref/` with sha256 records. sqlite behavior was probed directly on the corpus-pinned 3.53.4 shell rather than read from documentation, the lua collation claim was verified by file probe, the rust `str` ordering claim was read from the std source of the pinned rustc 1.99.0 toolchain, and every language claim in the lane sections is pinned by the lane's own source under `samples/ch41-*/` and its gate. No other online source was consulted for this chapter.

42. appendices: question index and sources

This appendix is the audit the book promised in its first chapter: every question in the inventory this book was commissioned from mapped to the labeled section that answers it. TDD sections exist as failing-test-first code under `make verify`, EWC sections walk code that runs under the same gate, DRILL sections are spoken answers with their follow-ups. A question missing from this table is a gap you can see, and a section without a label is one you can grep.

VERIFY: how to audit this table

Every row's section label is part of the chapter files under `chapters/`, appended to each section heading, and every TDD or EWC row names a workspace under `samples/` whose suite runs in the book's verify step. Grep the label, open the section, run the suite.

42.1. the question to section index

question	answered in	label
meta and landscape		
what changed in interviews 2024 to 2026, ai rounds, narration	landscape, ch 1, [the whole chapter]	DRILL
paradigms, oop versus fp, in go and c#	paradigms, ch 2, [oop sections, fp section]	EWC
mvc, mvvm, ecs	paradigms, ch 2, [where state lives]	DRILL
data structures and algorithms		
implement the seven core data structures	datastructures, ch 3, [all seven sections]	TDD
dfs, bfs, when each	graphs, ch 4, [dfs and bfs section]	TDD
shortest path on an unweighted graph	graphs, ch 4, [shortest path section]	TDD
topological sort, course schedule	graphs, ch 4, [kahn's section]	TDD
reverse a string	classics1, ch 5, [reverser section]	TDD
palindrome check	classics1, ch 5, [reverser section]	TDD
is prime, primes under n	classics1, ch 5, [primes section]	TDD
permutations, anagram detection	classics1, ch 5, [permutations section]	TDD
linked list reverse, middle, cycle	classics1, ch 5, [linked list section]	TDD
two sum, sorted and unsorted	classics1, ch 5, [two sum section]	TDD
count isolated islands	classics1, ch 5, [islands section]	TDD
climbing stairs, the dp ladder	classics2, ch 6, [the ladder section]	TDD
trie and autocomplete	classics2, ch 6, [trie section]	TDD

question	answered in	label
string manipulation, rle, unique char, anagram groups	classics2, ch 6, [string manipulation section]	TDD
javascript		
hoisting, scope, var let const	js-core, ch 7, [hoisting section]	EWC
closures, the loop capture gotcha	js-core, ch 7, [closures section]	EWC
operator precedence and coercion	js-core, ch 7, [precedence section]	EWC
null versus undefined versus nan, optionals	js-core, ch 7, [absences section]	EWC
prototypes and the class sugar	js-core, ch 7, [prototypes section]	EWC
the event loop, microtasks, timers	js-core, ch 7, [event loop section]	EWC
what is the dom, querySelector	js-dom, ch 8, [what the dom is]	EWC
center a div	js-dom, ch 8, [four ways section]	EWC
class versus arrow functions, this	js-dom, ch 8, [class versus arrow]	EWC
map filter reduce	js-dom, ch 8, [arrays section]	EWC
async await versus promise.all	js-dom, ch 8, [arrays section]	EWC
es6 features	js-dom, ch 8, [es6 shapes]	EWC
word count in one loop	js-fromscratch, ch 9, [word count]	TDD
an event emitter from scratch	js-fromscratch, ch 9, [emitter]	TDD
map filter reduce from scratch	js-fromscratch, ch 9, [array methods]	TDD
a promise from scratch, async await on it	js-fromscratch, ch 9, [promise subset]	TDD
a signal from scratch	js-fromscratch, ch 9, [signals core]	TDD
csv to mongodb migration	js-data, ch 10, [all three sections]	TDD, DRILL
react		
function versus class components	react-core, ch 11, [function versus class]	EWC
props, children, prop drilling	react-core, ch 11, [props and drilling]	EWC
how the virtual dom works	react-core, ch 11, [virtual dom]	EWC

question	answered in	label
the built-in hooks, custom hooks	react-hooks, ch 12, [hooks sections]	EWC
react-router	react-hooks, ch 12, [router section]	EWC
react 19, actions, use, useOptimistic, useActionState	react-hooks, ch 12, [react 19 section]	EWC
the react compiler, signals status	react-hooks, ch 12, [react 19 section]	EWC
context versus redux	react-state, ch 13, [context versus store]	EWC
rtk query caching	react-state, ch 13, [what rtk query automates]	EWC
graphql, rest, the n+1 problem	react-state, ch 13, [graphql section]	EWC
build a listing page against a public api	react-build, ch 14, [beats one and two]	TDD
a virtual dom from scratch	react-build, ch 14, [beat three]	TDD
go systems		
parallelism versus concurrency	go-runtime, ch 15, [definitions]	EWC
go scheduler versus os threads	go-runtime, ch 15, [scheduler section]	EWC
the garbage collector, go 1.26 gc changes	go-runtime, ch 15, [green tea section]	DRILL
design patterns, concurrent patterns, closures, generics	go-runtime, ch 15, [patterns, closures, generics sections]	EWC
context, timeouts, error wrapping	go-runtime, ch 15, [context section]	EWC
sync.Once singleton, race safety	go-runtime, ch 15, [singleton section]	TDD
mocking and dependency injection	go-testing, ch 16, [fakes and layers]	TDD
testing db, service, handler, callback layers	go-testing, ch 16, [the four layers]	TDD
golden-master testing with the -update flag	go-testing, ch 16, [the golden master, ported small]	EWC
bit-identical twins across languages	go-testing, ch 16, [determinism discipline section]	DRILL
acid versus base	db-answers, ch 17, [acid section]	DRILL
postgres cursors, limit offset, jsonb, money	db-answers, ch 17, [cursors section, money section]	DRILL, TDD
normalization and indexing	db-answers, ch 17, [normalization, indexing]	TDD

question	answered in	label
isolation levels, optimistic versus pessimistic	db-answers, ch 17, [locking section]	TDD, DRILL
mongo geo, point in polygon	db-answers, ch 17, [mongo geo]	TDD
http, tcp, udp, websocket, graphql	network-answers, ch 18, [stack, echo servers, ws]	TDD, DRILL
https, certificates, symmetric versus asymmetric	network-answers, ch 18, [https section]	DRILL
scaling with and without cloud	design-answers, ch 19, [scaling section]	DRILL
gateway, reverse proxy, kubernetes	design-answers, ch 19, [gateway section]	DRILL
bottlenecks, tracing, logging, metrics	design-answers, ch 19, [bottlenecks section]	DRILL
cap, event-driven, orchestration versus choreography	design-answers, ch 19, [cap section]	DRILL
producer consumer and offsets	design-answers, ch 19, [remaining inventory]	DRILL
queue before the database write	design-answers, ch 19, [queue first section]	DRILL
s3 listing, 10gb from postgres, 10gb file in go	design-answers, ch 19, [remaining inventory]	DRILL
cloud cron	design-answers, ch 19, [remaining inventory]	DRILL
a notification system	design-answers, ch 19, [remaining inventory]	DRILL
300k concurrent users	design-answers, ch 19, [the 300k sketch]	DRILL
parallel url fetcher, bounded workers	go-build1, ch 20, [fetcher section]	TDD
rest api with caching and rate limiting	go-build1, ch 20, [rest api section]	TDD
schema generator and parser	go-build1, ch 20, [schema section]	TDD
the client-side rate limiter against an upstream	go-build1, ch 20, [client-side limiter section]	TDD
inventory storefront, no oversell	go-build2, ch 21, [storefront section]	TDD
graceful shutdown, live deploy	go-build2, ch 21, [shutdown and deploy]	TDD
bolt a realtime feed onto the service	go-build2, ch 21, [the feed, step by step]	TDD

question	answered in	label
jetstream batch producer and consumer	go-streaming, ch 22, [produce, consume]	TDD
durable consumer, redelivery, offsets	go-streaming, ch 22, [durable and ack floor]	TDD
state that survives a broker restart	go-streaming, ch 22, [restart test]	TDD
kafka versus jetstream	go-streaming, ch 22, [the comparison paragraph]	TDD, DRILL
algorithms part 3 and weighted graphs		
longest substring without repeating characters	algopatterns, ch 23, [window section]	TDD
two pointers, sorted pair sum, in-place dedupe	algopatterns, ch 23, [two pointers section]	TDD
binary search and the rotated array	algopatterns, ch 23, [binary search section]	TDD
greedy, jump game, interval scheduling	algopatterns, ch 23, [greedy section]	TDD
backtracking, subsets and permutations	algopatterns, ch 23, [backtracking section]	TDD
bit manipulation, single number, counting bits	algopatterns, ch 23, [bits section]	TDD
sorting internals, which sort when	algopatterns, ch 23, [sorting section]	DRILL
dijkstra, heap-driven shortest path	weightedgraphs, ch 24, [dijkstra section]	TDD
union-find with path compression	weightedgraphs, ch 24, [union-find section]	TDD
kruskal and prim, the mst pair	weightedgraphs, ch 24, [mst section]	TDD
when bellman-ford and floyd-warshall	weightedgraphs, ch 24, [negative and all-pairs section]	DRILL
distributed and systems floors		
replication, leaders, followers, and lag	distributed-answers, ch 25, [replication section]	DRILL
sharding and partitioning strategies	distributed-answers, ch 25, [sharding section]	DRILL
consensus and raft, honestly	distributed-answers, ch 25, [raft section]	DRILL
crdts versus distributed transactions	distributed-answers, ch 25, [crdts section]	DRILL
delivery semantics and idempotency	distributed-answers, ch 25, [delivery section]	DRILL

question	answered in	label
stack, heap, and allocators	systems-answers, ch 26, [stack and heap section]	DRILL
virtual memory, pages, and the tlb	systems-answers, ch 26, [virtual memory section]	DRILL
processes versus threads	systems-answers, ch 26, [processes section]	DRILL
epoll and the event loop	systems-answers, ch 26, [epoll section]	DRILL
c networking, sockets end to end	systems-answers, ch 26, [sockets section]	DRILL
handbook floors: math, mining, languages		
asymptotics and solving recurrences	math-answers, ch 27, [asymptotics section]	DRILL
probability you can compute at the board	math-answers, ch 27, [probability section]	DRILL
counting and combinatorics	math-answers, ch 27, [counting section]	DRILL
reading statistics, baselines, p-values, overfitting	math-answers, ch 27, [statistics section]	DRILL
exact binomial, auc, and a bootstrap interval, run live	math-answers, ch 27, [statistics workspace section]	TDD
backtesting without lookahead	math-answers, ch 27, [backtesting section]	DRILL
guarded promotion and multiple testing in a game tool	math-answers, ch 27, [guarded promotion section]	DRILL
similarity and distance measures	mining-answers, ch 28, [similarity section]	DRILL
clustering, k-means, hierarchical, dbscan	mining-answers, ch 28, [clustering section]	DRILL
itemsets, apriori and fp-growth	mining-answers, ch 28, [itemsets section]	DRILL
cosine neighbors and smoothed bayes, run live	mining-answers, ch 28, [cosine neighbors section]	TDD
the levelwise pass over baskets, counted and pruned	mining-answers, ch 28, [levelwise pass section]	TDD
pagerank and the pipeline view	mining-answers, ch 28, [pagerank section]	DRILL
the gil and what it actually locks	python-answers, ch 29, [gil section]	DRILL

question	answered in	label
the data model, dunders and protocols	python-answers, ch 29, [data model section]	DRILL
decorators, closures, scopes	python-answers, ch 29, [decorators section]	DRILL
asyncio versus threads	python-answers, ch 29, [asyncio section]	DRILL
metatables and metamethods	lua-answers, ch 30, [metatables section]	DRILL
coroutines versus javascript async	lua-answers, ch 30, [coroutines section]	DRILL
closures and environments against javascript	lua-answers, ch 30, [closures section]	DRILL
async await machinery, state machines and context	csharp-answers, ch 31, [async section]	DRILL
linq as an embedded dsl	csharp-answers, ch 31, [linq section]	DRILL
value versus reference, boxing	csharp-answers, ch 31, [value and reference section]	DRILL
the gc generations against go's	csharp-answers, ch 31, [gc section]	DRILL
the web platform: css, rendering, security		
the box model, content-box versus border-box	css-answers, ch 32, [box model section]	EWC
the cascade, specificity, which rule wins	css-answers, ch 32, [cascade section]	TDD
flex, grid, grow and fr arithmetic	css-answers, ch 32, [flex and grid section]	EWC
responsive, media queries, breakpoints	css-answers, ch 32, [responsive section]	TDD
the rendering pipeline, layout, paint, composite	rendering-answers, ch 33, [layout paint composite section]	DRILL
reflow versus repaint versus compositing	rendering-answers, ch 33, [reflow section]	TDD
the accessibility tree, semantic html	rendering-answers, ch 33, [accessibility tree section]	TDD
aria, contrast, keyboard paths, wcag	rendering-answers, ch 33, [aria contrast section]	TDD
the owasp taxonomy, naming a vulnerability family	security-answers, ch 34, [the taxonomy section]	DRILL
xss and csrf, escaping by context, Same-	security-answers, ch 34, [xss and csrf section]	TDD

question	answered in	label
Site=Lax, can cors prevent csrf		
cors, the same-origin triple, preflight, the credentials error	security-answers, ch 34, [cors and the same-origin rule]	TDD
sql injection, concatenation versus placeholders, second-order	security-answers, ch 34, [sql injection section]	TDD
password storage lanes, 401 versus 403, idor, oauth oidc pkce	security-answers, ch 34, [authn, authz, tokens section]	DRILL
system design at altitude: estimation and prompts		
back of the envelope: the method and the sanity band	estimation-answers, ch 35, [the method section]	DRILL
the numbers every round expects, powers of two and the latency ladder	estimation-answers, ch 35, [the numbers section]	DRILL
storage, qps, headroom, five timed boards	estimation-answers, ch 35, [storage through boards sections]	DRILL
how to run a design prompt, the clock	design-prompts, ch 36, [the protocol section]	DRILL
design a url shortener	design-prompts, ch 36, [url shortener section]	TDD, DRILL
design a rate limiter, the distributed one	design-prompts, ch 36, [rate limiter section]	DRILL
design a news feed, fanout on write or read	design-prompts, ch 36, [news feed section]	DRILL
design a chat service, and long polling versus sse versus websockets	design-prompts, ch 36, [chat and delivery ladder sections]	DRILL
the human loop: behavioral, observed, negotiation, transfer		
tell me about yourself, the ninety second arc	behavioral-answers, ch 37, [tell me about yourself]	DRILL
the story cards, conflict, failure, ownership, proudest, wrong once	behavioral-answers, ch 37, [the story cards, the three scripts]	DRILL
why leaving, why us, weakness, why not hire	behavioral-answers, ch 37, [why leaving, why us, weakness]	DRILL
the gap, why no titles, five years, salary in a screen	behavioral-answers, ch 37, [the drill and the traps]	DRILL

question	answered in	label
the first five minutes, restating the problem, definition of done	observed-answers, ch 38, [the first five minutes]	DRILL
narration while writing, blanking, ai assist norms	observed-answers, ch 38, [narration, blanking, ai sections]	DRILL
timed mocks, the deck from this book, partner protocol	observed-answers, ch 38, [timed mocks from this book]	DRILL
take-home discipline, the debugging round, proving a fix	observed-answers, ch 38, [take-home, debugging]	DRILL
reading an offer, base, bonus, equity, benefits, scope	negotiation-answers, ch 39, [reading an offer section]	DRILL
anchoring, the offer conversation, no metrics as an asset	negotiation-answers, ch 39, [anchoring section]	DRILL
exploding offers, competing timelines, the acceleration ask	negotiation-answers, ch 39, [timelines section]	DRILL
the scripts: range answer, offer call, counter, accept, decline, after a no	negotiation-answers, ch 39, [scripts and closing sections]	DRILL
the transfer method, what transfers between languages	transfer-answers, ch 40, [the transfer method]	DRILL
our shop is java: jvm gc, virtual threads, streams, checked exceptions, spring	transfer-answers, ch 40, [java section]	DRILL
our shop is rust: ownership, option and result, no gc, the traps	transfer-answers, ch 40, [rust and traps sections]	DRILL
the performance floor: one problem, nine lanes		
the billion-row aggregation, the output contract, the exact integer-tenths arithmetic	onebrc, ch 41, [exact arithmetic, sqlite lane, full-scale table sections]	TDD
the nine-lane decision ladders, hash map versus table, windowed chunk scans, the parallel junctions	onebrc, ch 41, [the eight lane sections]	EWC

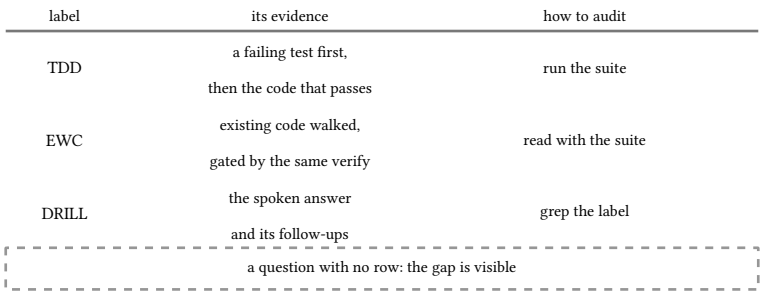


figure 187: every question maps to a label; a missing row is a visible gap

42.2. the workspaces behind the labels

workspace	contents	runner	tests
ch02-go, ch02-cs	paradigms in go and c#	go test, dotnet	7, 12
ch03-go, ch03-cs	the seven structures	go test, dotnet	25, 22
ch04-go	graph traversal and topo sort	go test	10
ch05-go, ch05-cs	classics part 1	go test, dotnet	17, 29
ch06-go, ch06-cs	classics part 2	go test, dotnet	10, 13
ch07-js	js core demos	node --test	22
ch08-js	js dom demos, styles, the centering choice	node --test	16
ch09-js	from-scratch implementables and the combinator ladder	node --test	40
ch10-js	csv to mongo migration	node --test	10
ch12-react	react workspace, react 19 + router 7	node --test	37
ch15-go	runtime, patterns, singleton	go test	16
ch16-go	four-layer testing on sqlite, golden master	go test	8
ch17-go	db answers on sqlite 3.53, geo	go test	9
ch18-go	tcp, udp, http, ws echo servers	go test	9
ch20-go	fetcher, rest api, schema, client limiter	go test	15
ch21-go	storefront, sse feed, deploys	go test	8
ch22-go	jetstream pipeline over embedded nats	go test	3
ch23-go	window, pointers, search, greedy, backtracking, bits	go test	13
ch24-go	dijkstra, union-find, kruskal, prim	go test	12
ch27-go	exact binomial, midranks, auc, bootstrap, holm and bh	go test	26
ch28-go	cosine neighbors, naive bayes, the levelwise pass	go test	22
ch32-js	css engine models and styles	node --test	14
ch33-js	pipeline, a11y, contrast models	node --test	10
ch34-js	security answer models, xss csrf cors sql headers	node --test	14
ch36-go	base62 codec, windowed counter allocator	go test	11
ch41-go	onebrc scan ladder, table against map, sharded merge	go test	15
ch41-cs	onebrc byte-key table, snapped parallel cuts	dotnet	28
ch41-js	onebrc buffer scan, windowed worker shards	node --test	25

The ch41 c, rust, java, lua, and sqlite lanes gate through `tools/run-1brc.ps1` byte diffs, and its python lane's 60 plain asserts in `test_onebrc.py` ride the same runner's py leg, counts this table does not carry.

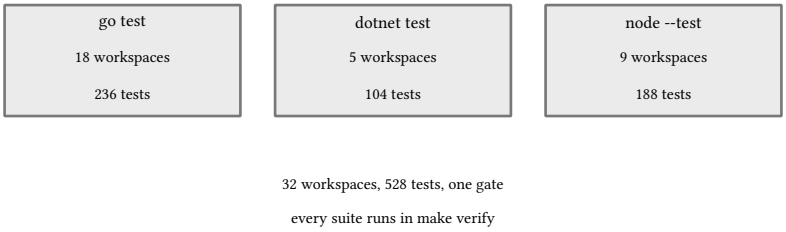


figure 188: 32 workspaces under three runners, one verify gate

42.3. pinned sources

Every online claim this book makes, with the access date it was verified:

claim	url	accessed
sqlite change log, 3.53.4 released 2026-07-24	https://www.sqlite.org/changes.html	2026-09-09
go 1.26 release notes	https://go.dev/doc/go1.26	2026-09-09
react 19, actions, use, and the compiler	https://react.dev/blog/2024/12/05/react-19	2026-09-09
tc39 signals proposal, stage tracker	https://github.com/tc39/proposal-signals	2026-09-09
nats jetstream concepts, consumers and offsets	https://docs.nats.io/nats-concepts/jetstream	2026-09-09
meta ai-enabled coding round, piloted october 2025, replaces one of two onsite coding rounds	https://interviewing.io/blog/how-to-use-ai-in-meta-s-ai-assisted-coding-interview-with-real-prompts-and-examples	2026-09-10
karat engineering interview trends 2026, 400 engineering leaders, 71 percent say ai makes technical skills harder to assess, 62 percent still ban ai in interviews, live technical interviews at 79 percent in the us	https://karat.com/engineering-interview-trends-2026/	2026-09-10
google reported pilot of an ai-assisted code comprehension round, rollout second half of 2026	https://www.entrepreneur.com/business-news/google-is-testing-a-new-rule-transform-job-interviews	2026-09-10
interviewing.io 100k interview analysis, code and problem solving outrank communication on the rubric	https://interviewing.io/blog/does-communication-matter-in-technical-interviewing-we-looked-at-100k-interviews-to-find-out	2026-09-10
2026 system design guides report a higher passing bar and ai infrastructure prompts inside design rounds	https://www.tryexponent.com/blog/system-design-interview-guide	2026-09-10
react compiler 1.0, stable october 7 2025, automatic memoization	https://react.dev/blog/2025/10/07/react-compiler-1	2026-09-10
react versions page, 19.3.0 latest, released 2026-09-09, 19.2.x patches through june 2026	https://react.dev/versions	2026-09-10
react api reference, usesyncexternalstore store binding	https://react.dev/reference/react/useSyncExternalStore	2026-09-10
rfc 6455, the websocket protocol, guid and handshake vector	https://www.rfc-editor.org/rfc/rfc6455	2026-09-10

claim	url	accessed
jsonplaceholder fake rest api, the posts endpoint the fixture captures	https://jsonplaceholder.typicode.com/posts?userId=1	2026-09-10
wcag 2.2, criteria 1.4.3 and 1.4.11, the 4.5:1 text and 3:1 non-text contrast thresholds	https://www.w3.org/TR/WCAG22/	2026-09-29
mdn, cross-origin resource sharing, the safelisted headers and the preflight triggers	https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS	2026-09-29
mdn, content-security-policy, unsafe-inline defeating script-src and the nonce overriding it	https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Content-Security-Policy	2026-09-29
mdn, strict-transport-security, max-age, includeSubDomains, and the preload list's one-year bar	https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security	2026-09-29
owasp top 10:2025, broken access control first, supply chain third, injection fallen to fifth	https://top10.owasp.org/	2026-09-29
owasp password storage cheat sheet, argon2id first at 19 MiB t=2 p=1, scrypt fallback, bcrypt legacy with the 72-byte cap	https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html	2026-09-29
rfc 6749, oauth 2.0, the four roles and four grants	https://www.rfc-editor.org/rfc/rfc6749	2026-09-29
openid foundation, how openid connect works, the id token as the outcome of an authentication process	https://openid.net/developers/how-connect-works/	2026-09-29
jep 444, virtual threads in jdk 21, pinning on synchronized blocks and native calls, no scheduler compensation, reentrantlock for hot paths	https://openjdk.org/jeps/444	2026-09-29
the rust book, ch 4, the three ownership rules and move semantics, s1 invalidated by let s2 = s1	https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html	2026-09-29
mdn, css length, the viewport units, vh equals lvh, dvh tracking the dynamic viewport as the mobile url bar moves	https://developer.mozilla.org/en-US/docs/Web/CSS/length	2026-09-29
mdn, justify-content, the safe overflow modifier, alignment falls back to start when the item overflows the container, a no-op when it fits	https://developer.mozilla.org/en-US/docs/Web/CSS/justify-content	2026-09-29

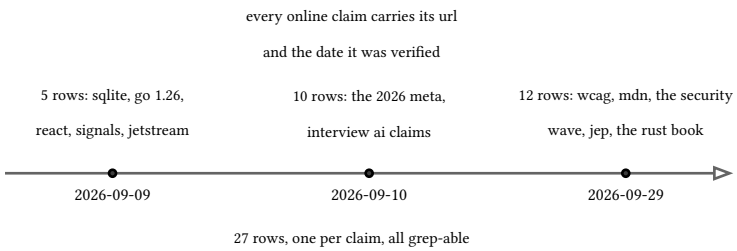


figure 189: claims pinned with urls and access dates, 2026-09-09/10/29